FPT UNIVERSITY

Capstone Project Report

Report 4 – Software Design Document

– Can Tho, October 2026 –

Table of Contents

I. Record of Changes **7**

II. Software Design Document **8**

1. System Design **8**

1.1 System Architecture 8

1.1.1 Layered Architecture 9

1.1.2 API Module Structure 10

1.2 Package Diagram 11

1.3 Assumptions and Risks 13

1.4 Design Constraints 14

1.4.1 Platform Compatibility 15

1.4.2 Integration Complexity 15

1.4.3 Performance Requirements 15

1.5 Technology Stack 15

1.5.1 NestJS Framework 16

1.5.2 Next.js and React 19 Frontend 17

1.5.3 Prisma ORM and PostgreSQL 17

1.5.4 Supporting Services 17

2. Database Design **17**

2.1 Table Relationships 18

2.2 Module Entity–Relationship Diagrams 27

3. Detailed Design **41**

3.1 Forum Management 41

3.1.1 Class Diagram 41

3.1.2 UC-14.01 – View Forum Category List 42

3.1.3 UC-14.02 – View Forum Category Detail 43

3.1.4 UC-14.03 – View Post List 44

3.1.5 UC-14.04 – View Post Detail 45

3.1.6 UC-14.05 – Create Post 46

3.1.7 UC-14.06 – Update Post 47

3.1.8 UC-14.07 – Delete Post 48

3.1.9 UC-14.08 – Like Post 49

3.1.10 UC-14.09 – Unlike Post 50

3.1.11 UC-14.10 – View Post Likes 51

3.1.12 UC-14.11 – View Post Comments . . . 52
3.1.13 UC-14.12 – Create Comment . . . 53
3.1.14 UC-14.13 – Update Comment . . . 54
3.1.15 UC-14.14 – Delete Comment . . . 55
3.1.16 UC-14.15 – Like Comment . . . 56
3.1.17 UC-14.16 – Unlike Comment . . . 57
3.1.18 UC-14.17 – Report Content . . . 58
3.2 Job Browsing . . . 59
3.2.1 Class Diagram . . . 59
3.2.2 UC-10.01 – View Job List . . . 60
3.2.3 UC-10.02 – View Job Detail . . . 61
3.3 Job Management . . . 61
3.3.1 Class Diagram . . . 62
3.3.2 UC-11.01 – View My Jobs . . . 63
3.3.3 UC-11.02 – Create Job . . . 64
3.3.4 UC-11.03 – Publish Job . . . 65
3.3.5 UC-11.04 – Update Job . . . 66
3.3.6 UC-11.05 – Delete Job . . . 67
3.3.7 UC-11.06 – Change Job Status . . . 68
3.4 Bookmarked Jobs Management . . . 68
3.4.1 Class Diagram . . . 69
3.4.2 UC-12.01 – View Bookmarked Job List . . . 70
3.4.3 UC-12.02 – Bookmark Job . . . 71
3.4.4 UC-12.03 – Remove Bookmark . . . 72
3.5 Verify JWT . . . 72
3.5.1 Class Diagram . . . 73
3.5.2 UC-00 – Verify JWT . . . 74
3.6 Authentication . . . 74
3.6.1 Class Diagram . . . 75
3.6.2 UC-01 – Login with Google . . . 76
3.6.3 UC-02 – Login . . . 77
3.6.4 UC-03 – Logout . . . 78
3.6.5 UC-04 – Register . . . 79
3.6.6 UC-05 – Forgot Password . . . 80
3.7 Role Management . . . 81
3.7.1 Class Diagram . . . 81
3.7.2 UC-06.01 – View Role List . . . 82
3.7.3 UC-06.02 – View Role Detail . . . 83

3.7.4 UC-06.03 – Create Role 84
3.7.5 UC-06.04 – Update Role 85
3.7.6 UC-06.05 – Delete Role 86
3.7.7 UC-06.06 – Update Role Permissions 87
3.8 Manage Conversation 88
3.8.1 Class Diagram 88
3.8.2 UC-27.01 – View Conversation List 89
3.8.3 UC-27.02 – Start Conversation 90
3.8.4 UC-27.03 – View Conversation Detail 91
3.8.5 UC-27.04 – Send Message 92
3.8.6 UC-27.05 – Mark Message as Read 93
3.8.7 UC-27.06 – Delete/Recall Message 94
3.8.8 UC-27.07 – Pin/Unpin Conversation 95
3.8.9 UC-27.08 – Send File 96
3.9 Saved Search Management 96
3.9.1 Class Diagram 97
3.9.2 UC-22.01 – View Saved Search List 98
3.9.3 UC-22.02 – View Saved Search Detail 99
3.9.4 UC-22.03 – Create Saved Search 100
3.9.5 UC-22.04 – Update Saved Search 101
3.9.6 UC-22.05 – Delete Saved Search 102
3.10 Job Alert Management 102
3.10.1 Class Diagram 103
3.10.2 UC-23.01 – View Job Alert List 104
3.10.3 UC-23.02 – View Job Alert Detail 105
3.10.4 UC-23.03 – Create Job Alert 106
3.10.5 UC-23.04 – Update Job Alert 107
3.10.6 UC-23.05 – Delete Job Alert 108
3.11 Permission Management 108
3.11.1 Class Diagram 109
3.11.2 UC-15.01 – View Permission List 109
3.11.3 UC-15.02 – View Permission Detail 110
3.12 Session Management 110
3.12.1 Class Diagram 111
3.12.2 UC-16.01 – View Session List 111
3.12.3 UC-16.02 – View Session Detail 112
3.12.4 UC-16.03 – Revoke Session 113
3.13 Forum Category Management 113

3.13.1 Class Diagram . . . 114
3.13.2 UC-17.01 – Create Forum Category . . . 115
3.13.3 UC-17.02 – Update Forum Category . . . 116
3.13.4 UC-17.03 – Delete Forum Category . . . 117
3.13.5 UC-17.06 – Restore Forum Category . . . 118
3.14 Manage Moderation . . . 118
3.14.1 Class Diagram . . . 119
3.14.2 UC-34.01 – View Moderation Queue . . . 119
3.14.3 UC-34.02 – View Moderation Item Detail . . . 120
3.14.4 UC-34.03 – Resolve Moderation Item . . . 121
3.15 Manage ID Verification . . . 122
3.15.1 Class Diagram . . . 122
3.15.2 UC-35.01 – View ID Verification Request List . . . 123
3.15.3 UC-35.02 – View ID Verification Request Detail . . . 124
3.15.4 UC-35.03 – Approve ID Verification . . . 125
3.15.5 UC-35.04 – Reject ID Verification . . . 126
3.16 Manage Forum Trash . . . 127
3.16.1 Class Diagram . . . 127
3.16.2 UC-36.01 – View Deleted Posts/Comments List . . . 128
3.16.3 UC-36.02 – View Deleted Posts/Comments Detail . . . 129
3.16.4 UC-36.03 – Restore Deleted Posts/Comments . . . 130
3.17 Proposal Management . . . 131
3.17.1 Class Diagram . . . 131
3.17.2 UC-32.01 – View My Proposal List . . . 133
3.17.3 UC-32.02 – View My Proposal Detail . . . 134
3.17.4 UC-32.03 – Submit Proposal . . . 135
3.17.5 UC-32.04 – Update Proposal . . . 136
3.17.6 UC-32.05 – Withdraw Proposal . . . 137
3.17.7 UC-32.06 – Accept Proposal . . . 138
3.17.8 UC-32.07 – Reject Proposal . . . 139
3.17.9 UC-32.08 – View Job Proposal List . . . 140
3.17.10 UC-32.09 – View Proposal Detail . . . 141
3.18 User Management . . . 141
3.18.1 Class Diagram . . . 142
3.18.2 UC-28.01 – View User List . . . 143
3.18.3 UC-28.02 – View User Detail . . . 144
3.18.4 UC-28.03 – Create User . . . 145
3.18.5 UC-28.04 – Update User . . . 146

3.18.6 UC-28.05 – Update Client Profile . . . 147
3.18.7 UC-28.06 – Update Freelancer Basic Info . . . 148
3.18.8 UC-28.07 – Update Freelancer Skills . . . 149
3.18.9 UC-28.08 – Update Freelancer Portfolio . . . 150
3.19 Banner Management . . . 150
3.19.1 Class Diagram . . . 151
3.19.2 UC-37.01 – View Banner List . . . 152
3.19.3 UC-37.02 – View Banner Detail . . . 153
3.19.4 UC-37.03 – Create Banner . . . 154
3.19.5 UC-37.04 – Update Banner . . . 155
3.19.6 UC-37.05 – Delete Banner . . . 156
3.20 Skill Management . . . 156
3.20.1 Class Diagram . . . 157
3.20.2 UC-38.01 – View Skill List . . . 158
3.20.3 UC-38.02 – View Skill Detail . . . 159
3.20.4 UC-38.03 – Create Skill . . . 160
3.20.5 UC-38.04 – Update Skill . . . 161
3.20.6 UC-38.05 – Delete Skill . . . 162
3.20.7 UC-38.06 – Restore Skill . . . 163
3.21 Expert Profile Management . . . 163
3.21.1 Class Diagram . . . 164
3.21.2 UC-28.09 – Update Expert Profile . . . 165
3.22 Job Category Management . . . 165
3.22.1 Class Diagram . . . 166
3.22.2 UC-46.01 – View Job Category List . . . 166
3.22.3 UC-46.02 – View Job Category Detail . . . 167
3.22.4 UC-46.03 – Create Job Category . . . 168
3.22.5 UC-46.04 – Update Job Category . . . 169
3.22.6 UC-46.05 – Delete Job Category . . . 170

I. Record of Changes

Date	A* M, D	In charge	Change Description

A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

1.1 System Architecture

Frevia follows a three-tier, deployment-separated architecture (Figure 1).

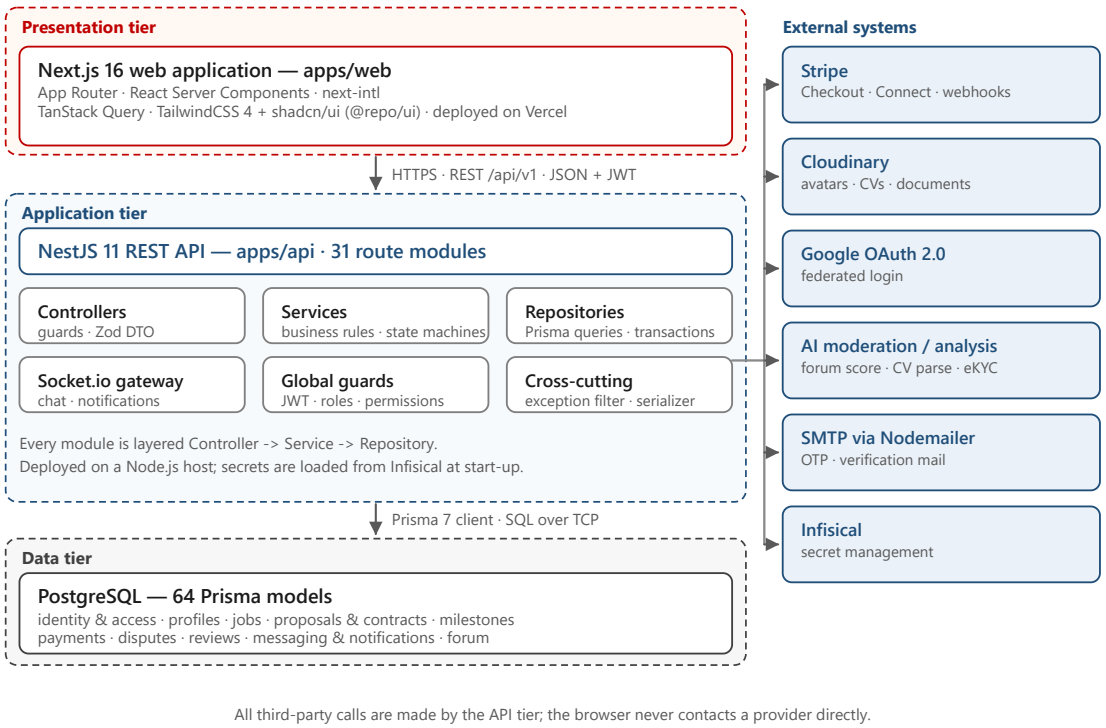

Figure 1: Overall architecture of the Frevia system

The three tiers are:

- **Presentation tier — Next.js 16 web application:** server components render public pages (job board, job detail, community forum, expert directory) and private dashboards; client components handle interactivity such as forms, likes, and chat. All server access goes through a single typed HTTP client that attaches the JWT access token and refreshes it transparently when it expires.
- **Application tier — NestJS REST API:** a modular monolith organized into 31 route modules. Every module is internally layered as Controller → Service → Repository. Controllers validate and serialize with shared Zod DTOs; services hold business rules and authorization checks; repositories encapsulate all Prisma queries.

- **Data tier — PostgreSQL via Prisma 7:** the relational store for 64 domain models. Binary files are never stored in the database but in Cloudinary; real-time transport for chat and notifications is handled by a Socket.io gateway beside the REST API.

The two applications are deployed independently (the web app on Vercel, the API on a Node.js host) and communicate only through the documented REST contract validated by the shared Zod schemas package.

Beside the three tiers, four external systems are connected to the application tier (Figure 1): **Stripe** (Checkout, Connect and webhooks) for escrow payments and payouts, **Cloudinary** for avatar, document and portfolio binaries, **Google OAuth** for federated login, and the **AI moderation/analysis service** that scores forum content, parses CVs and checks identity documents. E-mail (OTP and notifications) is delivered through SMTP with Nodemailer, and secrets are loaded from **Infisical** at start-up. None of these systems is called from the browser: the web application only talks to the API, which owns every credential and decides when a third party is contacted. The table below (Table 1) explains the API modules, and Section 1.2 explains every package of the repository.

1.1.1 Layered Architecture

The internal architecture of the API follows a strict layered pattern, enforced identically in every module (Figure 2):

- **Controller layer** — thin Routing Layer. Exposes REST endpoints, applies guards (authentication, roles, permissions), binds validated request DTOs, and delegates to a service. Controllers never query the database directly.
- **Service layer** — business logic layer. Implements use cases, ownership/authorization checks, state-machine transitions (proposal, contract, dispute), and cross-aggregation logic; it throws typed HTTP exceptions (`NotFoundException 404`, `ForbiddenException 403`, `ConflictException 409`) that a global exception filter converts to a uniform error response.
- **Repository layer** — data access layer. The only component allowed to touch Prisma; complex list queries (pagination + counting) are wrapped in `$transaction` for consistency.
- **Shared kernel** — the `@shared/types` workspace package contains Zod models, constants, permission names, and error codes shared with the web app.

Security flow: every request carries a JWT access token; the auth guard validates it and resolves the active role; role and permission guards authorize module-specific operations; refresh tokens are stored server-side in a `Session` row and can be revoked from the session management feature.

AI-assisted pipeline: user-generated content (forum posts/comments, CVs, identity documents) is passed through AI moderation or AI analysis services; forum content can be auto-approved when clean, or queued in `ModerationQueue` for administrator approval when suspicious.

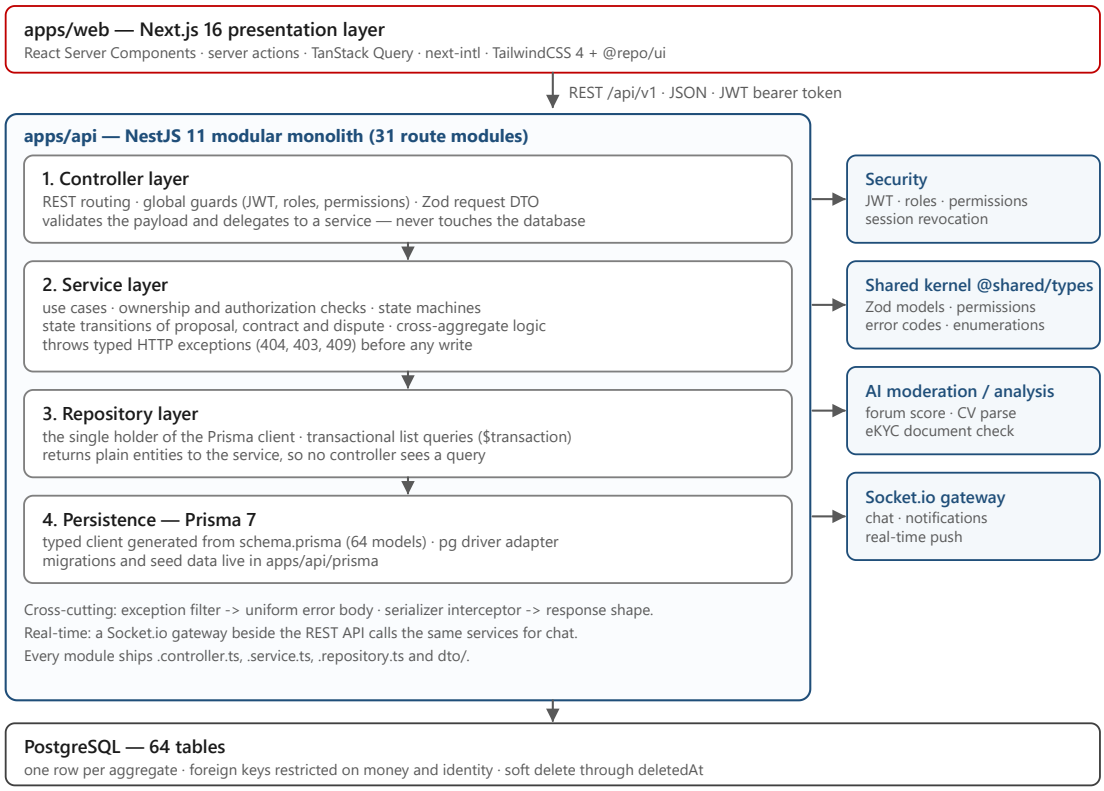

Figure 2: Layered system architecture with security and AI moderation flows

1.1.2 API Module Structure

Table 1 summarizes the 31 route modules of the API and their responsibilities.

Table 1: API route modules and responsibilities

Module	**Responsibility**
auth	registration, OTP, login, refresh token, logout, forgot password, Google OAuth, current account, role switching/joining
sessions	device session listing, detail, revocation
account-profile	general account profile, avatar, identity verification upload/AI check, client profile, social links, favorite freelancers, following
users	administrator user management and skill catalog
roles, permissions	RBAC role and permission administration
profiles	professional profile, CV upload/AI analysis, skills

Module	Responsibility
portfolios	portfolio item CRUD on profiles
expert-profile	expert profile management and public expert directory
profile-revisions	profile change request approval workflow
identity-verifications-admin	identity verification review
job-categories	public job category browsing
job-categories-admin	job category administration (soft delete + restore)
skills-admin	skill catalog administration (soft delete + restore)
browse-job	public job listing and job detail by slug
manage-job	client job management, bookmarks, skill search
proposals	proposal lifecycle: draft, submit, withdraw, interview, reject, accept
contract	contract lifecycle: create, update, sign, complete, cancel
milestone	milestone management inside a contract
milestone-file, milestone-submission	milestone attachments and work submission review
shared-file	contract shared workspace files
payments	Stripe Connect onboarding, platform fee, milestone funding, release, refund, dispute fee, transactions, webhook
disputes	dispute lifecycle (user side and admin arbitration side)
reviews	contract reviews and responses
conversations	direct messaging with real-time delivery
notifications	notification feed management
forums	forum categories and posts (public side)
forums-comment, forums-like, forums-reports	comments, likes, content reports
forums-admin	AI moderation queue, pending posts, trash restore, forum statistics
saved-search	saved job search filters
banners, banners-admin	advertisement banner delivery and administration

1.2 Package Diagram

The repository is a pnpm monorepo managed by Turborepo (Figure 3):

- `apps/web` — the Next.js application. Sub-packages group screens by actor (guest browsing, client, freelancer, admin), shared UI components (`@repo/ui`), server actions, and the i18n

message catalogs used by next-intl.

- `apps/api` — the NestJS application. `src/routes/*` contains the 31 domain modules described above; `src/shared` contains cross-cutting infrastructure: authentication guards and decorators, permission constants, global HTTP exception filter, Zod serializer interceptor, Cloudinary storage service, and Prisma client setup. Each domain module folder is organized as `<module>.controller.ts`, `<module>.service.ts`, `<module>.repository.ts`, DTO classes, and module-specific error factories (e.g., `forums.error.ts`).
- `packages/shared-type` — the single source of truth for API contracts: Zod request/response models (`model/*`), enumeration/constant definitions, and permission name lists, imported by both `apps/web` and `apps/api`.

Dependencies only point downwards: `web` and `api` both depend on `shared-type`; no route module imports another route module — cross-module data needs flow through services at the Prisma schema level.

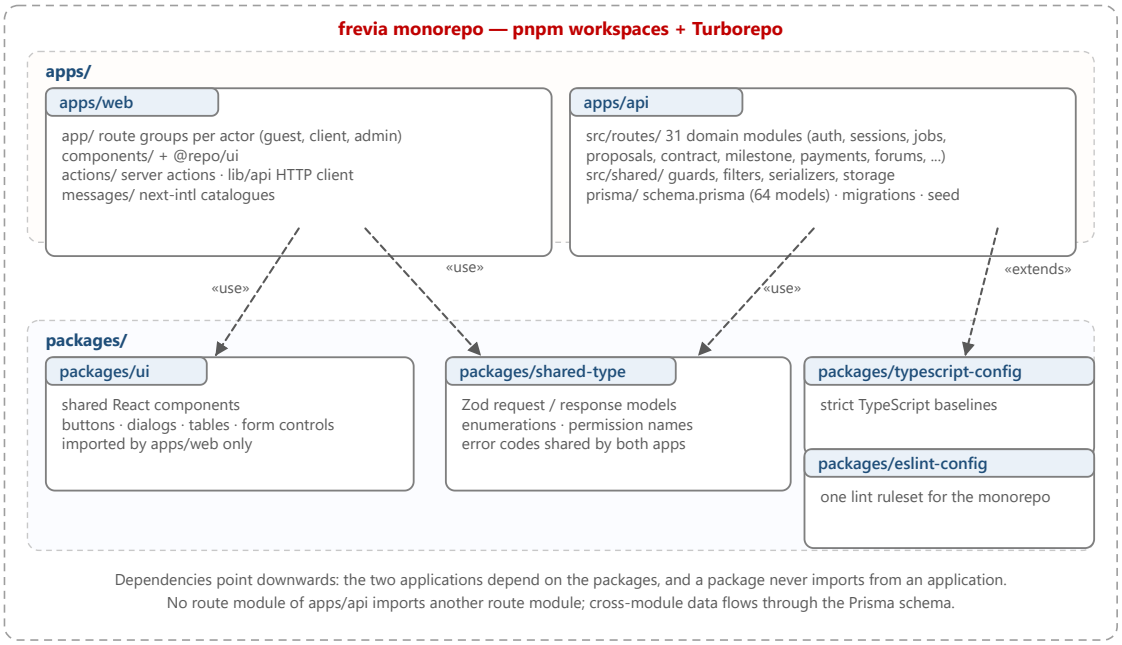

Figure 3: Package diagram of the Frevia monorepo

Package Descriptions

Table 2 lists every workspace package of the repository and its responsibility.

Table 2: Workspace packages and their responsibility

No	Package	Description
01	`apps/web`	Next.js 16 presentation layer (App Router, React Server Components, next-intl, TanStack Query, TailwindCSS 4 with shadcn/ui). Contains the screens per actor, the shared UI components, the server actions and the BFF route handlers that attach the access token.
02	`apps/api`	NestJS 11 application tier: 31 route modules laid out as Controller, Service, Repository and DTO files, together with the global guards, exception filter, Zod serializer interceptor and the Socket.io gateway used by messaging.
03	`apps/api/prisma`	Prisma 7 data layer: `schema.prisma` (64 models and their enums), migrations and seed data. The generated client is the only component allowed to query PostgreSQL.
04	`packages/shared-type`	Contract single source of truth (`@shared/types`): Zod request/response models, enumerations, permission names and error codes consumed by both applications.
05	`packages/ui`	Shared React component library (`@repo/ui`) imported by the web application so that buttons, dialogs, tables and form controls look and behave identically on every screen.
06	`packages/eslint-config`	Shared ESLint configuration (`@repo/eslint-config`) applied to every workspace so that the same rules run across the monorepo.
07	`packages/typescript-config`	Shared TypeScript configuration baselines (`@repo/typescript-config`) that the applications extend for strict compilation.

1.3 Assumptions and Risks

This system is designed based on the following assumptions:

- The Node.js runtime and the frameworks chosen for the project (NestJS 11 for the back-end and Next.js 16 for the frontend) will continue to be actively supported and maintained, ensuring long-term stability and compatibility with evolving technologies.
- The RESTful API architecture will remain a scalable and efficient choice for separating the Next.js presentation layer from the NestJS business layer.

- PostgreSQL, accessed through Prisma ORM 7, will continue to provide reliable, scalable, and secure relational data storage suitable for handling user identity, jobs, contracts, escrow payments, and forum data.
- Third-party services used by the platform — Stripe (payments and escrow), Cloudinary (file storage), Google (OAuth login), the AI moderation/analysis service, and mail delivery via Nodemailer — will stay available and API-compatible during the project lifetime.
- Modern web browsers (e.g., Chrome, Edge, Firefox, Safari) will continue to support the front-end standards required by Next.js/React 19, including HTML5, CSS3 (TailwindCSS), JavaScript ESM, and responsive rendering with React Server Components.
- The application will be deployed in environments with stable internet access to support real-time messaging (Socket.io) and online payment processing.
- Users (Guests, Clients, Freelancers, Experts, and Admins) are familiar with basic web application navigation: browsing job boards, filling forms, managing profiles, and following payment flows.

This system design is also based on the following risks:

- Any breaking change in the major versions of Next.js, NestJS, or Prisma could require refactoring of the user interface components or of the data access layer.
- Cross-browser inconsistencies or outdated device software may affect rendering or the real-time messaging experience.
- Cybersecurity threats such as credential stuffing, token hijacking, or database breaches could compromise sensitive personal and financial information if not proactively managed (the design therefore includes bcrypt-hashed passwords, JWT access/refresh tokens, session revocation, and role-based access control).
- Outages or policy changes of the integrated third-party services (Stripe, Cloudinary, Google OAuth, AI moderation) could temporarily disrupt payments, file uploads, login, or content moderation.
- Network connectivity issues could disrupt access to job proposals, contract collaboration, or checkout flows that rely on server interaction.

1.4 Design Constraints

This system should comply with the following constraints:

1.4.1 Platform Compatibility

The Frevia application must be compatible with modern web browsers (e.g., Chrome, Edge, Firefox, Safari) for all user roles including Guest, Client, Freelancer, Expert, and Admin. The frontend is implemented with Next.js 16 (App Router, React Server Components) styled with TailwindCSS 4 and shadcn/ui components; the backend is a NestJS 11 REST API. Responsive web design principles must be applied so that a consistent user experience is provided across desktop, tablet, and mobile browsers. Internationalization is achieved with next-intl so the interface can be served in multiple languages from the same code base.

1.4.2 Integration Complexity

The Frevia platform integrates a variety of components: the Next.js web application, the NestJS REST API, the PostgreSQL database (via Prisma 7 with the pg adapter), Stripe Checkout and Stripe Connect for escrow payments and payouts, Cloudinary for image and document storage, Socket.io for real-time chat and notifications, Google OAuth for federated login, e-mail for OTP delivery, and AI services for content moderation, CV parsing, and identity document verification. Communication between the client and the server is carried out through versioned REST endpoints secured with JWT access tokens and refresh tokens stored in dedicated session records; role-based access control restricts each operation to the allowed roles. Design decisions must keep these integrations loosely coupled behind services and repositories so that a provider can be replaced without rewriting business logic.

1.4.3 Performance Requirements

The application must remain responsive under concurrent usage by many freelancers and clients. Job browsing, proposal listing, forum feeds, and conversation histories must load promptly; therefore the design mandates server-side pagination for every list operation, database indexes on frequently filtered columns, optimized Prisma queries executed inside transactions for counted lists, and streaming React Server Components to reduce time-to-first-byte. Real-time features (messaging, notifications) must propagate events with minimal delay via Socket.io. Payment operations must reconcile eventual states through Stripe webhooks and explicit synchronization endpoints.

1.5 Technology Stack

Frevia adopts a RESTful API architecture to support secure, stateless, and scalable communication between the Next.js frontend and the NestJS backend. This approach provides:

- Modular, reusable API endpoints for account management, job posting, proposals, contracting, milestone escrow, dispute resolution, community forum, and administration.
- Easy extension for mobile applications or third-party integrations in the future.

- Secure access using JWT tokens with role-based authorization for every endpoint.
- Uniform use of standard HTTP methods (GET, POST, PATCH, PUT, DELETE), making the system intuitive to maintain.
- A clear separation between the presentation layer and the business logic layer.

All request/response payloads are validated and typed with shared Zod schemas packaged in `@shared/types`, which is consumed by both the frontend and the backend so that a single source of truth prevents contract drift between the two applications.

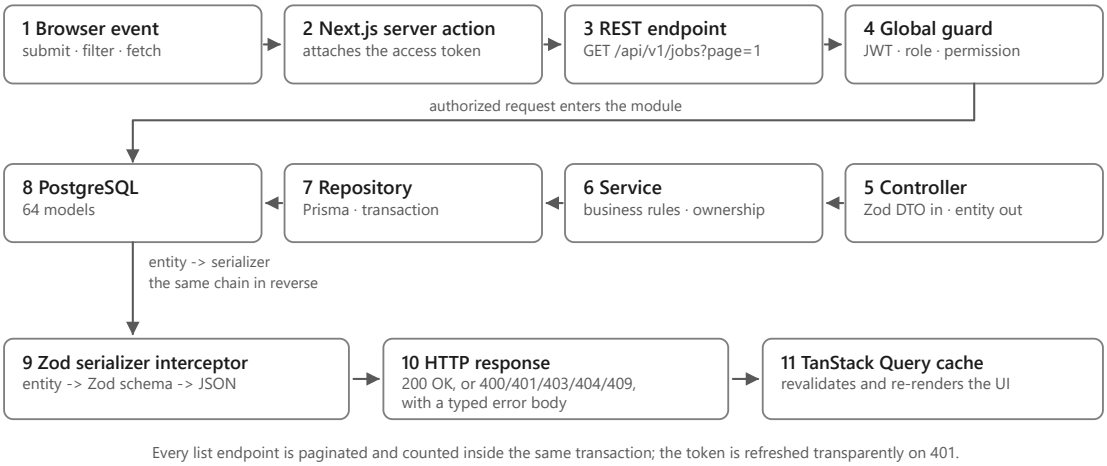

Figure 4: RESTful API communication between the Next.js frontend and the NestJS backend

1.5.1 NestJS Framework

The backend of Frevia is built with NestJS 11, a TypeScript-first, modular web framework running on Node.js. It is selected for its:

- Rich dependency-injection and module ecosystem, matching the Controller–Service–Repository layering.
- First-class decorator-based REST routing with guards and interceptors for authentication, roles, permissions, and response serialization.
- Excellent support for Zod schema validation (nestjs-zod) and typed DTOs.
- Component structure for WebSockets (Socket.io gateways) used by the messaging module.
- Straightforward integration with Prisma ORM and PostgreSQL for persistence.

1.5.2 Next.js and React 19 Frontend

The web client uses Next.js 16 with the App Router. React Server Components render public pages (job lists, job details, forum) directly from the server, while TanStack Query drives interactive islands (forms, likes, real-time chat). react-hook-form together with Zod resolvers validate user input with the same schemas the API uses.

1.5.3 Prisma ORM and PostgreSQL

Persistence relies on PostgreSQL accessed through Prisma 7. The schema defines 64 models covering identity and access, profiles, jobs, proposals, contracts and milestones, payments and disputes, reviews, messaging, the community forum, and platform configuration. Prisma's type-safe client keeps data access code verifiable at compile time.

1.5.4 Supporting Services

Payments and payouts use Stripe Checkout and Stripe Connect with webhook-driven reconciliation; binaries are stored in Cloudinary; emails (OTP, notifications) are sent with Nodemailer; Google login uses OAuth 2.0; environment configuration is loaded through Infisical so no secret is committed to the repository.

2. Database Design

Frevia persists every domain object in a single PostgreSQL database whose schema is declared with Prisma 7 in `apps/api/prisma/schema.prisma`. The schema contains 64 models together with their enumerations, every table has a surrogate integer primary key generated by the database (nine membership tables use a composite key instead), and deletion is either restricted by a foreign-key rule (identity and money-related rows are never removed) or implemented as a soft delete through a `deletedAt` column (content that an administrator must be able to restore: jobs, forum categories and posts, skills, banners, contract term templates). Binary files are not stored in the database: only their Cloudinary URL is kept in the owning row.

Figure 5 shows the entity–relationship diagram of the whole schema. Every table is drawn as a frameless box: the header carries the table name, each of the following rows is one column, and the key columns are flagged with `PK` (primary key) or `FK` (foreign key). A relationship is a line that leaves the many-side table with a crow's foot and ends on the one-side table with a single bar, so it reads from the row that holds the foreign key to the row it references.

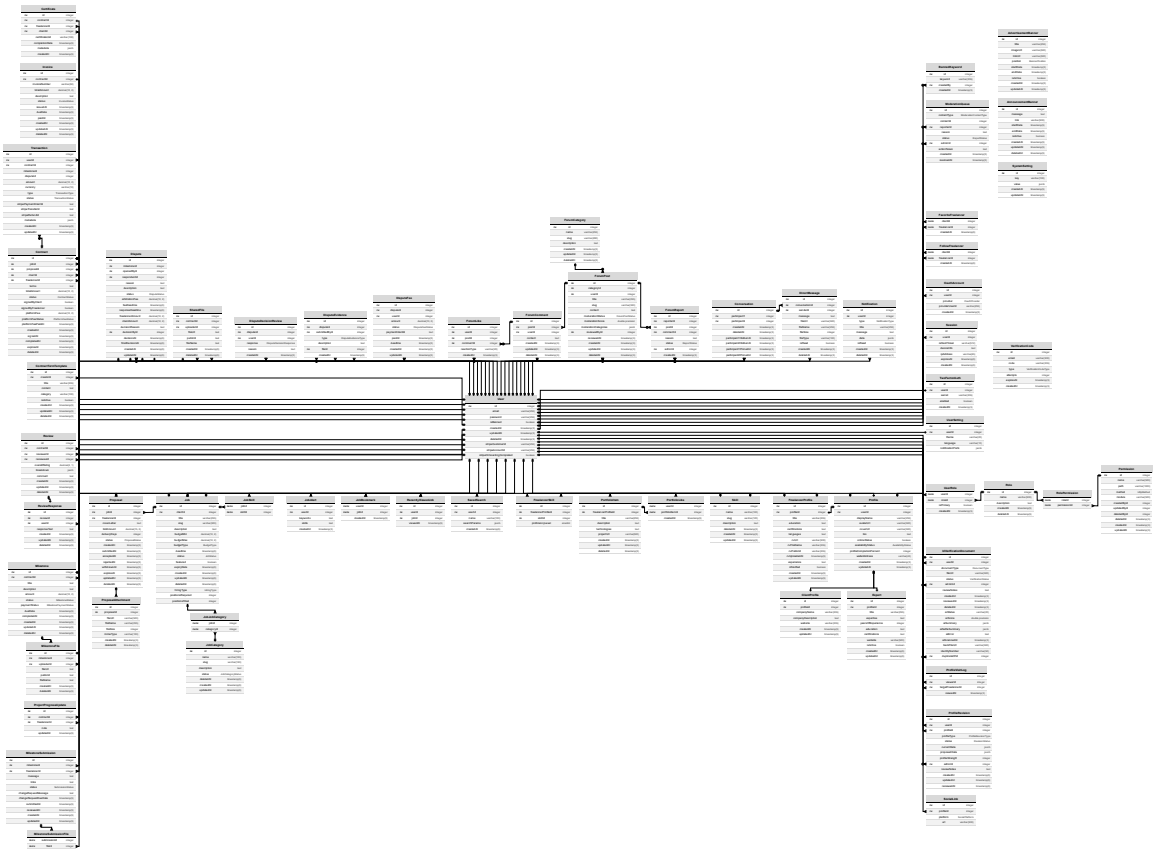

Figure 5: Entity–relationship diagram of the Frevia database

2.1 Table Relationships

The 64 tables are grouped into nine families; inside a family the relationships are one-to-many, and membership tables carry the many-to-many links:

- **Identity and access** — `User` is the root of the schema. `UserRole` links an account to a `Role`, and `RolePermission` links a role to a `Permission`, which yields the RBAC model enforced by the guards. Authentication adds `Session` (one row per device), `VerificationCode` (OTP for registration and password reset), `OauthAccount` (Google login), `TwoFactorAuth` and `UserSetting`.

- **Profiles** — `Profile` is shared by every account; `FreelancerProfile` and `ClientProfile` extend it with role-specific columns, `Expert` publishes an expert entry in the public directory, and `FreelancerSkill` joins a freelancer to the `Skill` catalogue. `PortfolioItem`, `SocialLink`, `Certificate`, `IdVerificationDocument` and `ProfileRevision` hang from the profile.

- **Jobs** — `Job` belongs to a client and is classified through `JobJobCategory` (`JobCategory`) and `JobSkill` (`Skill`). `JobBookmark`, `RecentlyViewedJob`, `SavedSearch` and `JobAlert` record the browsing behaviour of a freelancer.

- **Proposals and contracts** — `Proposal` references one `Job` and one freelancer; accepting it creates exactly one `Contract`, which carries `ContractTermTemplate` snapshots and a list of `Milestone` rows. `ProjectProgressUpdate`, `SharedFile` and `MilestoneFile` document the work in progress, while `MilestoneSubmission` (with `MilestoneSubmissionFile`) records the deliverable and its review decision.
- **Payments** — `Transaction` stores every Stripe movement (funding, release, refund, platform fee, dispute fee) with its payment-intent identifier, and `Invoice` holds the invoice issued per contract. Money is never derived: the escrow balance of a milestone is the sum of its transactions.
- **Disputes** — `Dispute` is opened on a contract or a milestone, `DisputeEvidence` collects the attachments of both parties, `DisputeFee` records the fee charged to open it, and `DisputeDecisionReview` stores the request of a party that asks for the decision to be reconsidered.
- **Reviews** — one `Review` per completed contract, answered by at most one `ReviewResponse`.
- **Messaging and notifications** — `Conversation` groups the `DirectMessage` rows exchanged by two accounts (pin and read state are kept on the message and on the conversation), and `Notification` feeds the in-app bell.
- **Community forum and configuration** — `ForumCategory` contains `ForumPost`, a post contains `ForumComment`, and `ForumLike` / `ForumReport` attach reactions and reports to posts or comments. Administrators work through `ModerationQueue` and `BannedKeyword`; platform-wide configuration lives in `SystemSetting`, `AnnouncementBanner` and `AdvertisementBanner`.

Table Descriptions

Table 3 describes the purpose of every table of the schema, in the order in which the models are declared in `schema.prisma`, together with its primary key and its foreign keys.

Table 3: Table descriptions of the Frevia database

No	Table	Description
01	`Role`	Role catalogue of the RBAC model (admin, client, freelancer, expert); carries the role code, name and description. • **Primary keys:** `id` • **Foreign keys:** —

No	Table	Description
02	UserRole	Assignment of a role to an account, so one account can hold several roles and switch between them. • **Primary keys:** userId, roleId • **Foreign keys:** userId → User, roleId → Role
03	Permission	Catalogue of route-level permissions (module.action names) used by the permission guard. • **Primary keys:** id • **Foreign keys:** —
04	RolePermission	Permissions granted to a role; drives the effective permission list of every signed-in account. • **Primary keys:** roleId, permissionId • **Foreign keys:** roleId → Role, permissionId → Permission
05	User	Account identity: e-mail, username, bcrypt password hash, status, verification and ban flags, preferred language and timestamps. • **Primary keys:** id • **Foreign keys:** —
06	VerificationCode	One-time code (OTP) issued for e-mail verification or password reset, with its purpose, expiry and consumed flag. • **Primary keys:** id • **Foreign keys:** —
07	Session	Server-side session per device: refresh-token hash, device/user-agent information, last activity and revocation time. • **Primary keys:** id • **Foreign keys:** userId → User
08	OauthAccount	Link between an account and an external identity provider (Google) for federated login. • **Primary keys:** id • **Foreign keys:** userId → User
09	TwoFactorAuth	Two-factor authentication secret and enabled flag of an account (one row per account). • **Primary keys:** id • **Foreign keys:** userId → User
10	UserSetting	Per-account preferences: interface theme, language and the JSON notification preferences. • **Primary keys:** id • **Foreign keys:** userId → User

No	Table	Description
11	`Profile`	Base profile shared by all accounts: display name, avatar, cover, headline and biography. • **Primary keys:** `id` • **Foreign keys:** `userId` → `User`
12	`FreelancerProfile`	Professional data of a freelancer: professional title, overview, hourly rate, availability and total earnings. • **Primary keys:** `id` • **Foreign keys:** `profileId` → `Profile`
13	`FreelancerSkill`	Many-to-many link between a freelancer profile and a skill, with the self-declared proficiency level. • **Primary keys:** `id` • **Foreign keys:** `freelancerProfileId` → `FreelancerProfile`, `skillId` → `Skill`
14	`ClientProfile`	Client-side data: company name, company size, industry and billing information. • **Primary keys:** `id` • **Foreign keys:** `profileId` → `Profile`
15	`Expert`	Expert entry published in the public expert directory, with specialisation and verification state. • **Primary keys:** `id` • **Foreign keys:** `profileId` → `Profile`
16	`ProfileRevision`	Pending change request on a profile, reviewed and approved or rejected by an administrator. • **Primary keys:** `id` • **Foreign keys:** `userId` → `User`, `adminId` → `User`, `profileId` → `Profile`
17	`PortfolioItem`	Portfolio entry of a profile: title, description, media URL, project link and ordering position. • **Primary keys:** `id` • **Foreign keys:** `freelancerProfileId` → `FreelancerProfile`
18	`SocialLink`	External link of a profile (website, LinkedIn, GitHub, Behance) shown on the public profile. • **Primary keys:** `id` • **Foreign keys:** `profileId` → `Profile`
19	`IdVerificationDocument`	Identity document uploaded for eKYC, with the AI check result and the administrator decision. • **Primary keys:** `id` • **Foreign keys:** `userId` → `User`, `adminId` → `User`, `duplicateOfId` → `IdVerificationDocument`

No	Table	Description
20	ProfileVisitLog	View log of a freelancer profile (viewerId, targetFreelancerId, viewedAt); source of the profile statistics. • **Primary keys:** id • **Foreign keys:** viewerId → User, targetFreelancerId → User
21	JobCategory	Hierarchical job category (parent reference, slug, name, icon) with soft-delete support. • **Primary keys:** id • **Foreign keys:** —
22	Job	Job posting: slug, title, description, budget range, duration, deadline, visibility, status and the owning client. • **Primary keys:** id • **Foreign keys:** clientId → User
23	JobJobCategory	Many-to-many link between a job and the job categories it is published in. • **Primary keys:** jobId, categoryId • **Foreign keys:** jobId → Job, categoryId → JobCategory
24	Skill	Skill catalogue shared by freelancer profiles and jobs, with soft-delete support. • **Primary keys:** id • **Foreign keys:** —
25	JobSkill	Skills required by a job, with the required proficiency level. • **Primary keys:** jobId, skillId • **Foreign keys:** jobId → Job, skillId → Skill
26	JobBookmark	Jobs saved by a freelancer. • **Primary keys:** userId, jobId • **Foreign keys:** userId → User, jobId → Job
27	RecentlyViewedJob	Recently opened job postings of an account, with the viewing timestamp. • **Primary keys:** id • **Foreign keys:** userId → User, jobId → Job
28	SavedSearch	Stored job filter criteria (keyword, categories, budget, skills) that the freelancer can reapply. • **Primary keys:** id • **Foreign keys:** userId → User

No	Table	Description
29	JobAlert	Alert configuration attached to a saved search (frequency, channel, active flag). • **Primary keys:** id • **Foreign keys:** userId → User
30	Proposal	Proposal submitted by a freelancer for a job: cover letter, bid amount, estimated duration and lifecycle status. • **Primary keys:** id • **Foreign keys:** jobId → Job, freelancerId → User
31	ProposalAttachment	File attached to a proposal (portfolio sample, reference document) stored in Cloudinary. • **Primary keys:** id • **Foreign keys:** proposalId → Proposal
32	Contract	Contract created when a proposal is accepted: client, freelancer, total amount, terms, signature state, start/end dates and status. • **Primary keys:** id • **Foreign keys:** jobId → Job, proposalId → Proposal, clientId → User, freelancerId → User
33	ContractTermTemplate	Reusable contract term template owned by a user, with category, content and soft-delete support. • **Primary keys:** id • **Foreign keys:** creatorId → User
34	Milestone	Milestone inside a contract: title, description, amount, due date, escrow status, progress percentage and approval state. • **Primary keys:** id • **Foreign keys:** contractId → Contract
35	MilestoneFile	Reference file attached to a milestone and shared with both parties. • **Primary keys:** id • **Foreign keys:** milestoneId → Milestone, uploaderId → User
36	MilestoneSubmission	Work submitted for a milestone, with the submitted note, timestamp and the client review decision. • **Primary keys:** id • **Foreign keys:** milestoneId → Milestone, freelancerId → User

No	Table	Description
37	MilestoneSubmissionFile	Join table linking a submission to the milestone files it delivers. • **Primary keys:** submissionId, fileId • **Foreign keys:** submissionId → MilestoneSubmission, fileId → MilestoneFile
38	Invoice	Invoice of a contract (one per contract): invoice number, total amount, status, issued, due and paid dates. • **Primary keys:** id • **Foreign keys:** contractId → Contract
39	Certificate	Completion certificate generated when a contract is finished: unique certificate UID, completion date and JSON metadata. • **Primary keys:** id • **Foreign keys:** contractId → Contract, freelancerId → User, clientId → User
40	Dispute	Dispute opened on a contract or a milestone: reason, description, opened by, status, resolution and decision timestamps. • **Primary keys:** id • **Foreign keys:** milestoneId → Milestone, openedById → User, respondentId → User, decisionById → User
41	DisputeFee	Fee charged to open a dispute, with its amount, payment state and the transaction that settled it. • **Primary keys:** id • **Foreign keys:** disputeId → Dispute, userId → User
42	DisputeEvidence	Evidence attached by either party to a dispute (file, description, uploader). • **Primary keys:** id • **Foreign keys:** disputeId → Dispute, submittedById → User, fileId → SharedFile
43	DisputeDecisionReview	Request of a party to reconsider a dispute decision, with the response, the reason and the timestamp. • **Primary keys:** id • **Foreign keys:** disputeId → Dispute, userId → User

No	Table	Description
44	Transaction	Payment movement reconciled with Stripe (funding, release, refund, platform fee, dispute fee): type, amount, status, payment-intent id and timestamps. • **Primary keys:** id • **Foreign keys:** userId → User, contractId → Contract
45	Review	Review written after a contract is completed: rating, comment, author, recipient and the reviewed contract. • **Primary keys:** id • **Foreign keys:** contractId → Contract, reviewerId → User, revieweeId → User
46	ReviewResponse	Reply of the reviewed party to a review (at most one per review). • **Primary keys:** id • **Foreign keys:** reviewId → Review, userId → User
47	ForumCategory	Forum category: slug, name, description, display order, post counter and soft-delete support. • **Primary keys:** id • **Foreign keys:** —
48	ForumPost	Forum post: title, content, author, category, moderation status, AI score, reviewer and soft-delete support. • **Primary keys:** id • **Foreign keys:** categoryId → ForumCategory, userId → User, reviewedById → User
49	ForumComment	Comment on a forum post; comments are one level deep and can be soft-deleted. • **Primary keys:** id • **Foreign keys:** postId → ForumPost, userId → User
50	ForumLike	Reaction of a member on a post or a comment; a member can like a target only once. • **Primary keys:** id • **Foreign keys:** userId → User, postId → ForumPost, commentId → ForumComment
51	ForumReport	Content report filed by a member: reason, description, status and the administrator who handled it. • **Primary keys:** id • **Foreign keys:** reporterId → User, postId → ForumPost, commentId → ForumComment, adminId → User

No	Table	Description
52	Notification	In-app notification of an account: type, title, body, target link and read state. • **Primary keys:** id • **Foreign keys:** userId → User
53	AnnouncementBanner	Platform announcement shown in the application: message, link, display window, active flag and soft delete. • **Primary keys:** id • **Foreign keys:** —
54	ModerationQueue	Queue of AI-flagged or user-reported content waiting for an administrator decision, with the item type, the score and the assignee. • **Primary keys:** id • **Foreign keys:** reporterId → User, adminId → User
55	SystemSetting	Platform-wide configuration values stored as key/value rows. • **Primary keys:** id • **Foreign keys:** —
56	BannedKeyword	Keyword list used by the moderation filter before the AI service is called. • **Primary keys:** id • **Foreign keys:** createdBy → User
57	AdvertisementBanner	Advertising banner with image, target URL, position, schedule and click counter. • **Primary keys:** id • **Foreign keys:** —
58	FavoriteFreelancer	Freelancers marked as favourite by a client account. • **Primary keys:** clientId, freelancerId • **Foreign keys:** clientId → User, freelancerId → User
59	FollowFreelancer	Follow relationship between two accounts, used by the activity feed. • **Primary keys:** clientId, freelancerId • **Foreign keys:** clientId → User, freelancerId → User
60	PortfolioLike	Like given on a portfolio item. • **Primary keys:** userId, portfolioItemId • **Foreign keys:** userId → User, portfolioItemId → PortfolioItem

No	Table	Description
61	ProjectProgressUpdate	Progress note posted by the freelancer on a contract, with the update timestamp. • **Primary keys:** id • **Foreign keys:** contractId → Contract, freelancerId → User
62	SharedFile	File shared in the workspace of a contract (both parties can upload and read). • **Primary keys:** id • **Foreign keys:** contractId → Contract, uploaderId → User
63	Conversation	Direct conversation between two accounts, with last-message cache and the pin state of each participant. • **Primary keys:** id • **Foreign keys:** participant1 → User, participant2 → User
64	DirectMessage	Message of a conversation: sender, content, attachments, read state, recall (soft delete) flag and timestamp. • **Primary keys:** id • **Foreign keys:** conversationId → Conversation, senderId → User

2.2 Module Entity–Relationship Diagrams

The whole schema is easier to navigate when it is split along the functional modules used in Report 3. Each figure below shows one module together with the tables that the module references, so every foreign key of the module is visible in the same picture. The notation is the one of Figure 5: the crow's foot marks the many side of a relationship and the single bar marks the one side.

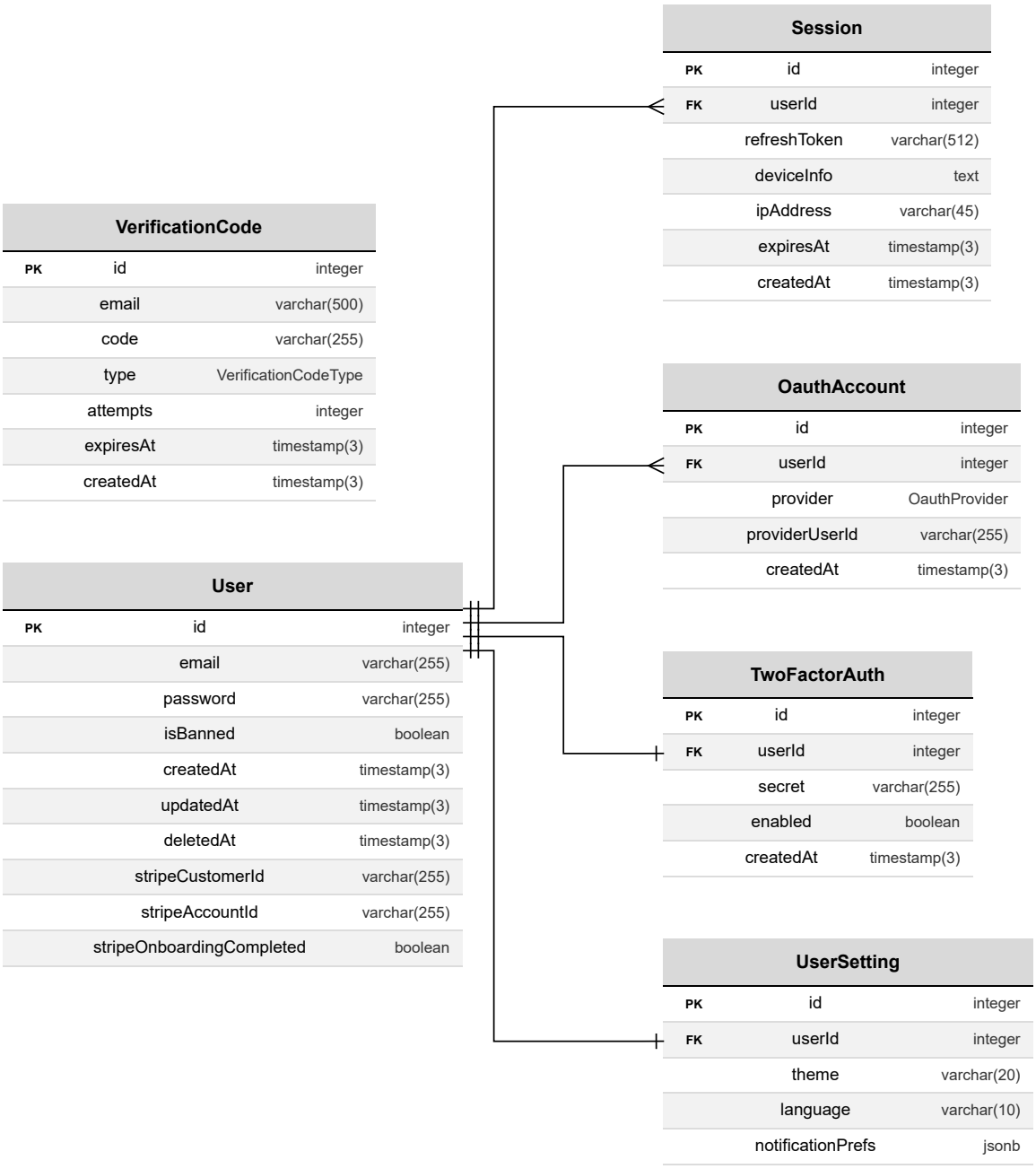

Figure 6: ERD1 — Authentication & Sessions (`User`, `Session`, `OauthAccount`, `TwoFactorAuth`, `UserSetting`, `VerificationCode`)

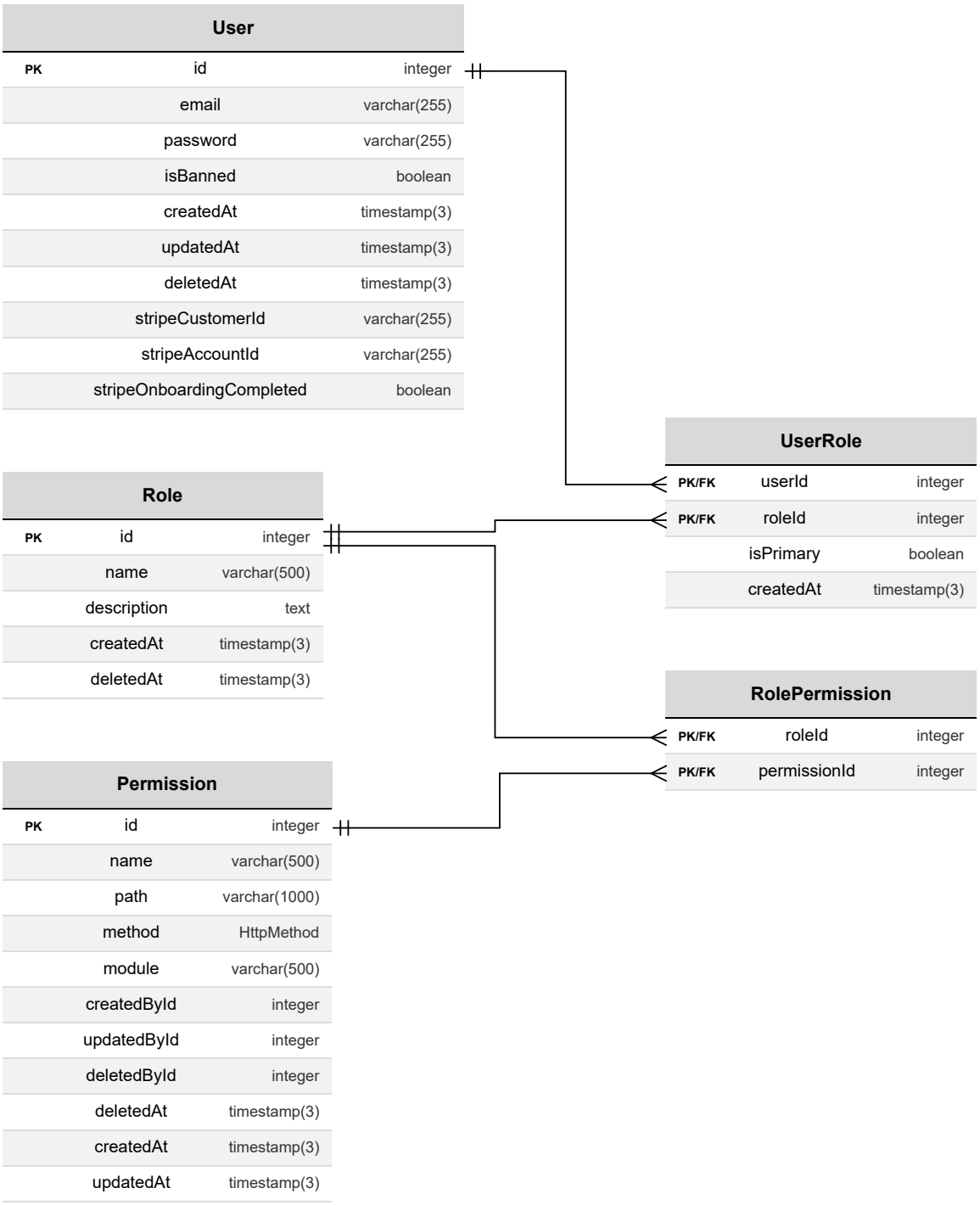

Figure 7: ERD2 — Role-Based Access Control (`User`, `Role`, `UserRole`, `Permission`, `RolePermission`)

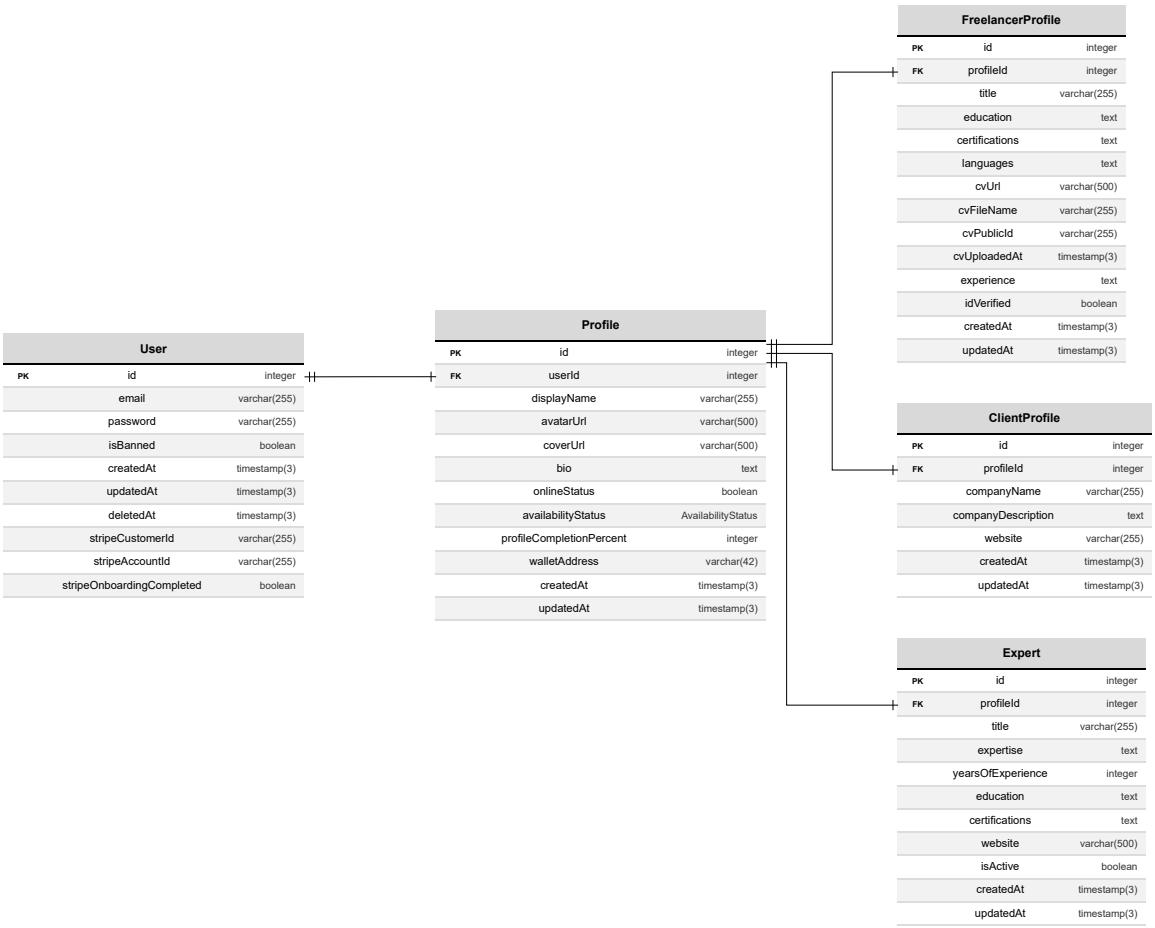

Figure 8: ERD3 — Profile Core (`User`, `Profile`, `FreelancerProfile`, `ClientProfile`, `Expert`)

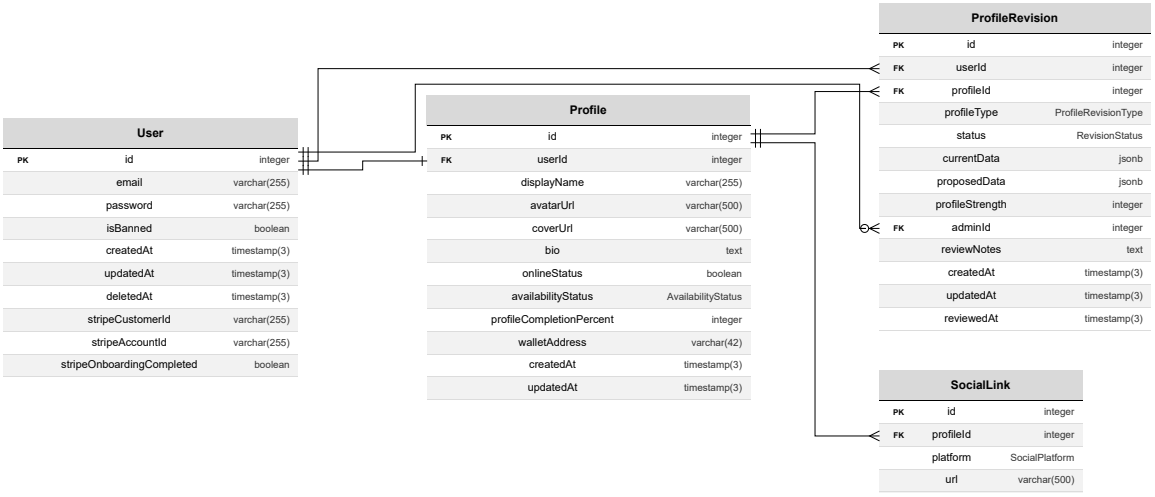

Figure 9: ERD4 — Profile History & Social Links (`User`, `Profile`, `ProfileRevision`, `SocialLink`)

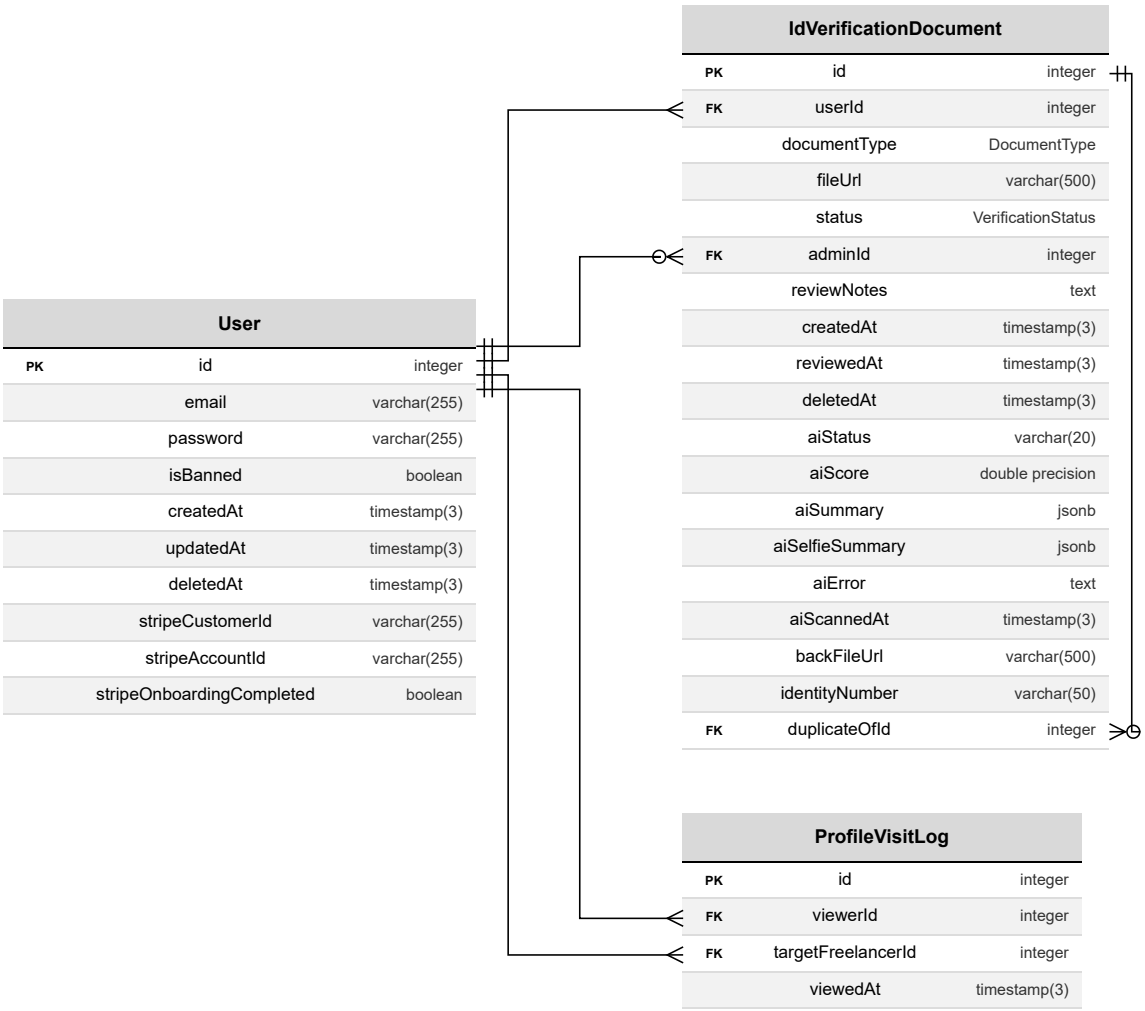

Figure 10: ERD5 — Identity Verification & Profile Visits (`User`, `IdVerificationDocument`, `ProfileVisitLog`)

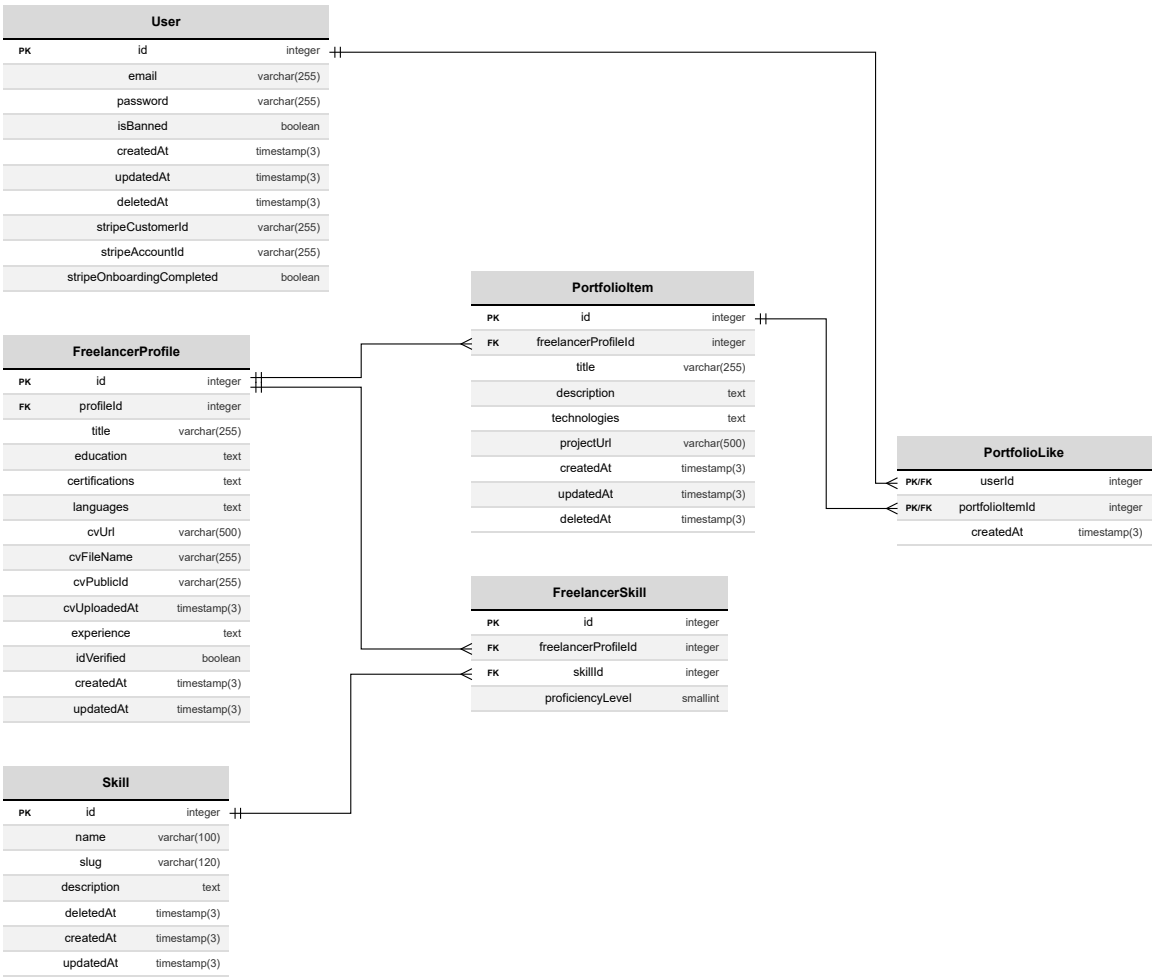

Figure 11: ERD6 — Portfolio & Skills (`User`, `FreelancerProfile`, `FreelancerSkill`, `Skill`, `PortfolioItem`, `PortfolioLike`)

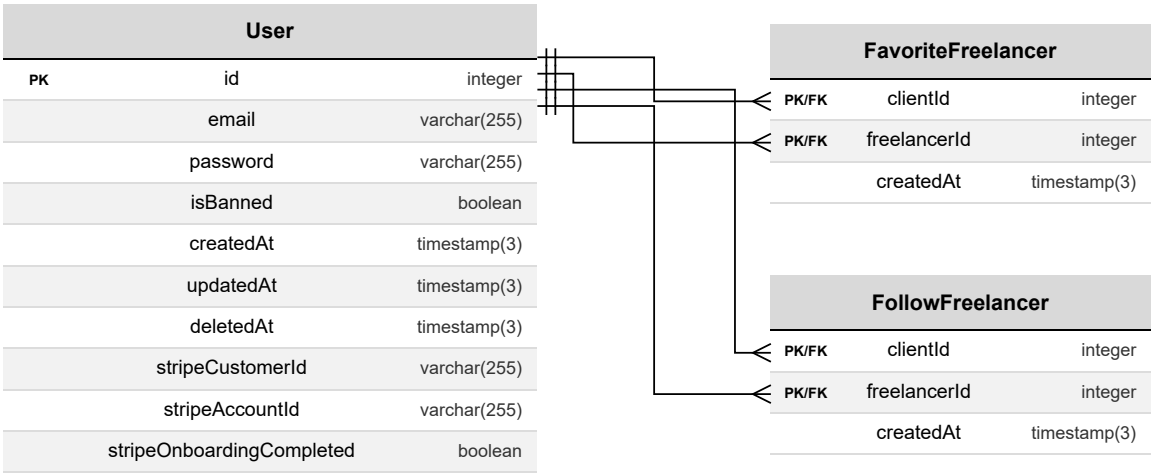

Figure 12: ERD7 — Favorites & Follows (`User`, `FavoriteFreelancer`, `FollowFreelancer`)

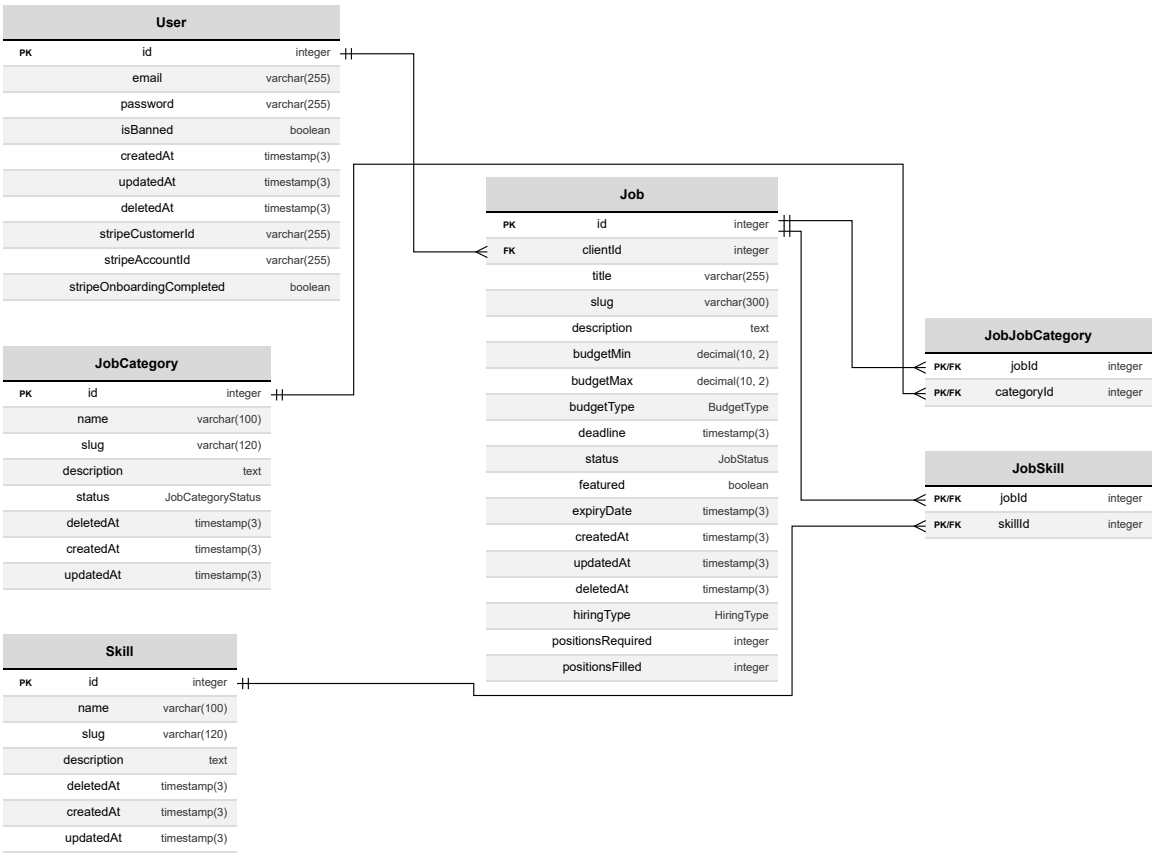

Figure 13: ERD8 — Job Posting (`User`, `Job`, `JobCategory`, `JobJobCategory`, `JobSkill`, `Skill`)

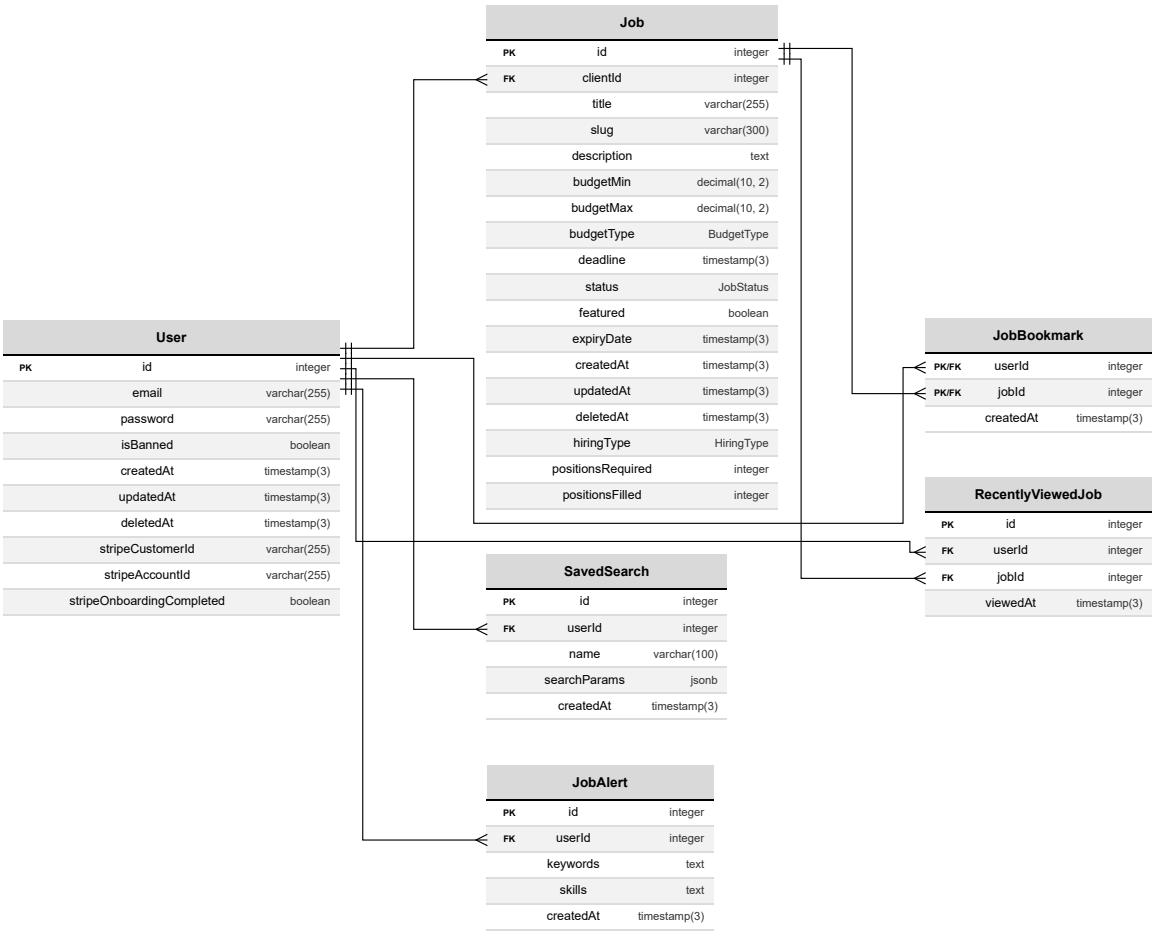

Figure 14: ERD9 — Job Discovery & Alerts (`User`, `Job`, `JobBookmark`, `RecentlyViewedJob`, `SavedSearch`, `JobAlert`)

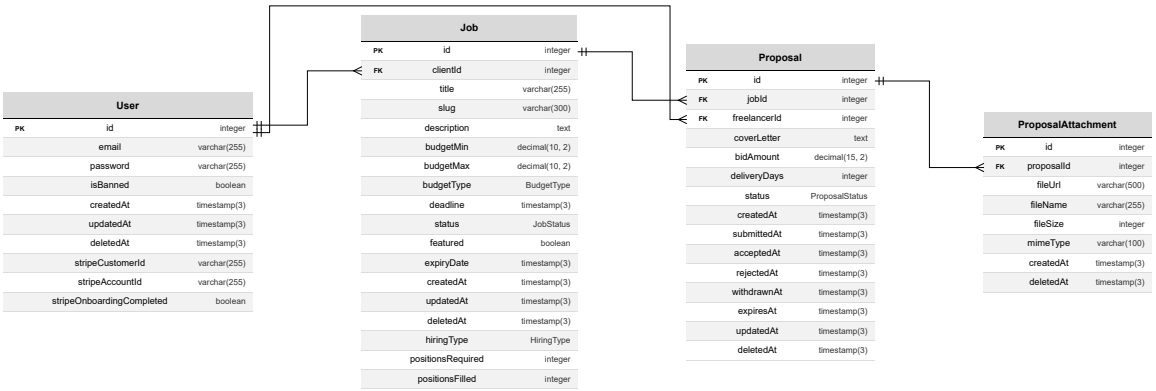

Figure 15: ERD10 — Proposal (`Job`, `User`, `Proposal`, `ProposalAttachment`)

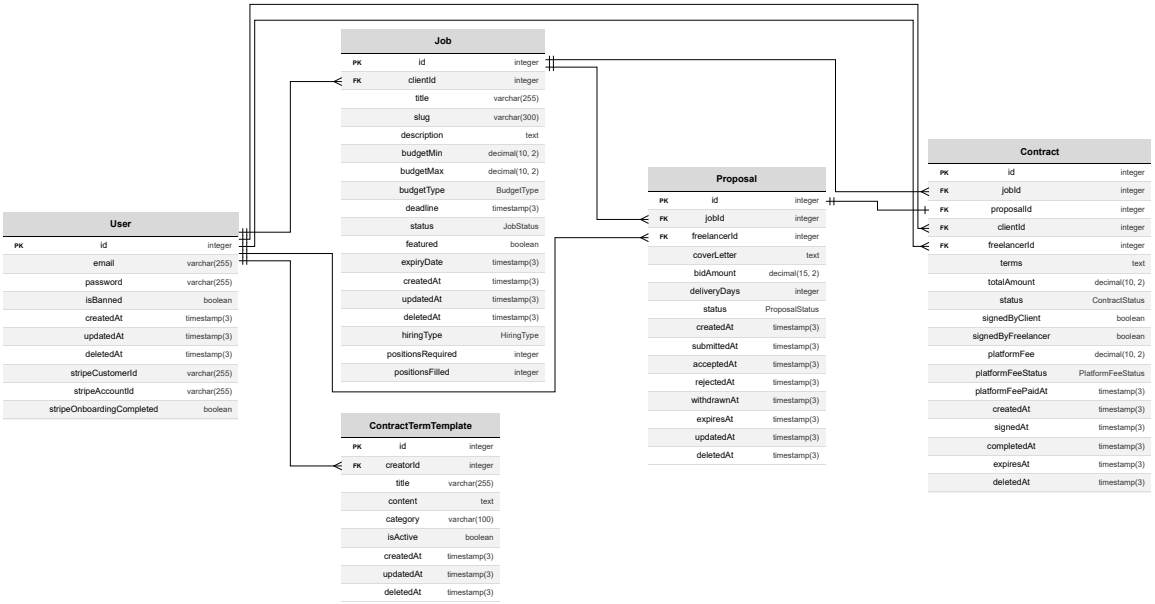

Figure 16: ERD11 — Contract (`Job`, `Proposal`, `User`, `Contract`, `ContractTermTemplate`)

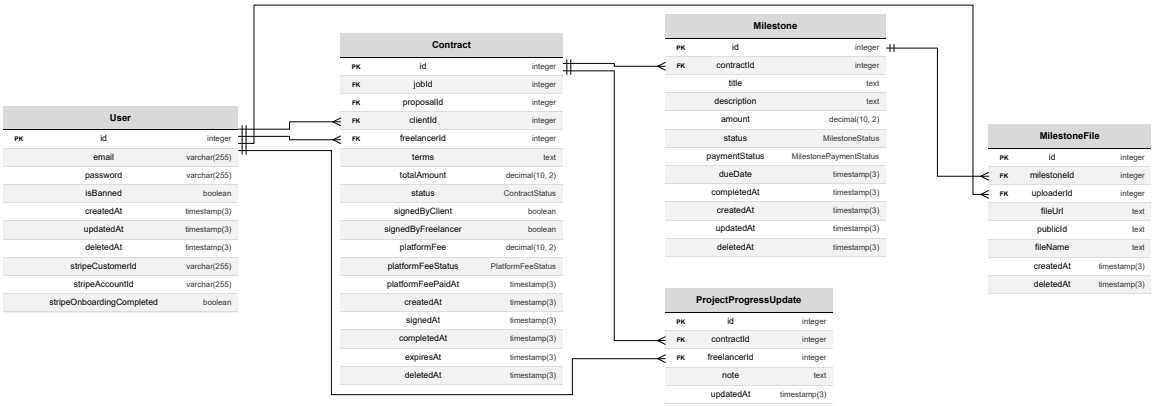

Figure 17: ERD12 — Milestone & Project Progress (`User`, `Contract`, `Milestone`, `MilestoneFile`, `ProjectProgressUpdate`)

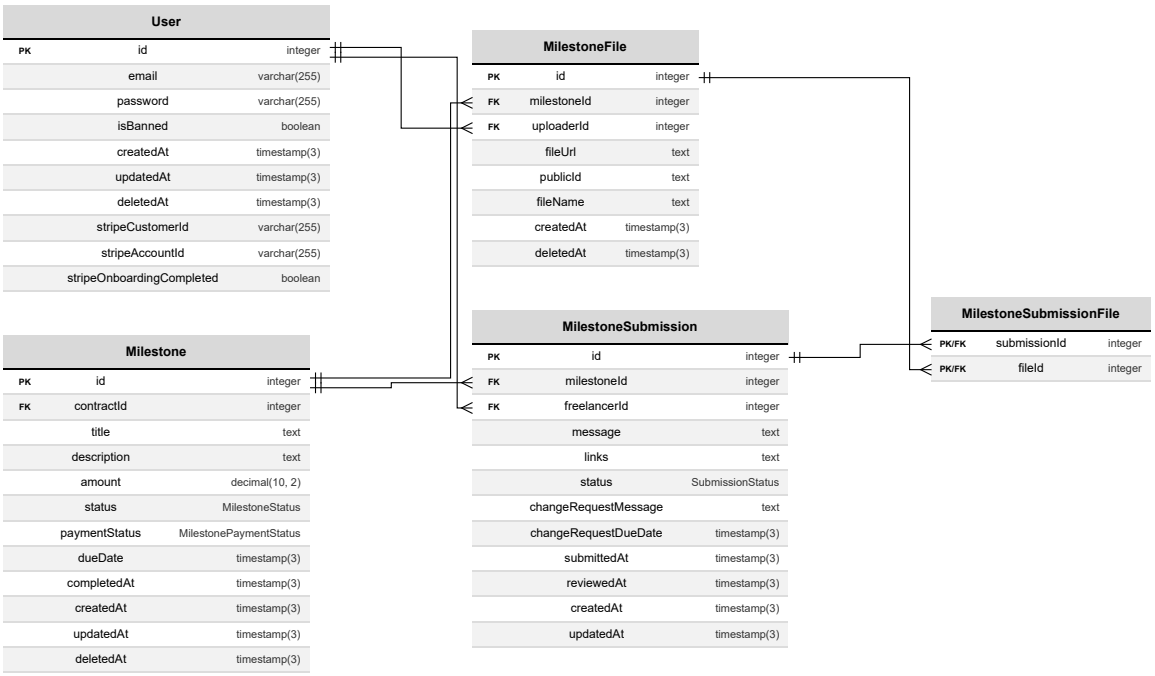

Figure 18: ERD13 — Milestone Submission (`User`, `Milestone`, `MilestoneFile`, `MilestoneSubmission`, `MilestoneSubmissionFile`)

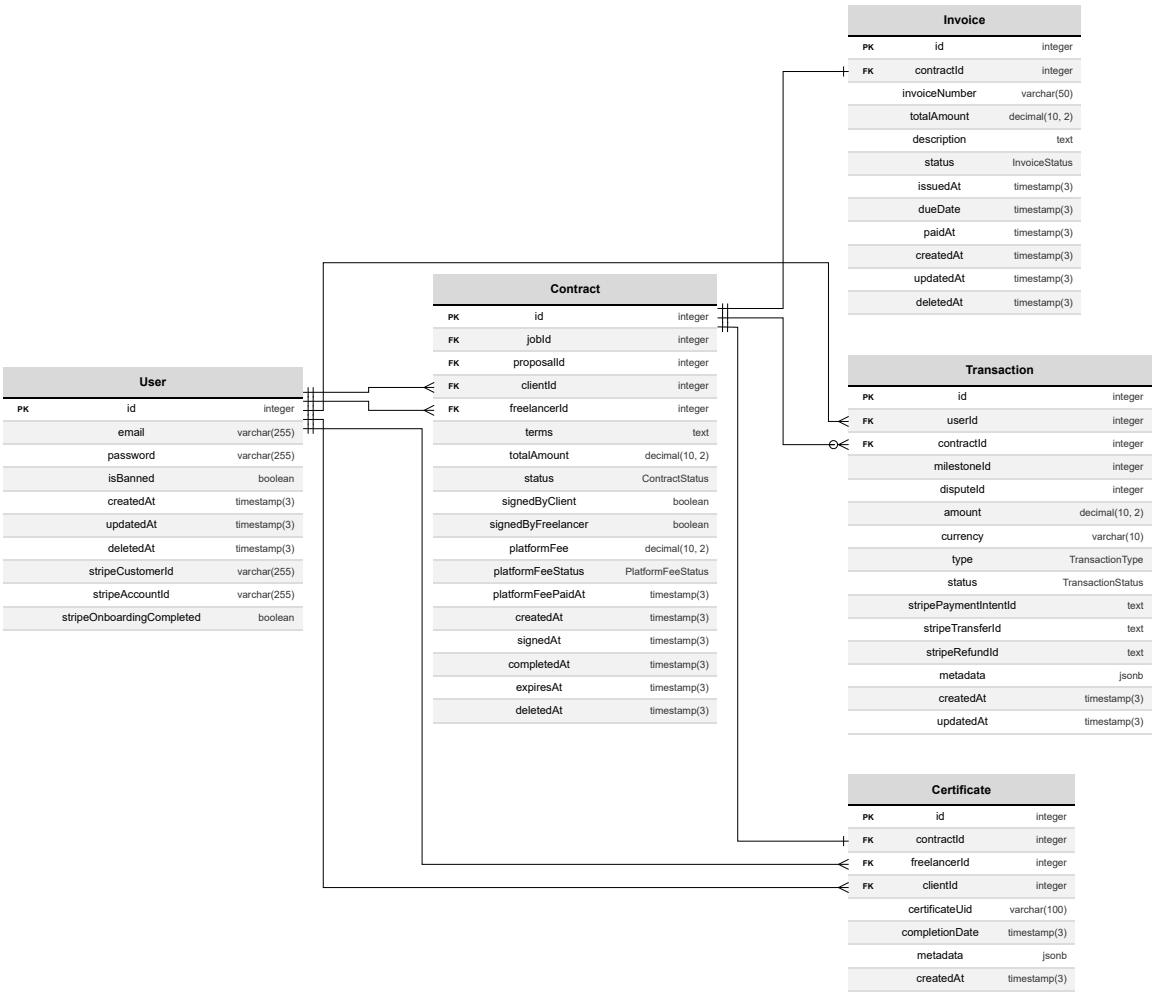

Figure 19: ERD14 — Escrow, Payments & Certificates (`User`, `Contract`, `Invoice`, `Transaction`, `Certificate`)

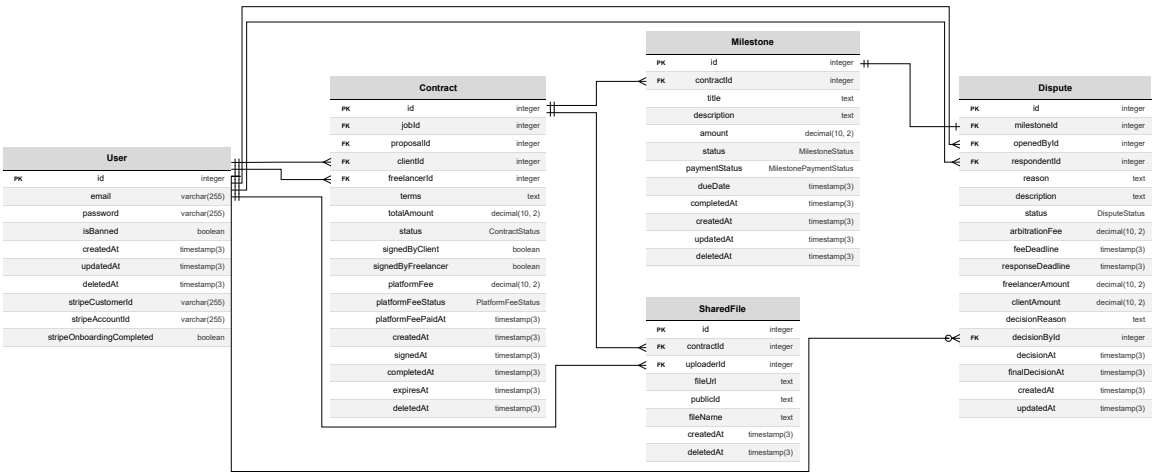

Figure 20: ERD15 — Dispute Core (`Contract`, `Milestone`, `User`, `SharedFile`, `Dispute`)

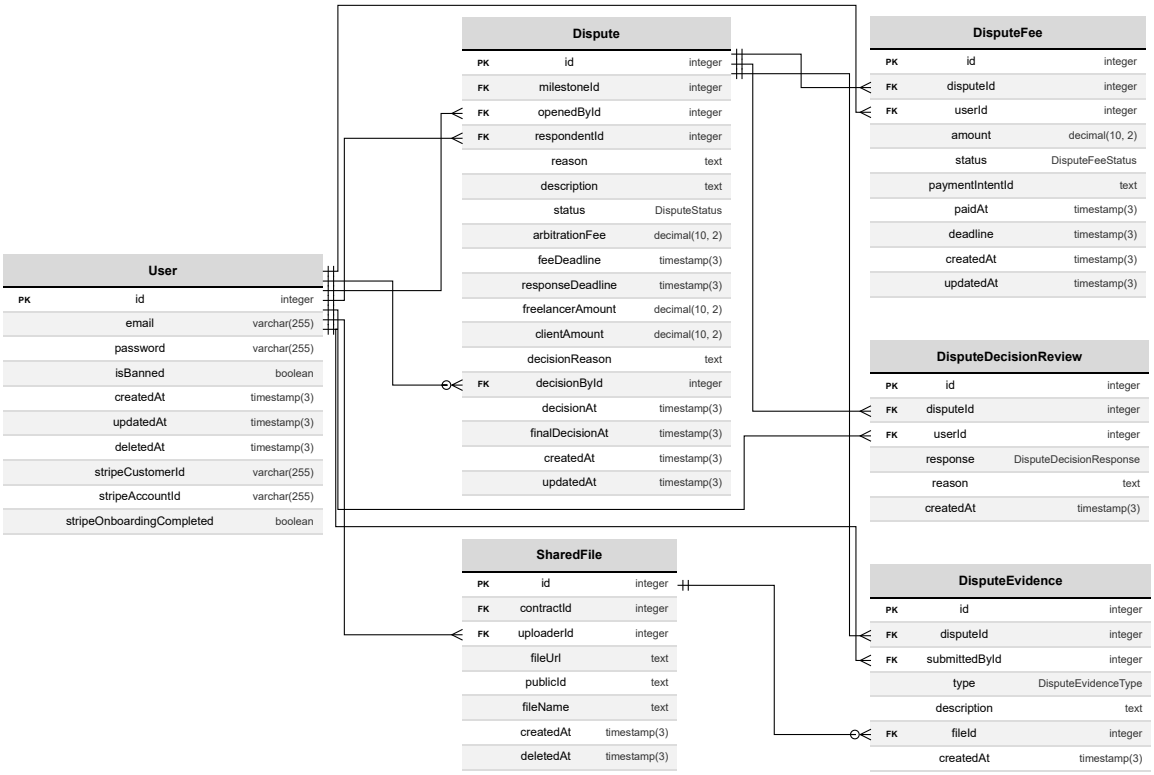

Figure 21: ERD16 — Dispute Resolution (`User`, `Dispute`, `DisputeFee`, `DisputeEvidence`, `SharedFile`, `DisputeDecisionReview`)

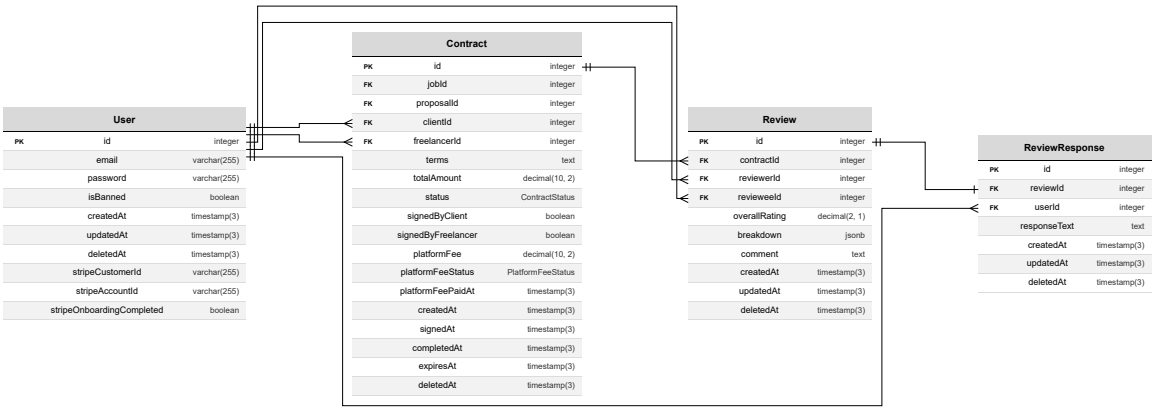

Figure 22: ERD17 — Reviews & Reputation (`Contract`, `User`, `Review`, `ReviewResponse`)

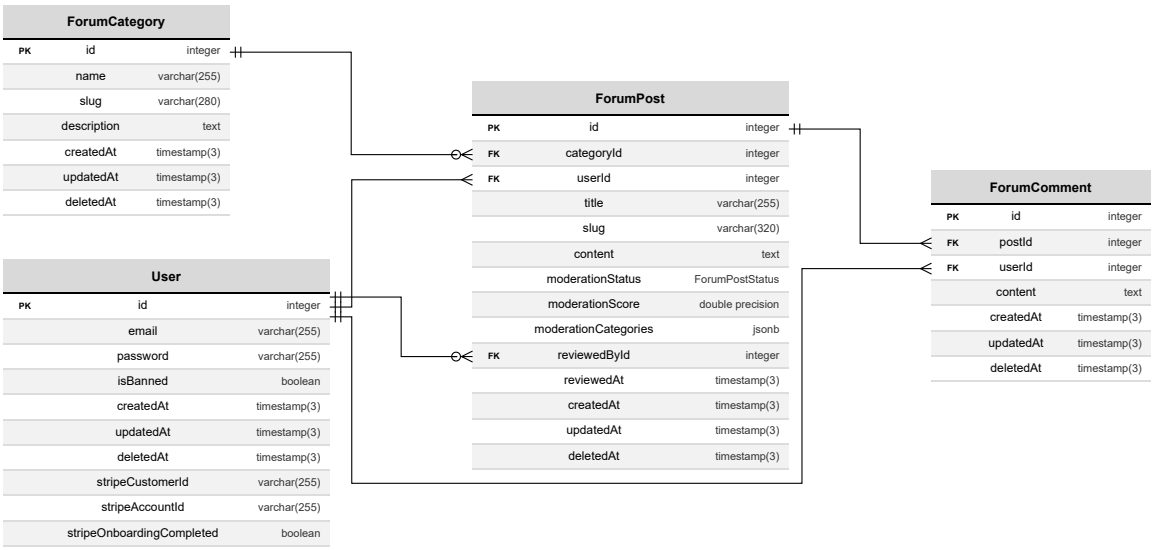

Figure 23: ERD18 — Forum Posts & Comments (`ForumCategory`, `ForumPost`, `User`, `ForumComment`)

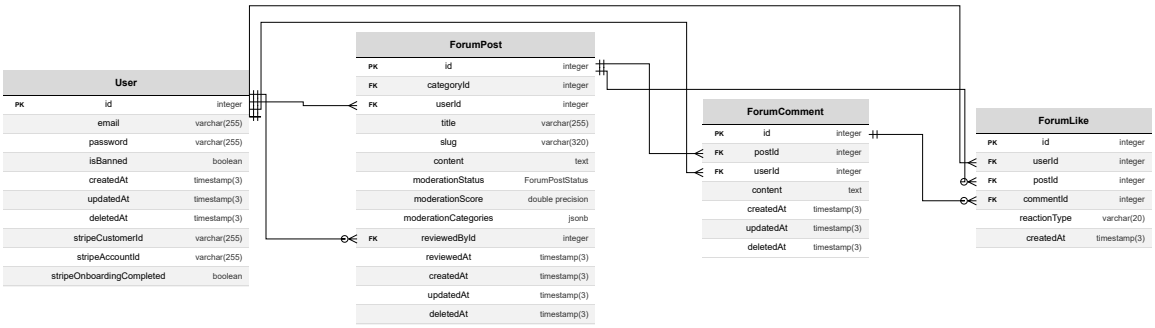

Figure 24: ERD19 — Forum Likes (`User`, `ForumPost`, `ForumComment`, `ForumLike`)

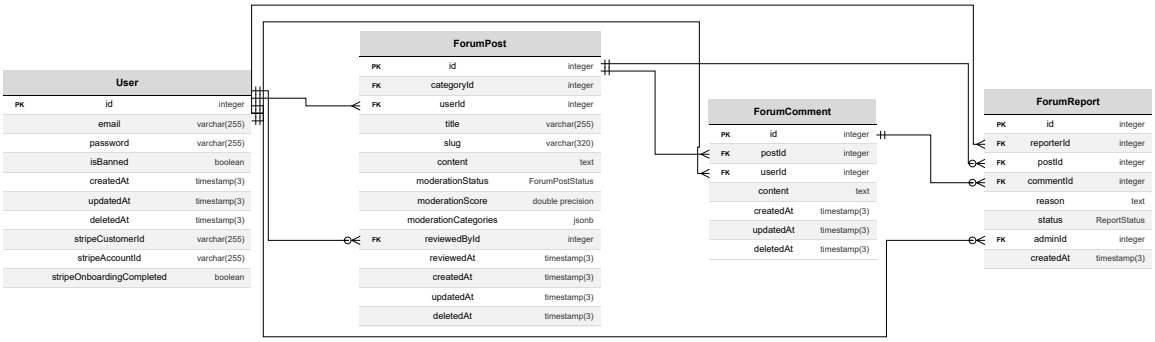

Figure 25: ERD20 — Forum Reports & Moderation (`User`, `ForumPost`, `ForumComment`, `ForumReport`)

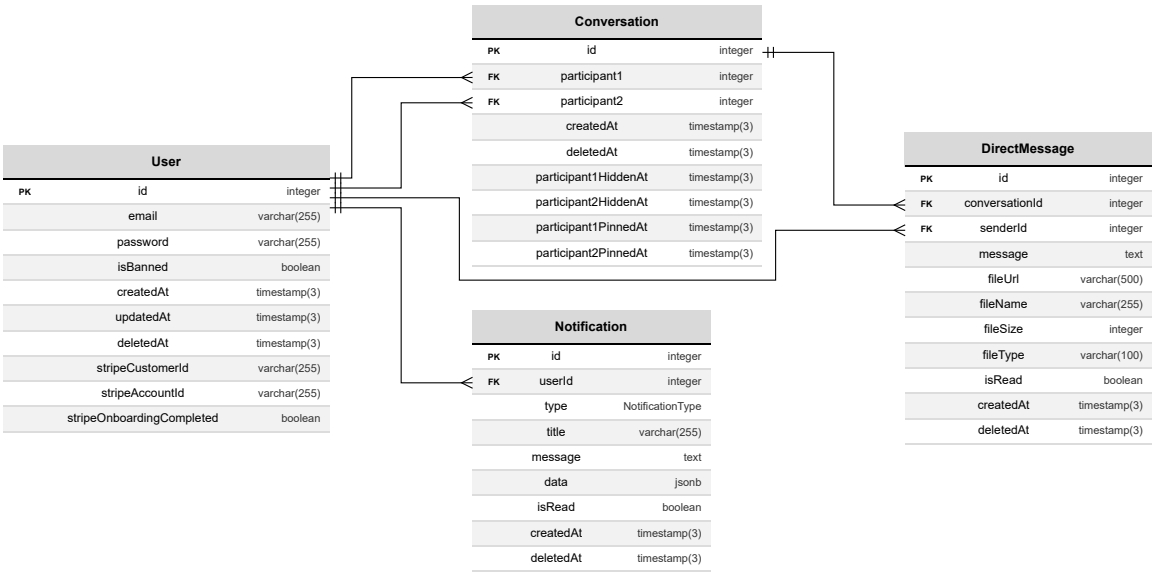

Figure 26: ERD21 — Messaging & Notifications (`User`, `Conversation`, `DirectMessage`, `Notification`)

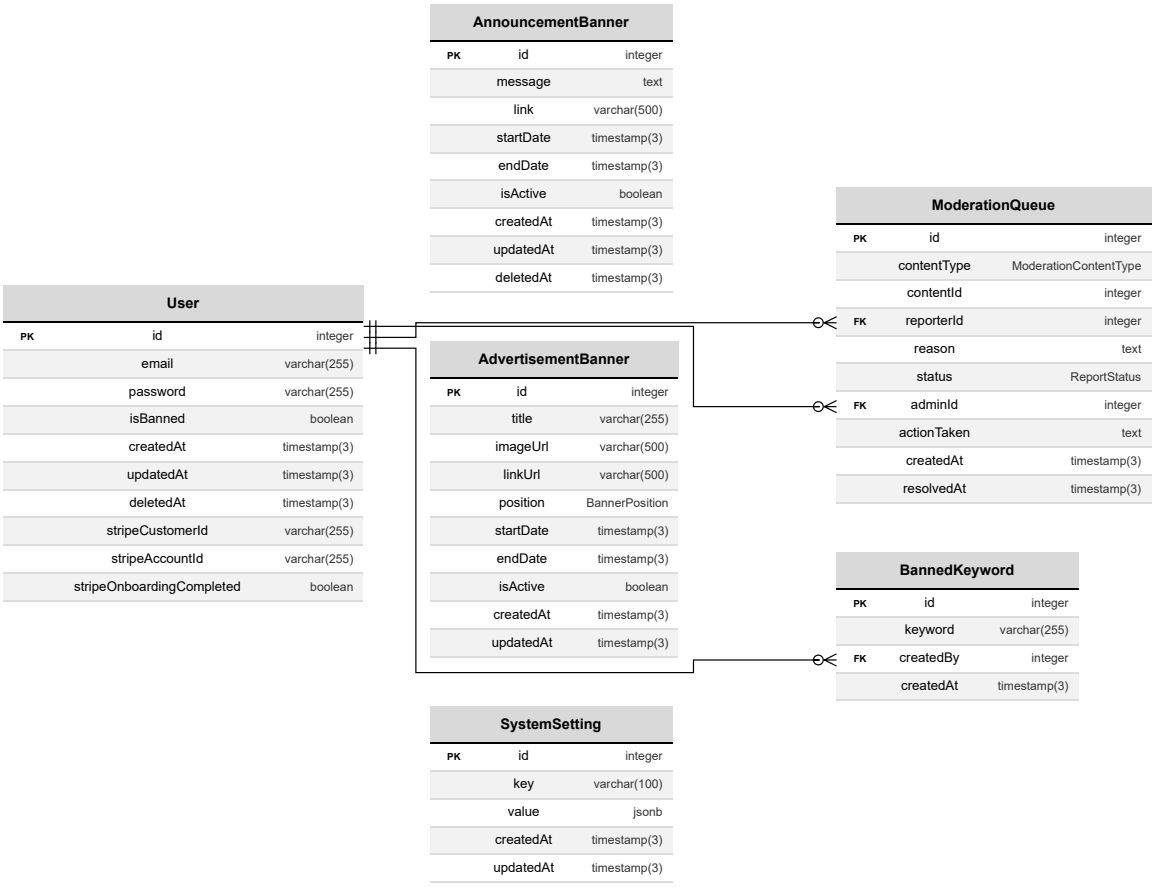

Figure 27: ERD22 — Administration & System (`User`, `ModerationQueue`, `BannedKeyword`, `AnnouncementBanner`, `AdvertisementBanner`, `SystemSetting`)

3. Detailed Design

This chapter presents the detailed design of every feature of the system. Each section (3.x) covers one feature and contains the class diagram of the feature (3.x.1) followed by its sequence diagrams (3.x.2, 3.x.3, ...), one for each use case of the feature. Where several functions are built from the same classes and follow the same interaction pattern, the diagrams are presented once for the representative function and the other functions refer to that section.

3.1 Forum Management

Forum Management covers the UC-14 community forum: browsing forum categories and posts, publishing and maintaining posts and comments, toggling likes on posts and comments, and reporting content that breaks the forum rules.

3.1.1 Class Diagram

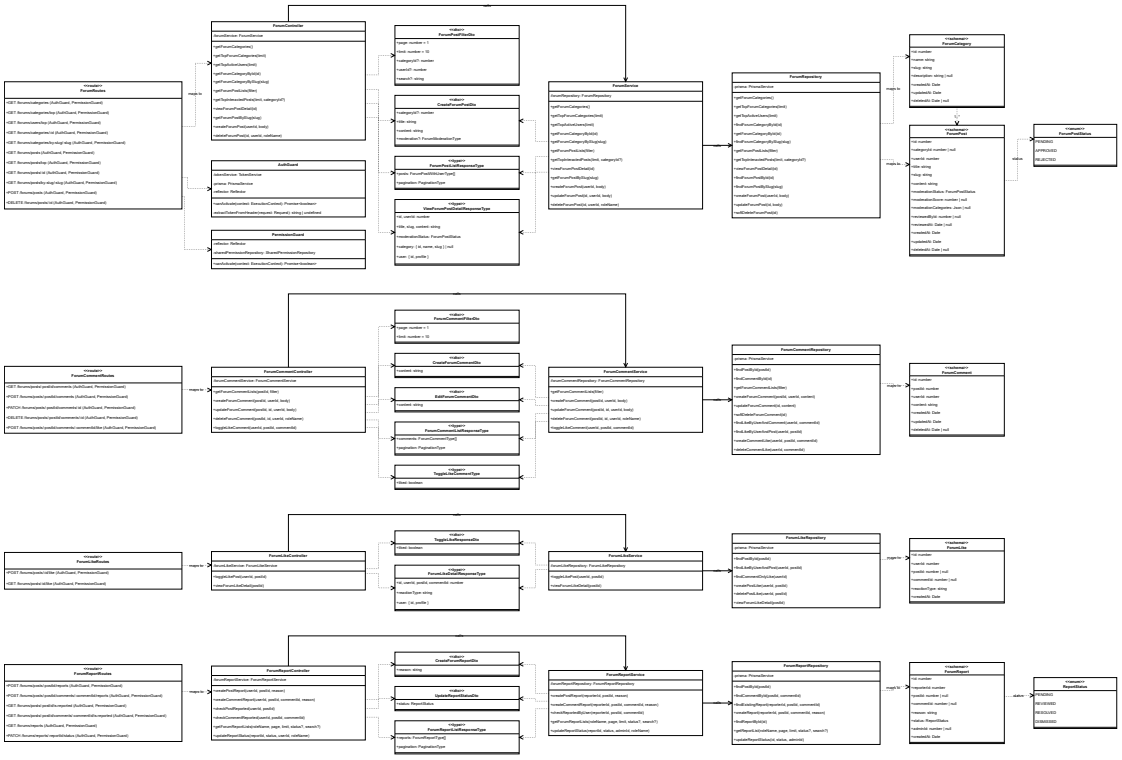

Figure 28: Class diagram of Forum Management

3.1.2 UC-14.01 – View Forum Category List

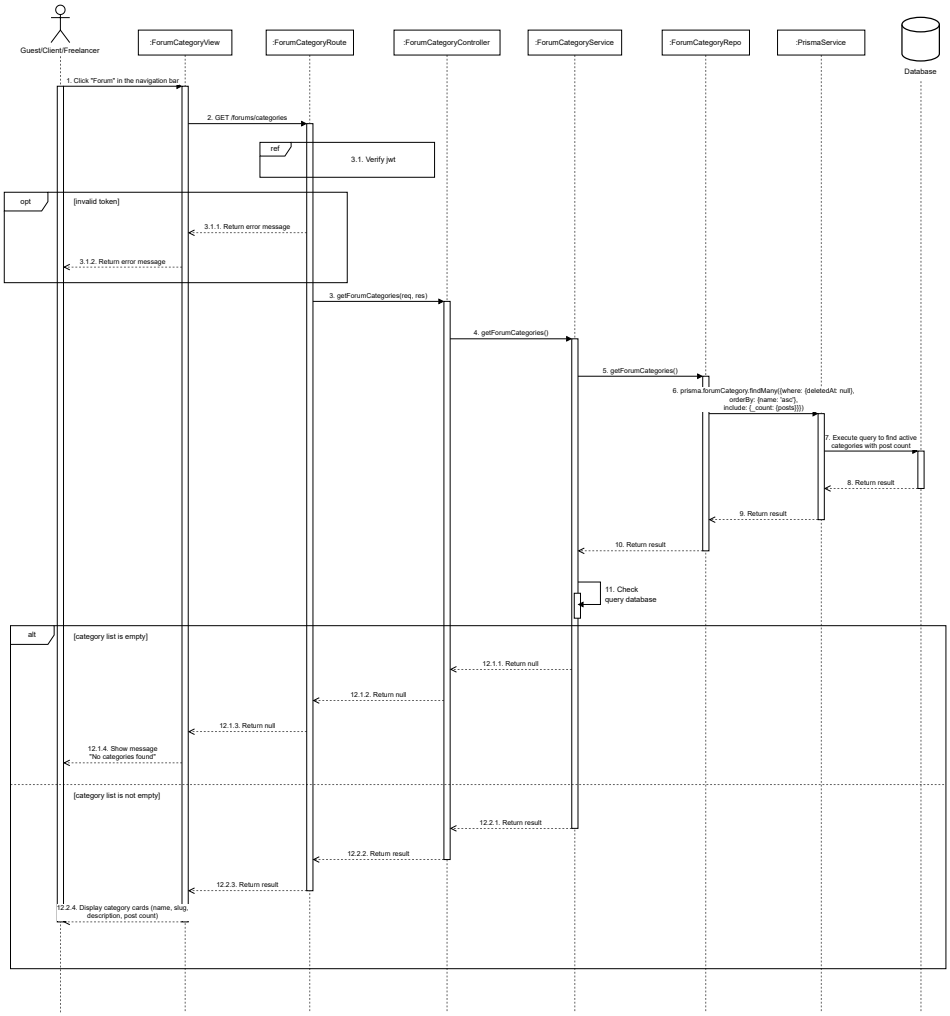

Figure 29: Sequence diagram of UC-14.01 View Forum Category List

3.1.3 UC-14.02 – View Forum Category Detail

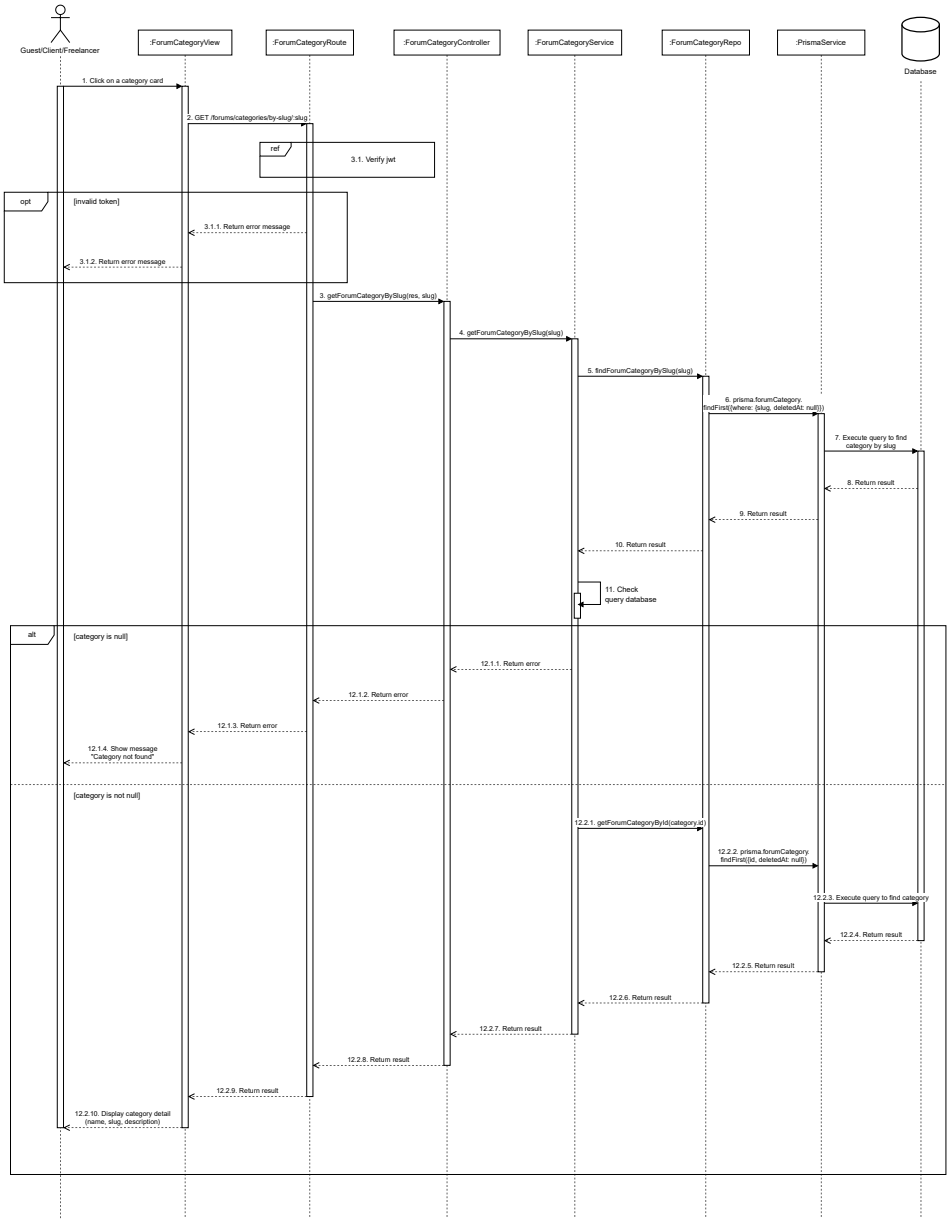

Figure 30: Sequence diagram of UC-14.02 View Forum Category Detail

3.1.4 UC-14.03 – View Post List

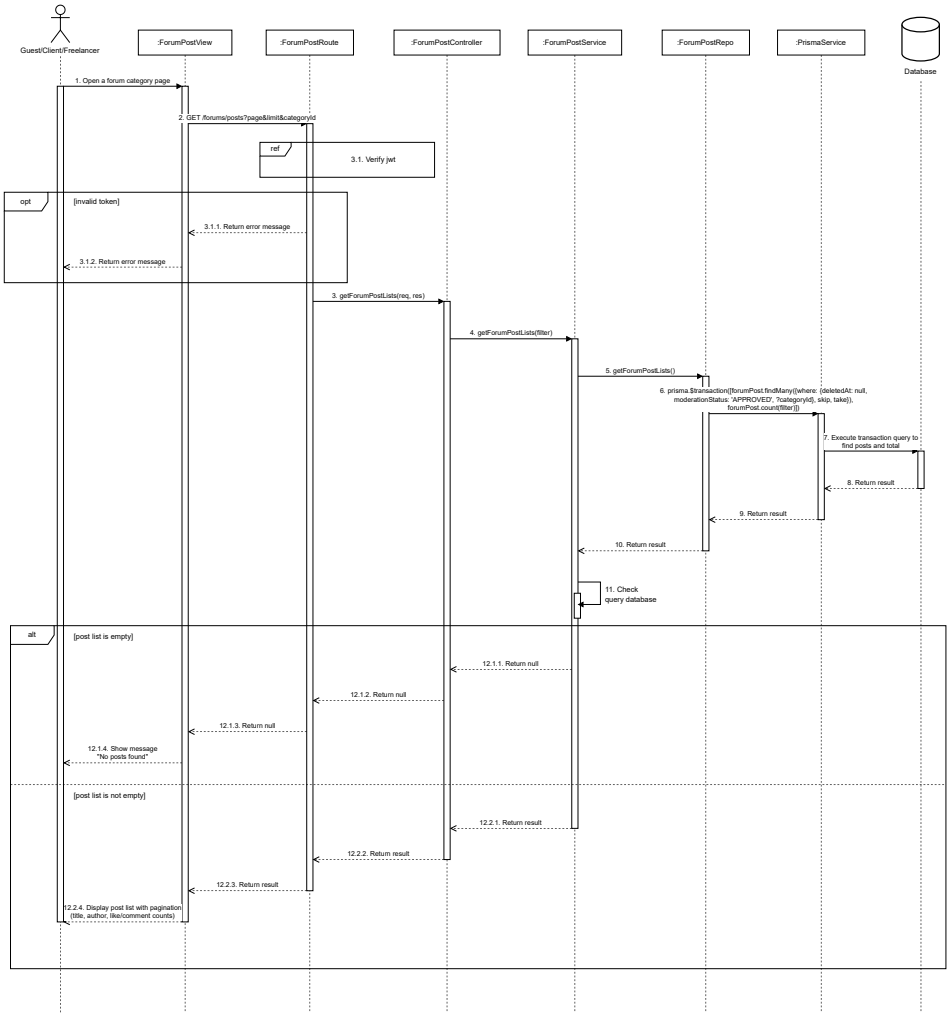

Figure 31: Sequence diagram of UC-14.03 View Post List

3.1.5 UC-14.04 – View Post Detail

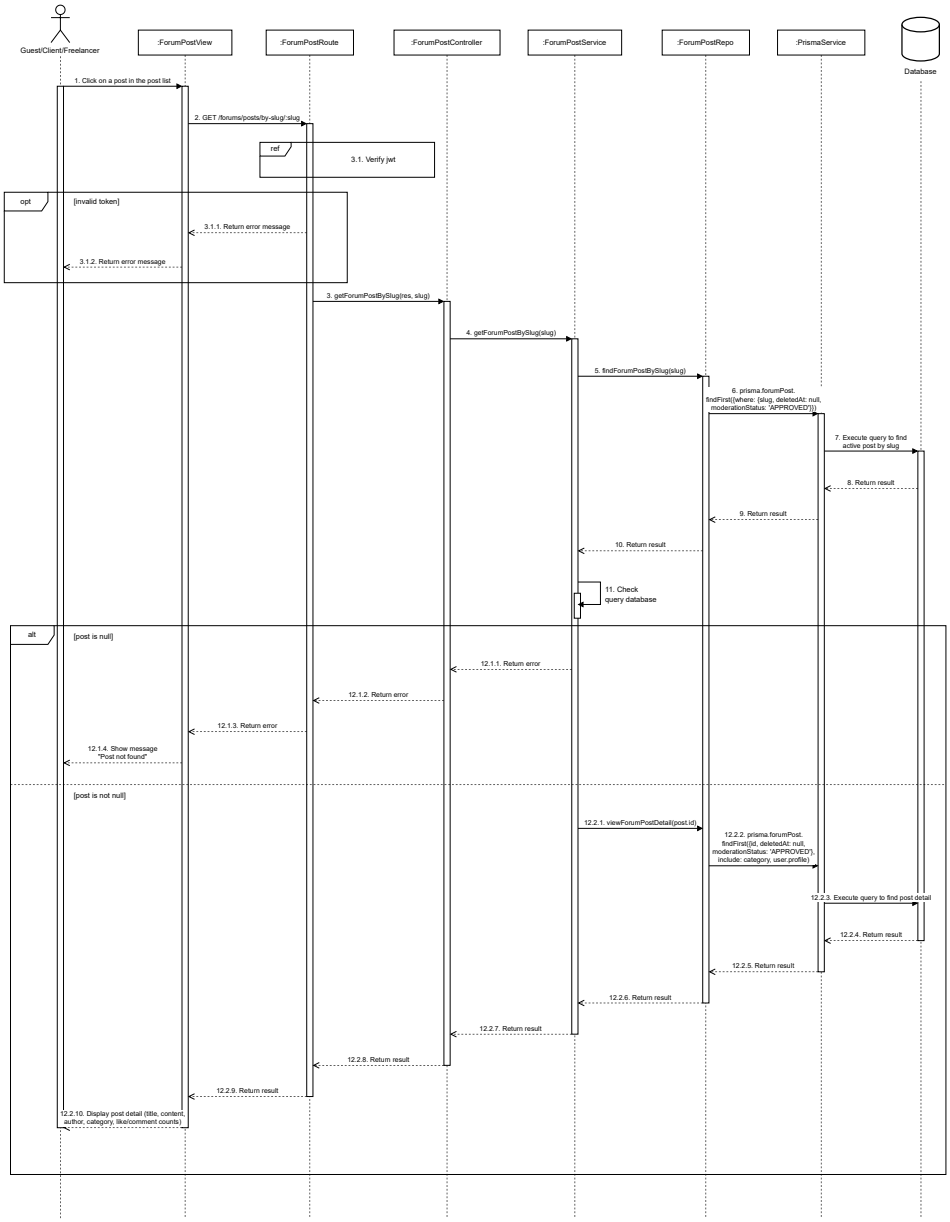

Figure 32: Sequence diagram of UC-14.04 View Post Detail

3.1.6 UC-14.05 – Create Post

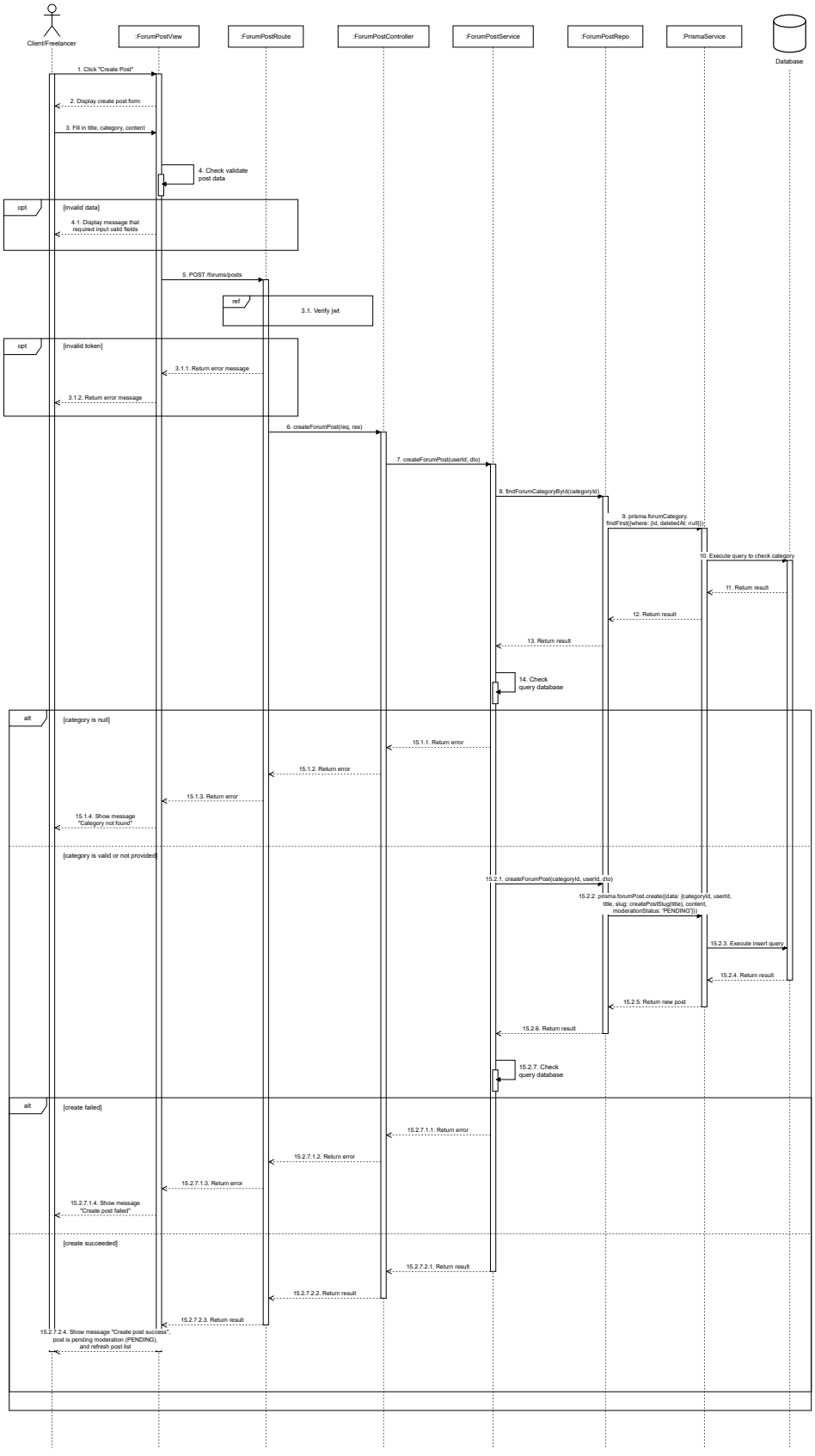

Figure 33: Sequence diagram of UC-14.05 Create Post

3.1.7 UC-14.06 – Update Post

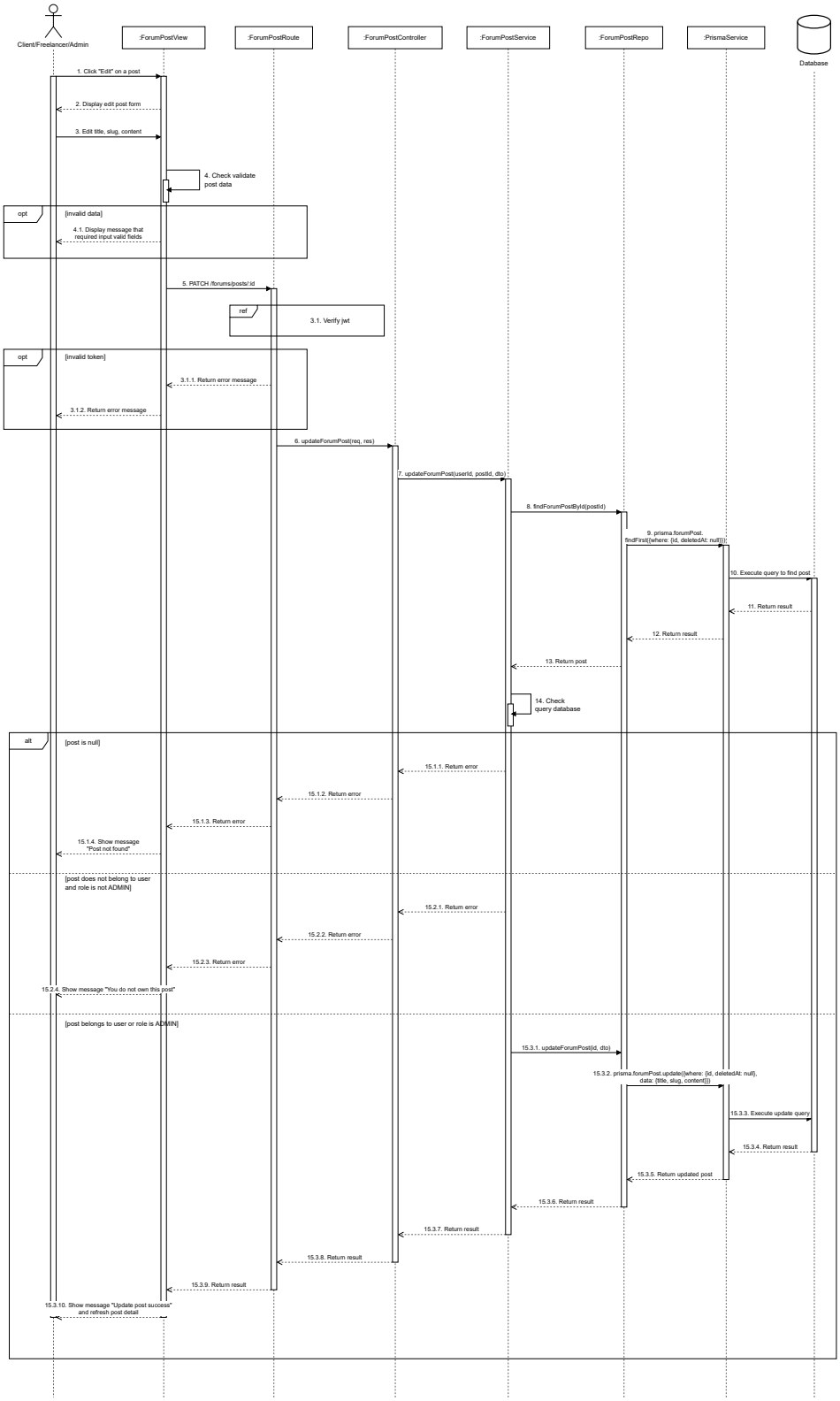

Figure 34: Sequence diagram of UC-14.06 Update Post

3.1.8 UC-14.07 – Delete Post

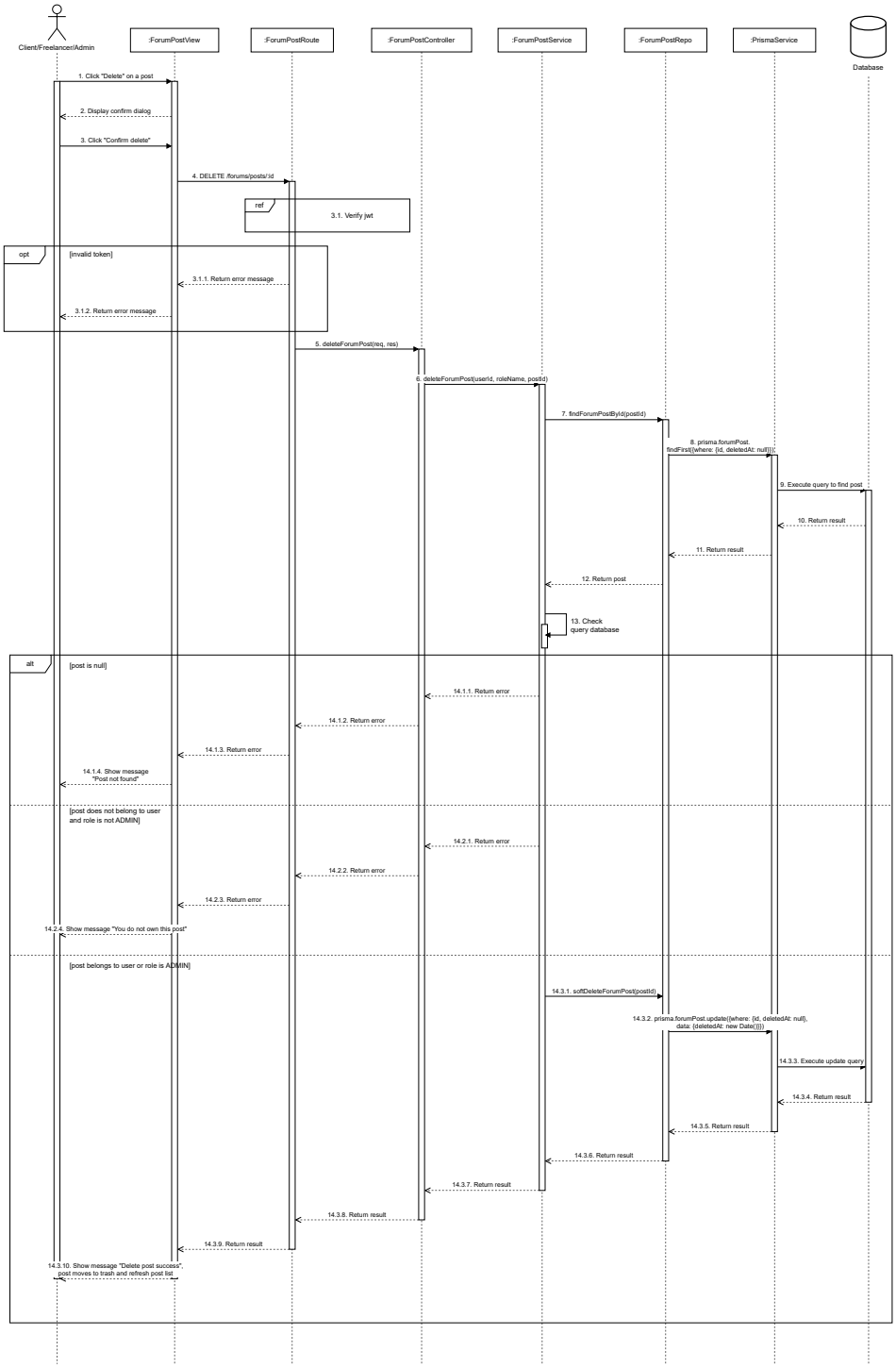

Figure 35: Sequence diagram of UC-14.07 Delete Post

3.1.9 UC-14.08 – Like Post

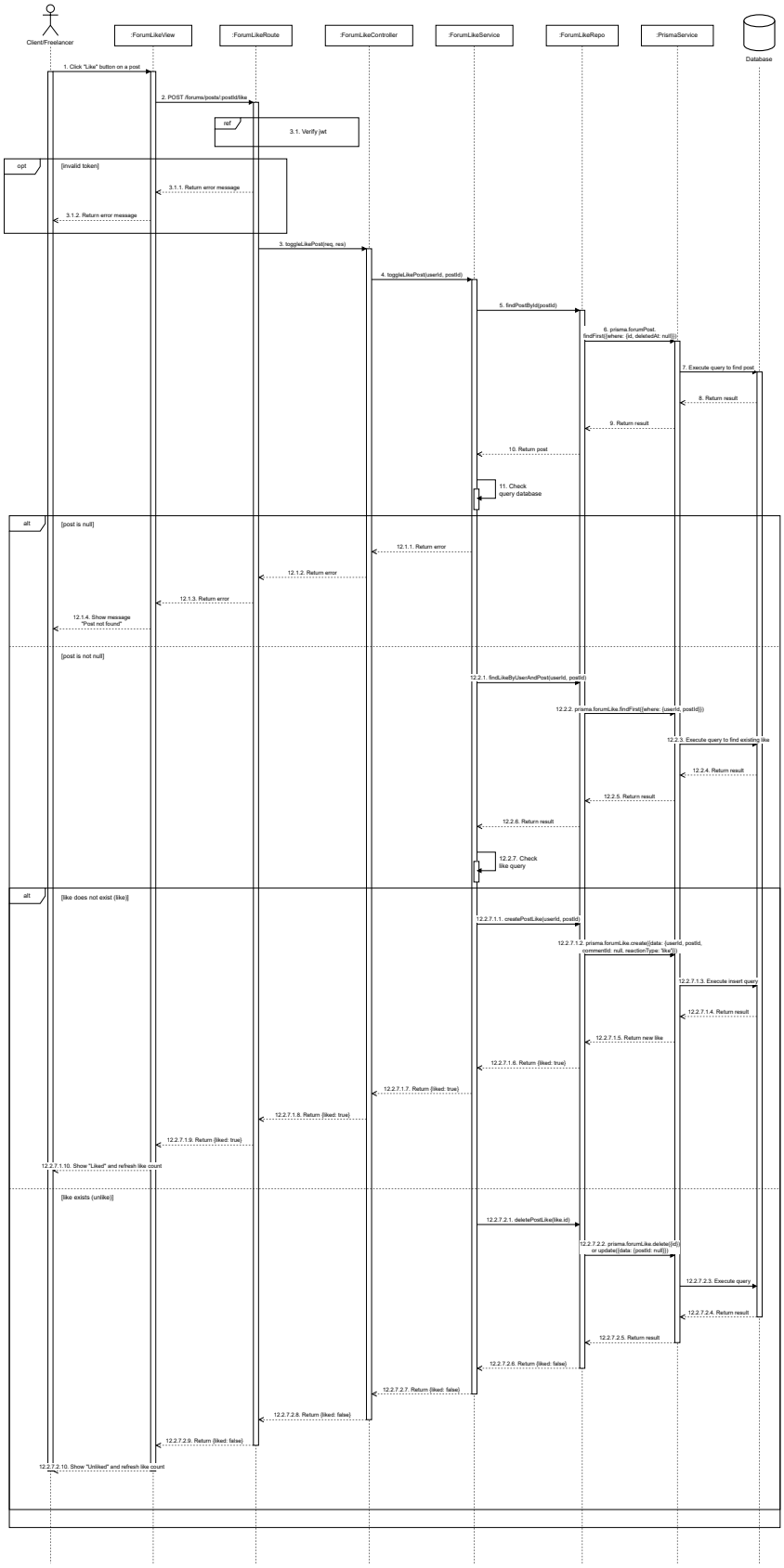

Figure 36: Sequence diagram of UC-14.08 Like Post

3.1.10 UC-14.09 – Unlike Post

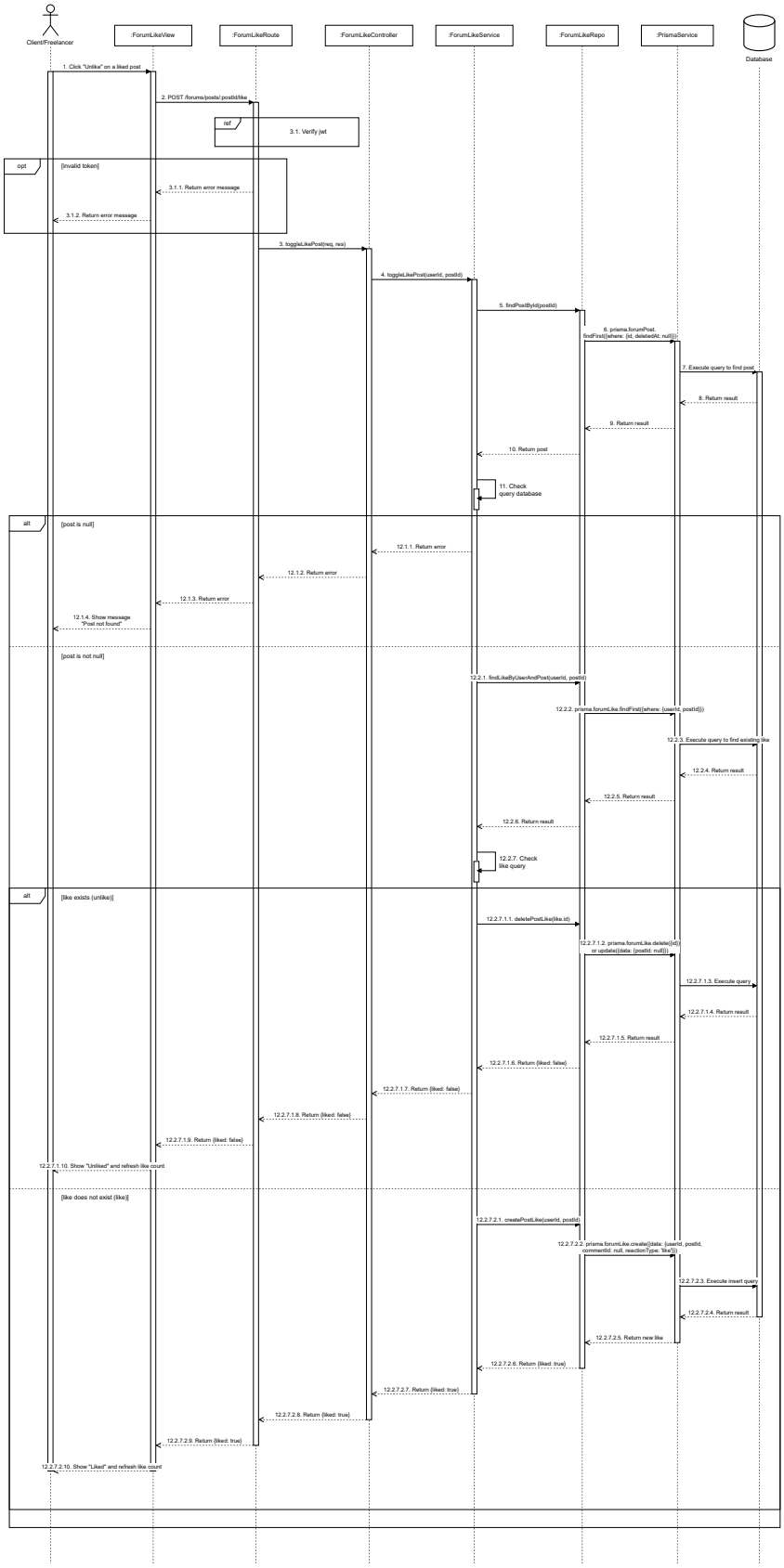

Figure 37: Sequence diagram of UC-14.09 Unlike Post

3.1.11 UC-14.10 – View Post Likes

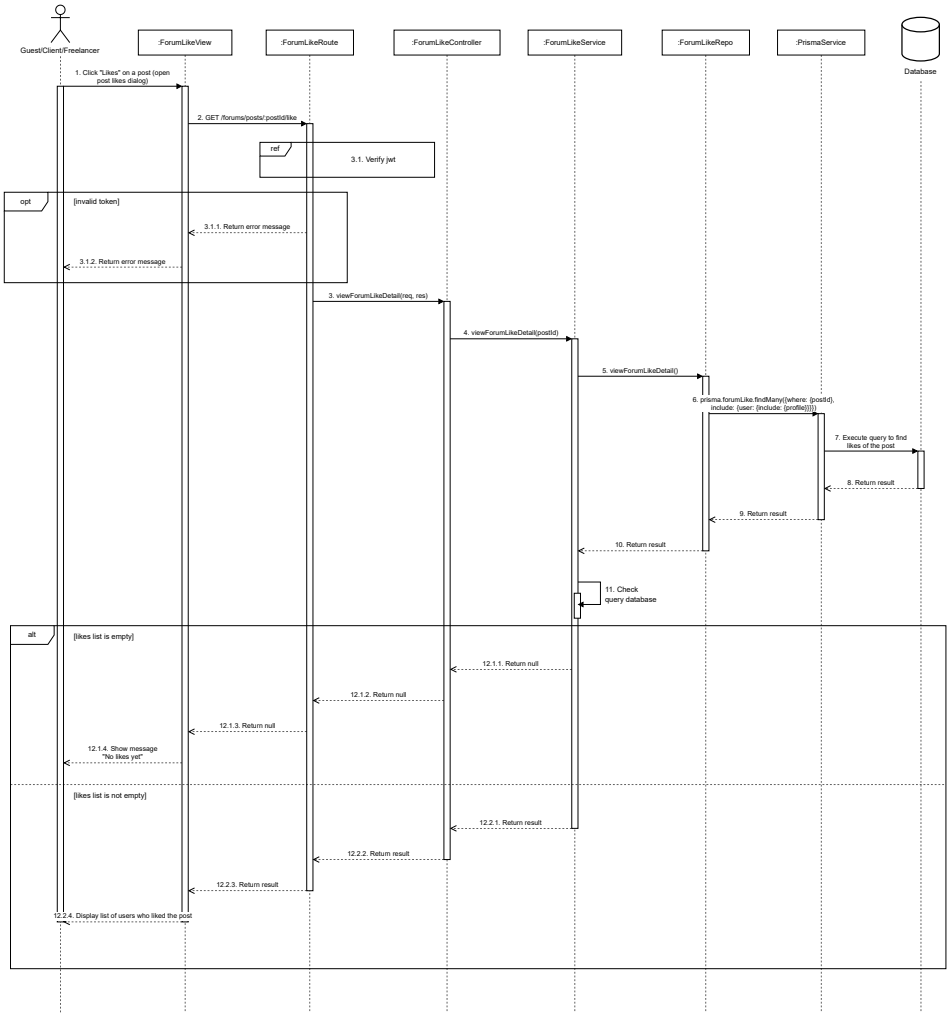

Figure 38: Sequence diagram of UC-14.10 View Post Likes

3.1.12 UC-14.11 – View Post Comments

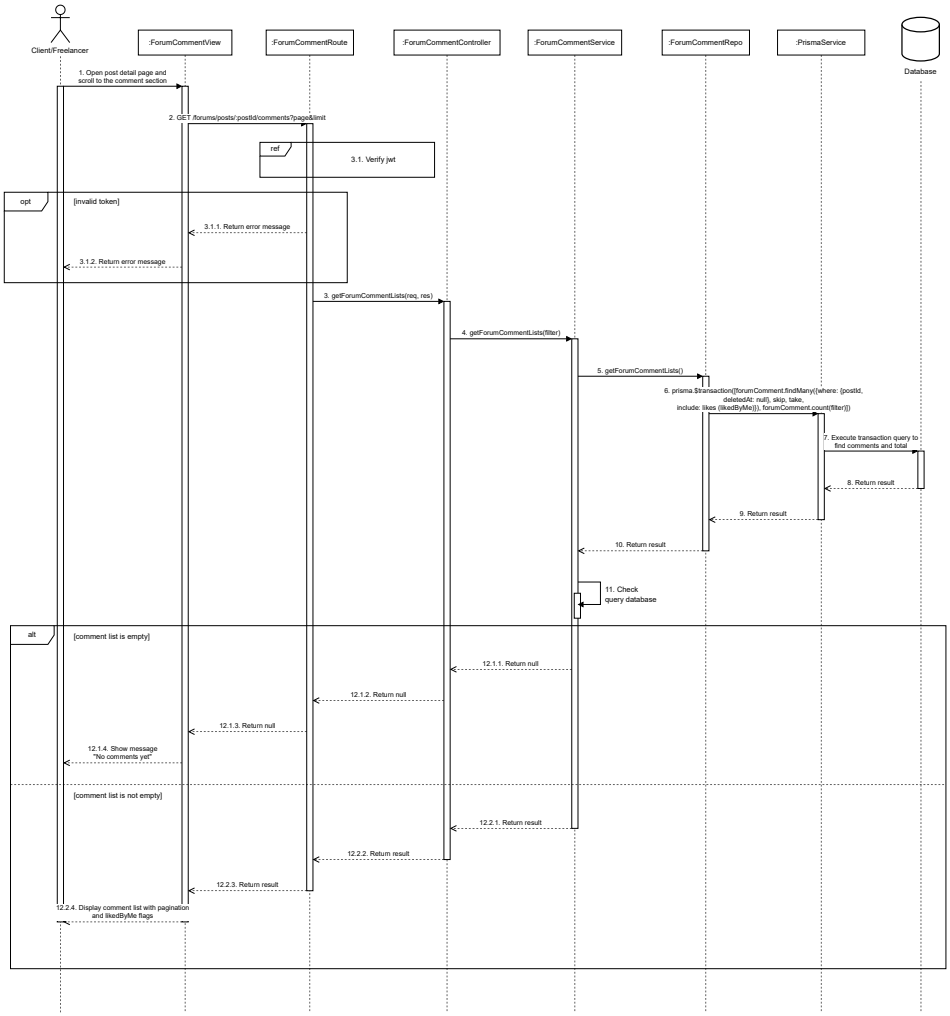

Figure 39: Sequence diagram of UC-14.11 View Post Comments

3.1.13 UC-14.12 – Create Comment

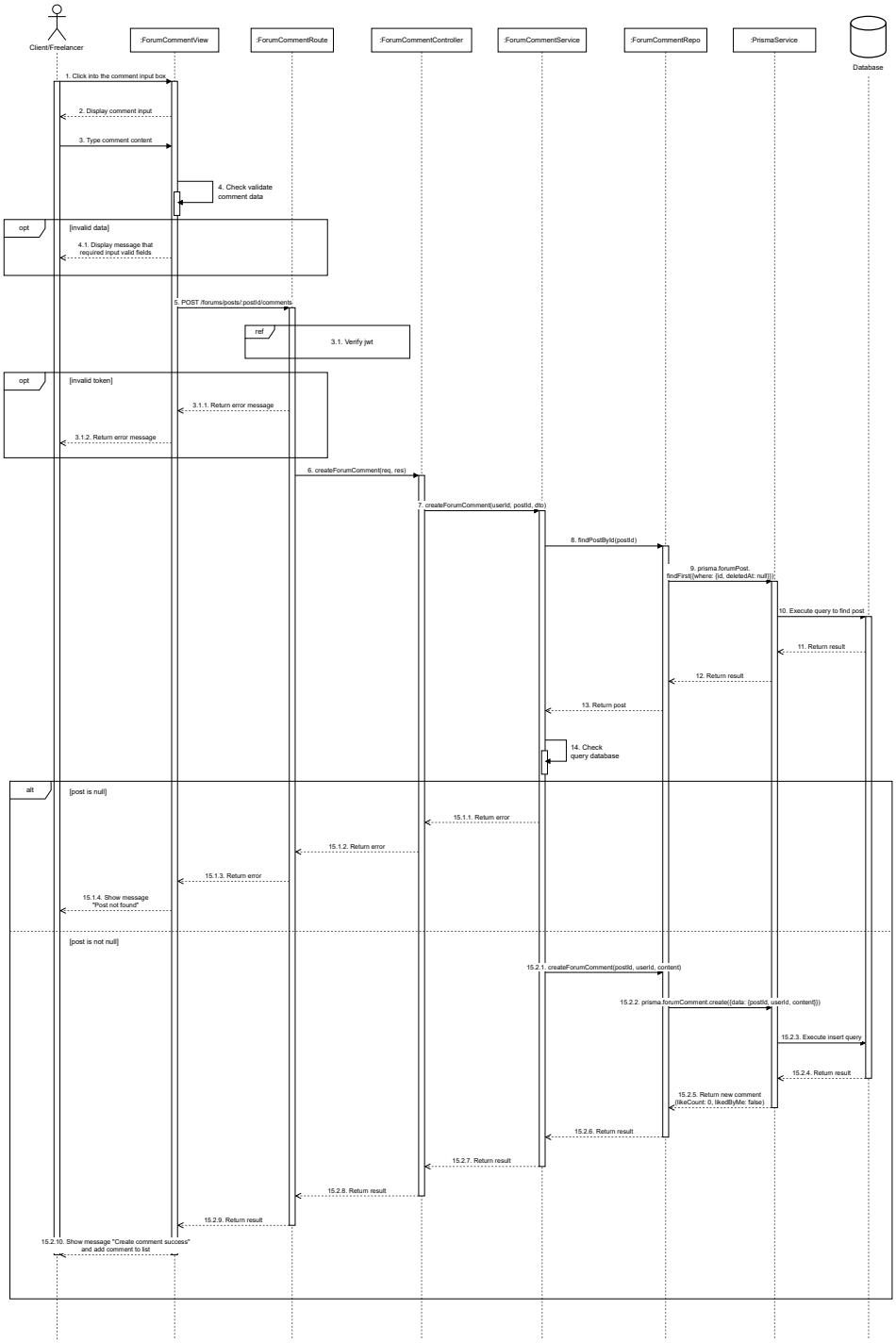

Figure 40: Sequence diagram of UC-14.12 Create Comment

3.1.14 UC-14.13 – Update Comment

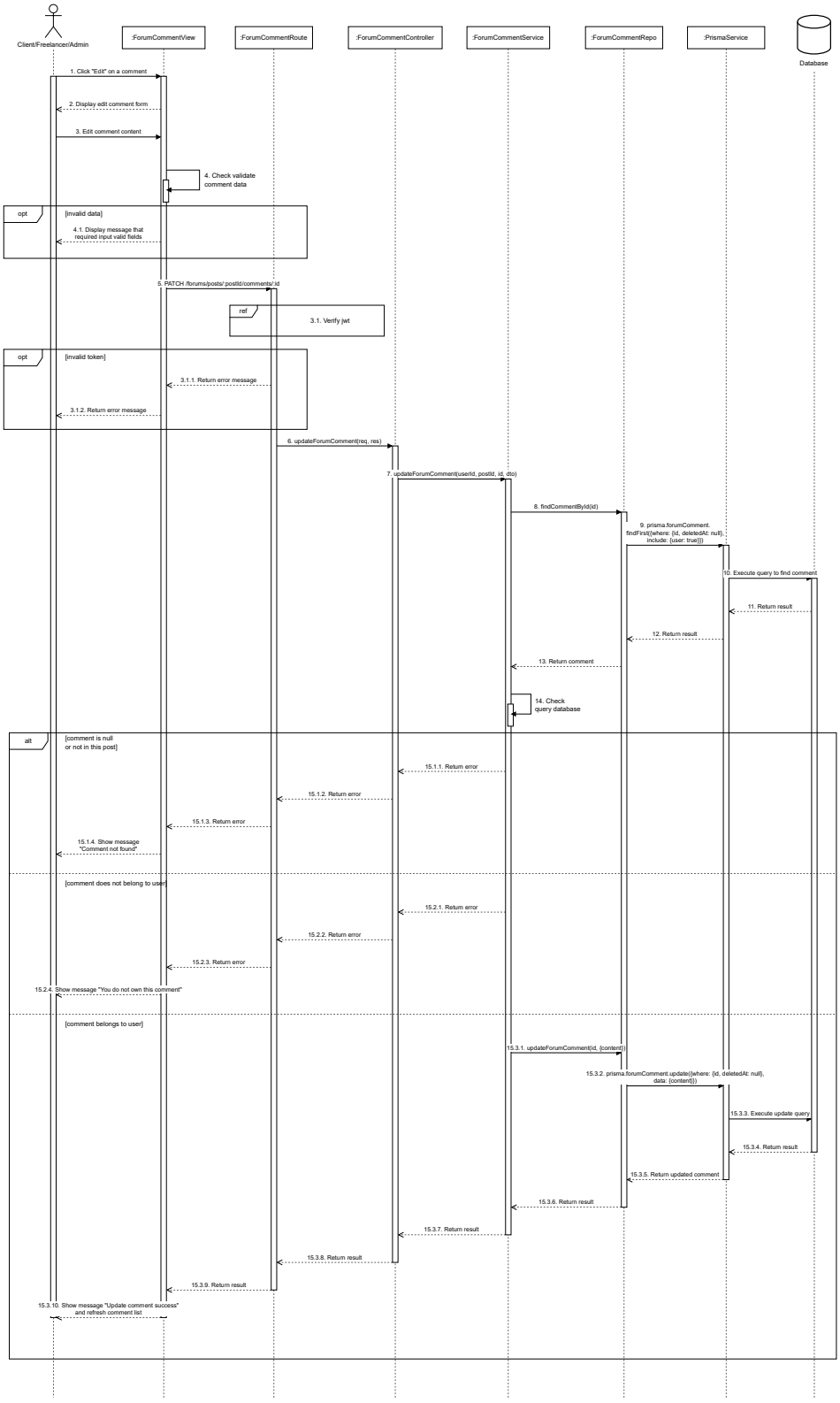

Figure 41: Sequence diagram of UC-14.13 Update Comment

3.1.15 UC-14.14 – Delete Comment

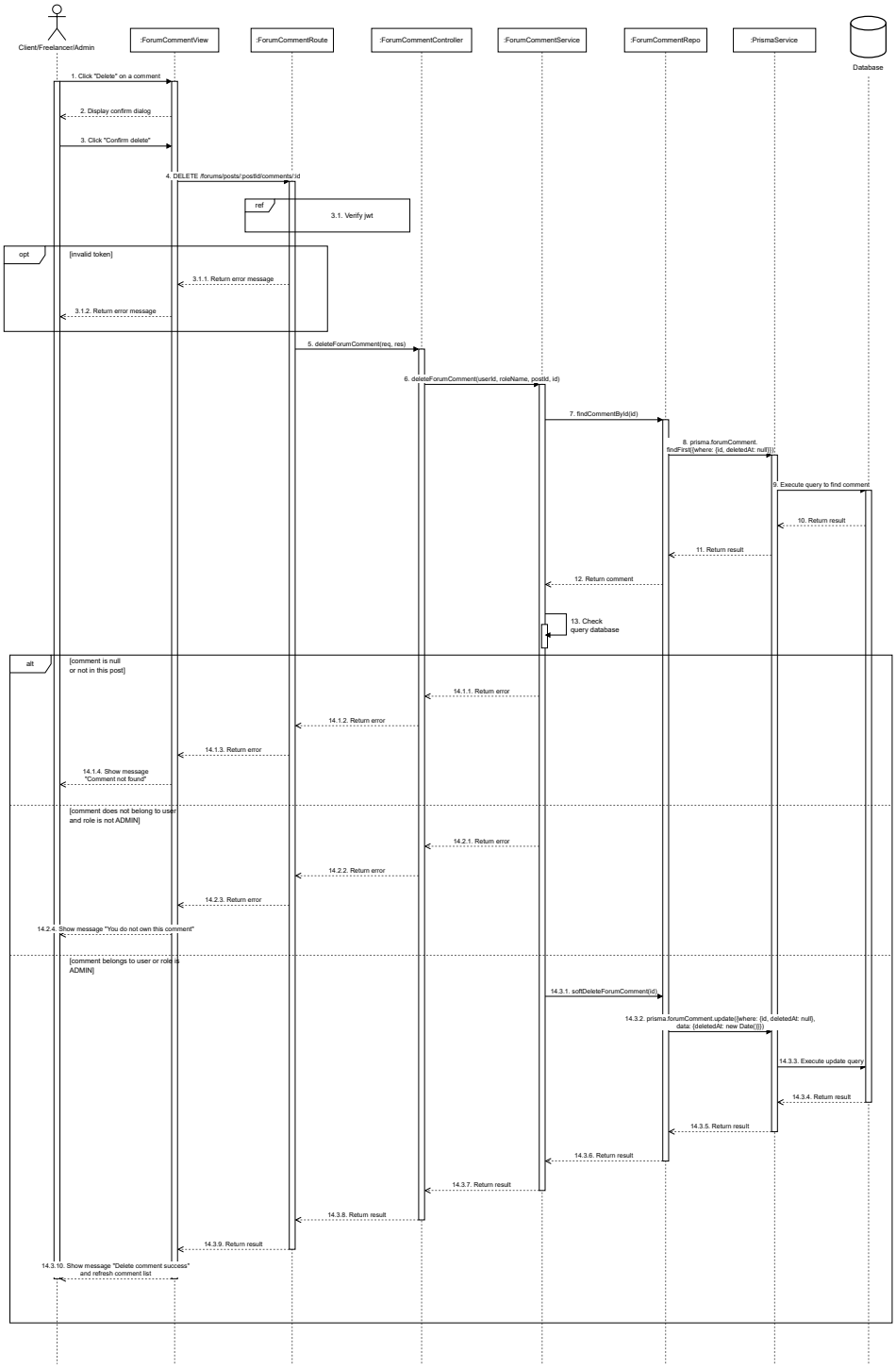

Figure 42: Sequence diagram of UC-14.14 Delete Comment

3.1.16 UC-14.15 – Like Comment

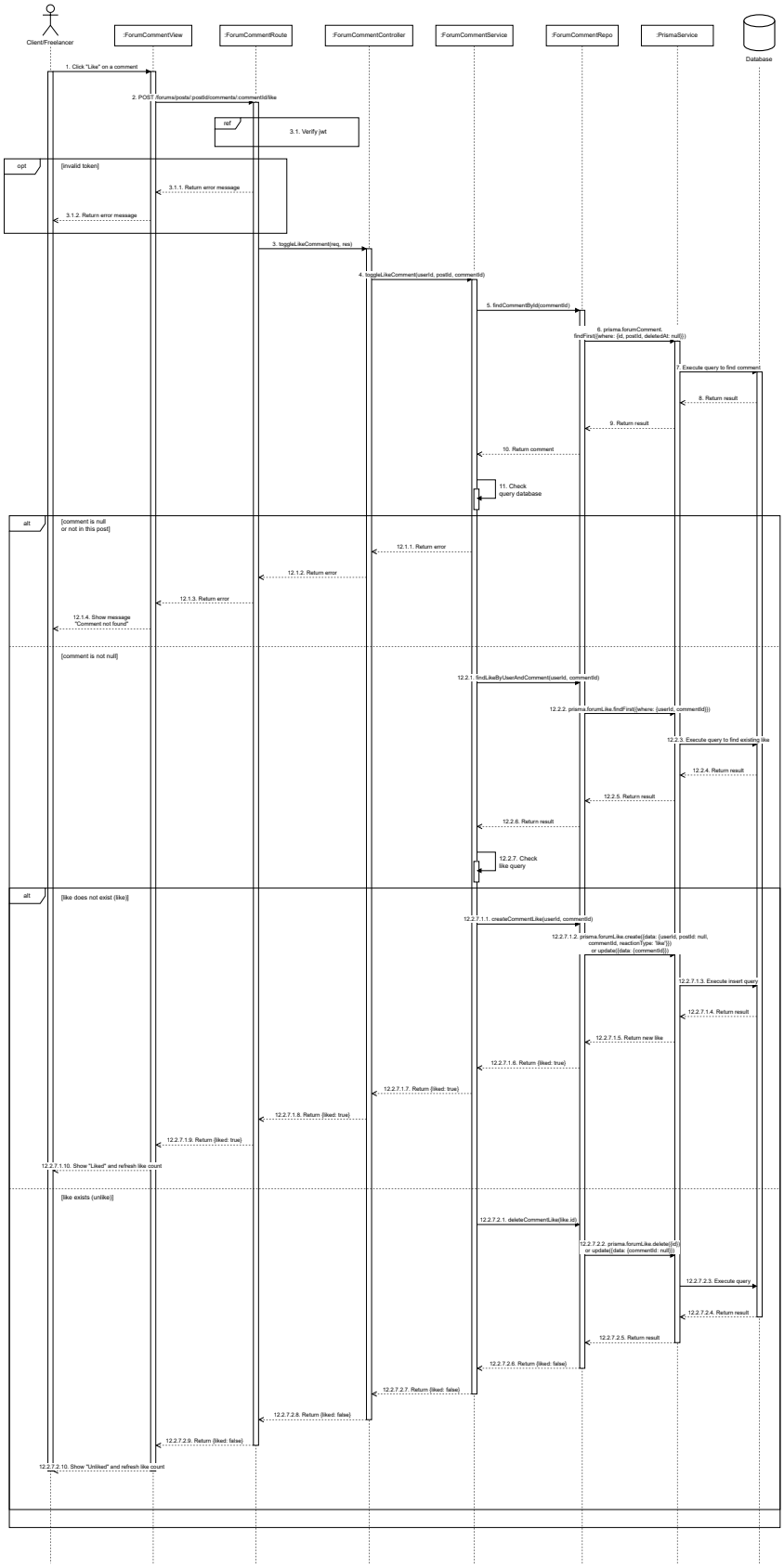

Figure 43: Sequence diagram of UC-14.15 Like Comment

3.1.17 UC-14.16 – Unlike Comment

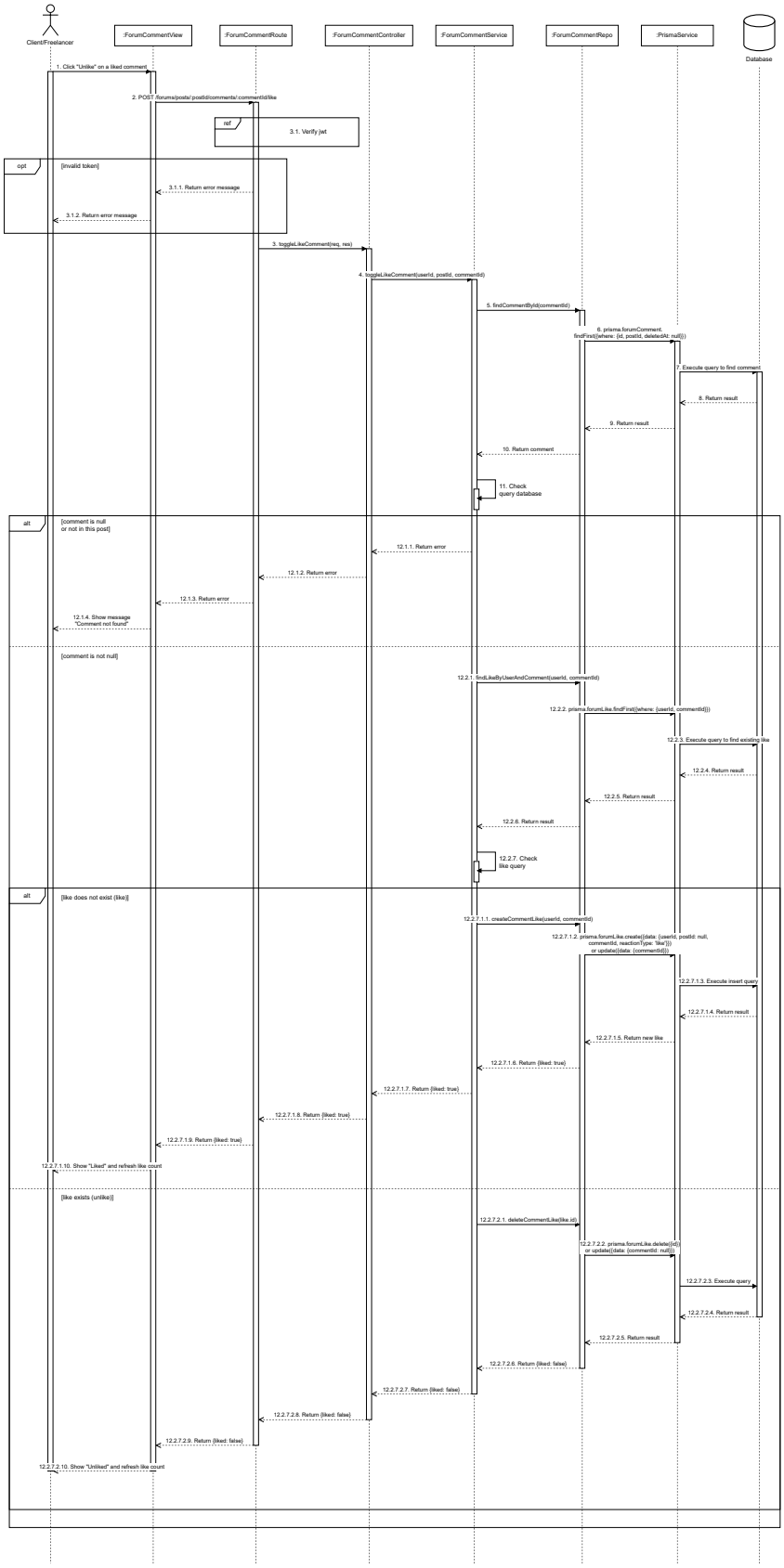

Figure 44: Sequence diagram of UC-14.16 Unlike Comment

3.1.18 UC-14.17 – Report Content

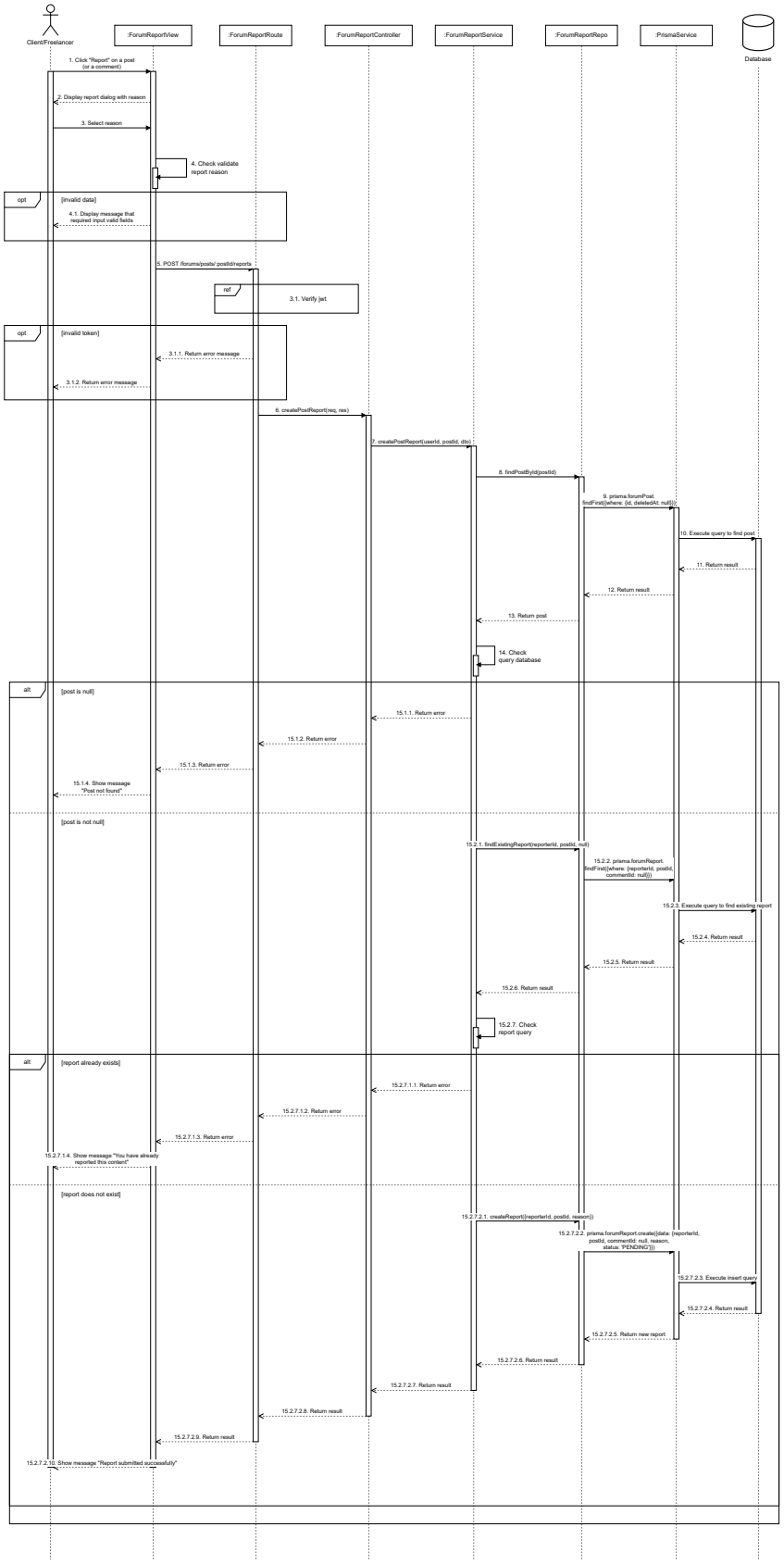

Figure 45: Sequence diagram of UC-14.17 Report Content

3.2 Job Browsing

Job Browsing allows guests and registered users to search, filter, and view the list of available jobs on the platform, as well as view comprehensive job details.

3.2.1 Class Diagram

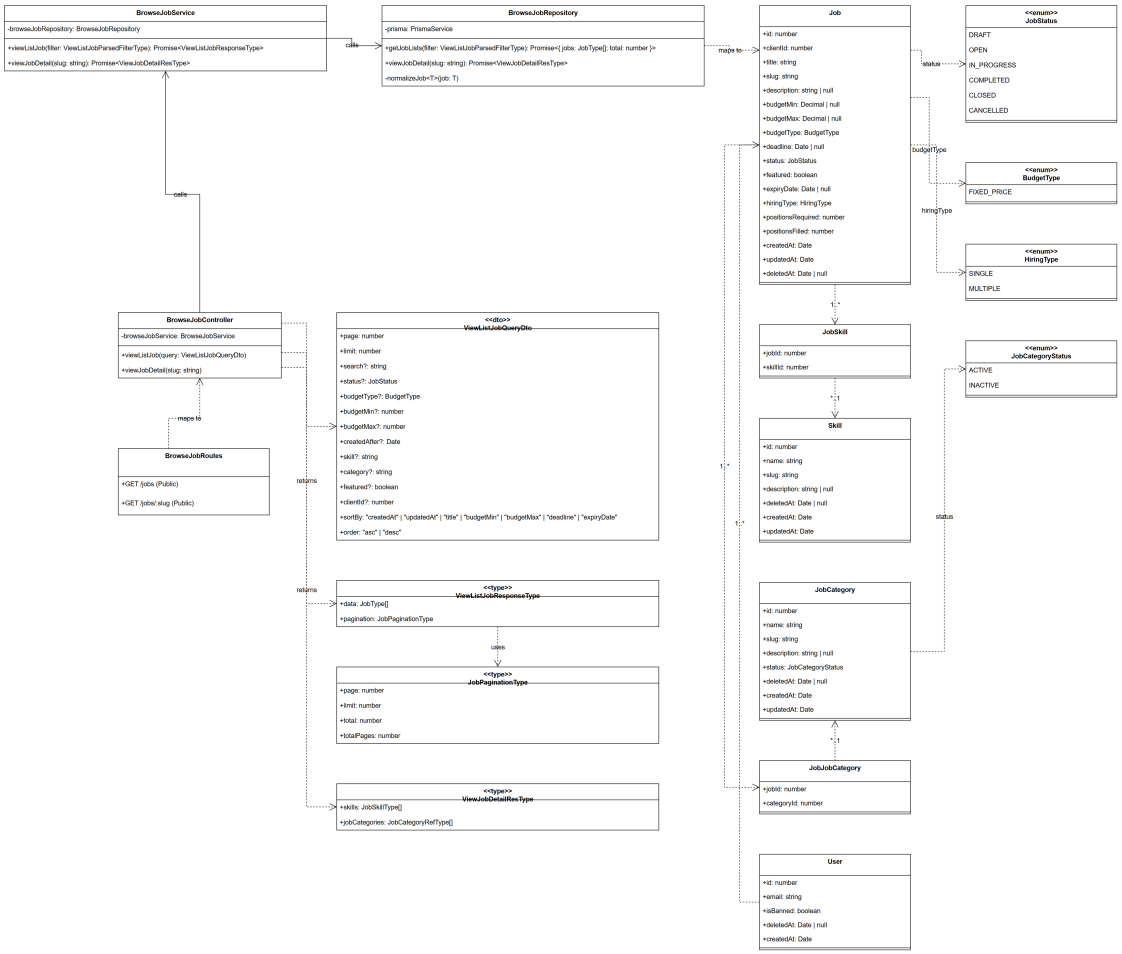

Figure 46: Class diagram of Job Browsing

3.2.2 UC-10.01 – View Job List

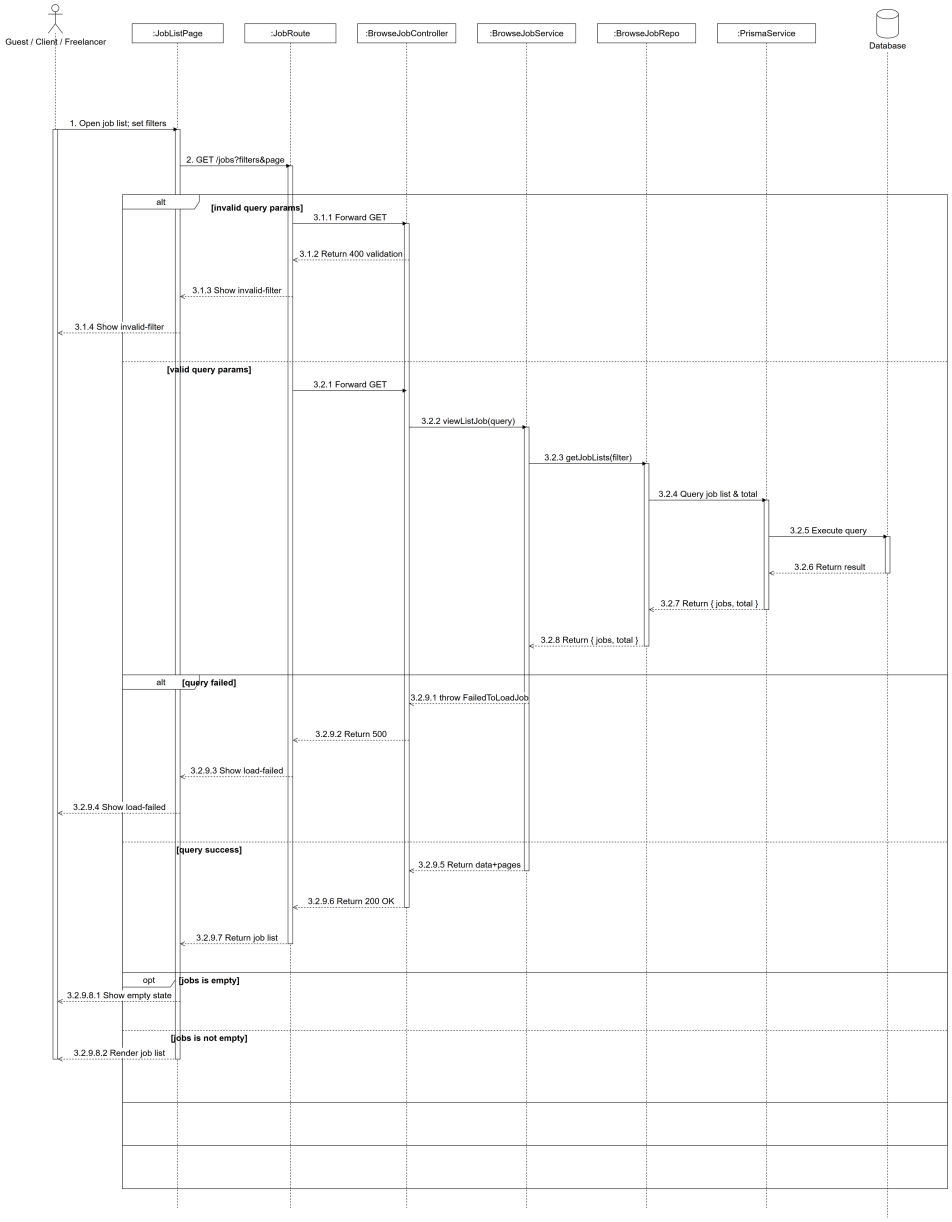

Figure 47: Sequence diagram of UC-10.01 View Job List

3.2.3 UC-10.02 – View Job Detail

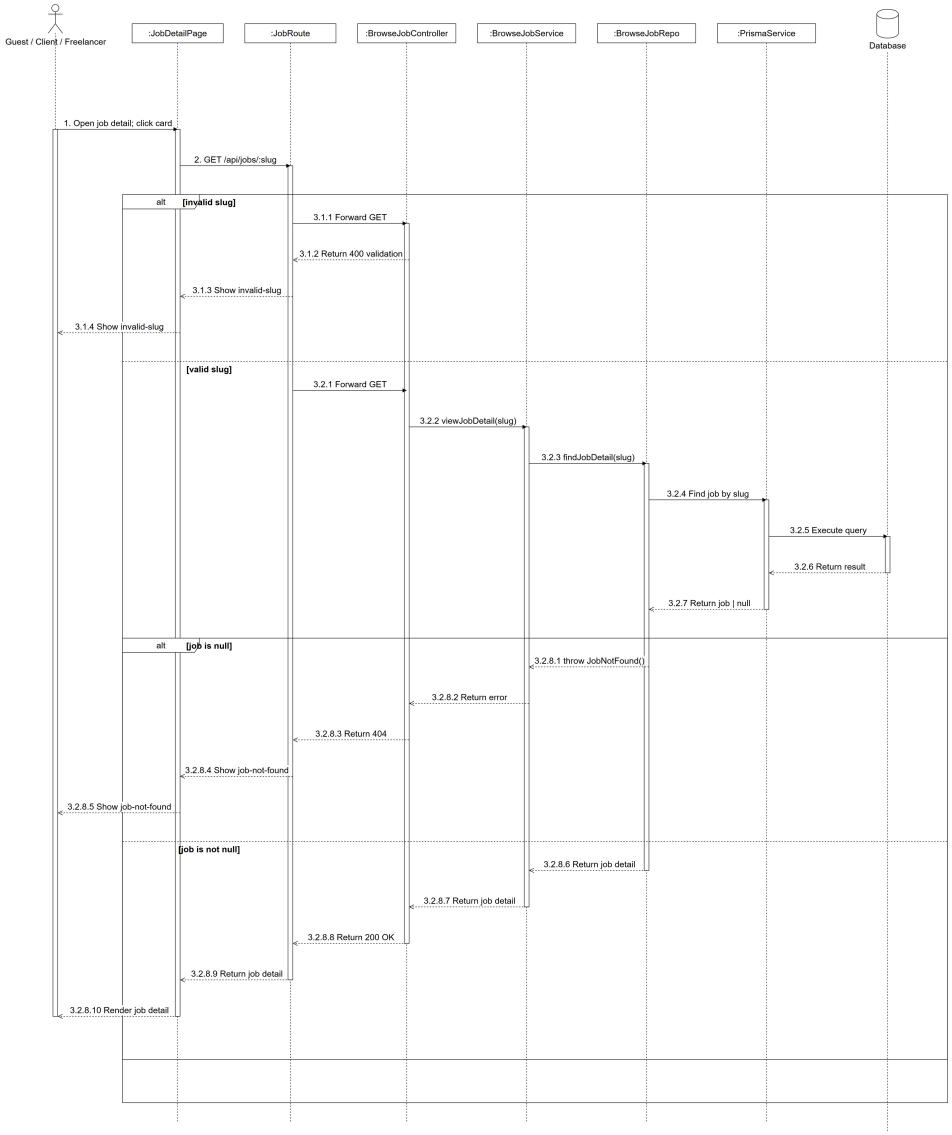

Figure 48: Sequence diagram of UC-10.02 View Job Detail

3.3 Job Management

Job Management enables clients to manage their posted jobs throughout their lifecycle, including creating new jobs, editing job parameters, publishing drafts, closing completed or cancelled jobs, and soft-deleting job postings.

3.3.1 Class Diagram

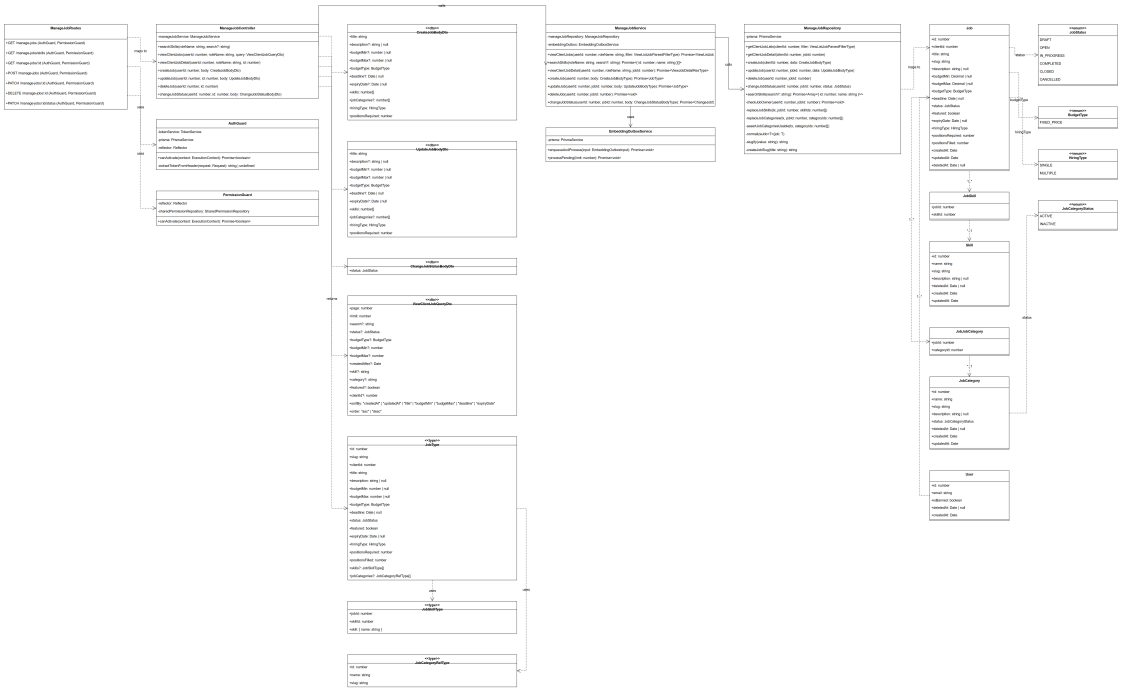

Figure 49: Class diagram of Job Management

3.3.2 UC-11.01 – View My Jobs

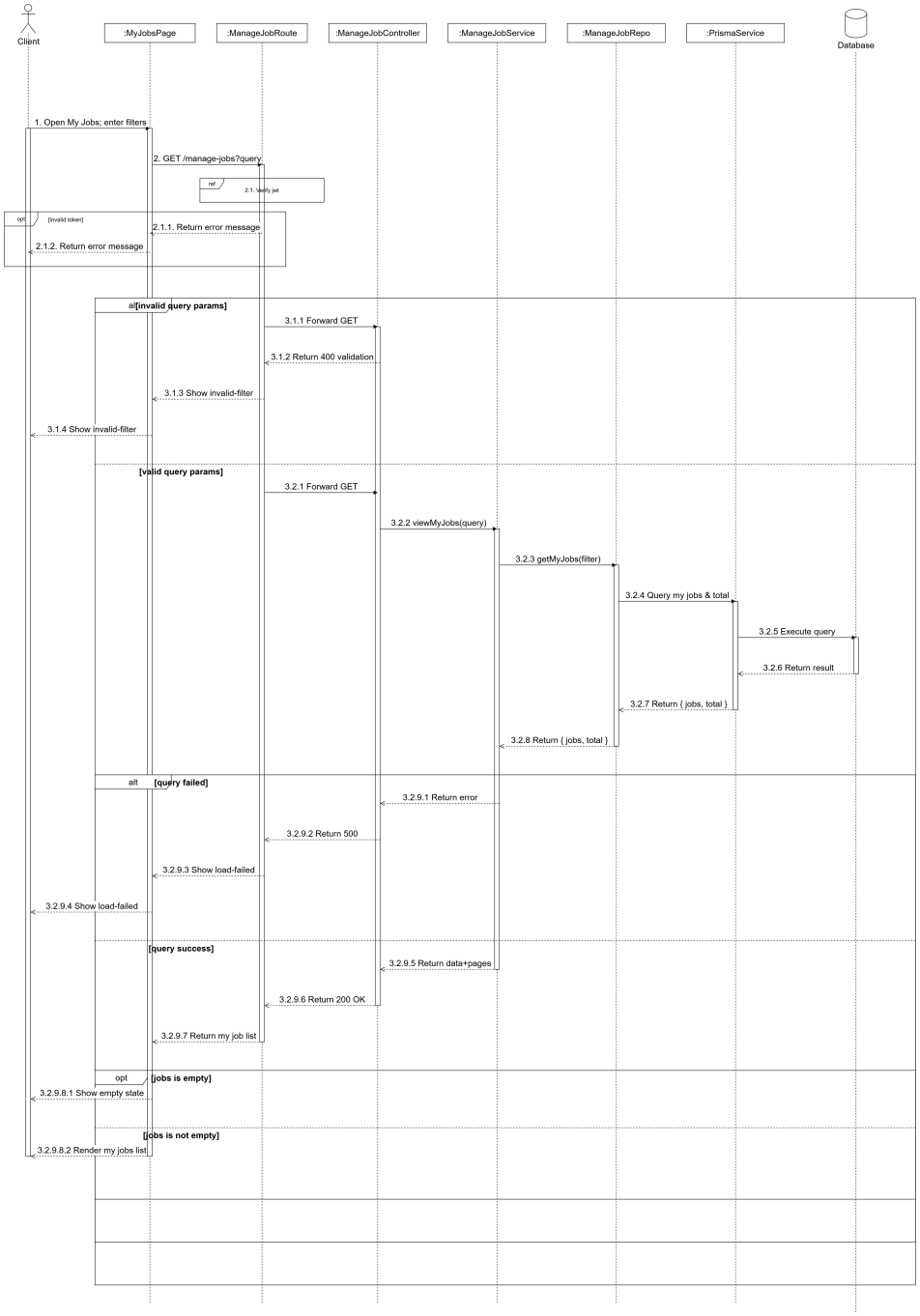

Figure 50: Sequence diagram of UC-11.01 View My Jobs

3.3.3 UC-11.02 – Create Job

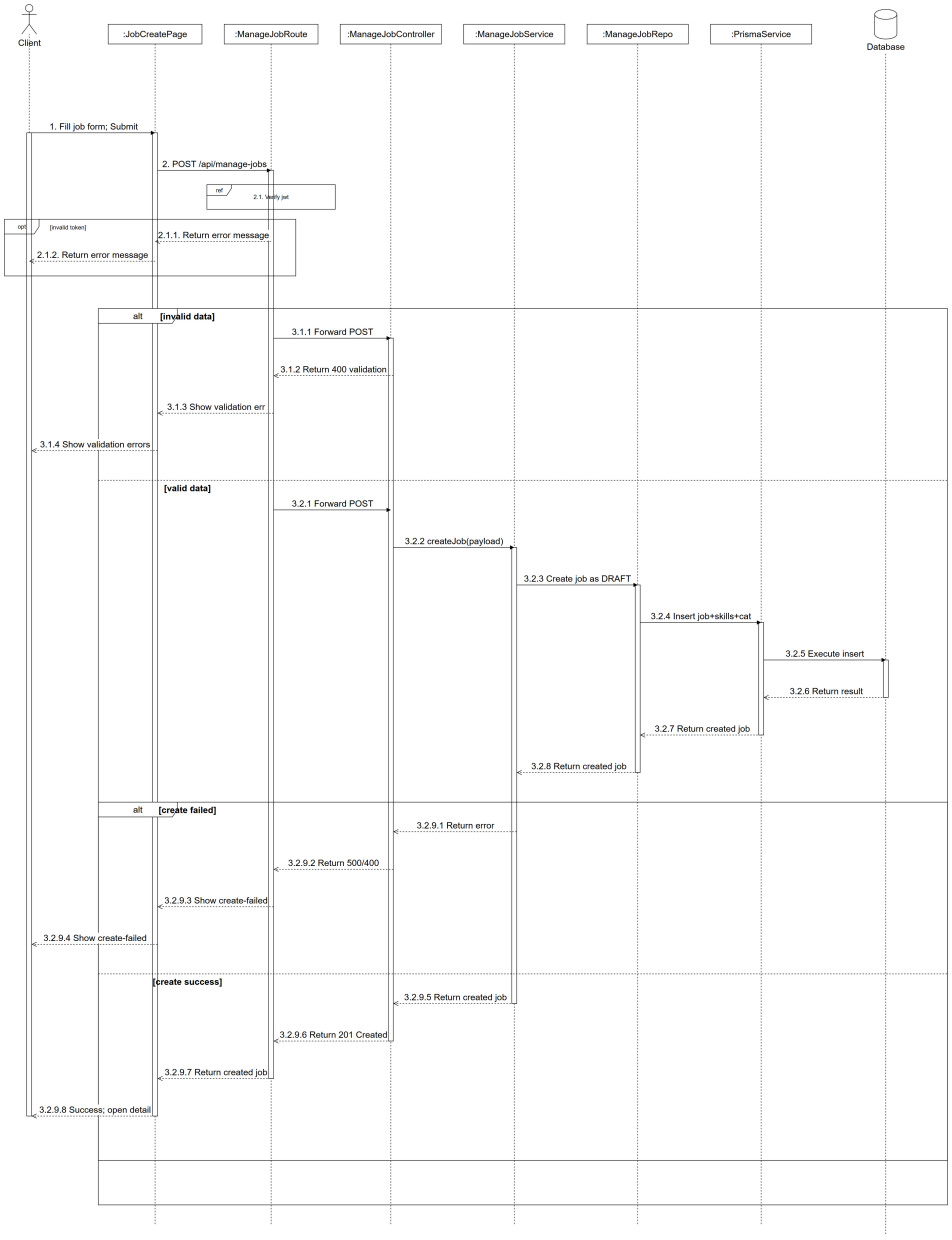

Figure 51: Sequence diagram of UC-11.02 Create Job

3.3.4 UC-11.03 – Publish Job

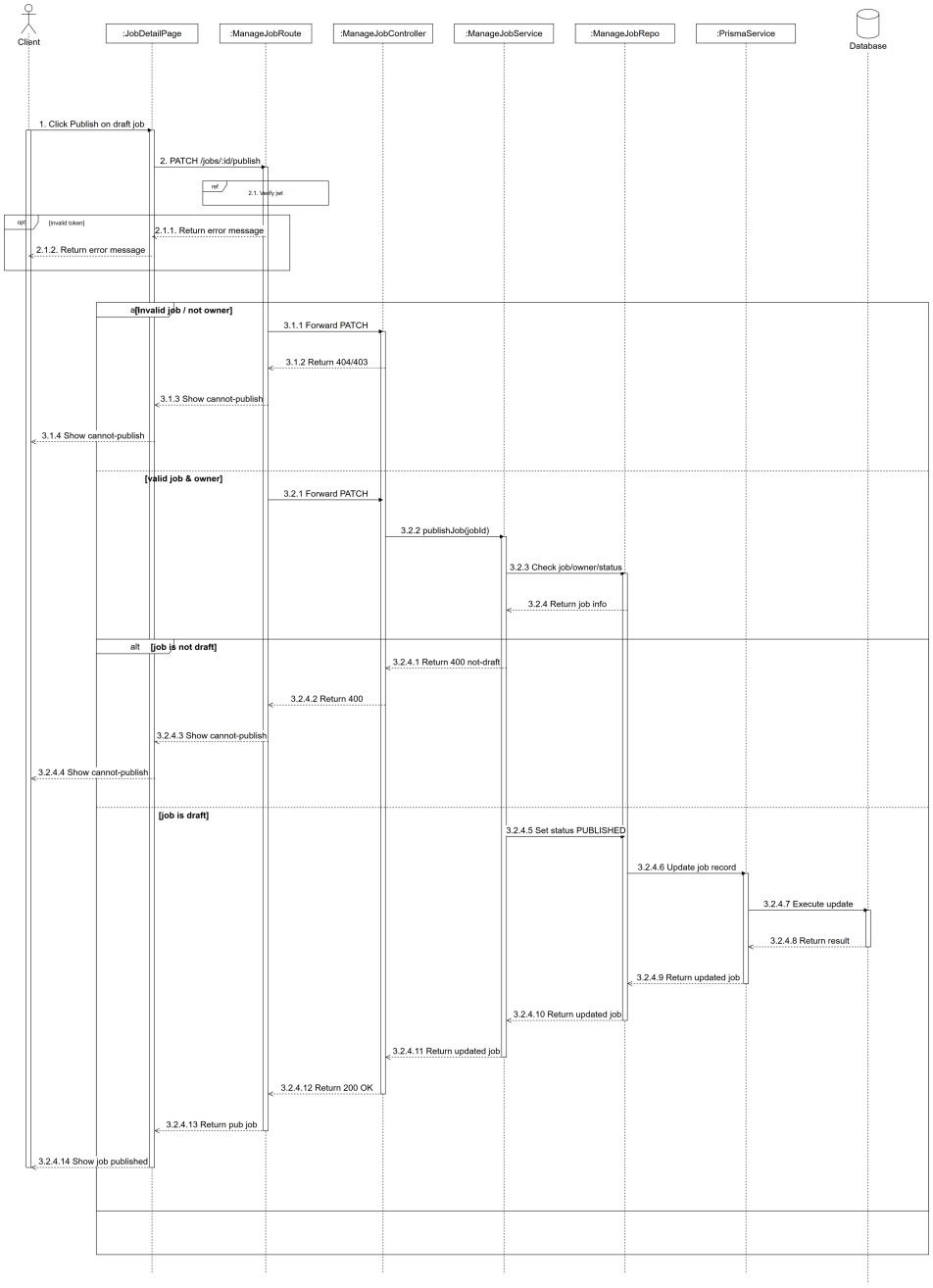

Figure 52: Sequence diagram of UC-11.03 Publish Job

3.3.5 UC-11.04 – Update Job

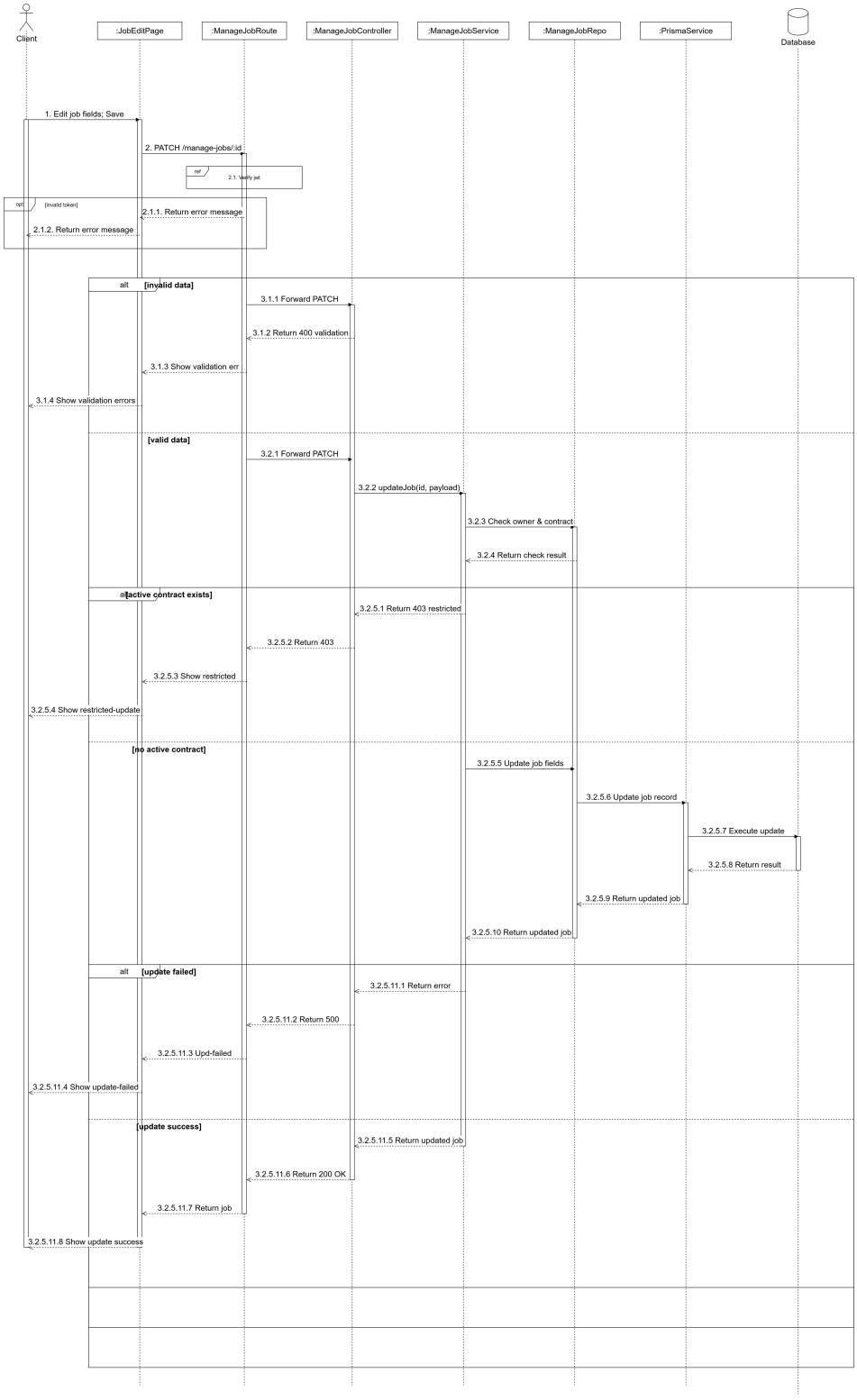

Figure 53: Sequence diagram of UC-11.04 Update Job

3.3.6 UC-11.05 – Delete Job

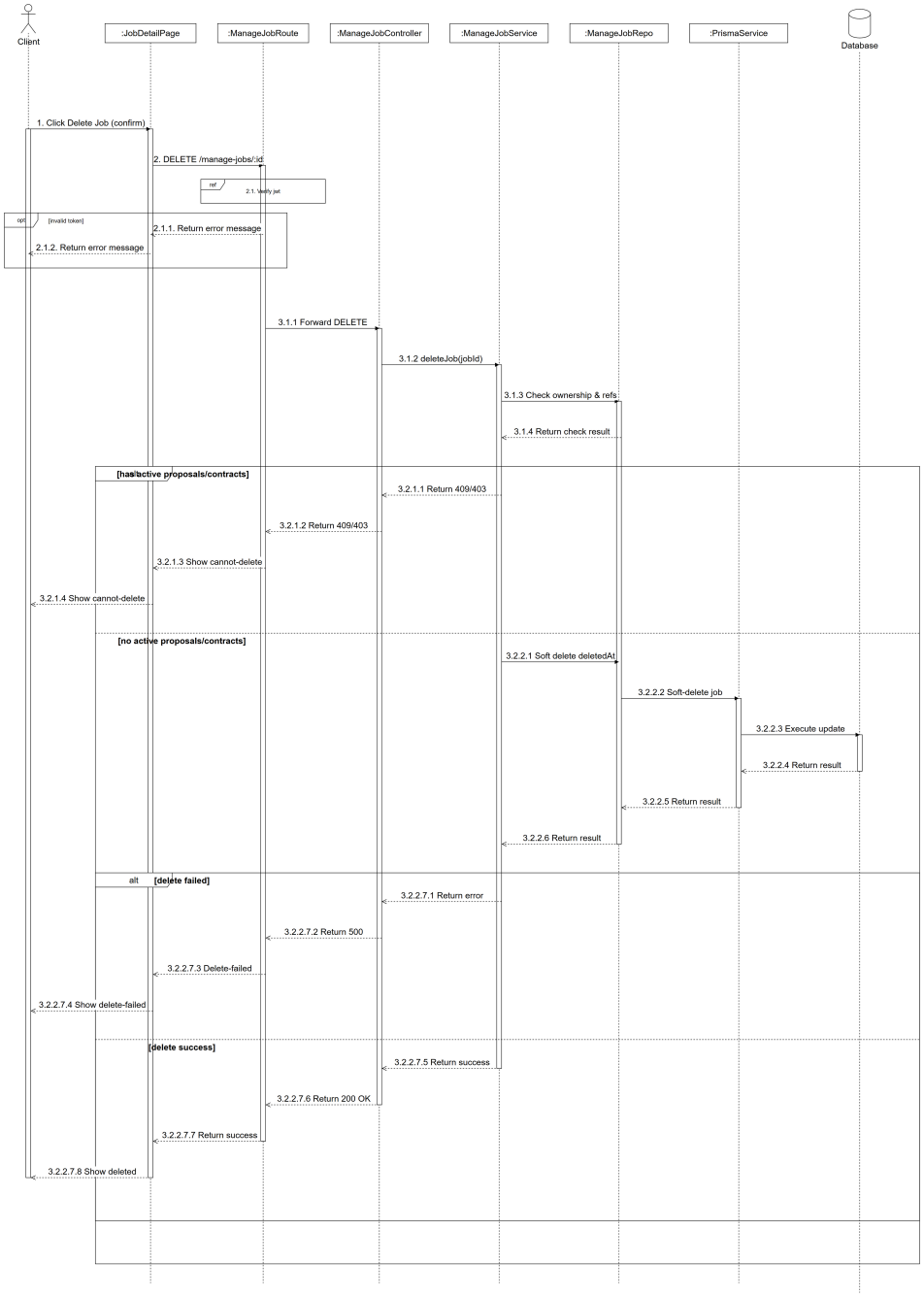

Figure 54: Sequence diagram of UC-11.05 Delete Job

3.3.7 UC-11.06 – Change Job Status

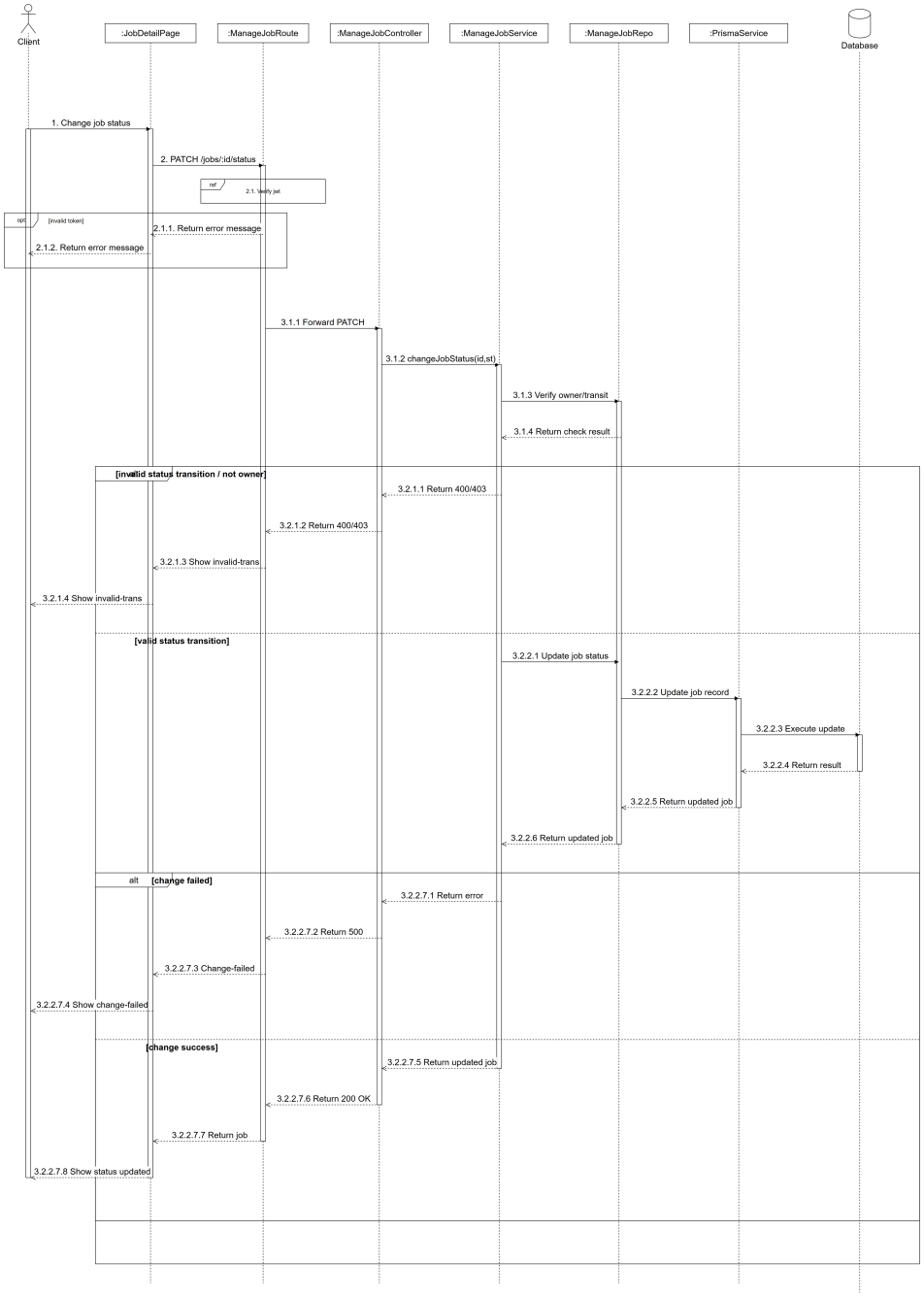

Figure 55: Sequence diagram of UC-11.06 Change Job Status

3.4 Bookmarked Jobs Management

Bookmarked Jobs Management allows freelancers to save interesting job postings for later review, inspect their list of bookmarked jobs, and remove bookmarks when no longer needed.

3.4.1 Class Diagram

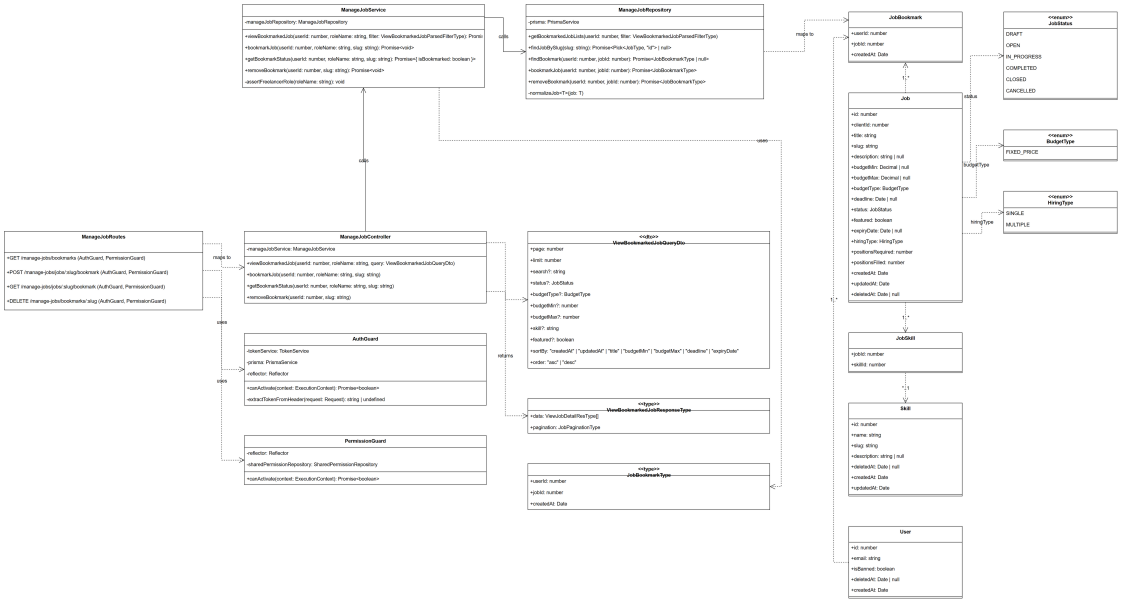

Figure 56: Class diagram of Bookmarked Jobs Management

3.4.2 UC-12.01 – View Bookmarked Job List

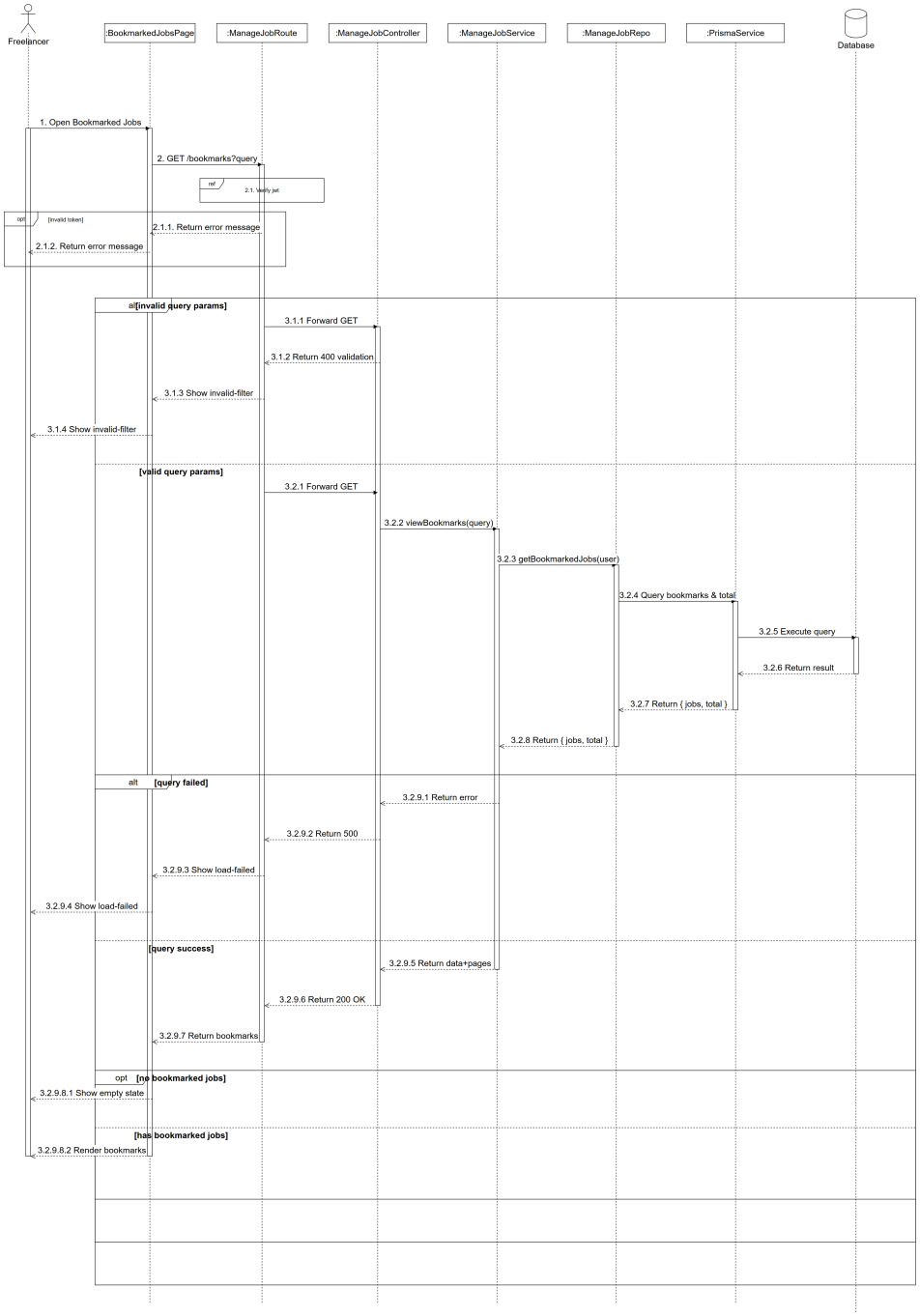

Figure 57: Sequence diagram of UC-12.01 View Bookmarked Job List

3.4.3 UC-12.02 – Bookmark Job

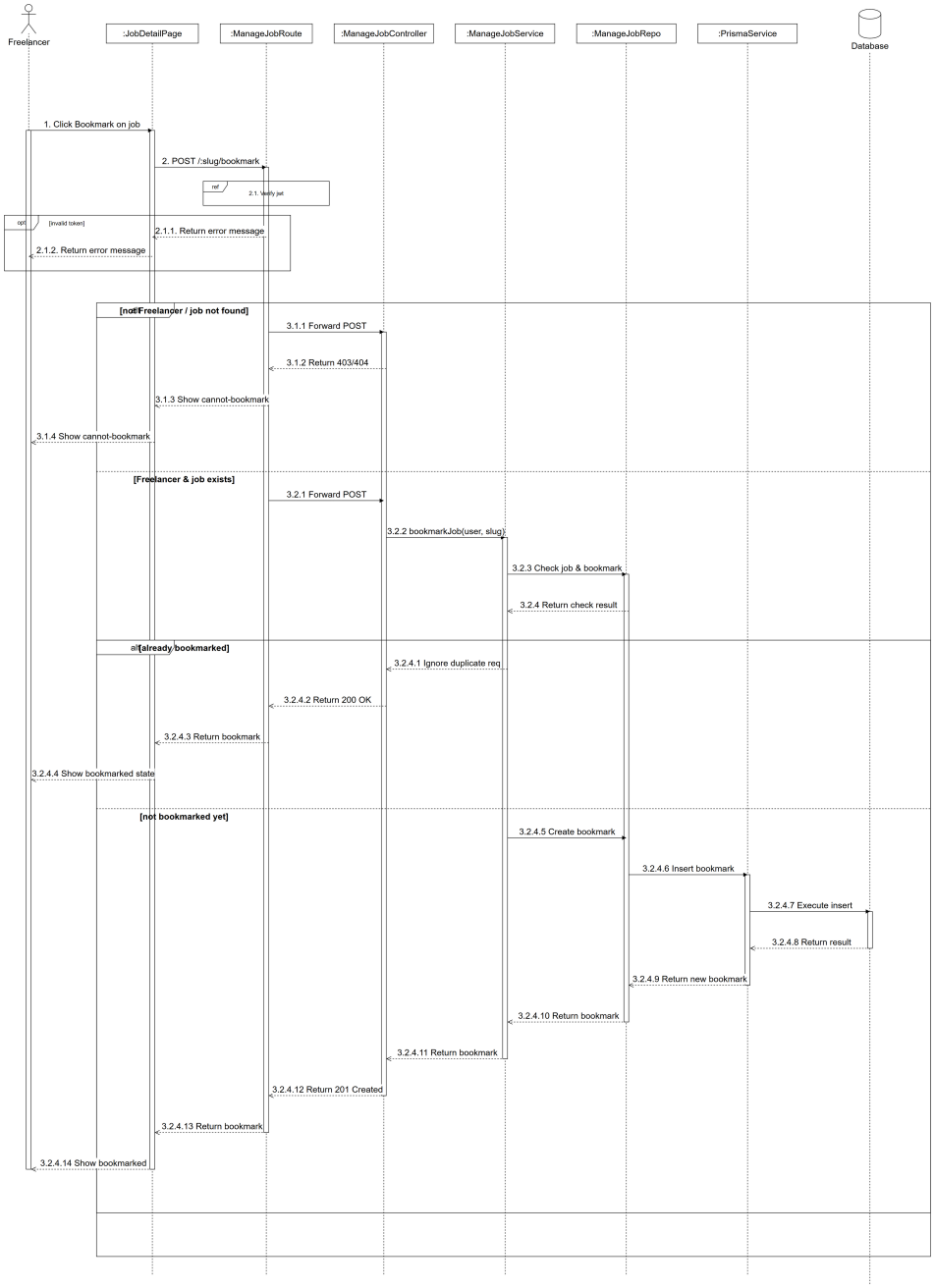

Figure 58: Sequence diagram of UC-12.02 Bookmark Job

3.4.4 UC-12.03 – Remove Bookmark

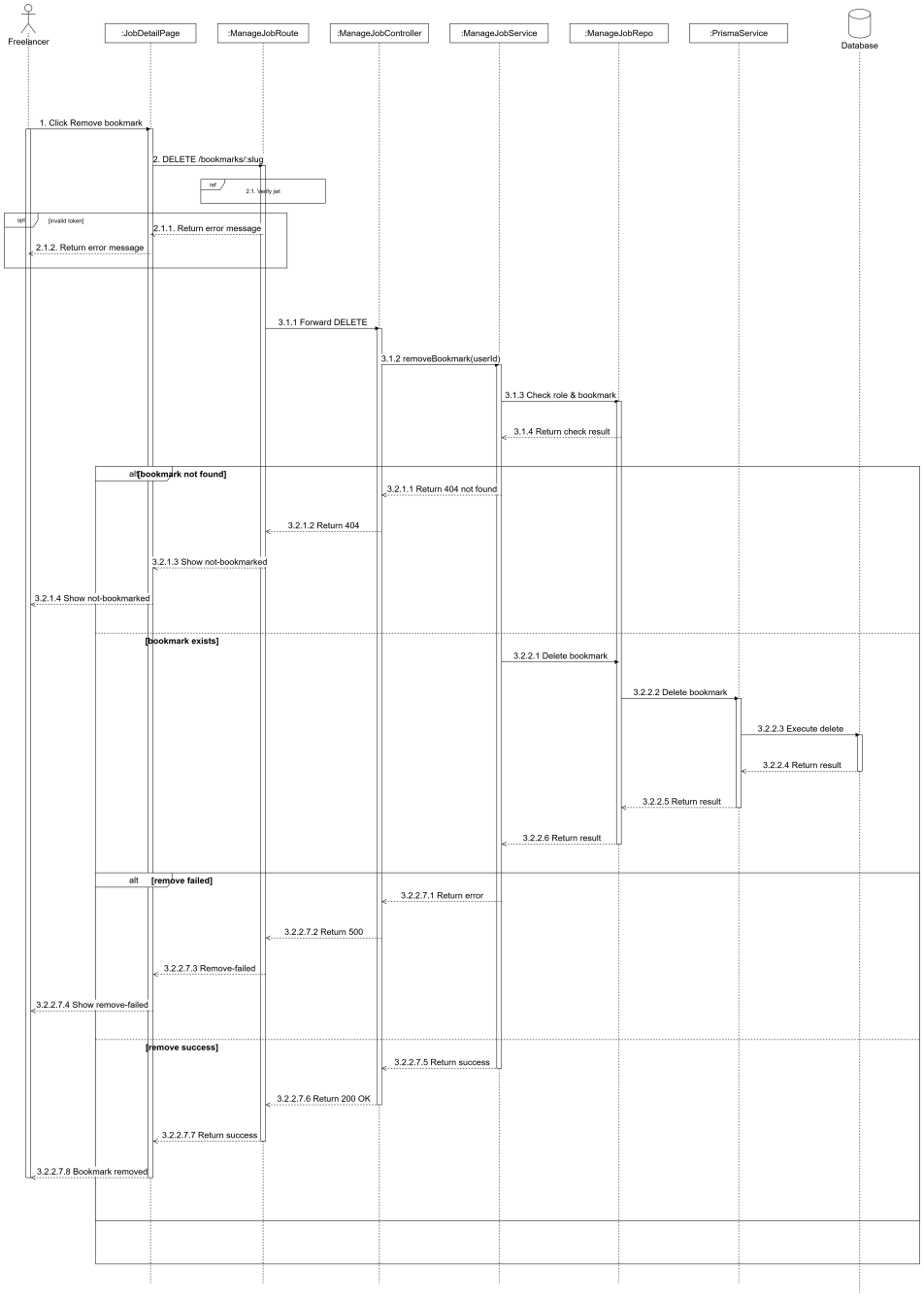

Figure 59: Sequence diagram of UC-12.03 Remove Bookmark

3.5 Verify JWT

Verify JWT is the shared security guard protecting API endpoints across all modules. Every incoming HTTP request with a Bearer token is intercepted, validated for cryptographic signature and expiration time, and verified against the database to guarantee the associated session remains active and has not been revoked.

3.5.1 Class Diagram

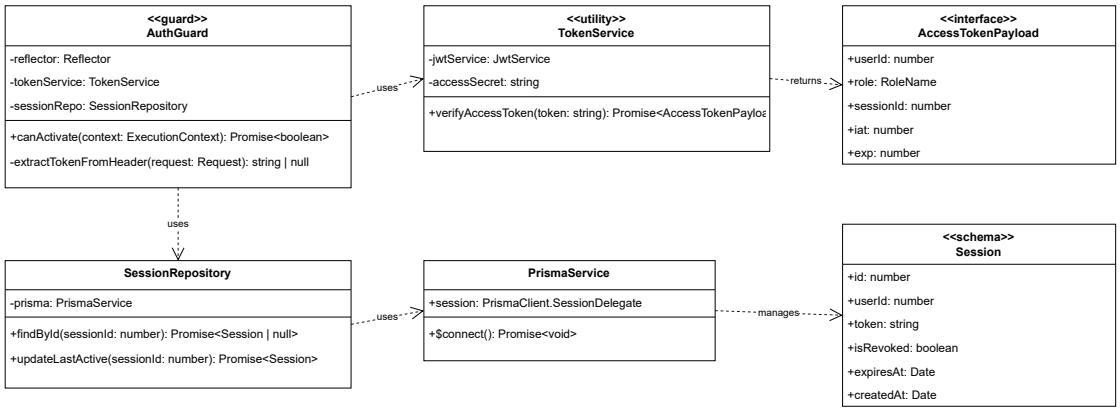

Figure 60: Class diagram of Verify JWT

3.5.2 UC-00 – Verify JWT

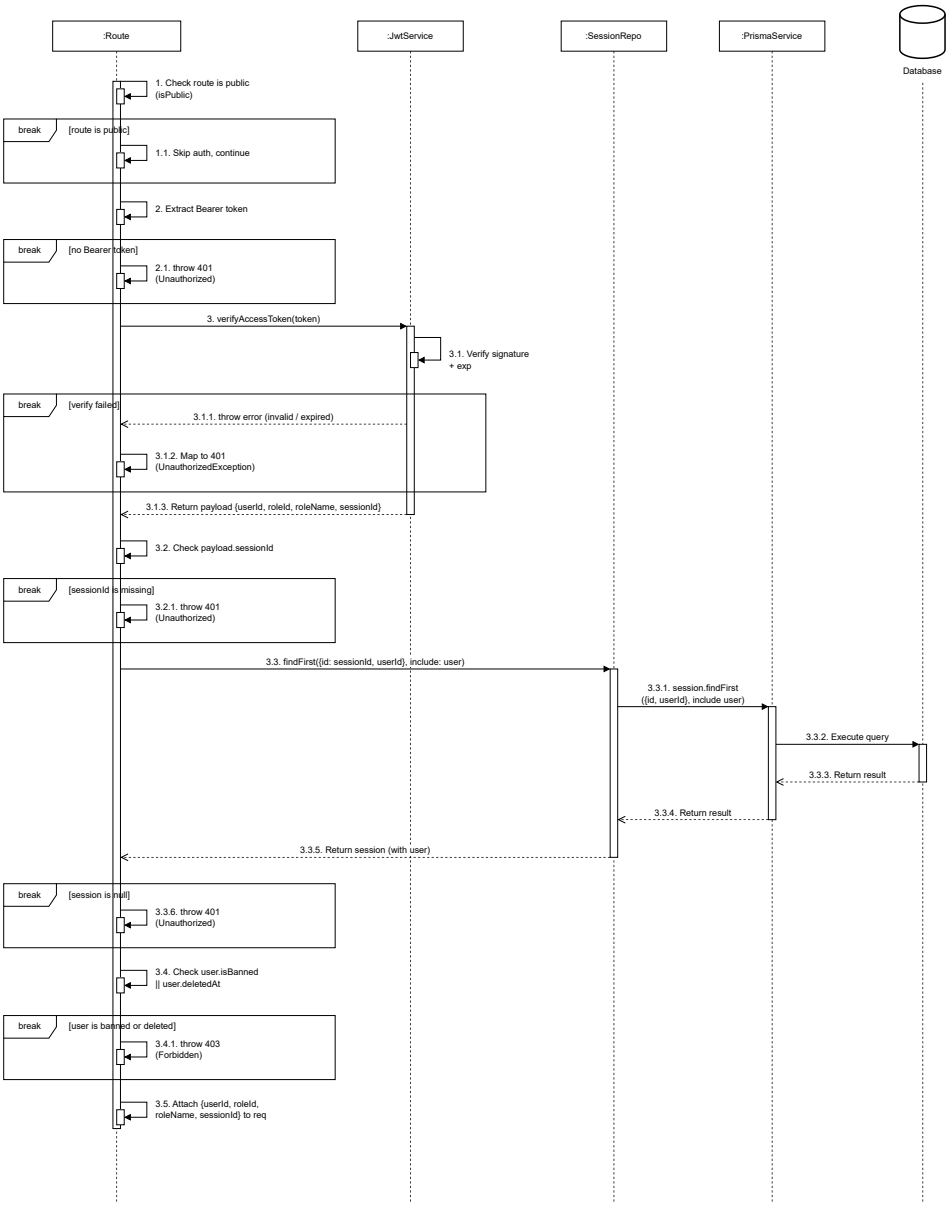

Figure 61: Sequence diagram of UC-00 Verify JWT

3.6 Authentication

The authentication module covers account registration, email/password and Google OAuth login, logout, password recovery, and the JWT/session flow shared by all protected endpoints.

3.6.1 Class Diagram

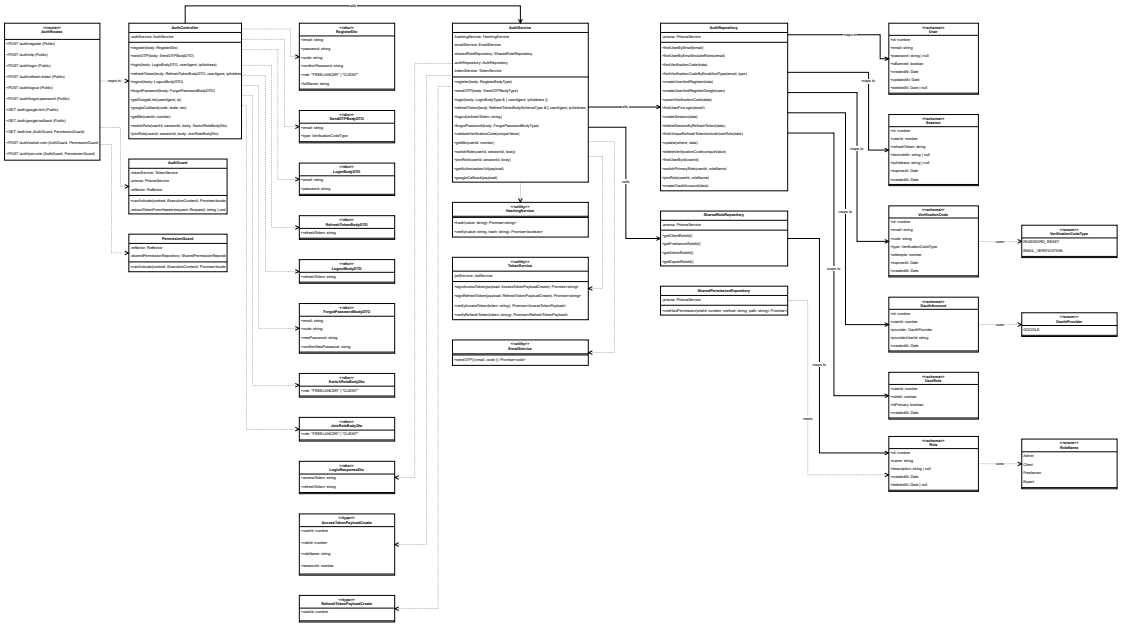

Figure 62: Class diagram of Authentication

3.6.2 UC-01 – Login with Google

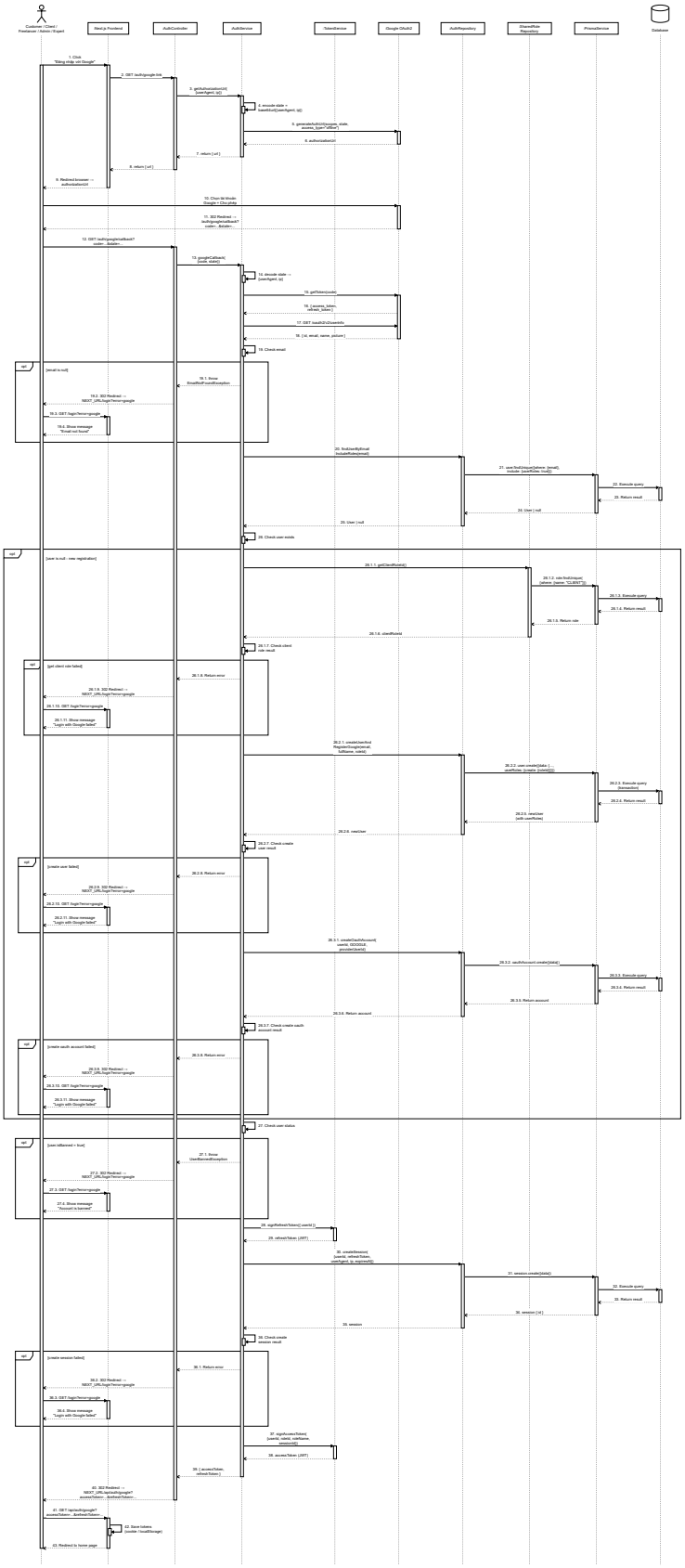

Figure 63: Sequence diagram of UC-01 Login with Google

3.6.3 UC-02 – Login

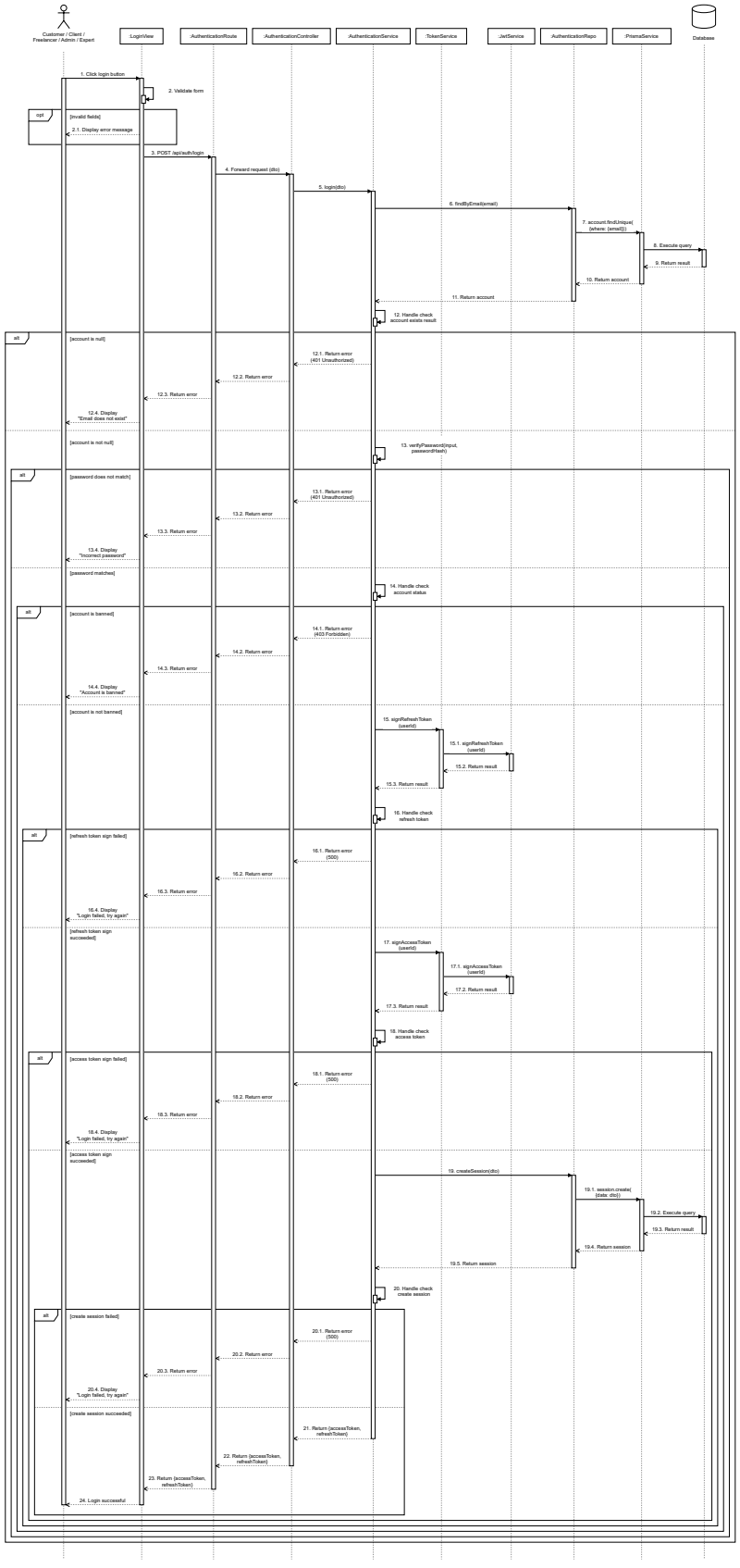

Figure 64: Sequence diagram of UC-02 Login

3.6.4 UC-03 – Logout

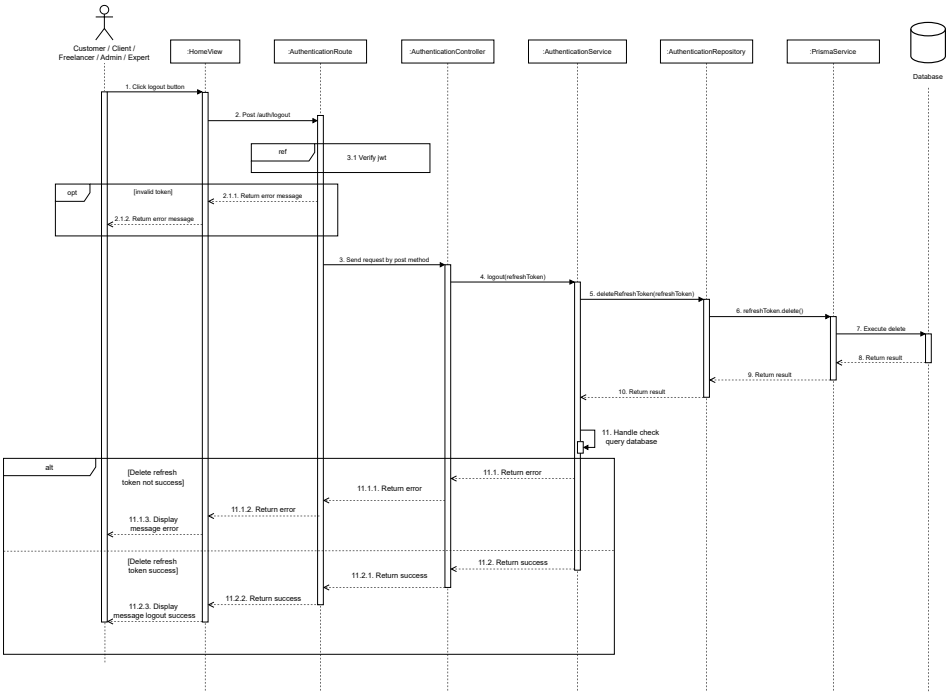

Figure 65: Sequence diagram of UC-03 Logout

3.6.5 UC-04 – Register

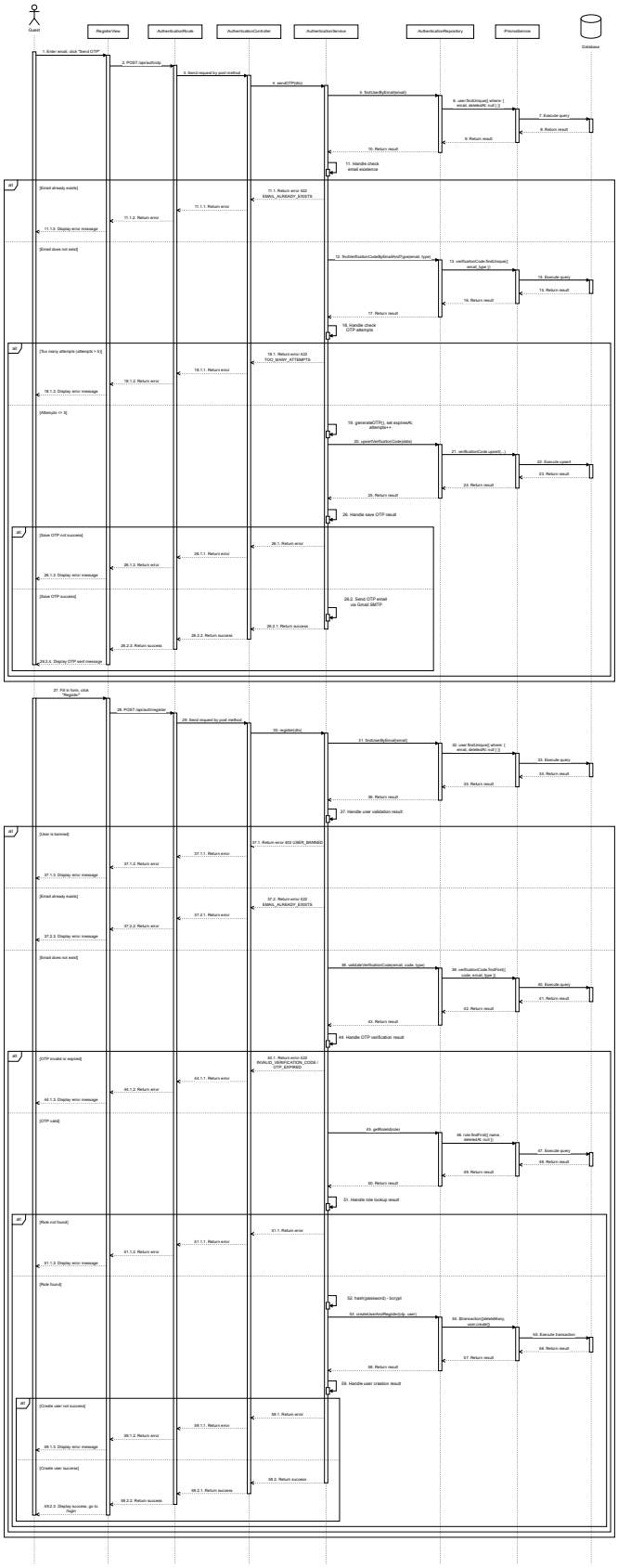

Figure 66: Sequence diagram of UC-04 Register

3.6.6 UC-05 – Forgot Password

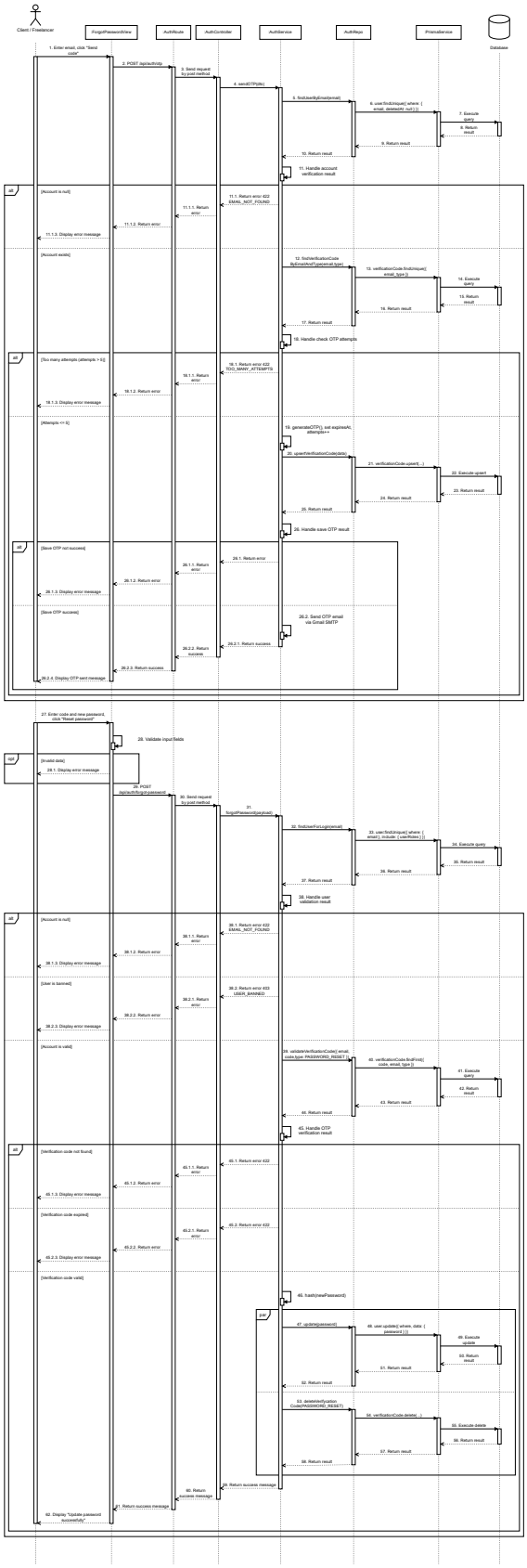

Figure 67: Sequence diagram of UC-05 Forgot Password

3.7 Role Management

Role Management lets an administrator inspect the role catalogue, maintain role metadata, and replace the permission set assigned to a role.

3.7.1 Class Diagram

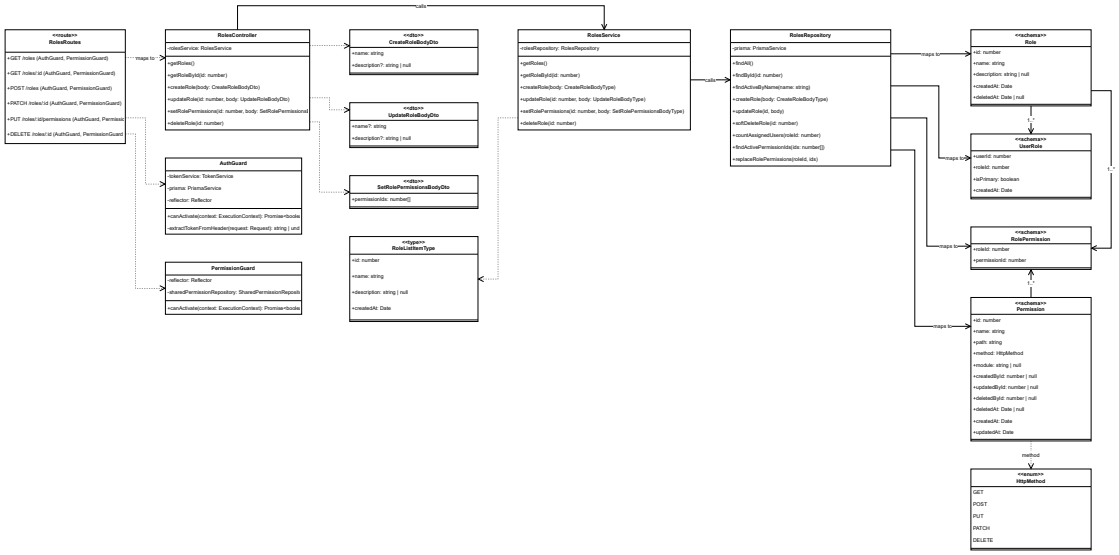

Figure 68: Class diagram of Role Management

3.7.2 UC-06.01 – View Role List

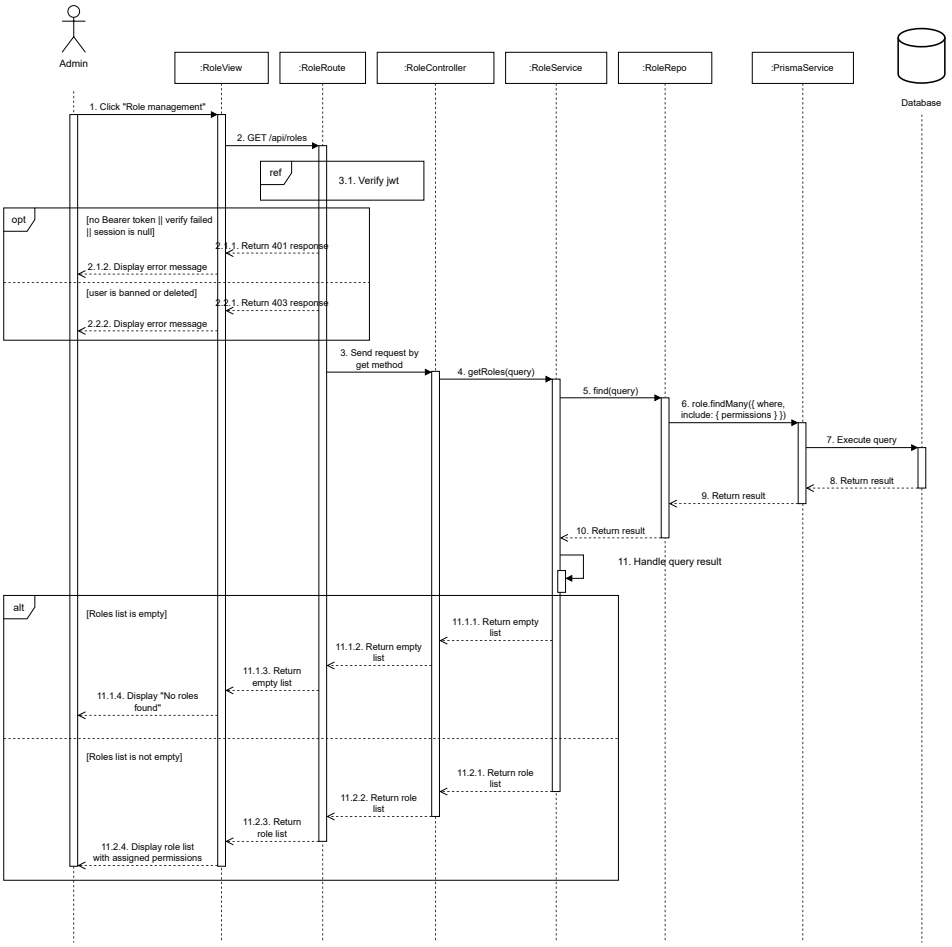

Figure 69: Sequence diagram of UC-06.01 View Role List

3.7.3 UC-06.02 – View Role Detail

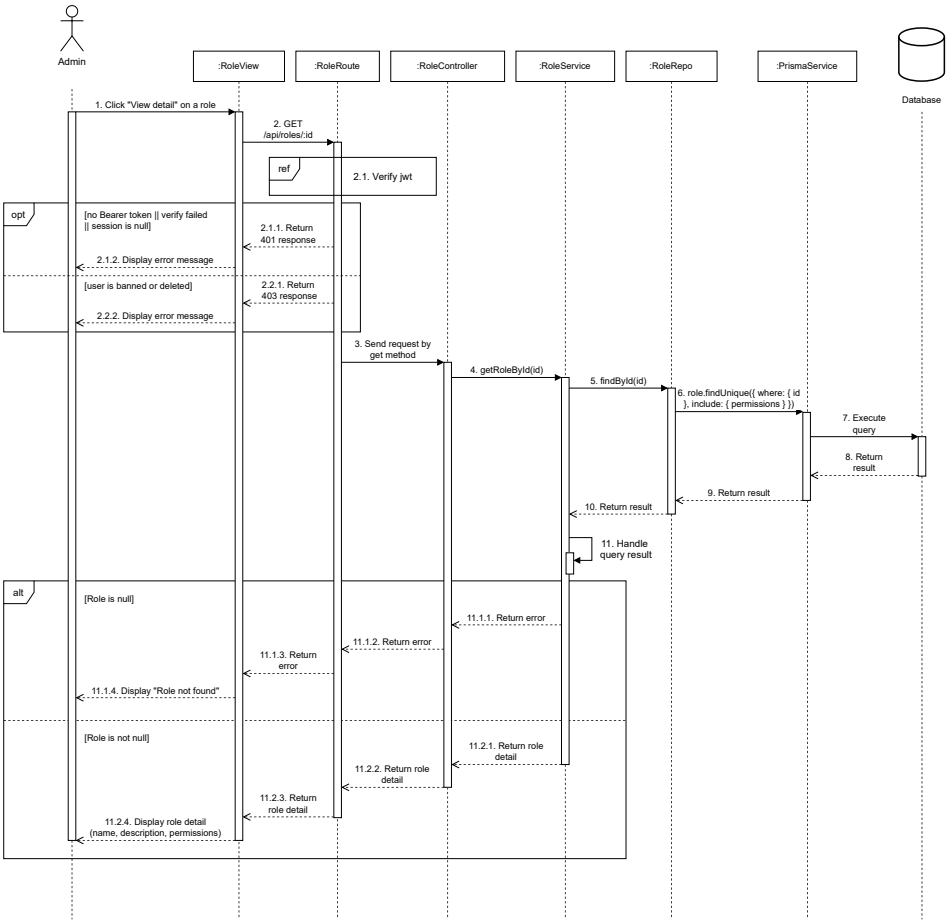

Figure 70: Sequence diagram of UC-06.02 View Role Detail

3.7.4 UC-06.03 – Create Role

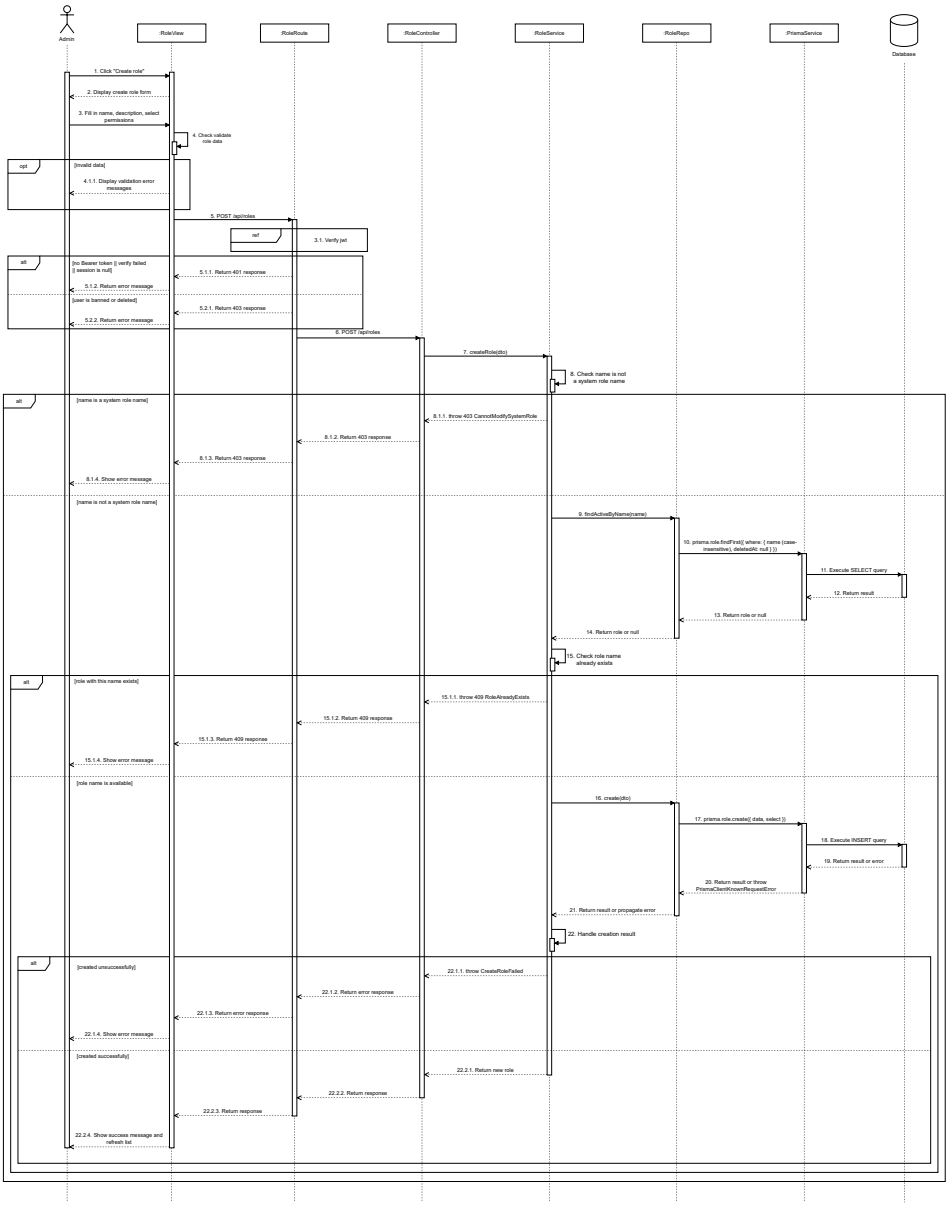

Figure 71: Sequence diagram of UC-06.03 Create Role

3.7.5 UC-06.04 – Update Role

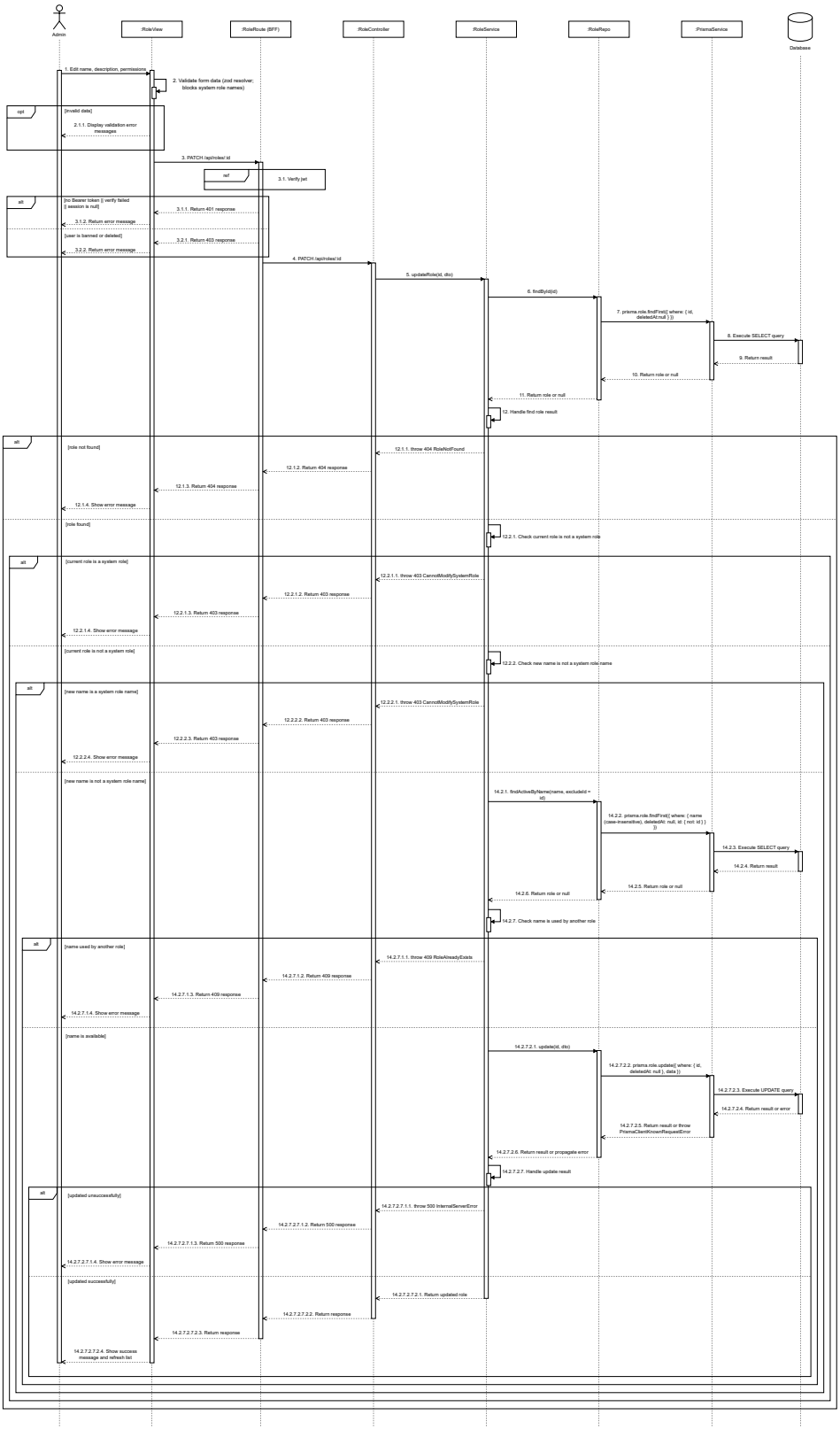

Figure 72: Sequence diagram of UC-06.04 Update Role

3.7.6 UC-06.05 – Delete Role

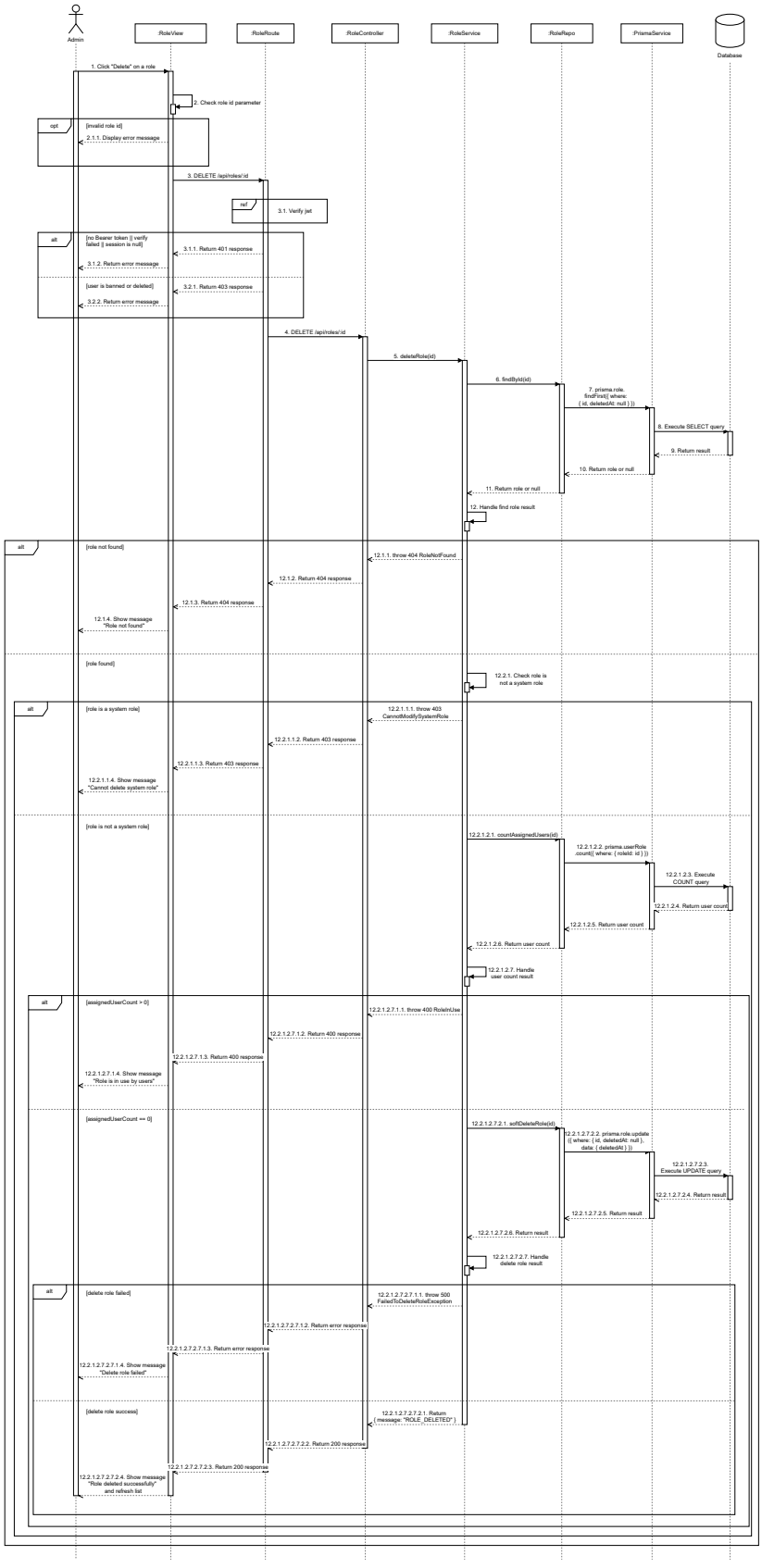

Figure 73: Sequence diagram of UC-06.05 Delete Role

3.7.7 UC-06.06 – Update Role Permissions

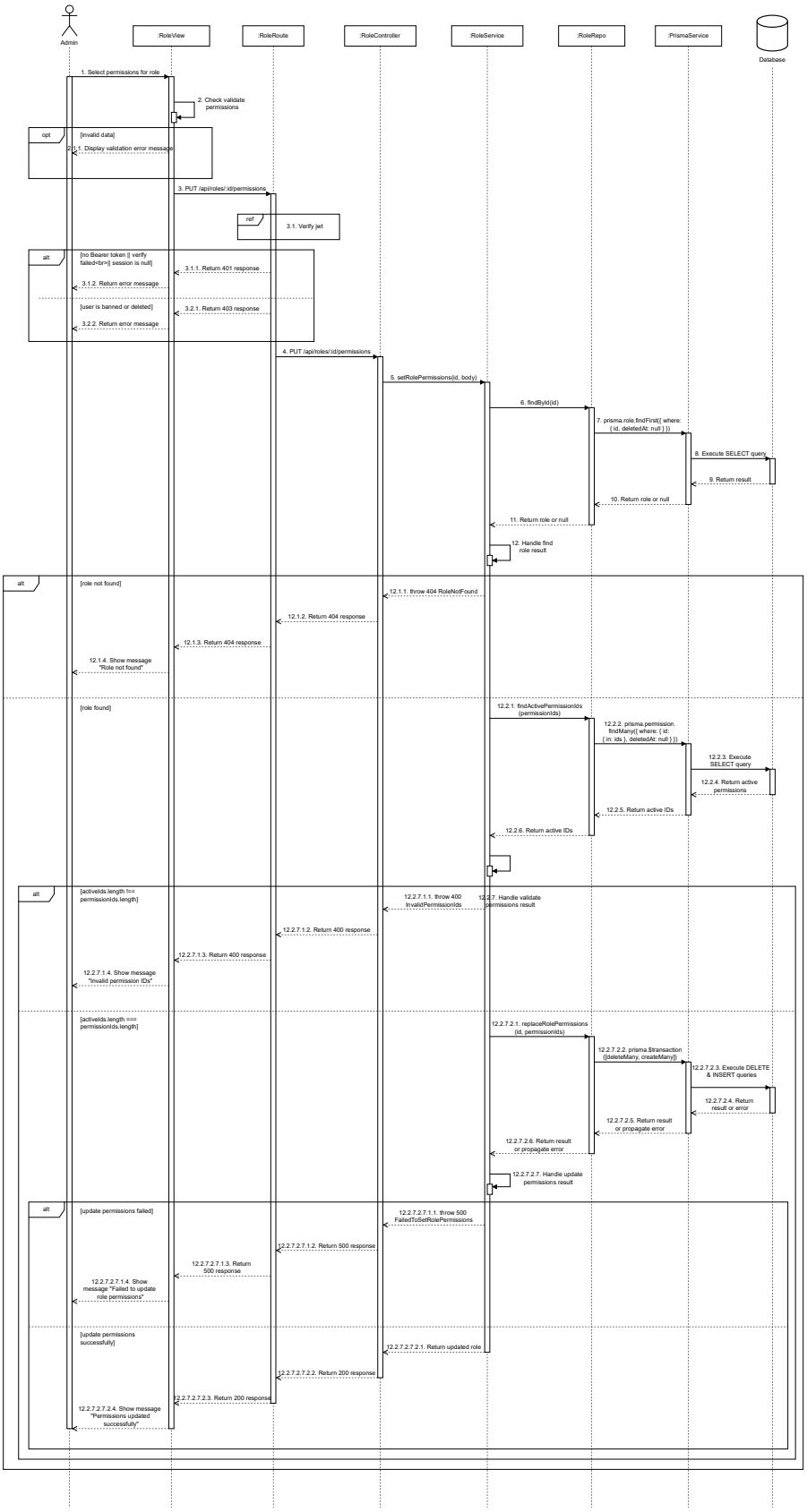

Figure 74: Sequence diagram of UC-06.06 Update Role Permissions

3.8 Manage Conversation

Manage Conversation covers the UC-27 private messaging module: listing and starting one-to-one conversations, opening a conversation, sending text messages and files, marking messages as read, recalling a message, and pinning or unpinning a conversation. Delivery is served both by the REST endpoints and by the realtime gateway in the same namespace.

3.8.1 Class Diagram

Figure 75: Class diagram of Manage Conversation

3.8.2 UC-27.01 – View Conversation List

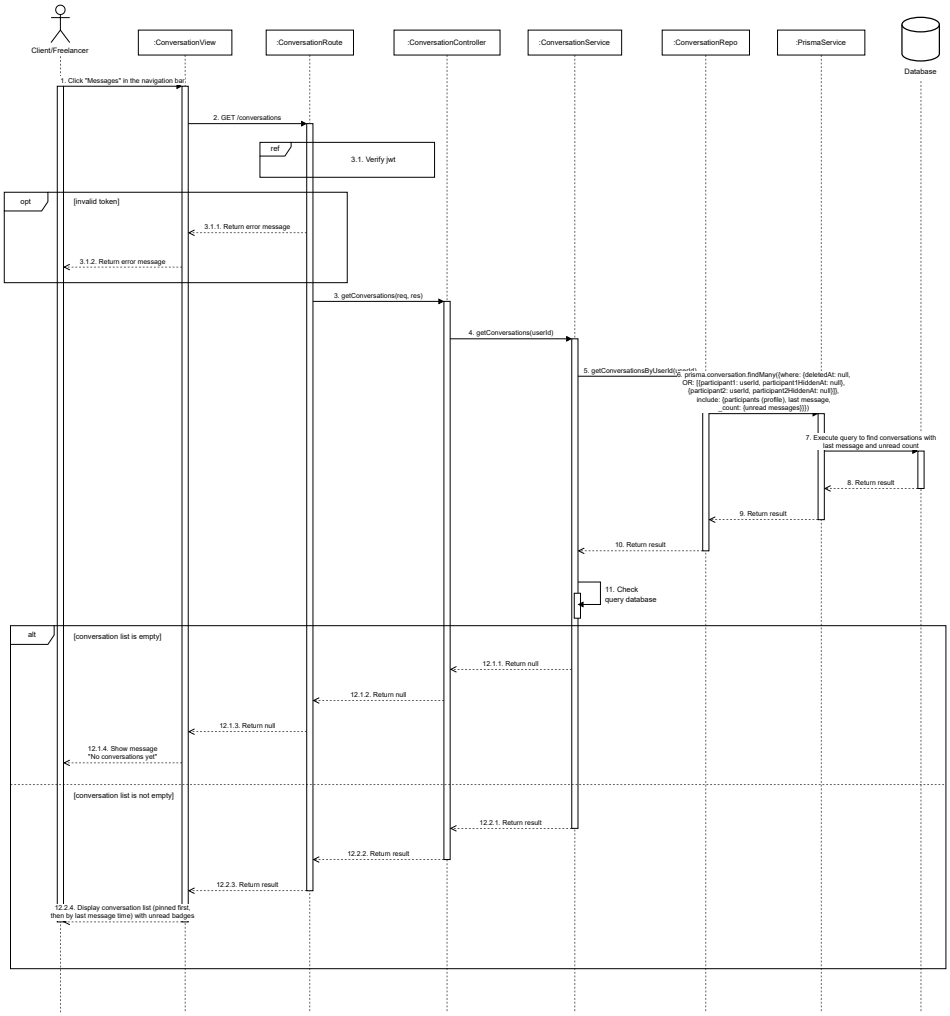

Figure 76: Sequence diagram of UC-27.01 View Conversation List

3.8.3 UC-27.02 – Start Conversation

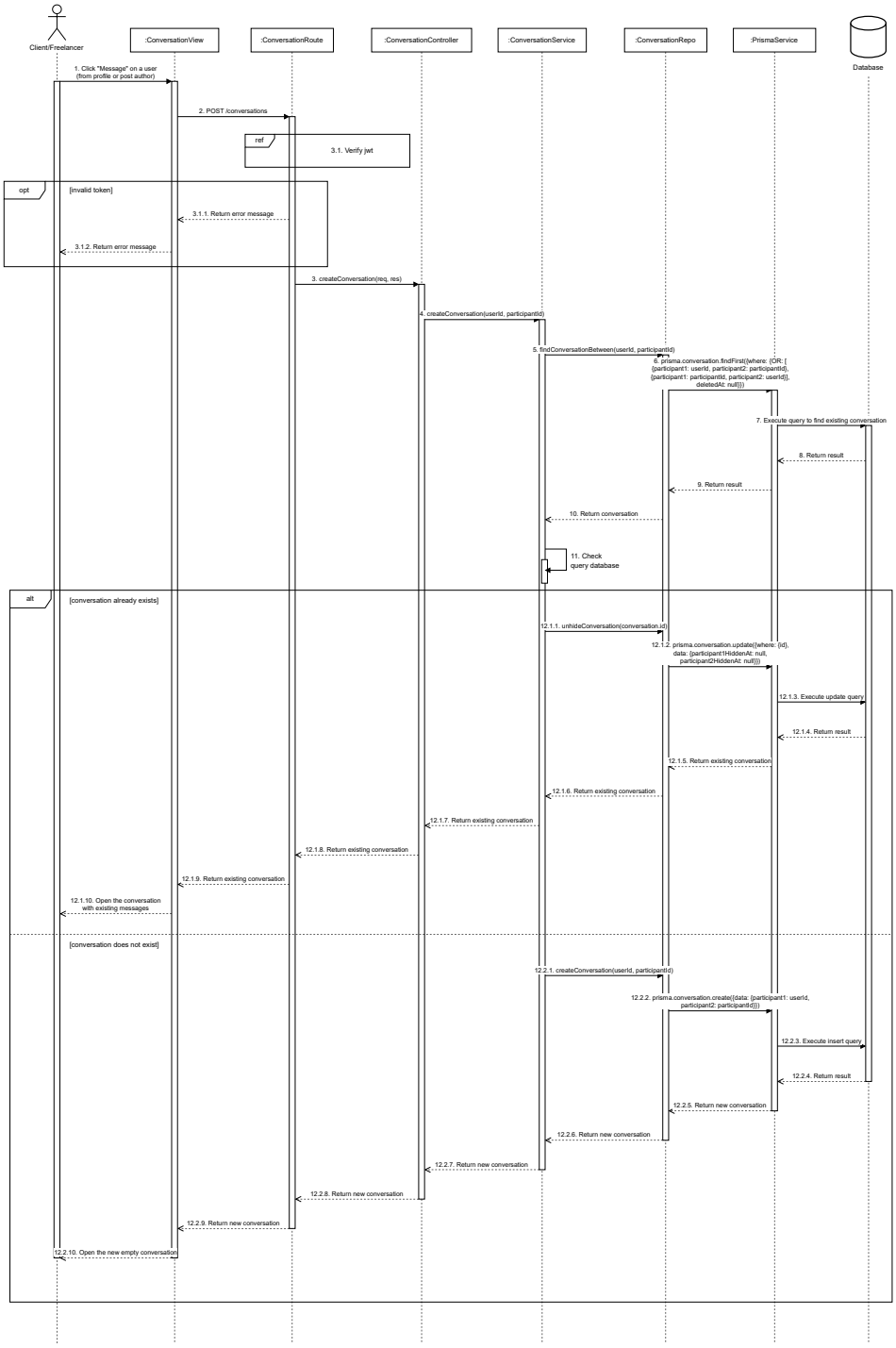

Figure 77: Sequence diagram of UC-27.02 Start Conversation

3.8.4 UC-27.03 – View Conversation Detail

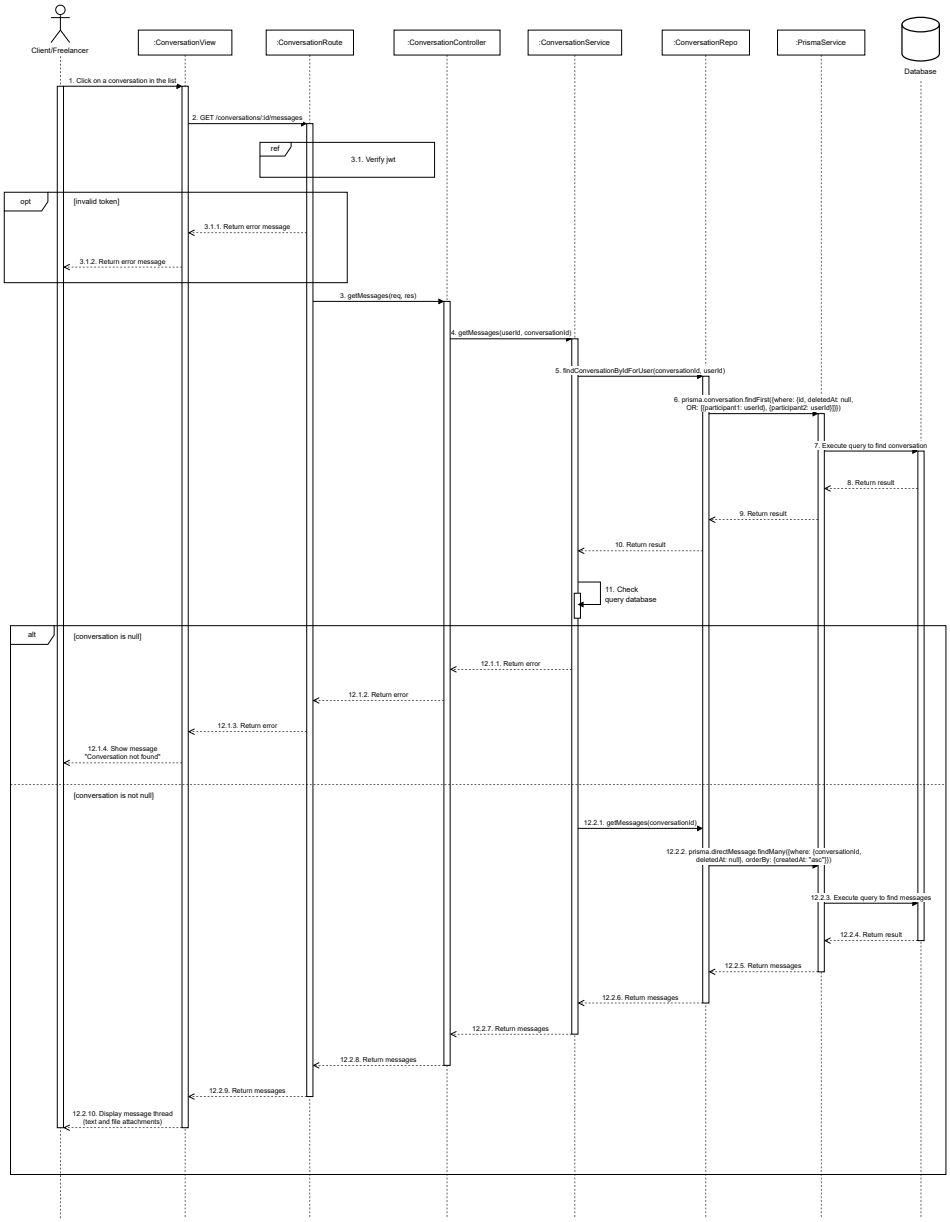

Figure 78: Sequence diagram of UC-27.03 View Conversation Detail

3.8.5 UC-27.04 – Send Message

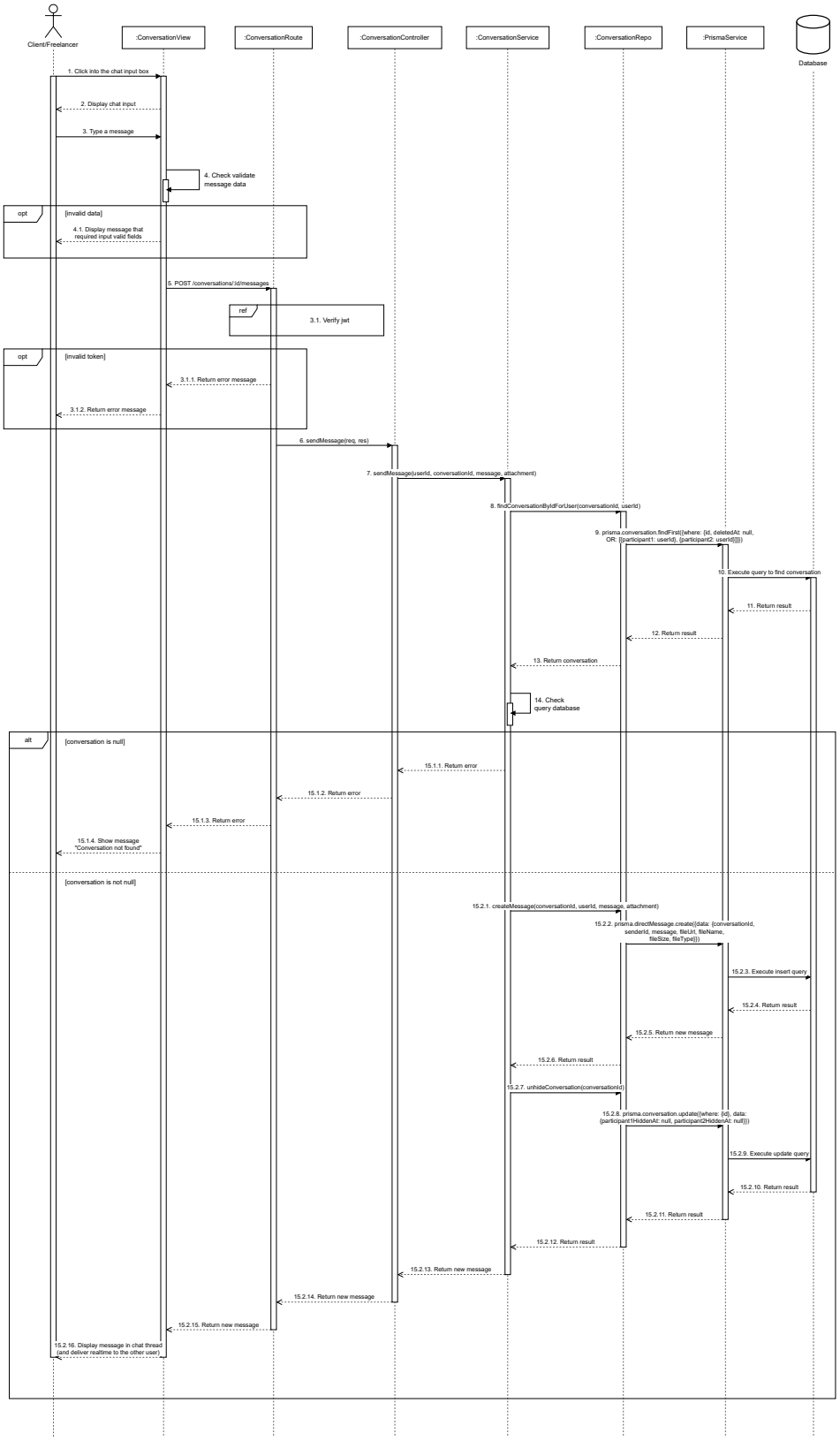

Figure 79: Sequence diagram of UC-27.04 Send Message

3.8.6 UC-27.05 – Mark Message as Read

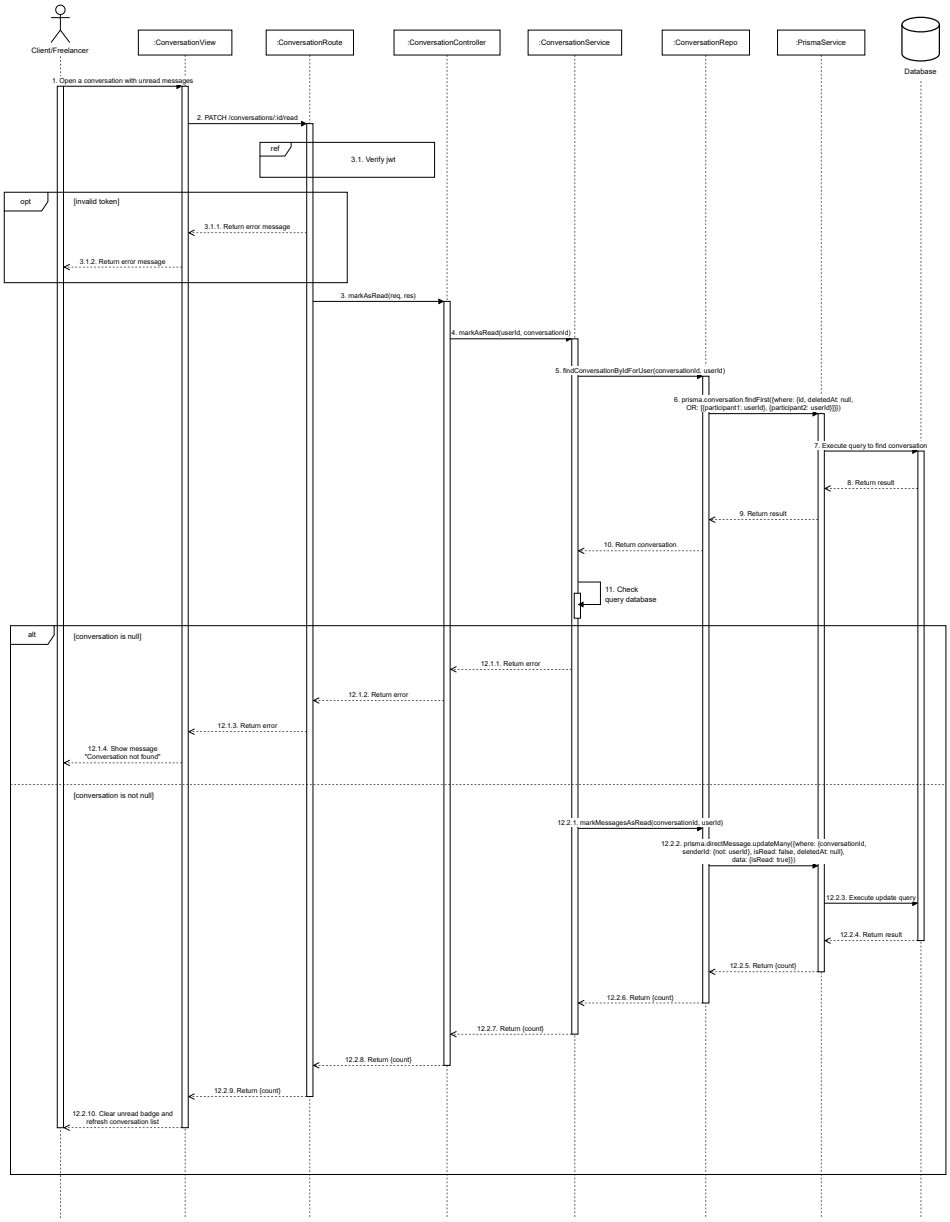

Figure 80: Sequence diagram of UC-27.05 Mark Message as Read

3.8.7 UC-27.06 – Delete/Recall Message

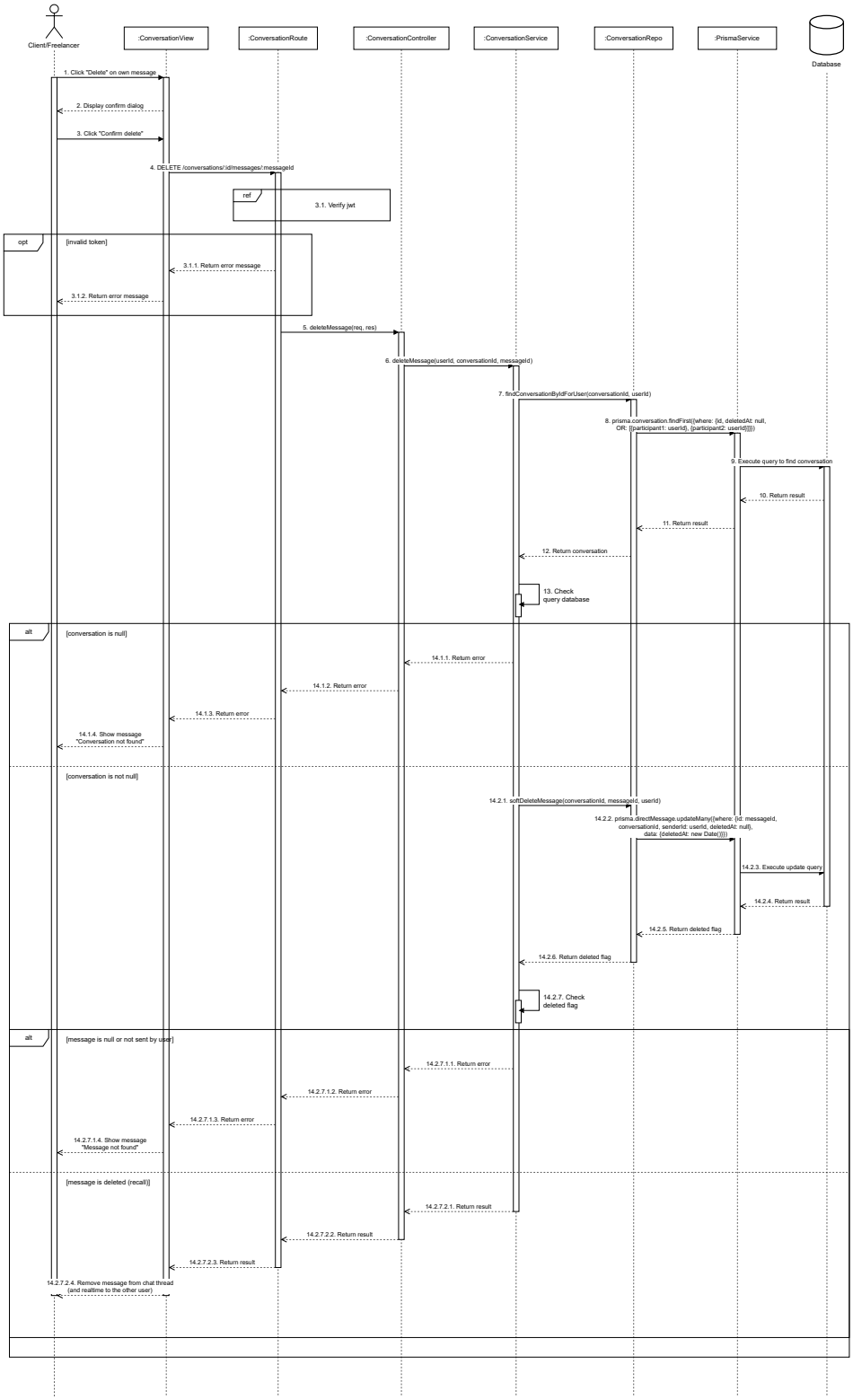

Figure 81: Sequence diagram of UC-27.06 Delete/Recall Message

3.8.8 UC-27.07 – Pin/Unpin Conversation

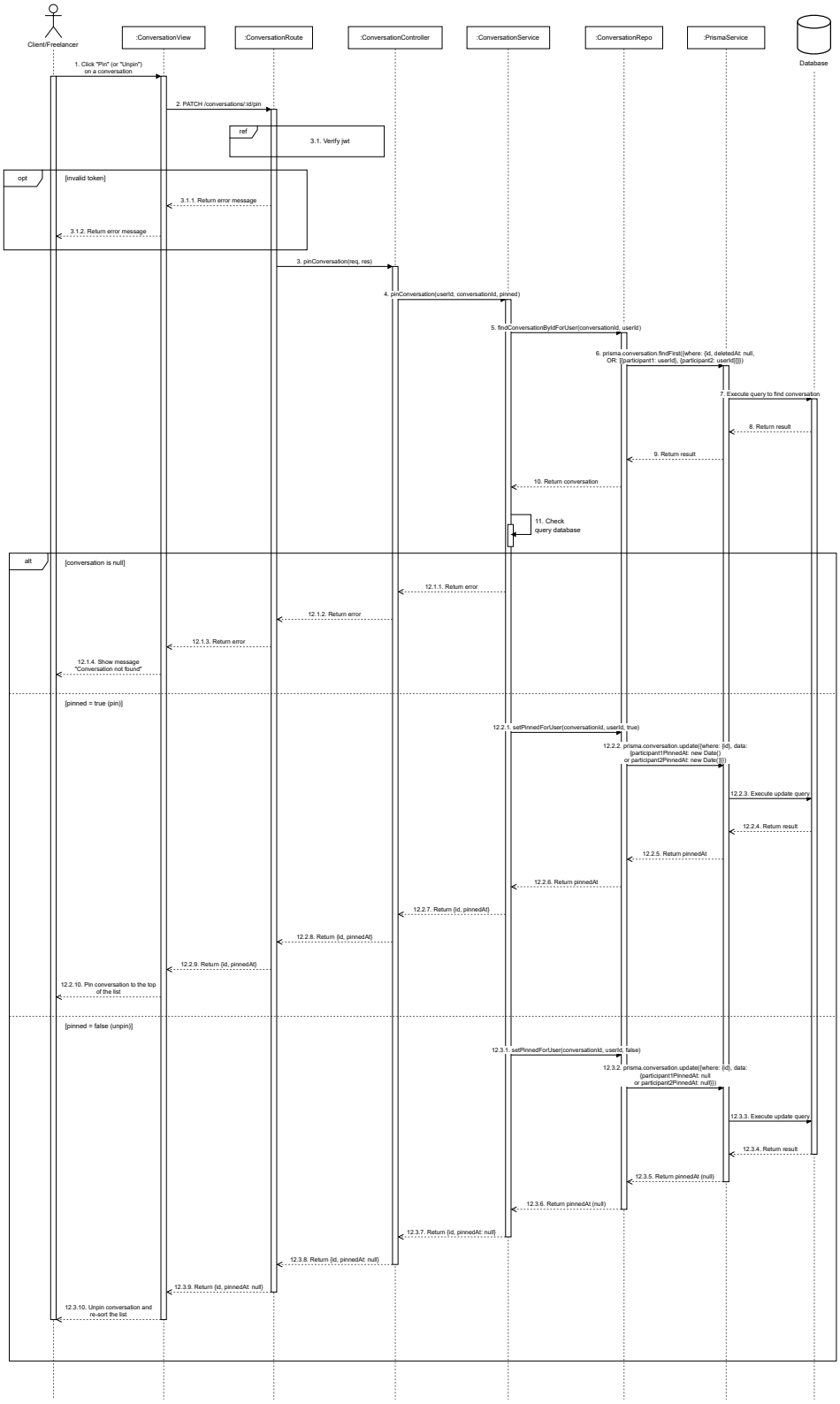

Figure 82: Sequence diagram of UC-27.07 Pin/Unpin Conversation

3.8.9 UC-27.08 – Send File

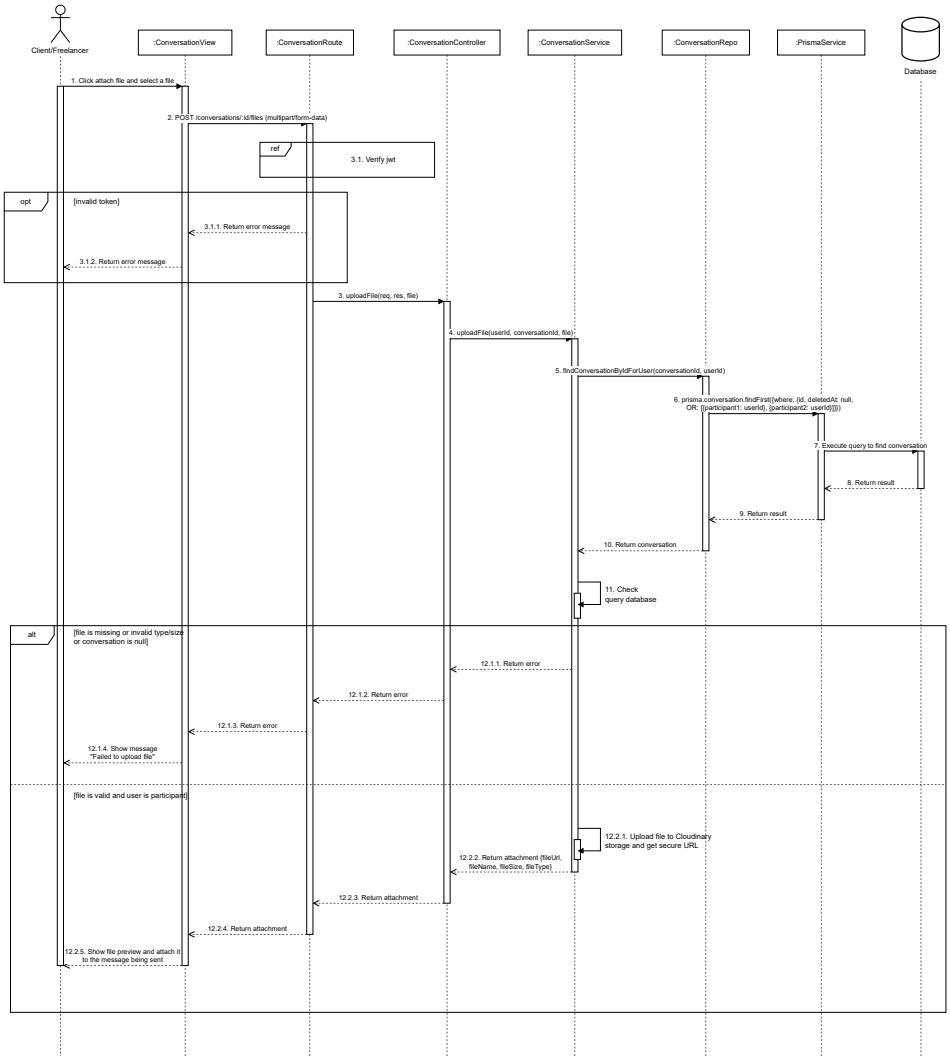

Figure 83: Sequence diagram of UC-27.08 Send File

3.9 Saved Search Management

Saved Search Management allows freelancers to save complex job filter criteria, view their list of saved searches, inspect search details, update filter parameters, and remove obsolete searches.

3.9.1 Class Diagram

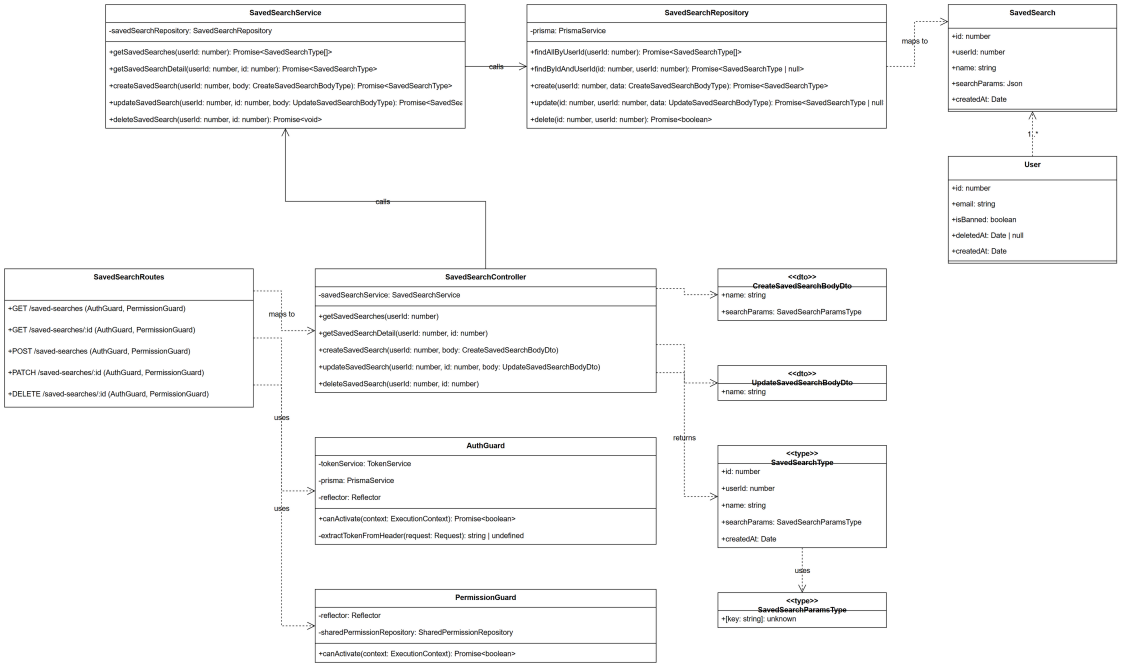

Figure 84: Class diagram of Saved Search Management

3.9.2 UC-22.01 – View Saved Search List

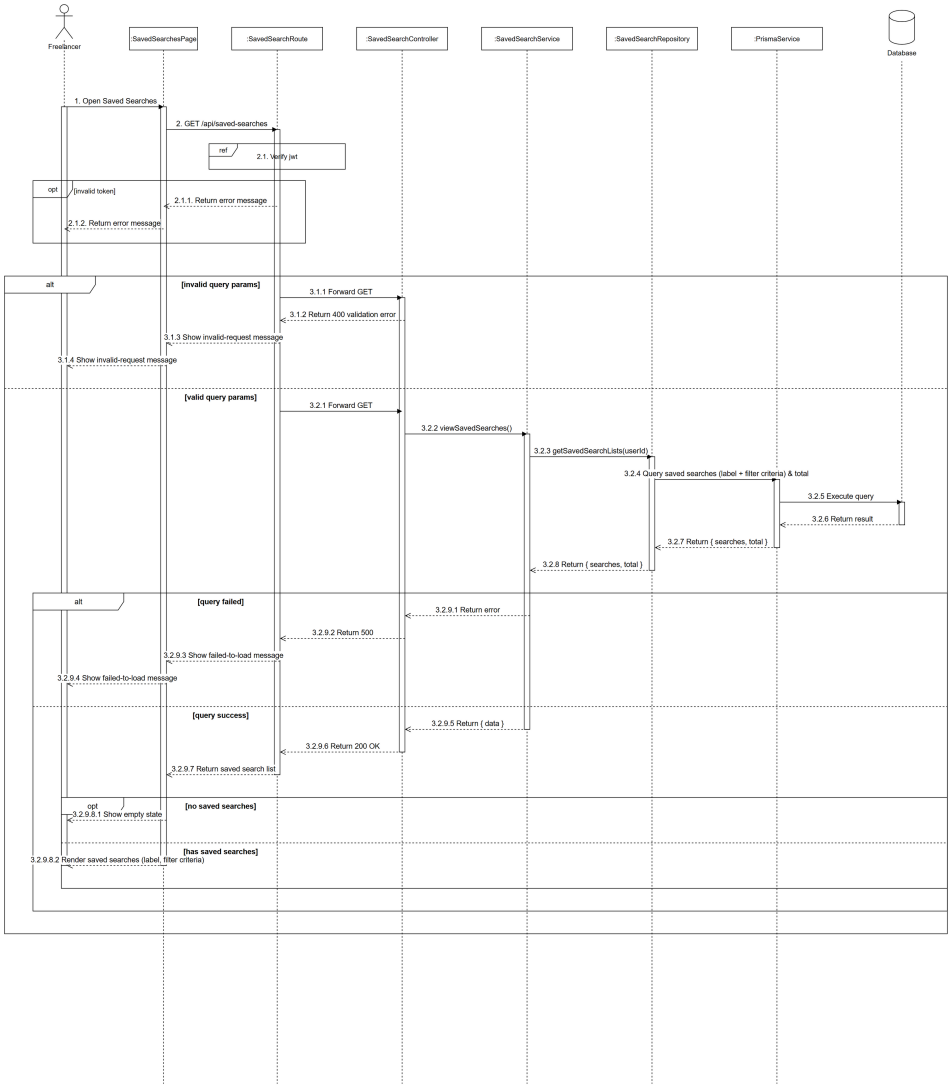

Figure 85: Sequence diagram of UC-22.01 View Saved Search List

3.9.3 UC-22.02 – View Saved Search Detail

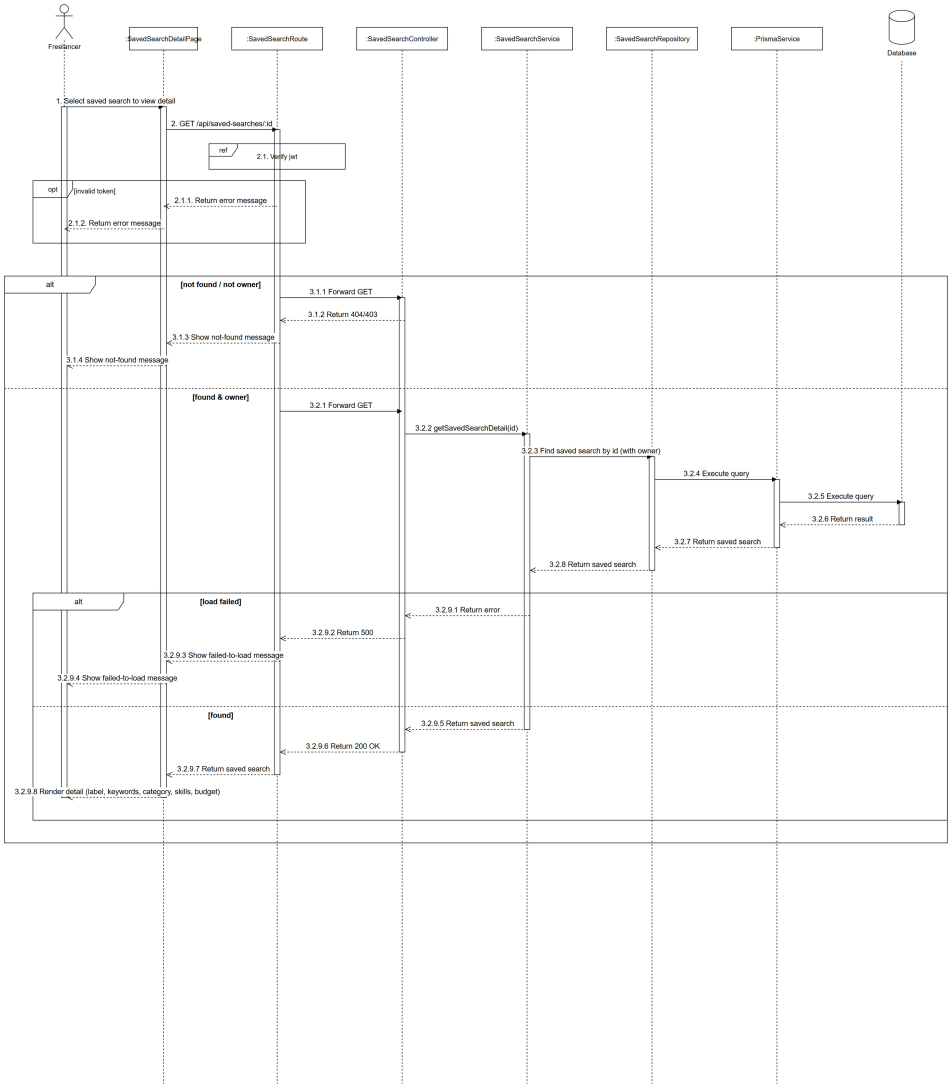

Figure 86: Sequence diagram of UC-22.02 View Saved Search Detail

3.9.4 UC-22.03 – Create Saved Search

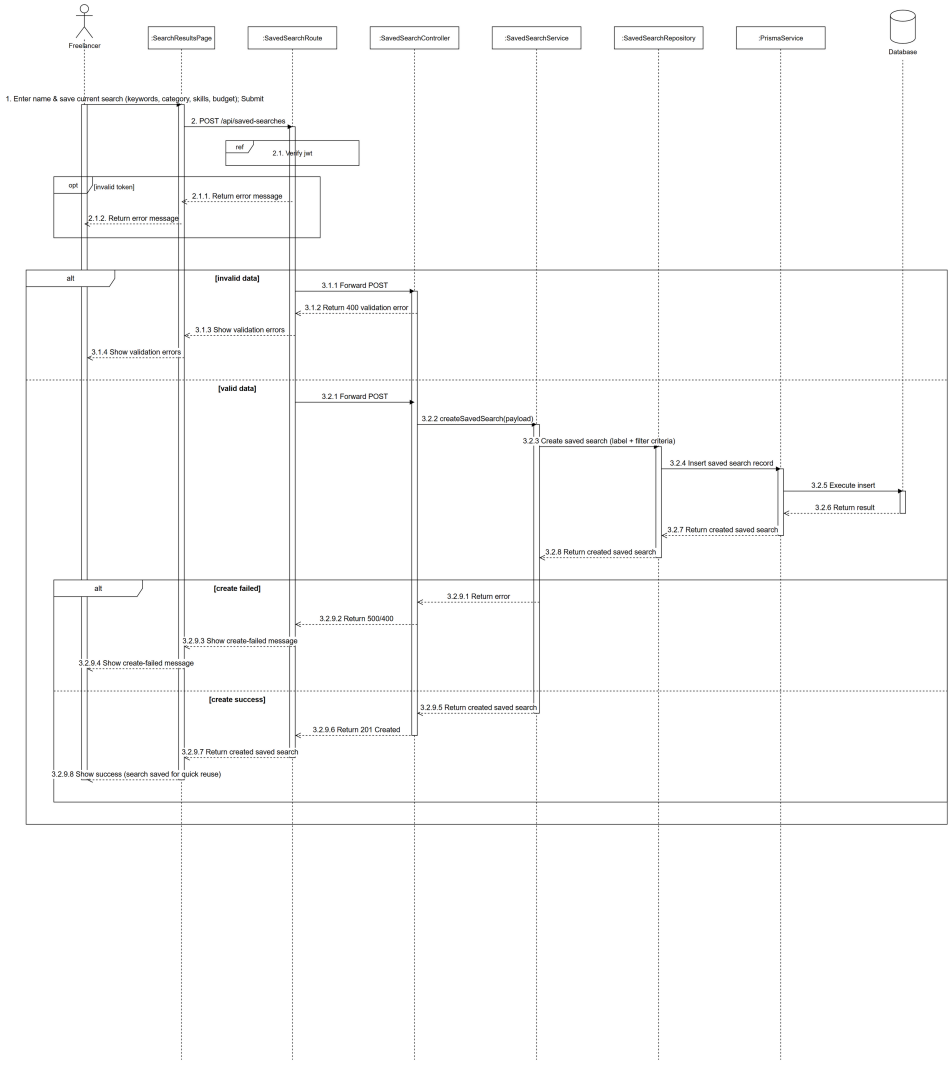

Figure 87: Sequence diagram of UC-22.03 Create Saved Search

3.9.5 UC-22.04 – Update Saved Search

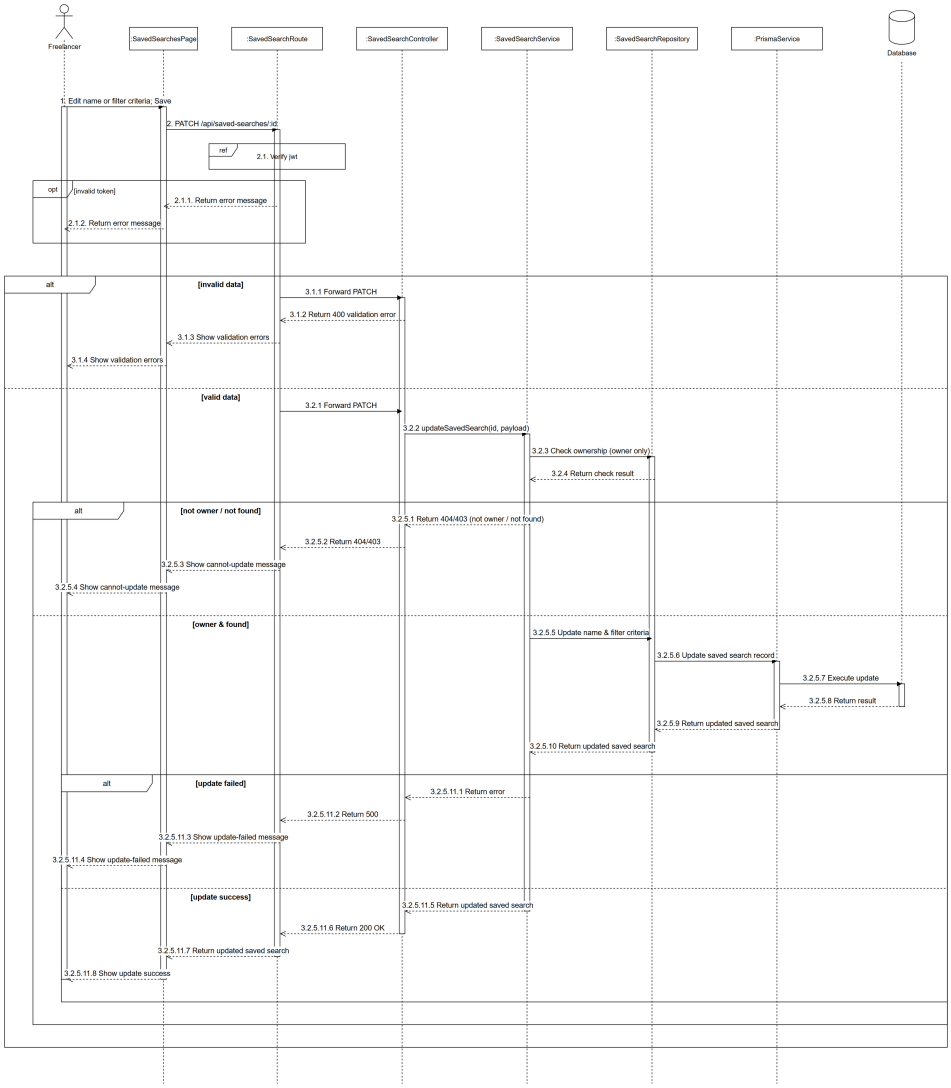

Figure 88: Sequence diagram of UC-22.04 Update Saved Search

3.9.6 UC-22.05 – Delete Saved Search

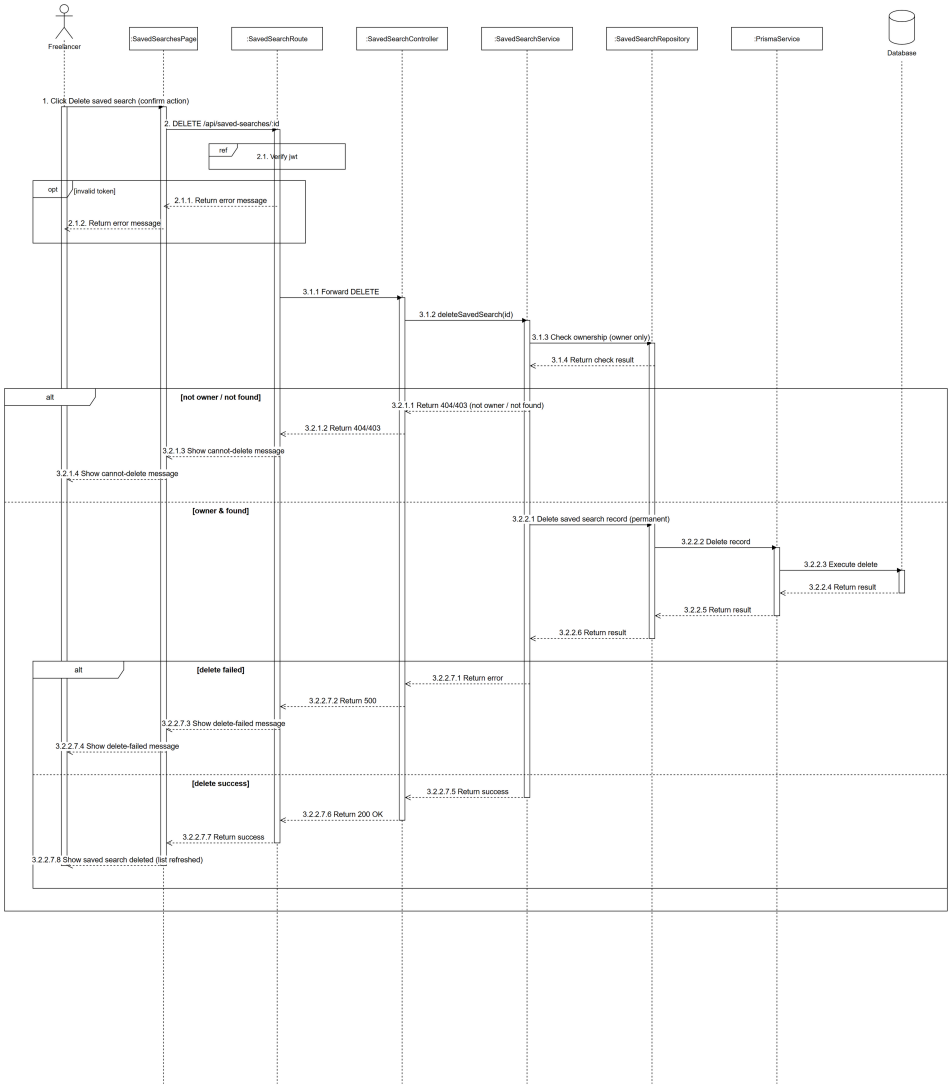

Figure 89: Sequence diagram of UC-22.05 Delete Saved Search

3.10 Job Alert Management

Job Alert Management enables freelancers to configure automated notifications based on job keywords, budget ranges, and category criteria, ensuring they receive prompt alerts when matching jobs are published.

3.10.1 Class Diagram

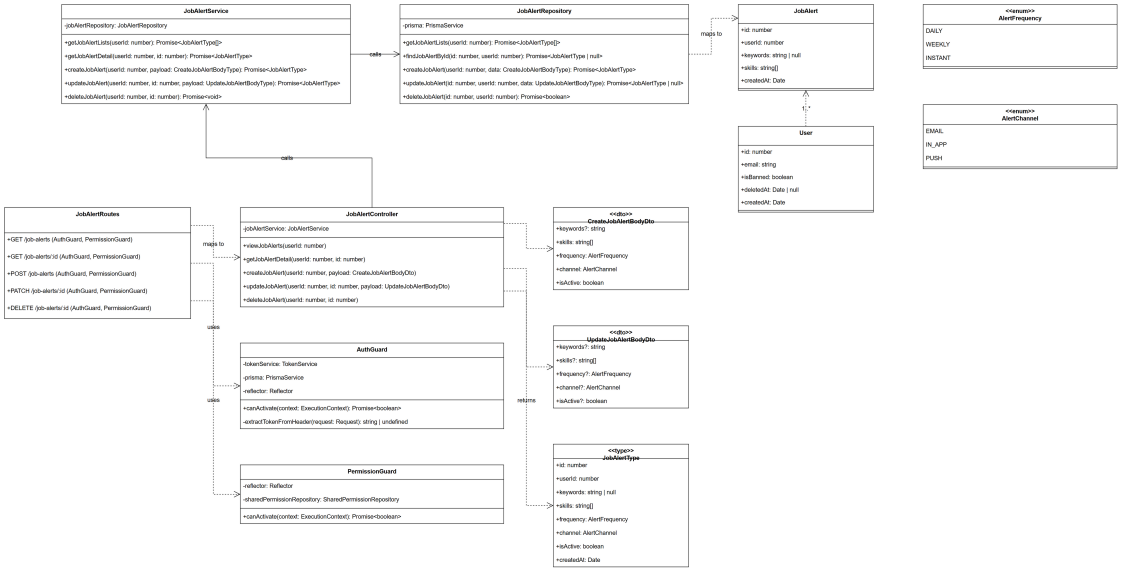

Figure 90: Class diagram of Job Alert Management

3.10.2 UC-23.01 – View Job Alert List

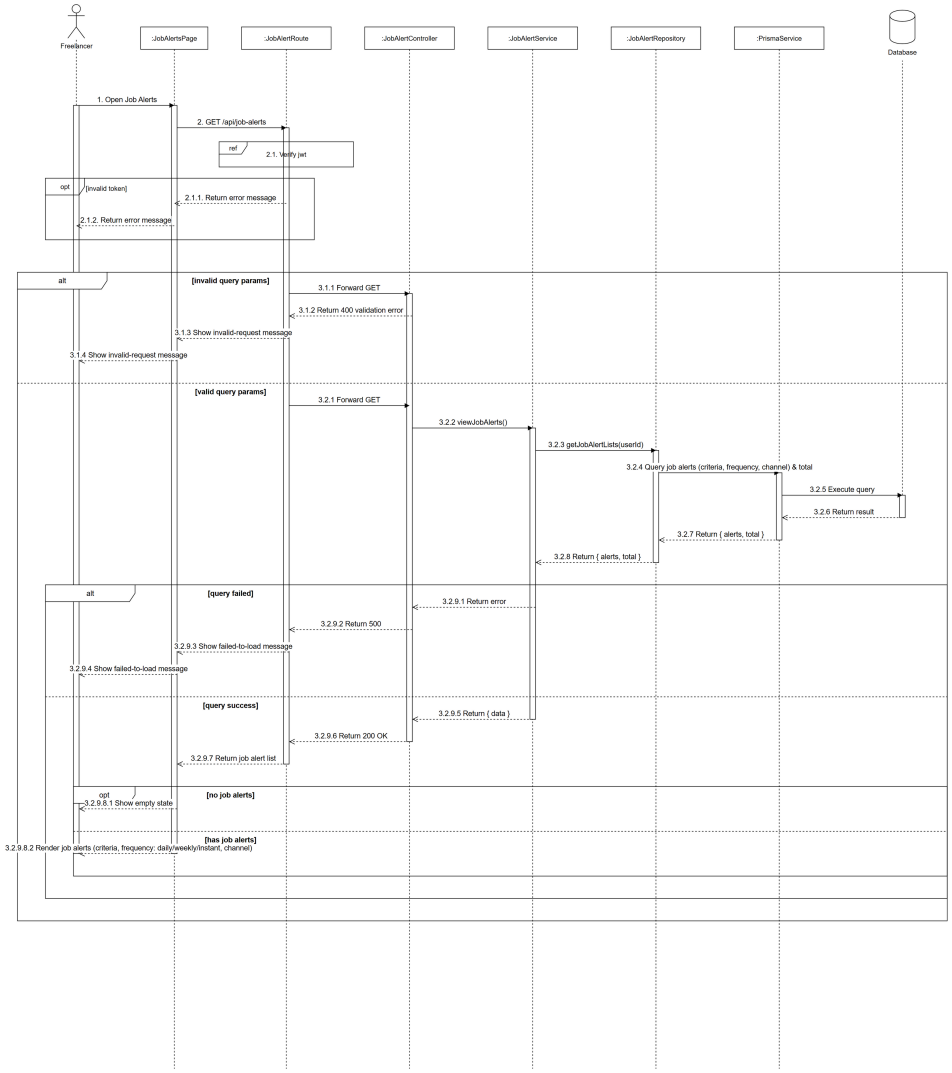

Figure 91: Sequence diagram of UC-23.01 View Job Alert List

3.10.3 UC-23.02 – View Job Alert Detail

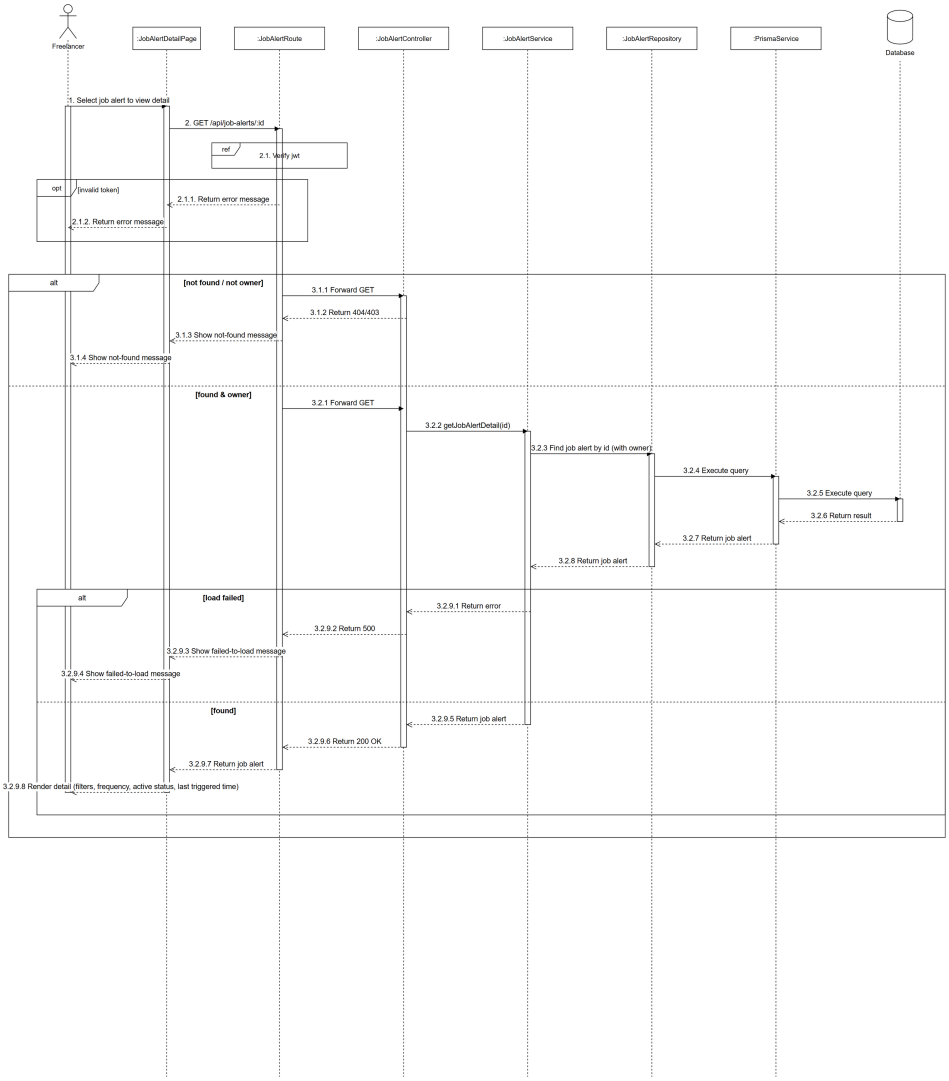

Figure 92: Sequence diagram of UC-23.02 View Job Alert Detail

3.10.4 UC-23.03 – Create Job Alert

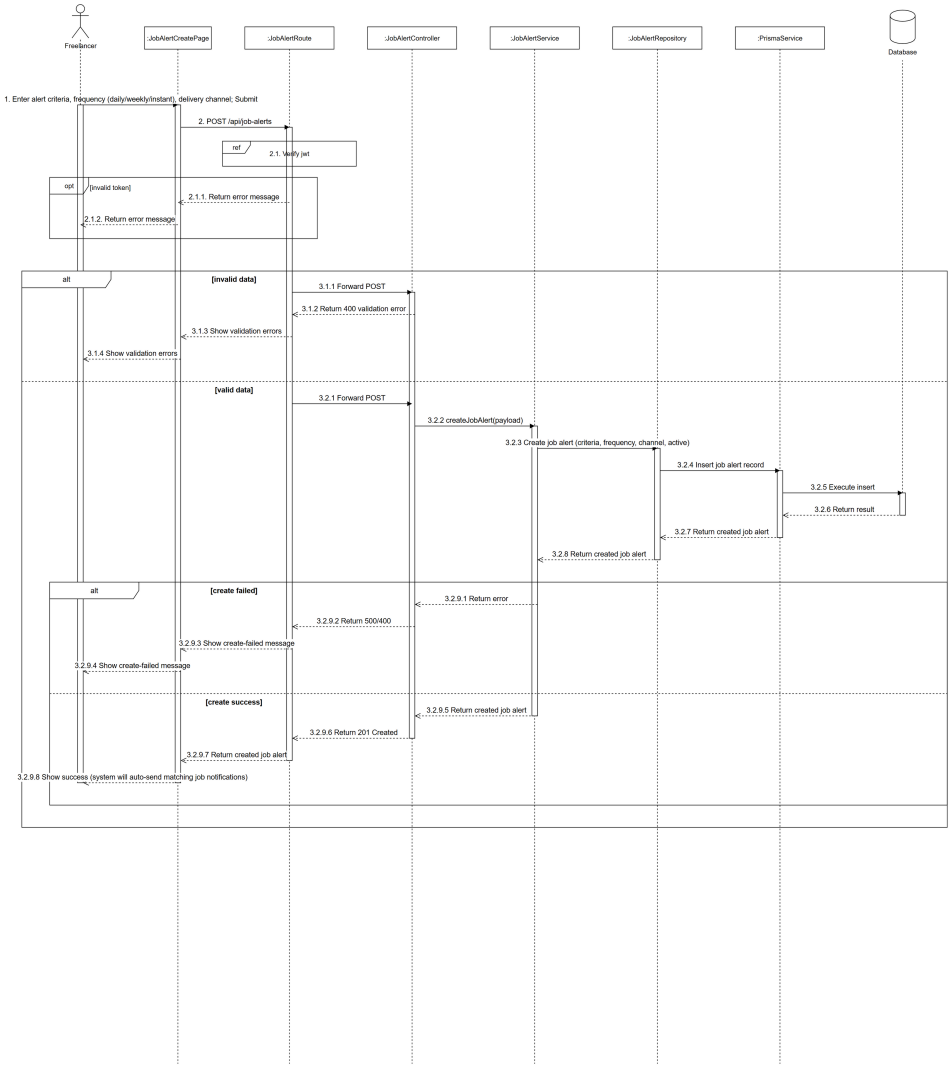

Figure 93: Sequence diagram of UC-23.03 Create Job Alert

3.10.5 UC-23.04 – Update Job Alert

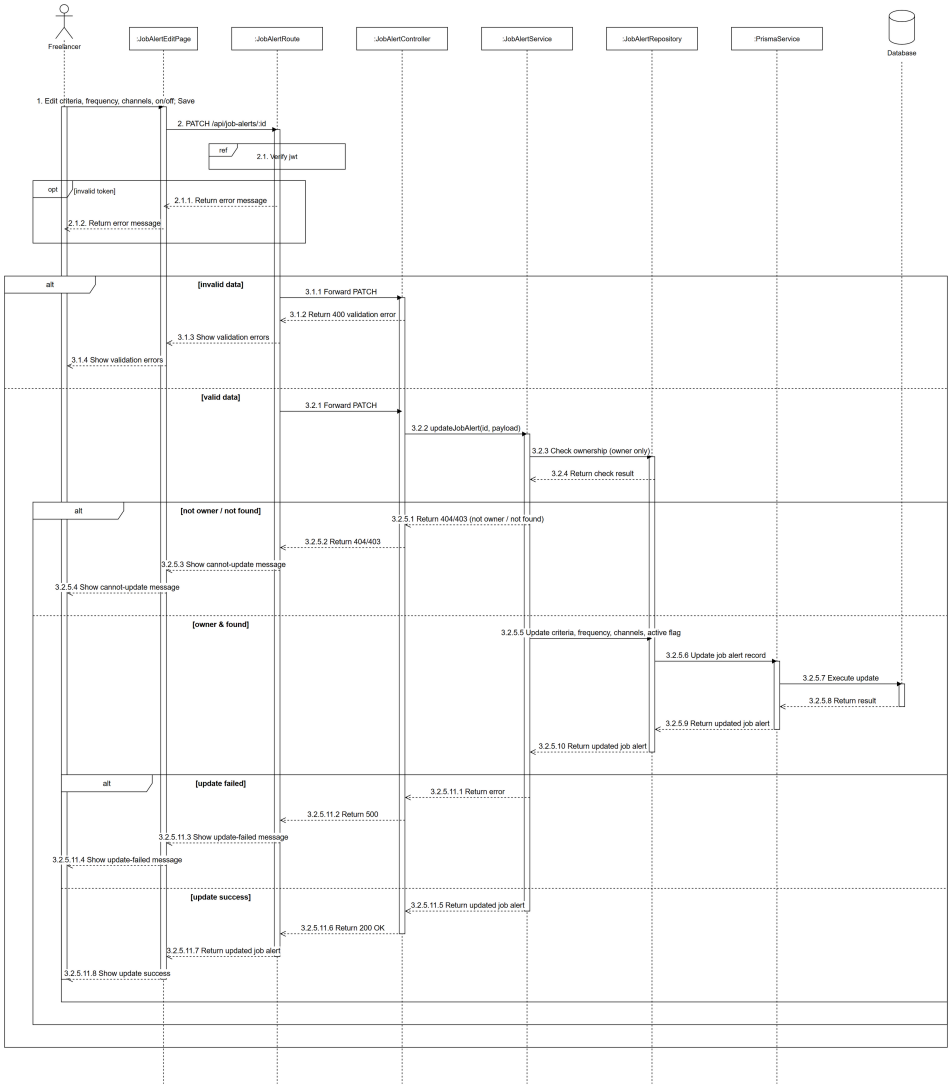

Figure 94: Sequence diagram of UC-23.04 Update Job Alert

3.10.6 UC-23.05 – Delete Job Alert

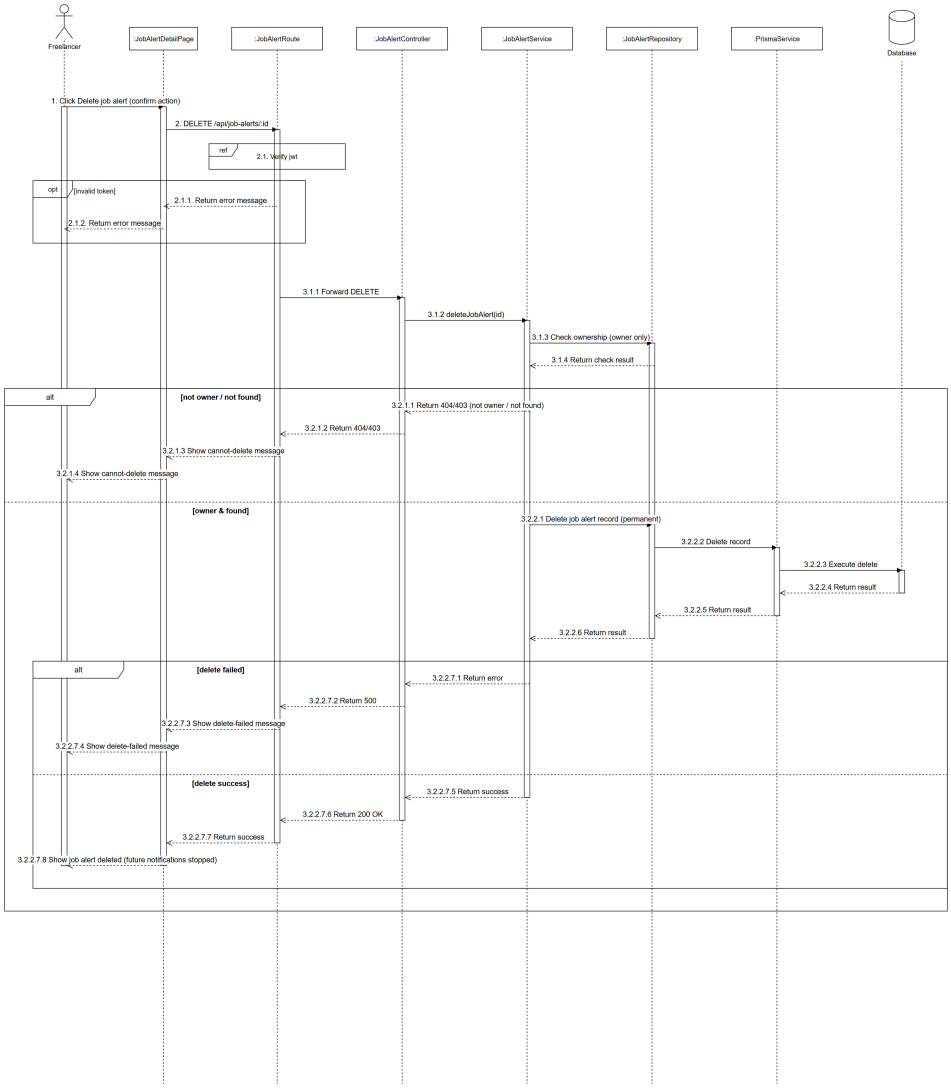

Figure 95: Sequence diagram of UC-23.05 Delete Job Alert

3.11 Permission Management

Permission Management exposes the route-level permission catalogue so that an administrator can inspect the permissions available for role assignment.

3.11.1 Class Diagram

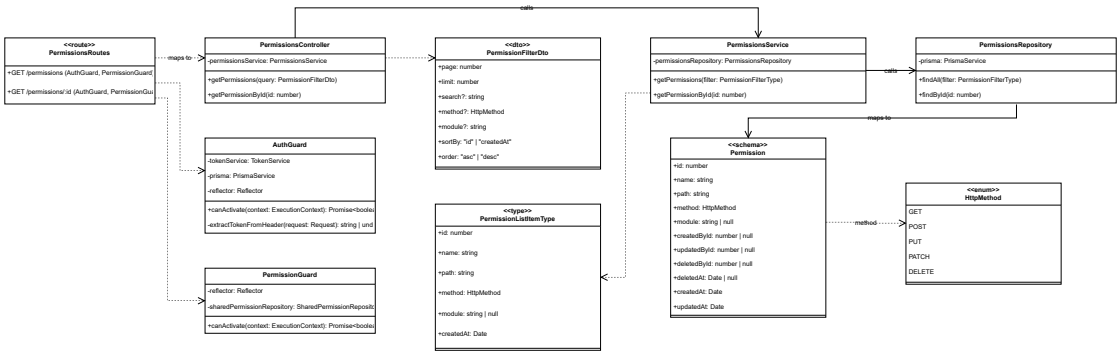

Figure 96: Class diagram of Permission Management

3.11.2 UC-15.01 – View Permission List

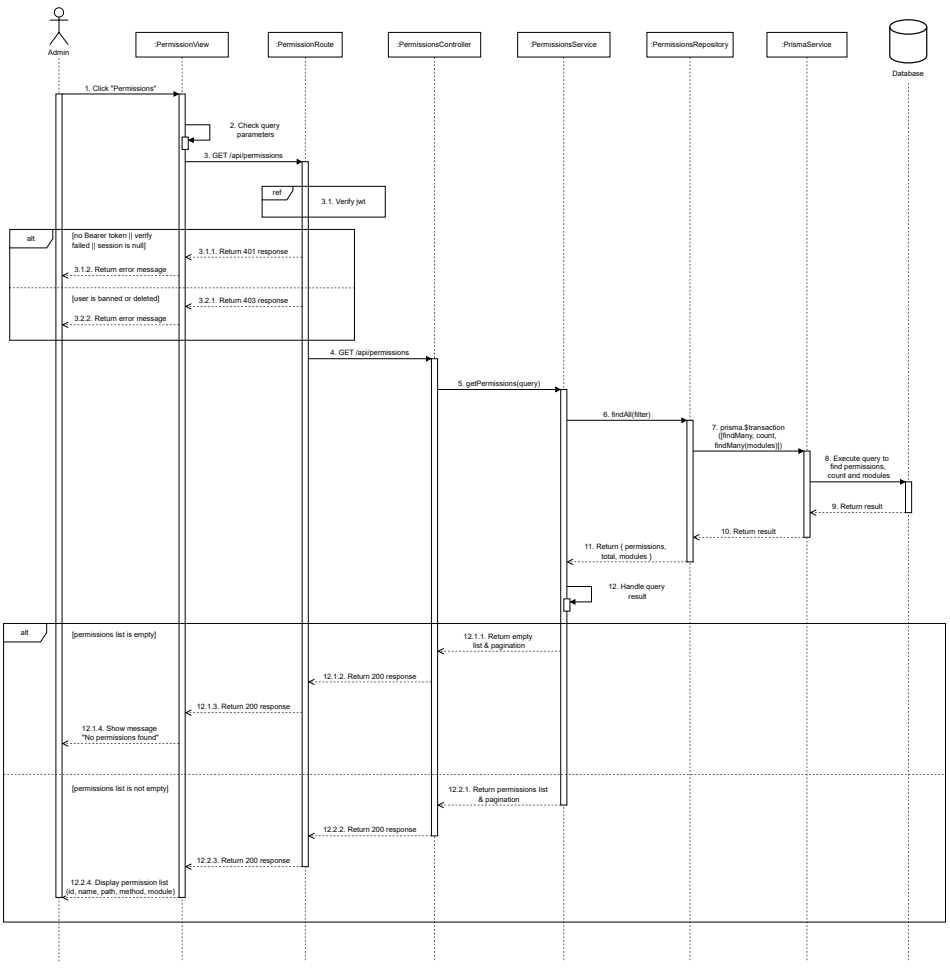

Figure 97: Sequence diagram of UC-15.01 View Permission List

3.11.3 UC-15.02 – View Permission Detail

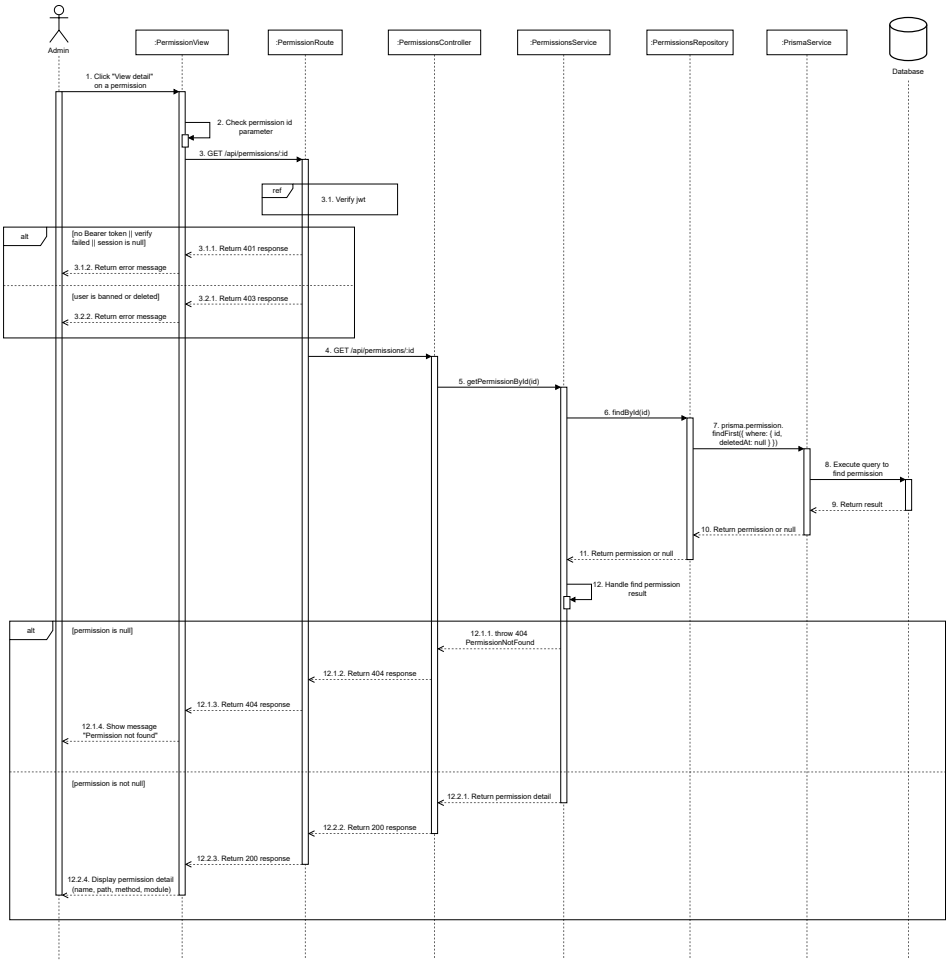

Figure 98: Sequence diagram of UC-15.02 View Permission Detail

3.12 Session Management

Session Management gives an authenticated user visibility over the devices that currently hold a refresh-token session and allows a session to be revoked remotely.

3.12.1 Class Diagram

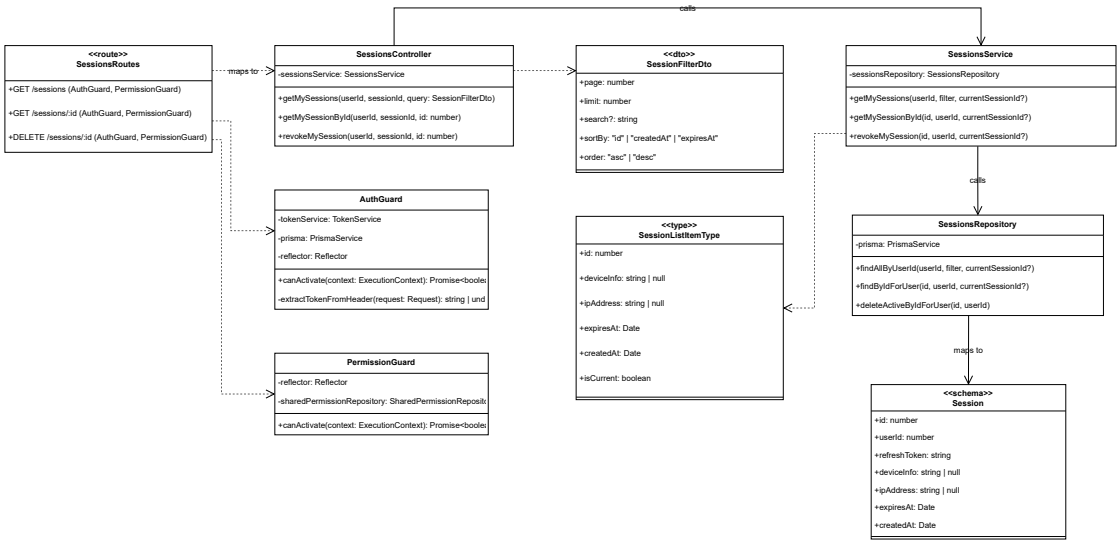

Figure 99: Class diagram of Session Management

3.12.2 UC-16.01 – View Session List

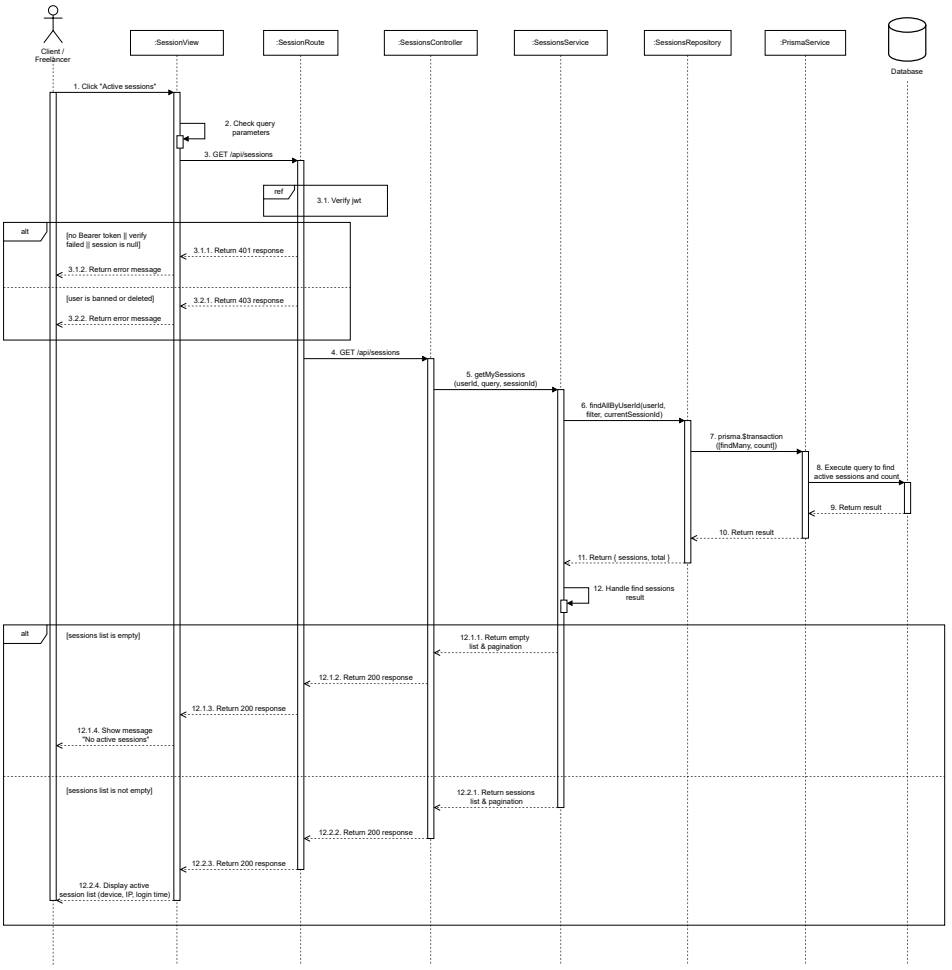

Figure 100: Sequence diagram of UC-16.01 View Session List

3.12.3 UC-16.02 – View Session Detail

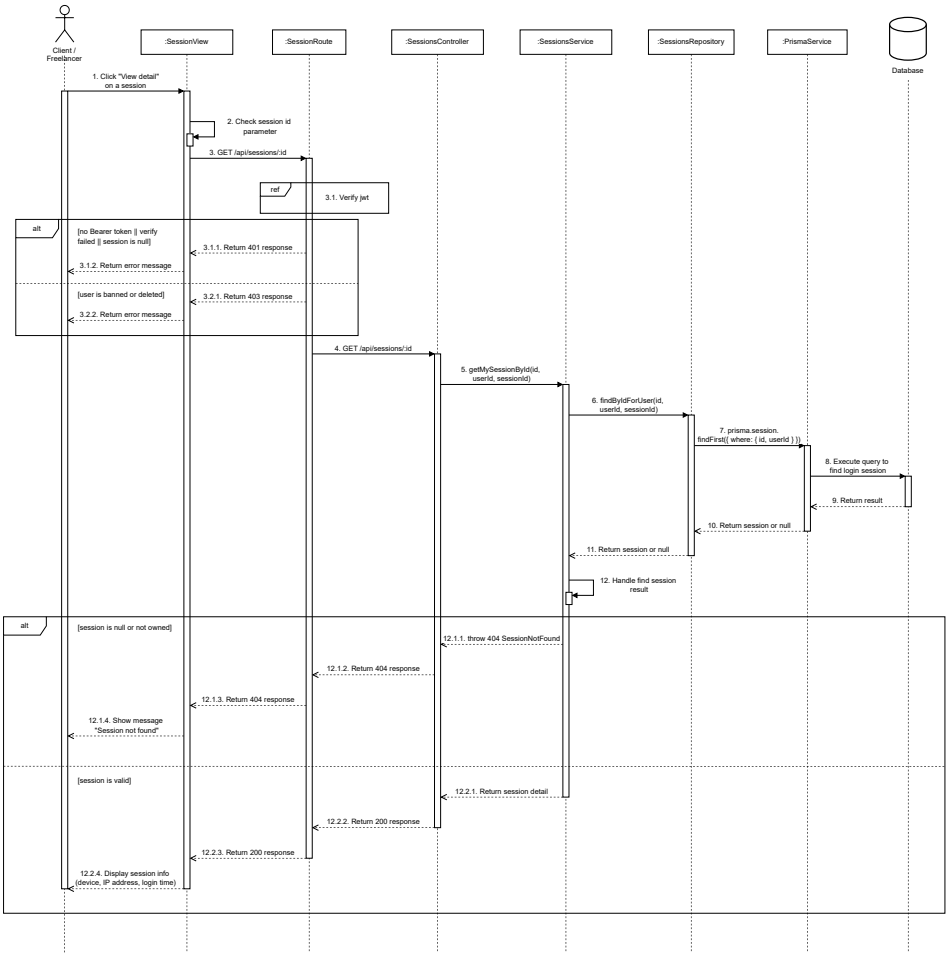

Figure 101: Sequence diagram of UC-16.02 View Session Detail

3.12.4 UC-16.03 – Revoke Session

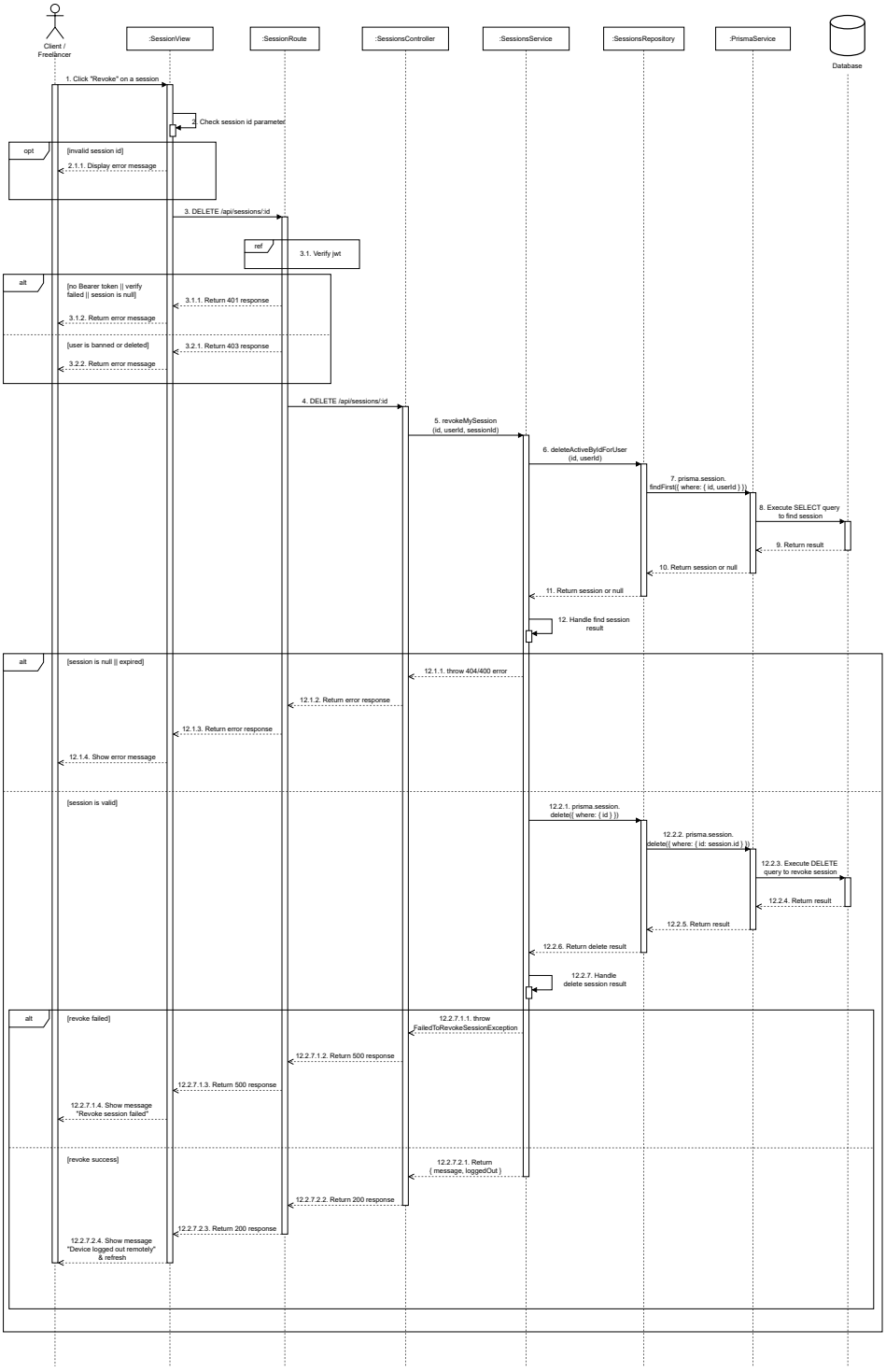

Figure 102: Sequence diagram of UC-16.03 Revoke Session

3.13 Forum Category Management

Forum Category Management covers the administrator-side category catalogue and the public category views used by the forum. Deletion is a soft delete, so a category can be restored later.

3.13.1 Class Diagram

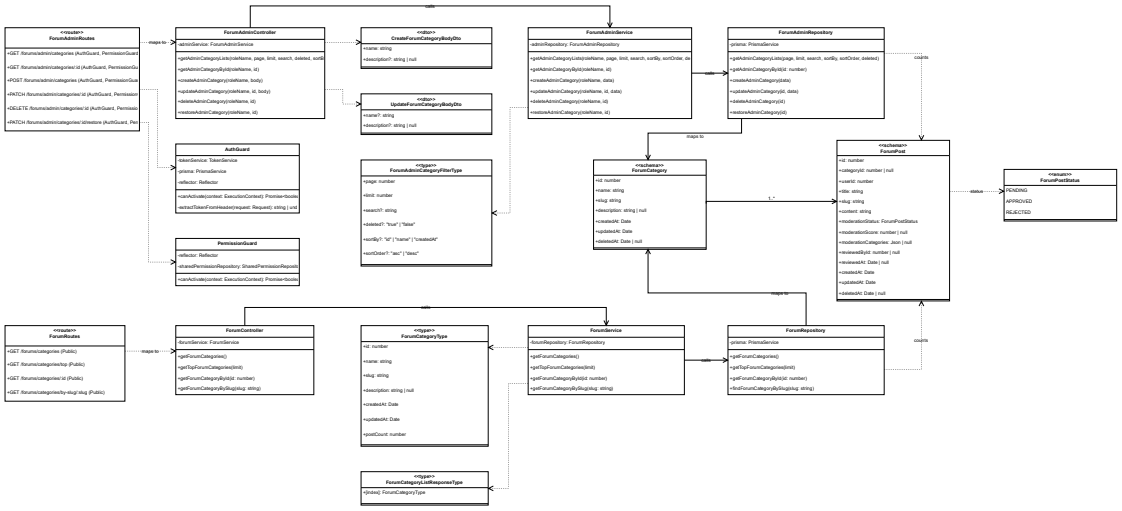

Figure 103: Class diagram of Forum Category Management

3.13.2 UC-17.01 – Create Forum Category

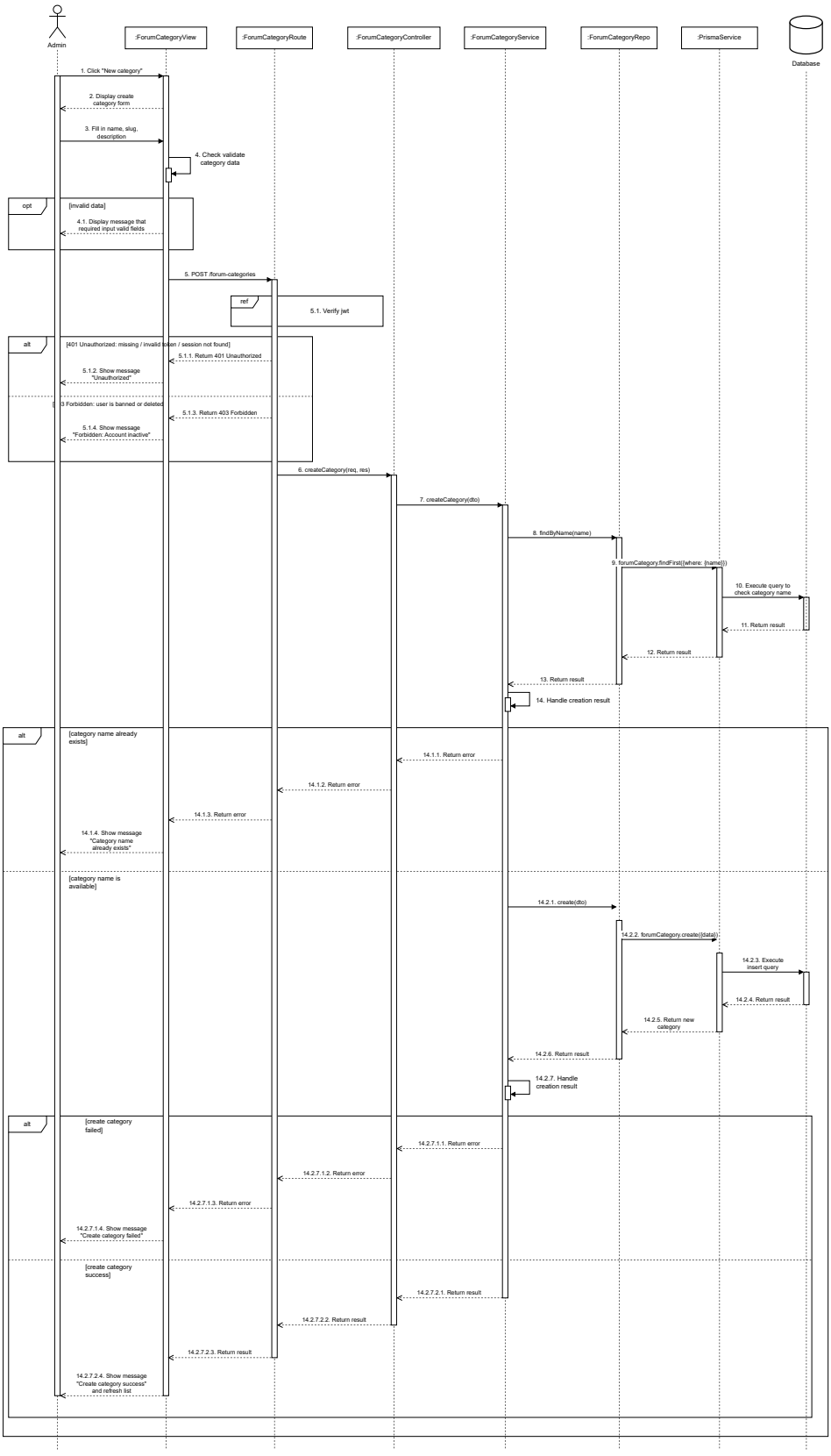

Figure 104: Sequence diagram of UC-17.01 Create Forum Category

3.13.3 UC-17.02 – Update Forum Category

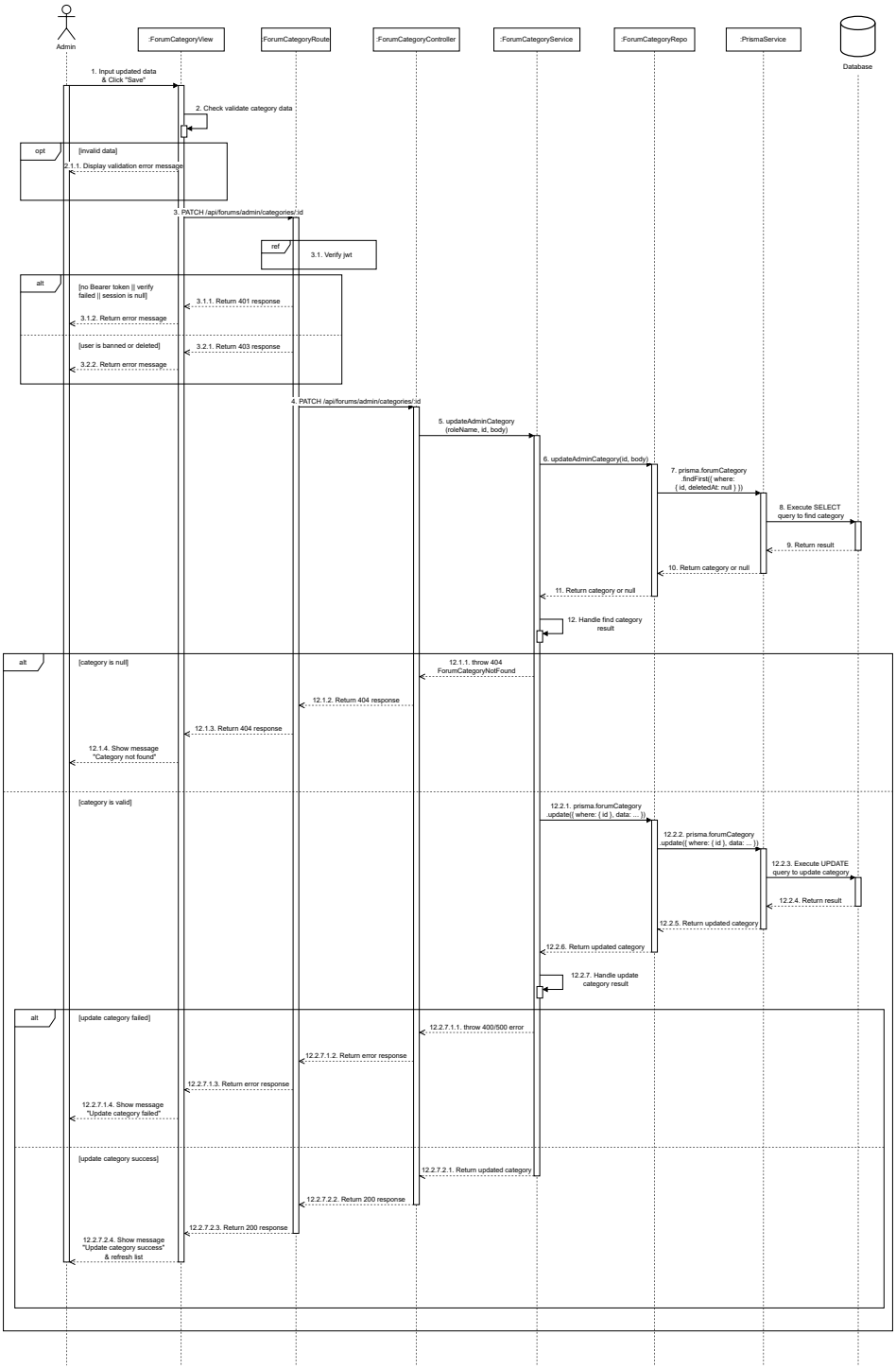

Figure 105: Sequence diagram of UC-17.02 Update Forum Category

3.13.4 UC-17.03 – Delete Forum Category

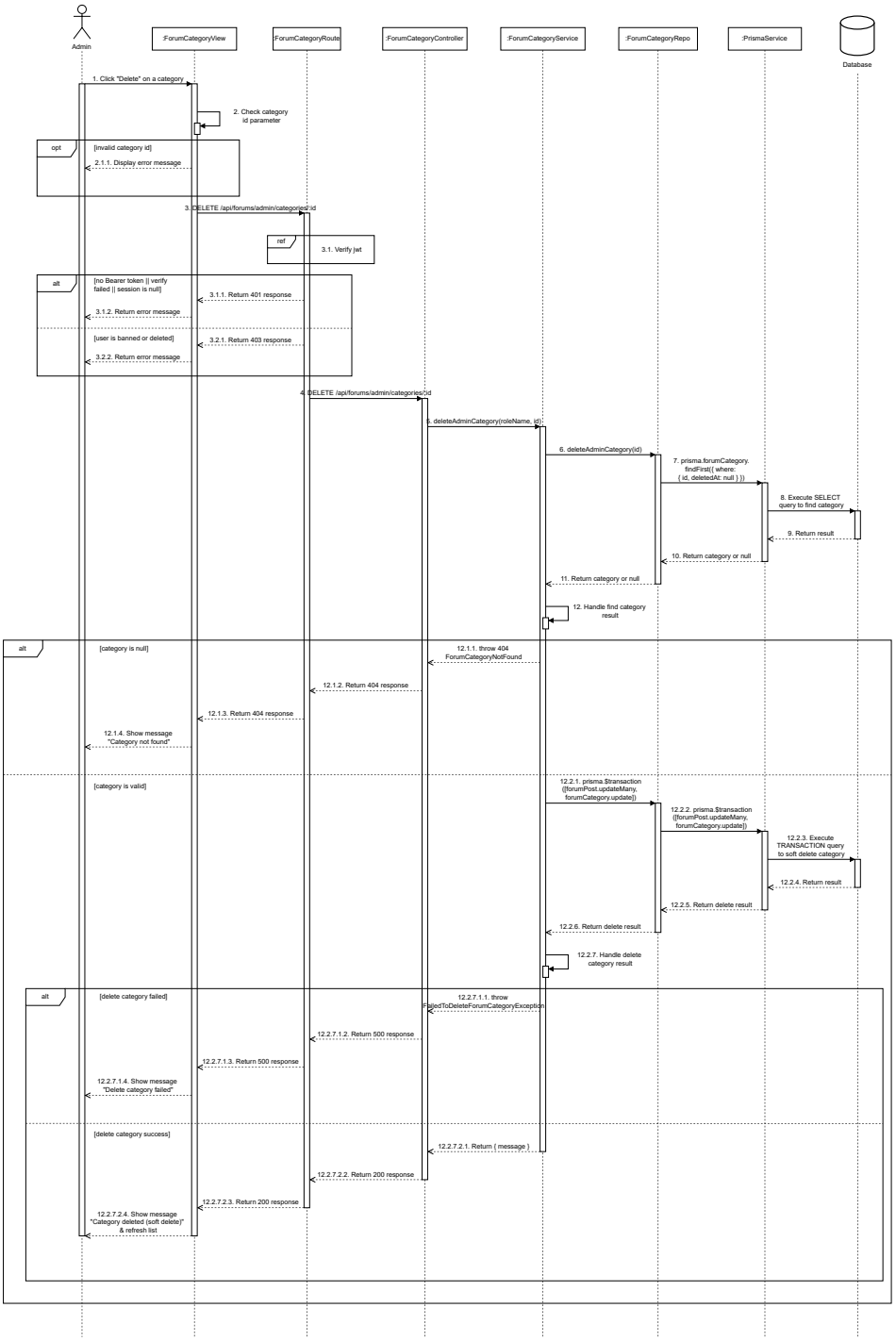

Figure 106: Sequence diagram of UC-17.03 Delete Forum Category

3.13.5 UC-17.06 – Restore Forum Category

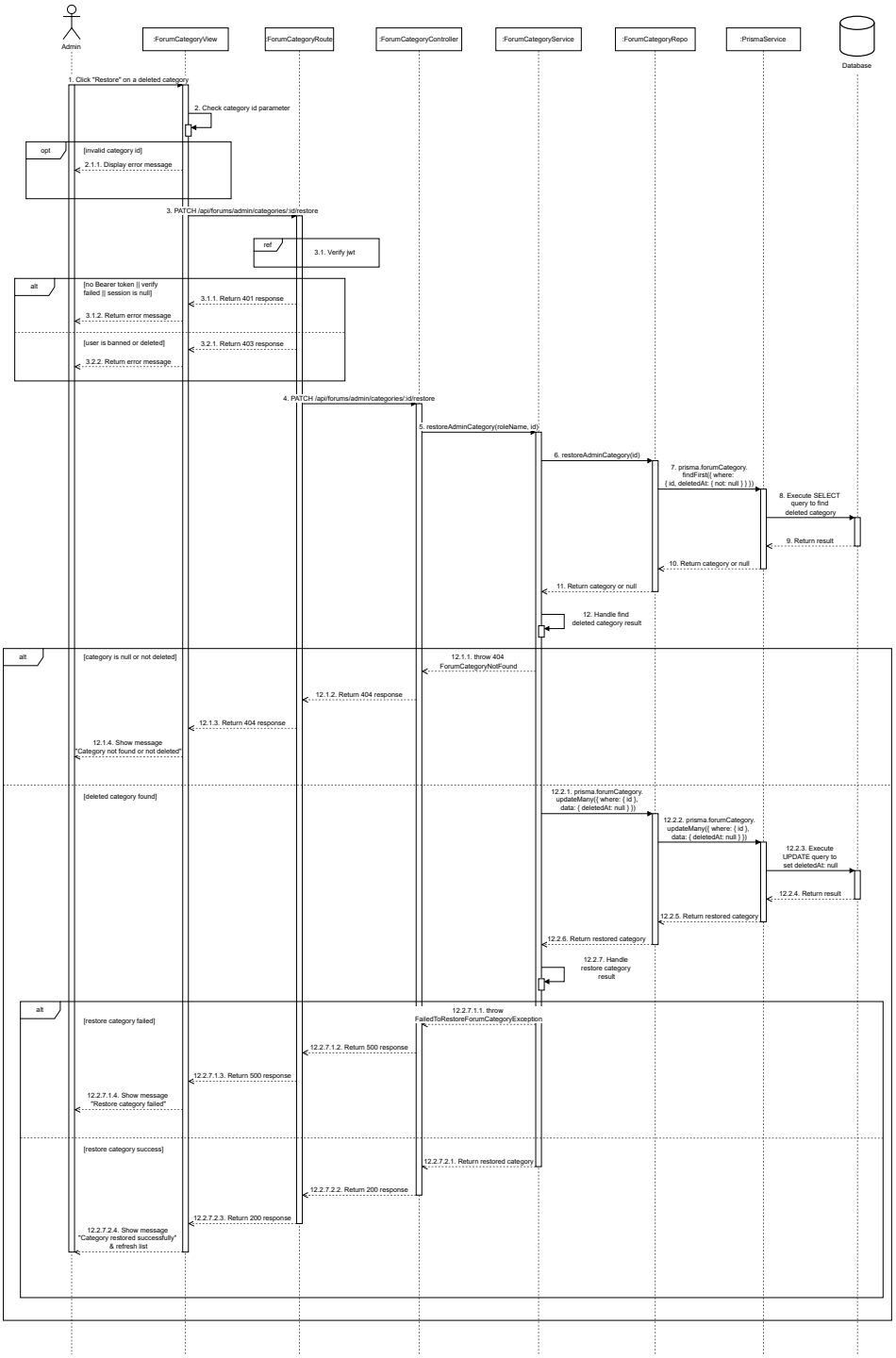

Figure 107: Sequence diagram of UC-17.06 Restore Forum Category

3.14 Manage Moderation

Manage Moderation covers the UC-34 review workflow of the administration console: listing the posts waiting for review, opening a flagged post and approving or rejecting it. A review writes the moderation status, score and reviewer back to the forum post.

3.14.1 Class Diagram

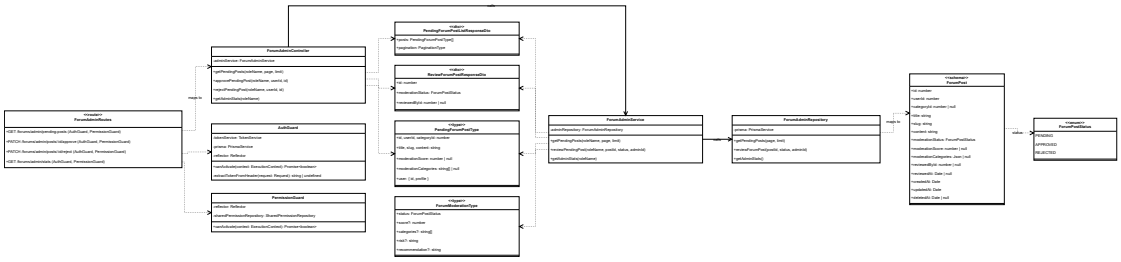

Figure 108: Class diagram of Manage Moderation

3.14.2 UC-34.01 – View Moderation Queue

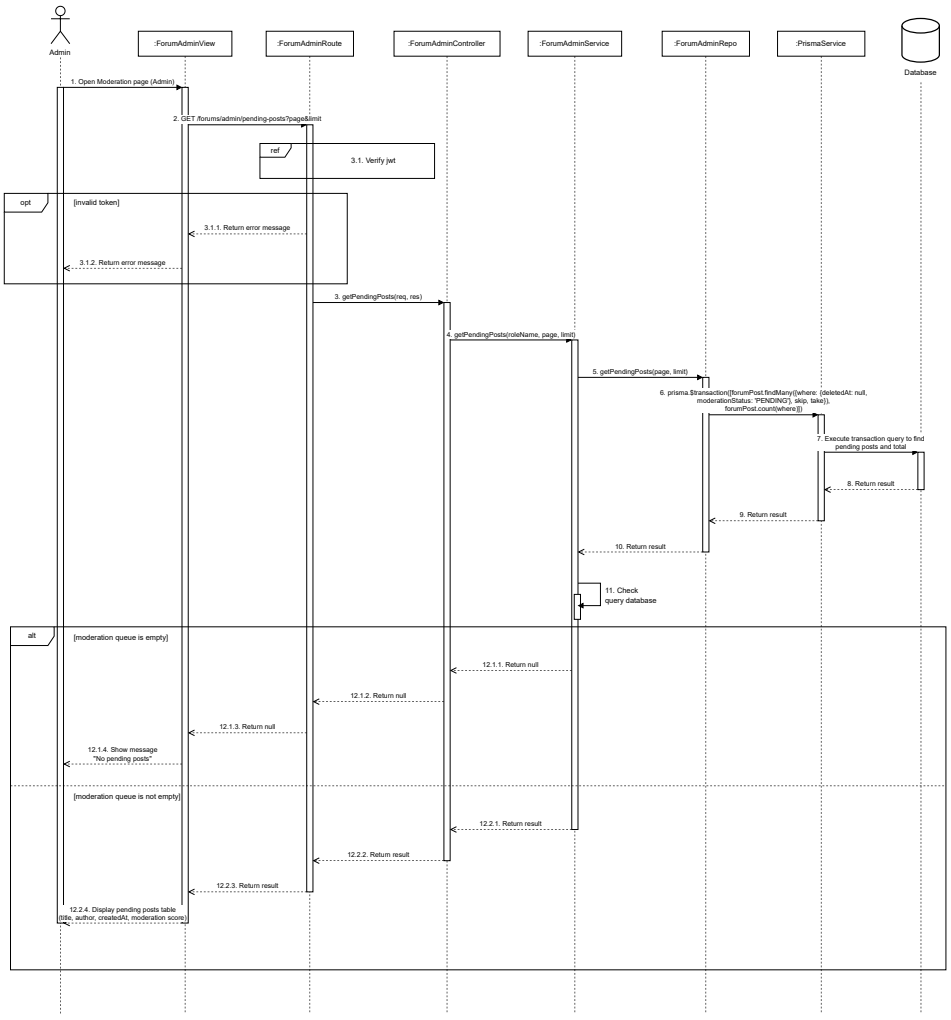

Figure 109: Sequence diagram of UC-34.01 View Moderation Queue

3.14.3 UC-34.02 – View Moderation Item Detail

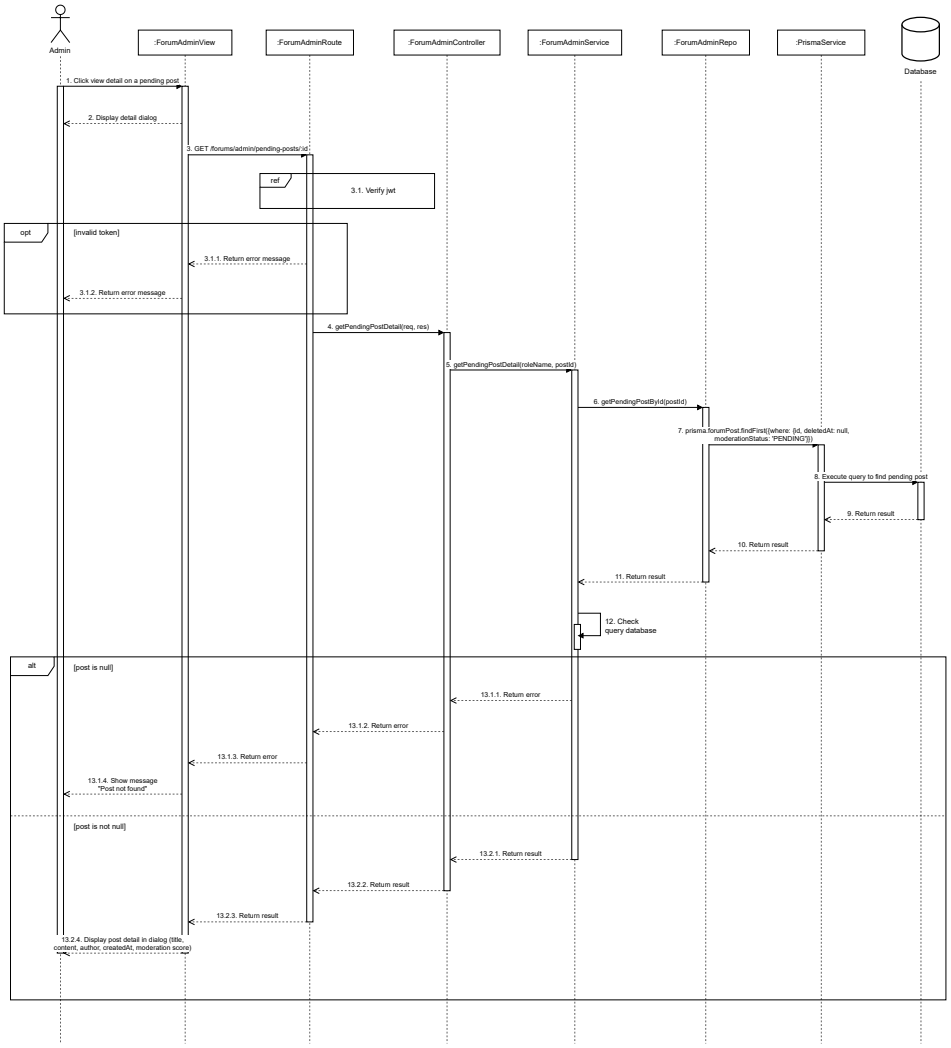

Figure 110: Sequence diagram of UC-34.02 View Moderation Item Detail

3.14.4 UC-34.03 – Resolve Moderation Item

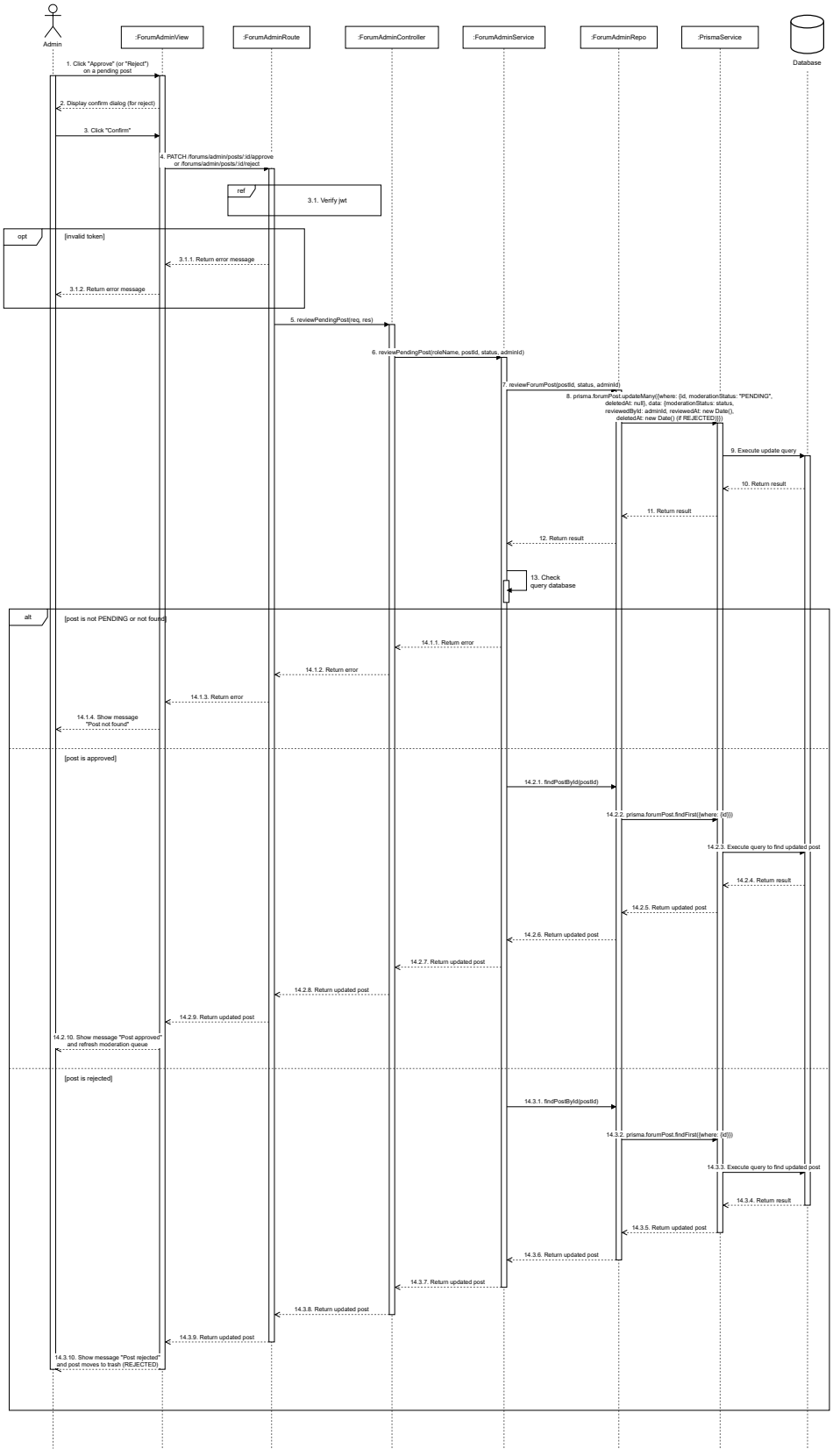

Figure 111: Sequence diagram of UC-34.03 Resolve Moderation Item

3.15 Manage ID Verification

Manage ID Verification covers the UC-35 identity review workflow: listing the submitted identity documents, opening a request with its uploaded file, and approving or rejecting the request. A decision updates the verification status and the reviewer of the identity document.

3.15.1 Class Diagram

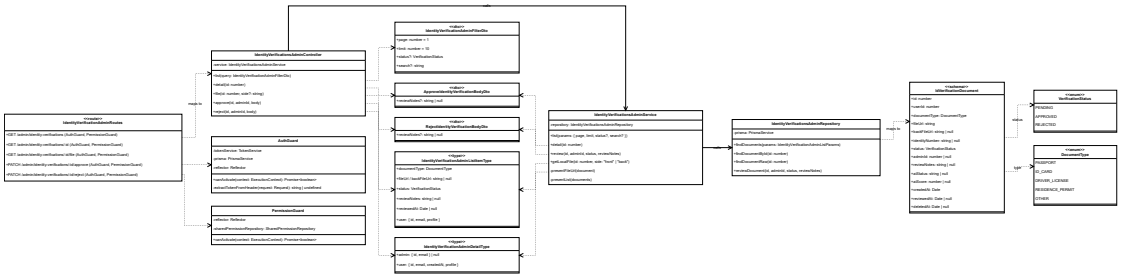

Figure 112: Class diagram of Manage ID Verification

3.15.2 UC-35.01 – View ID Verification Request List

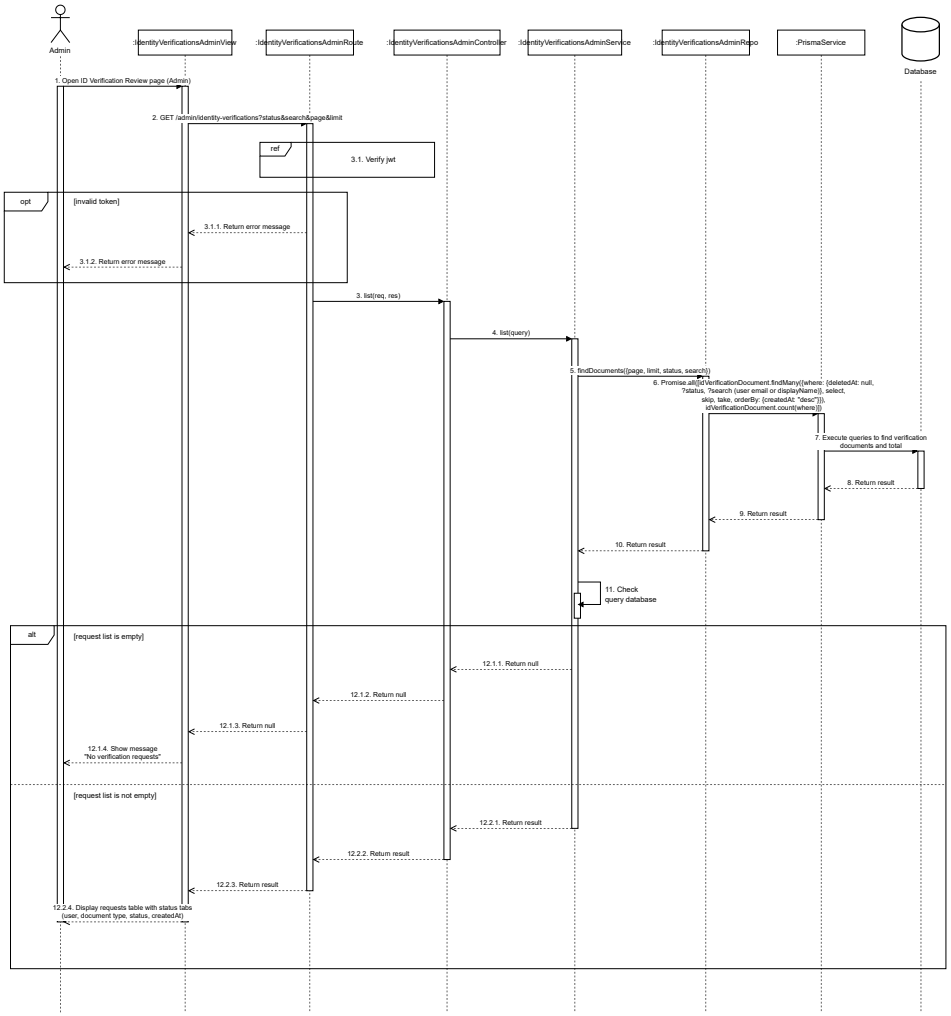

Figure 113: Sequence diagram of UC-35.01 View ID Verification Request List

3.15.3 UC-35.02 – View ID Verification Request Detail

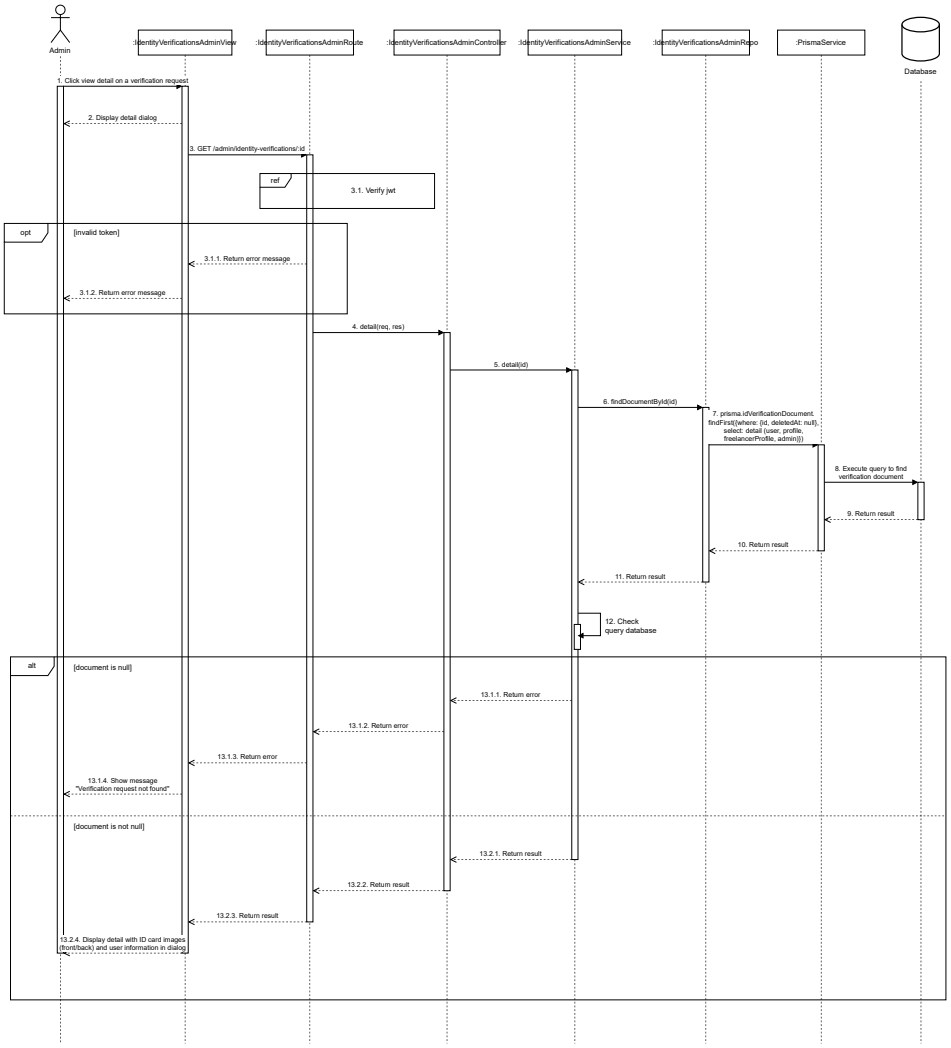

Figure 114: Sequence diagram of UC-35.02 View ID Verification Request Detail

3.15.4 UC-35.03 – Approve ID Verification

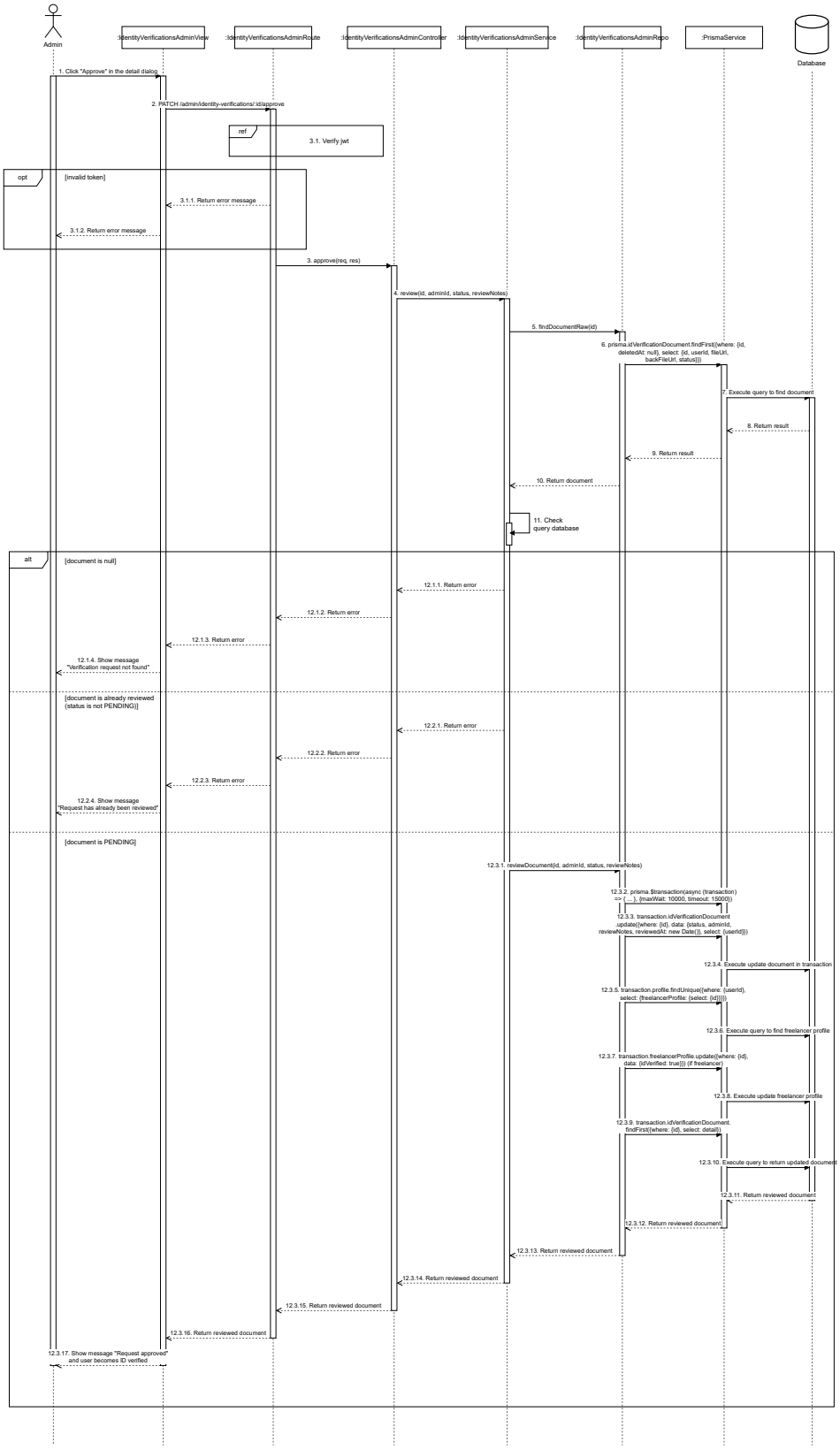

Figure 115: Sequence diagram of UC-35.03 Approve ID Verification

3.15.5 UC-35.04 – Reject ID Verification

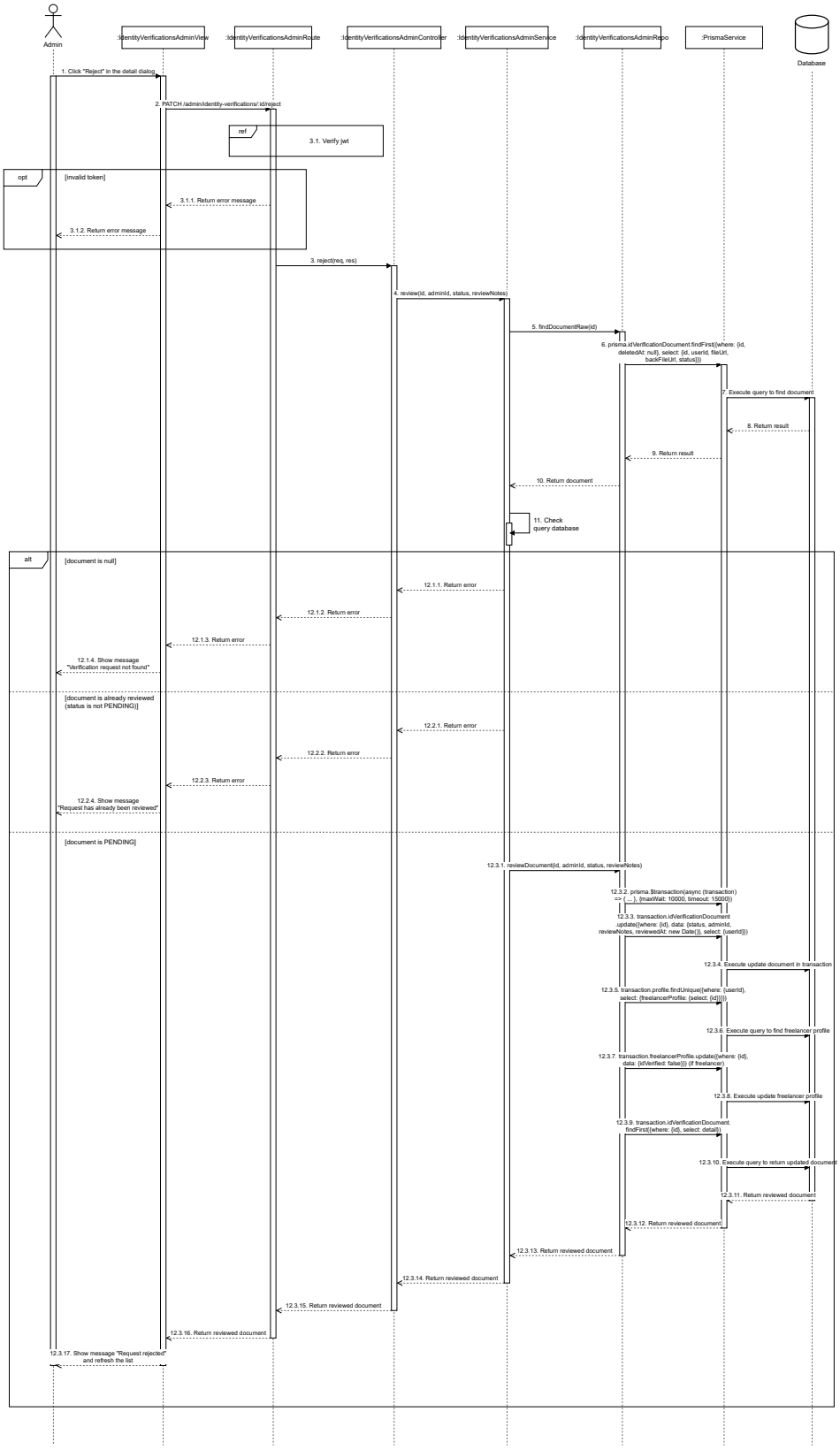

Figure 116: Sequence diagram of UC-35.04 Reject ID Verification

3.16 Manage Forum Trash

Manage Forum Trash covers the UC-36 recovery workflow: listing the soft-deleted posts and comments kept for review, opening one deleted item, and restoring it so that it becomes visible in the forum again.

3.16.1 Class Diagram

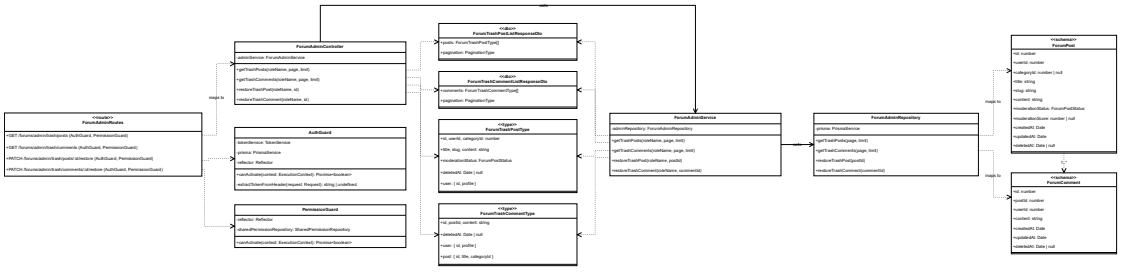

Figure 117: Class diagram of Manage Forum Trash

3.16.2 UC-36.01 – View Deleted Posts/Comments List

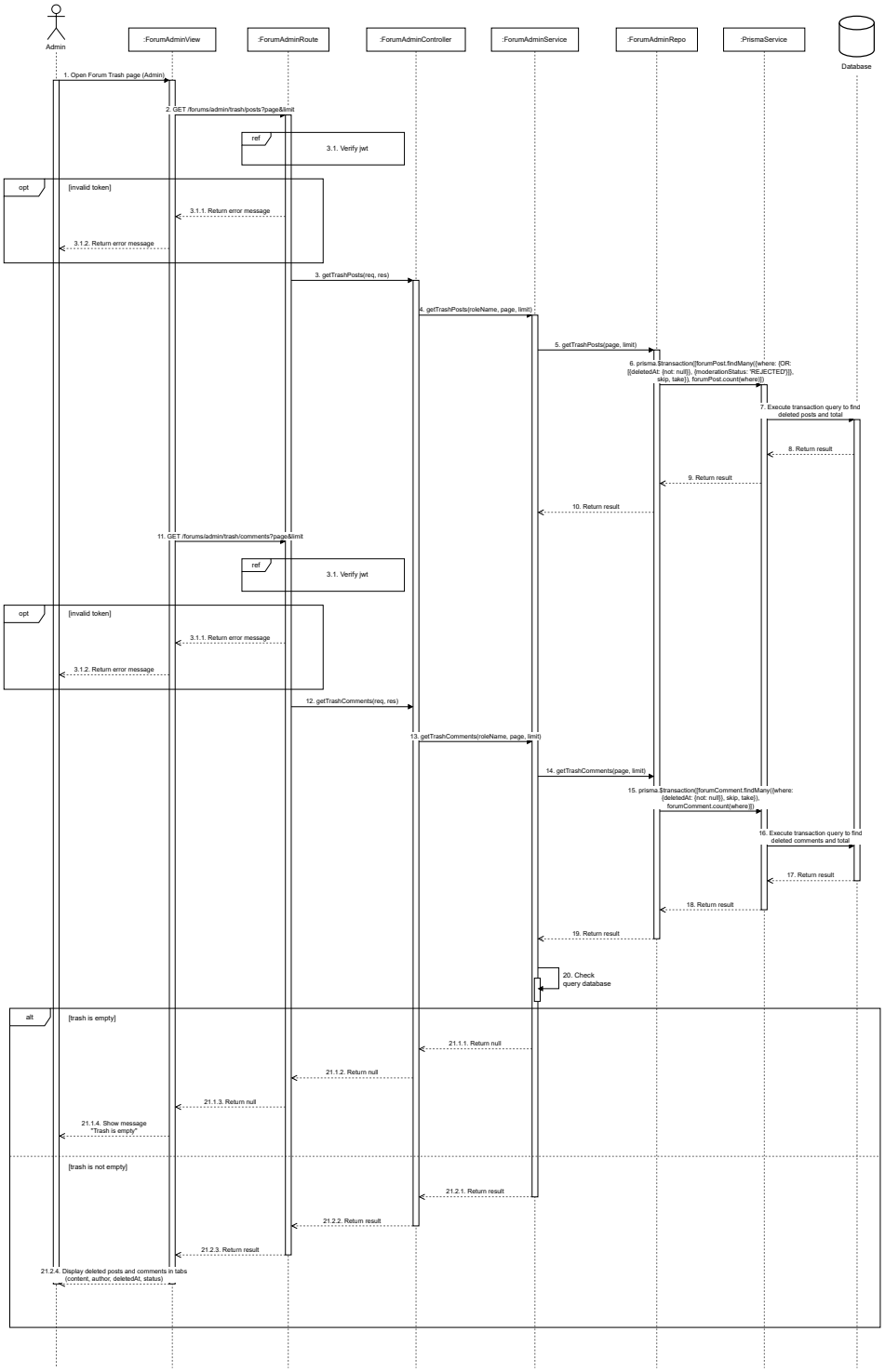

Figure 118: Sequence diagram of UC-36.01 View Deleted Posts/Comments List

3.16.3 UC-36.02 – View Deleted Posts/Comments Detail

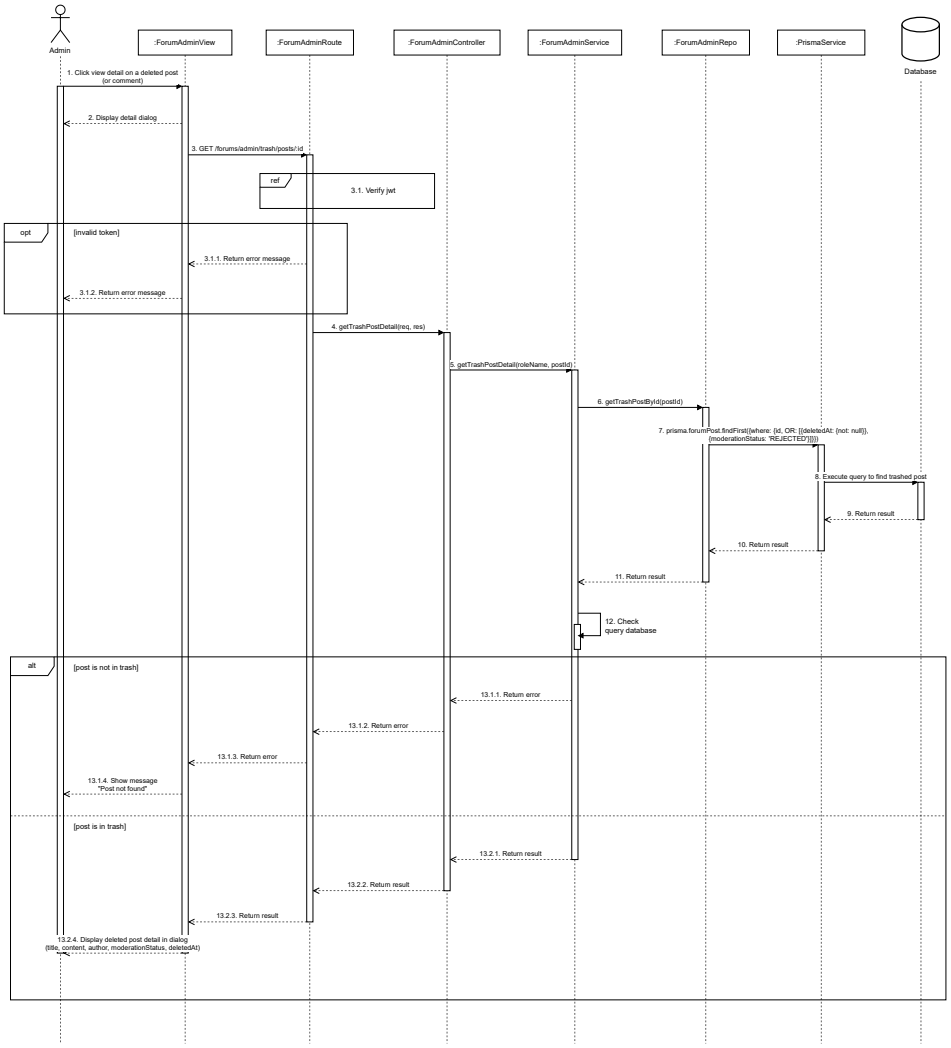

Figure 119: Sequence diagram of UC-36.02 View Deleted Posts/Comments Detail

3.16.4 UC-36.03 – Restore Deleted Posts/Comments

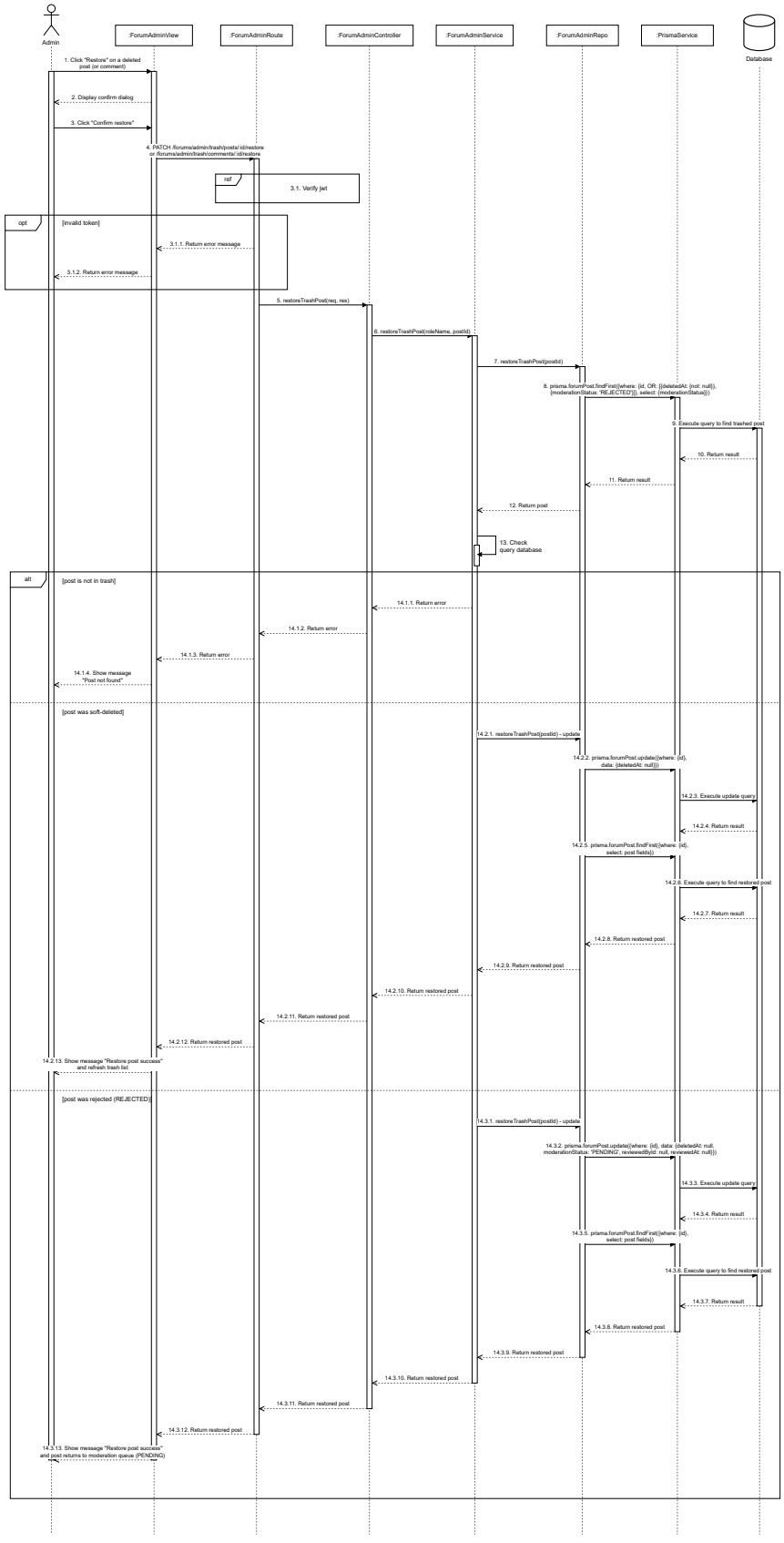

Figure 120: Sequence diagram of UC-36.03 Restore Deleted Posts/Comments

3.17 Proposal Management

Proposal Management covers the full proposal workflow between freelancers and clients. Freelancers can submit proposals with cover letters, bid amounts, and delivery estimates, view their submitted proposals, modify pending proposals, or withdraw them. Clients can view all proposals submitted for their jobs, inspect detailed freelancer bids, and accept or reject proposals to initiate contracting.

3.17.1 Class Diagram

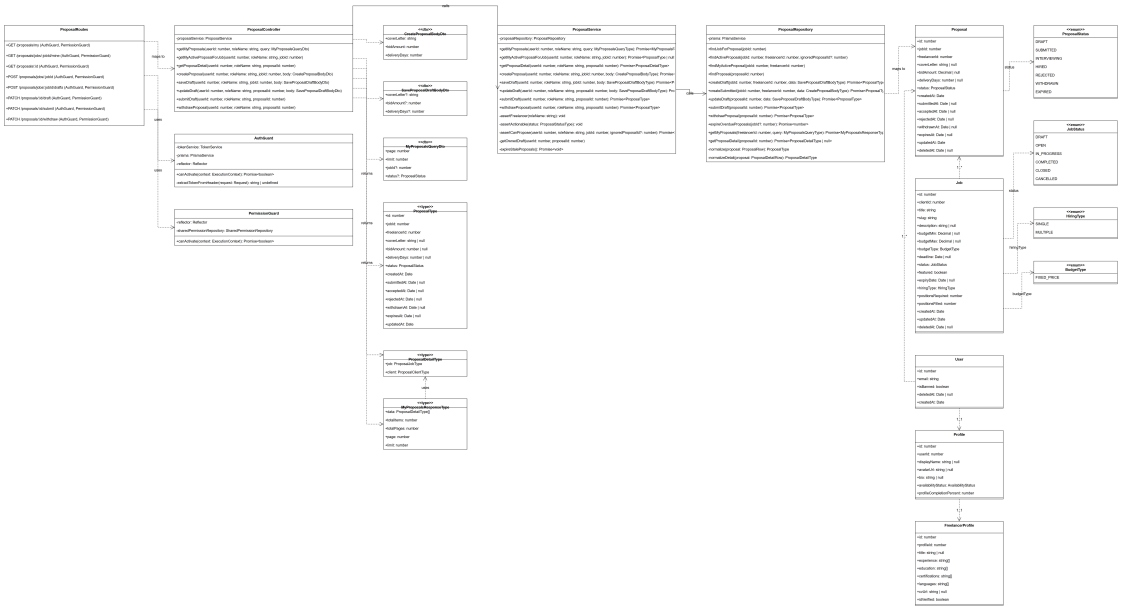

Figure 121: Class diagram of Freelancer Proposal Management

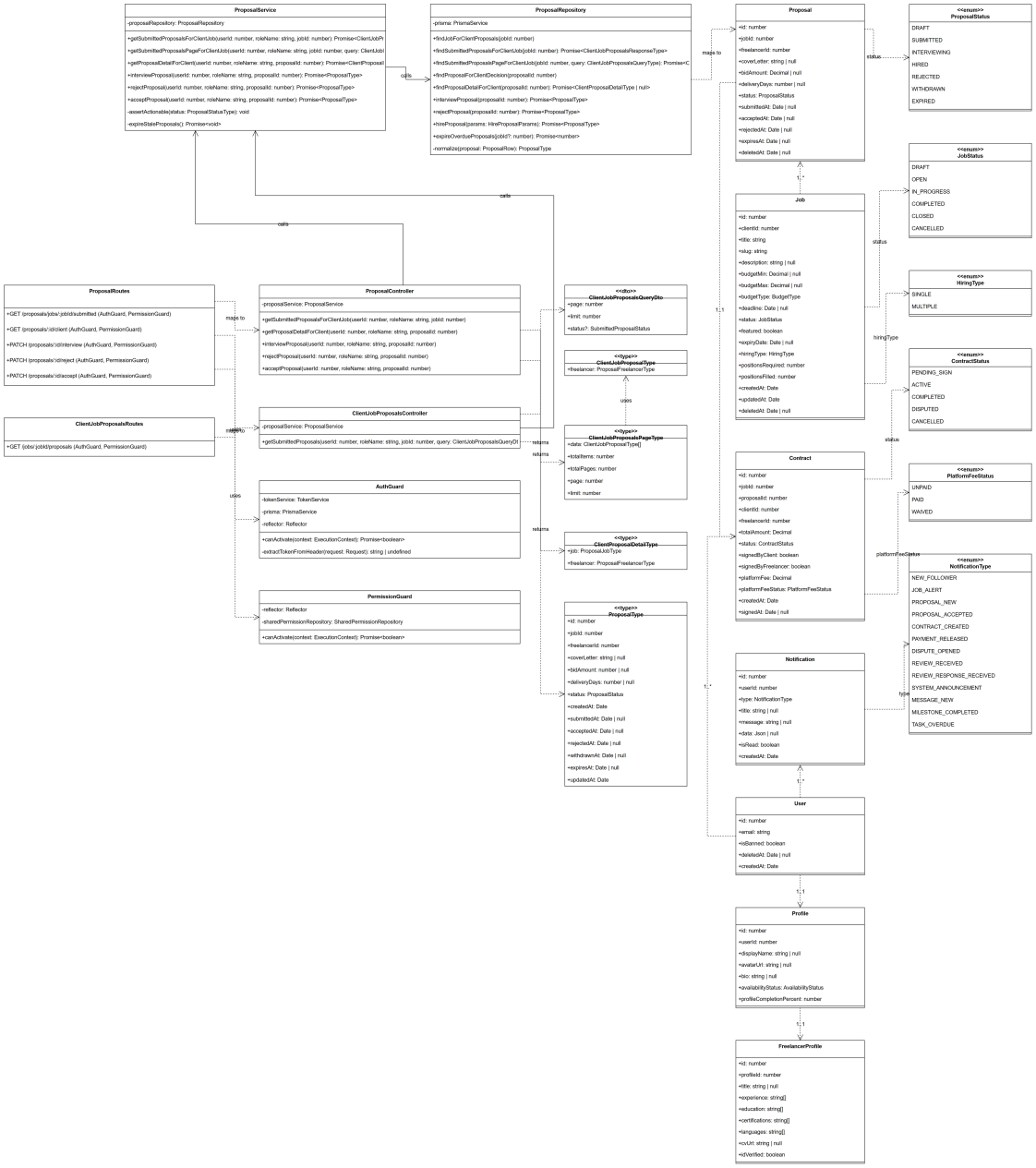

Figure 122: Class diagram of Client Proposal Management

3.17.2 UC-32.01 – View My Proposal List

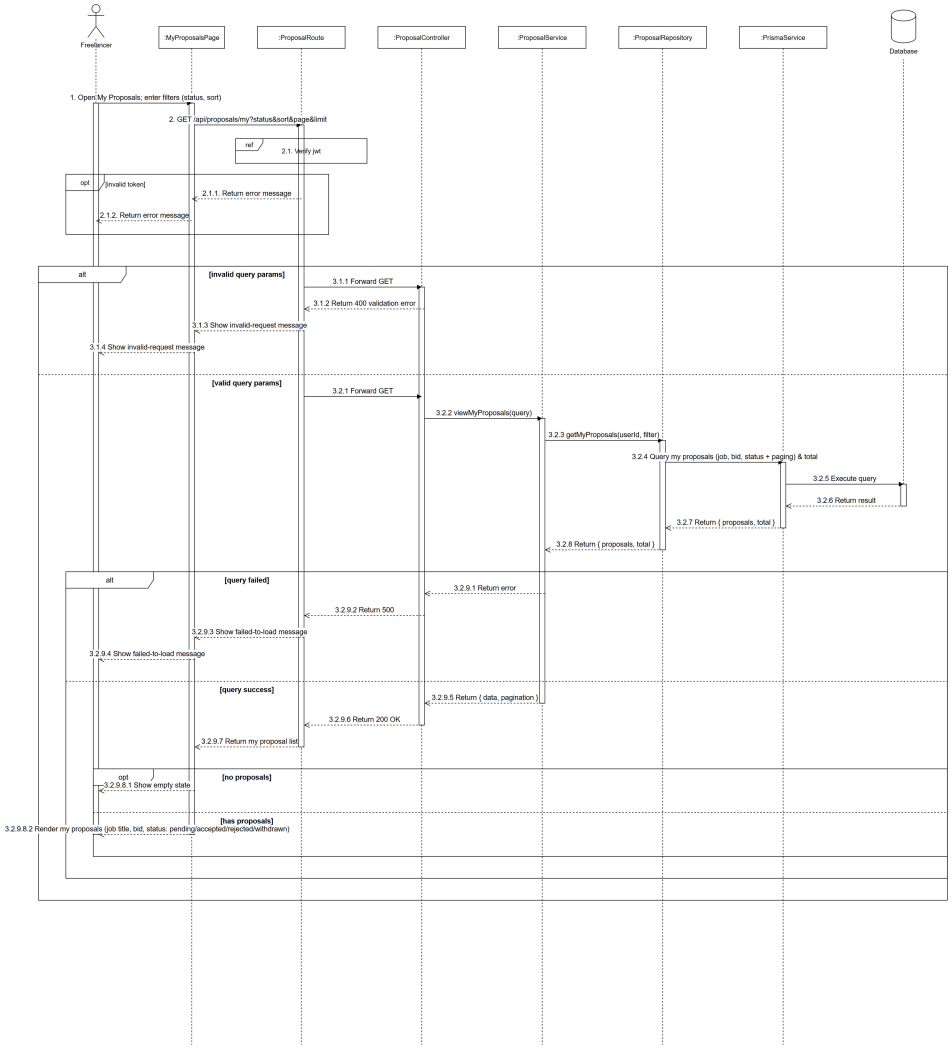

Figure 123: Sequence diagram of UC-32.01 View My Proposal List

3.17.3 UC-32.02 – View My Proposal Detail

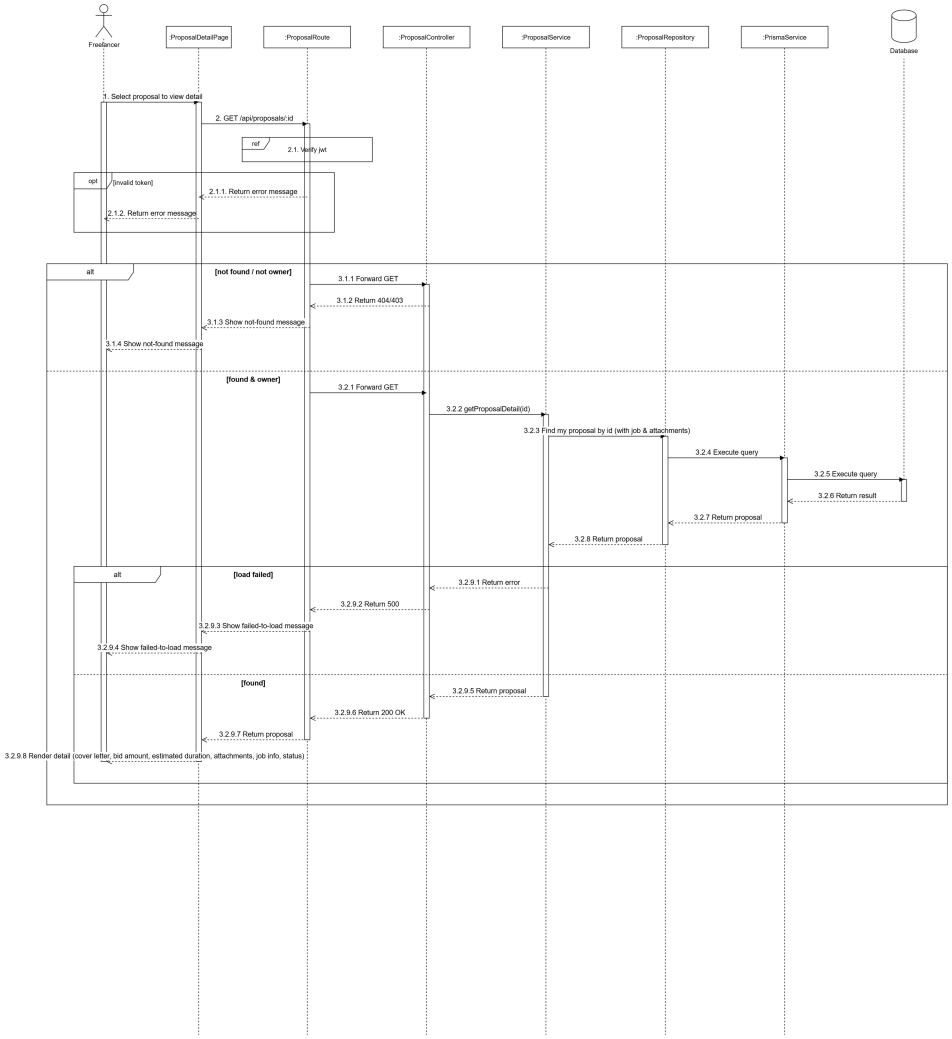

Figure 124: Sequence diagram of UC-32.02 View My Proposal Detail

3.17.4 UC-32.03 – Submit Proposal

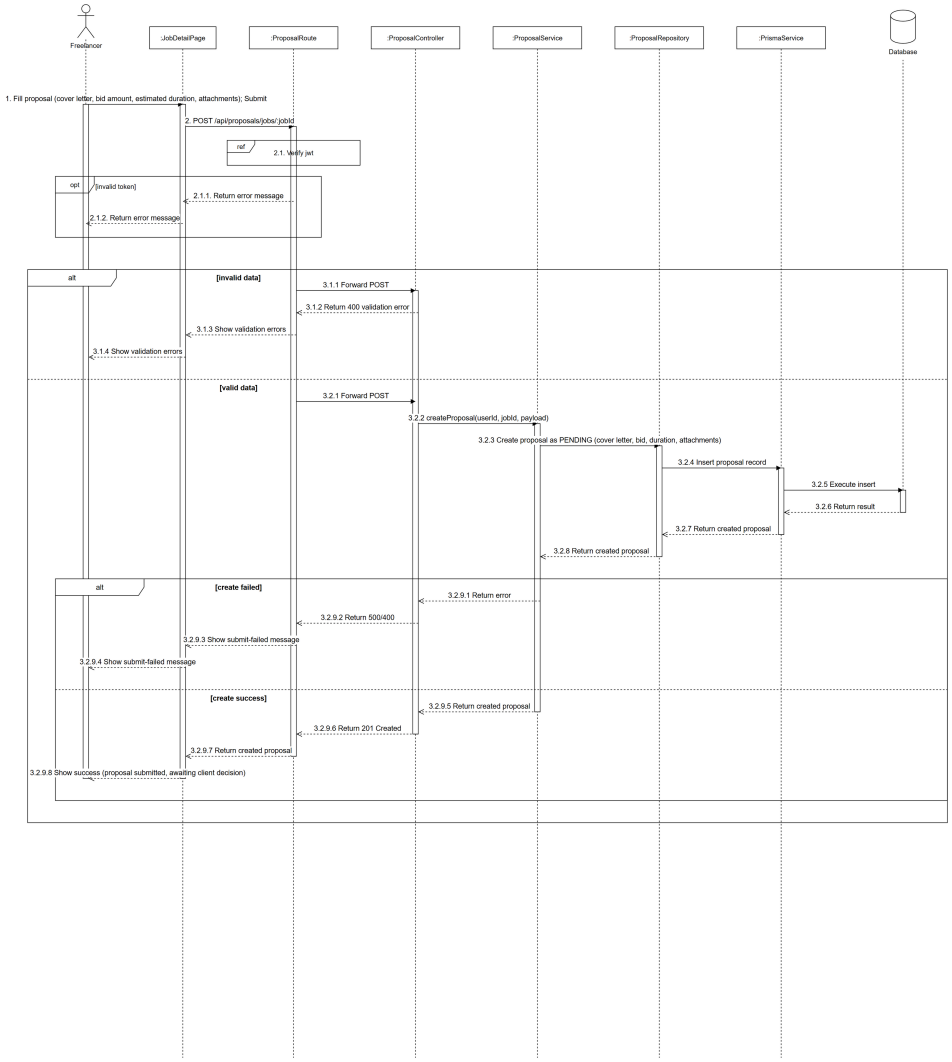

Figure 125: Sequence diagram of UC-32.03 Submit Proposal

3.17.5 UC-32.04 – Update Proposal

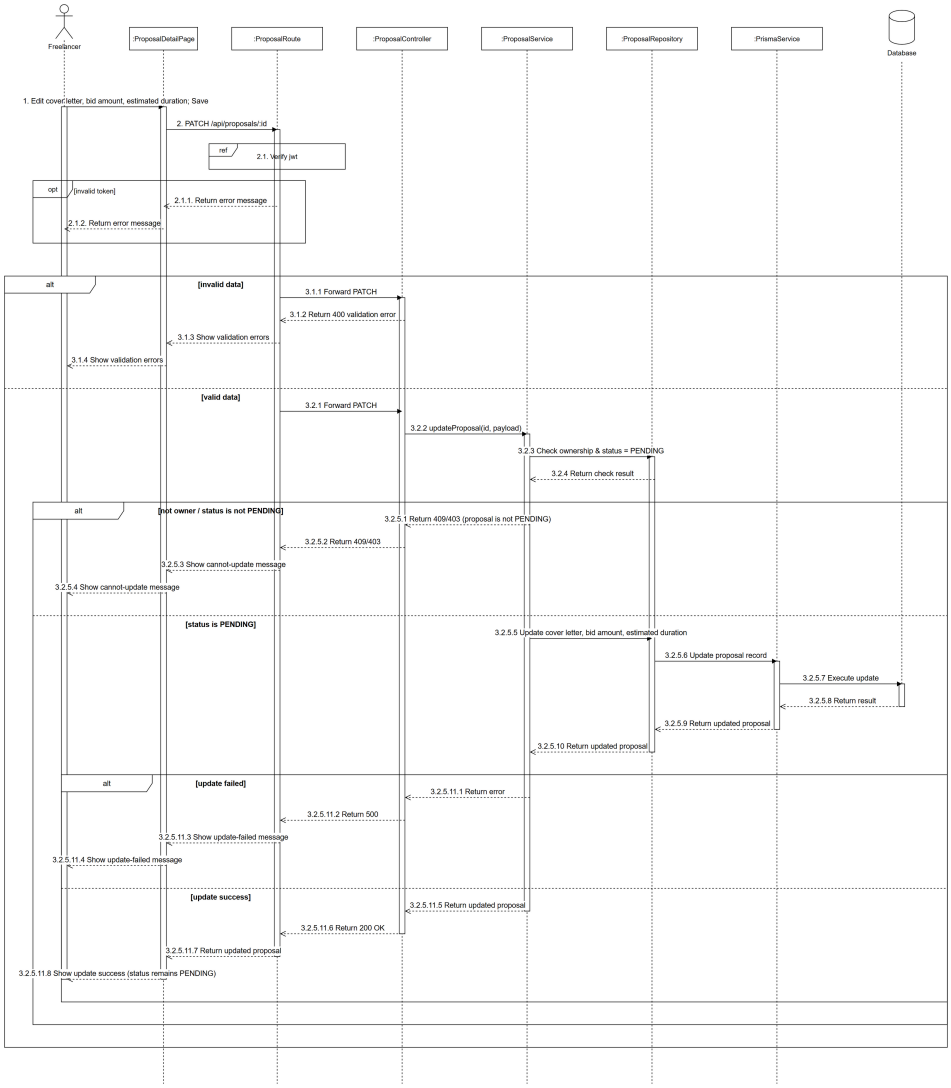

Figure 126: Sequence diagram of UC-32.04 Update Proposal

3.17.6 UC-32.05 – Withdraw Proposal

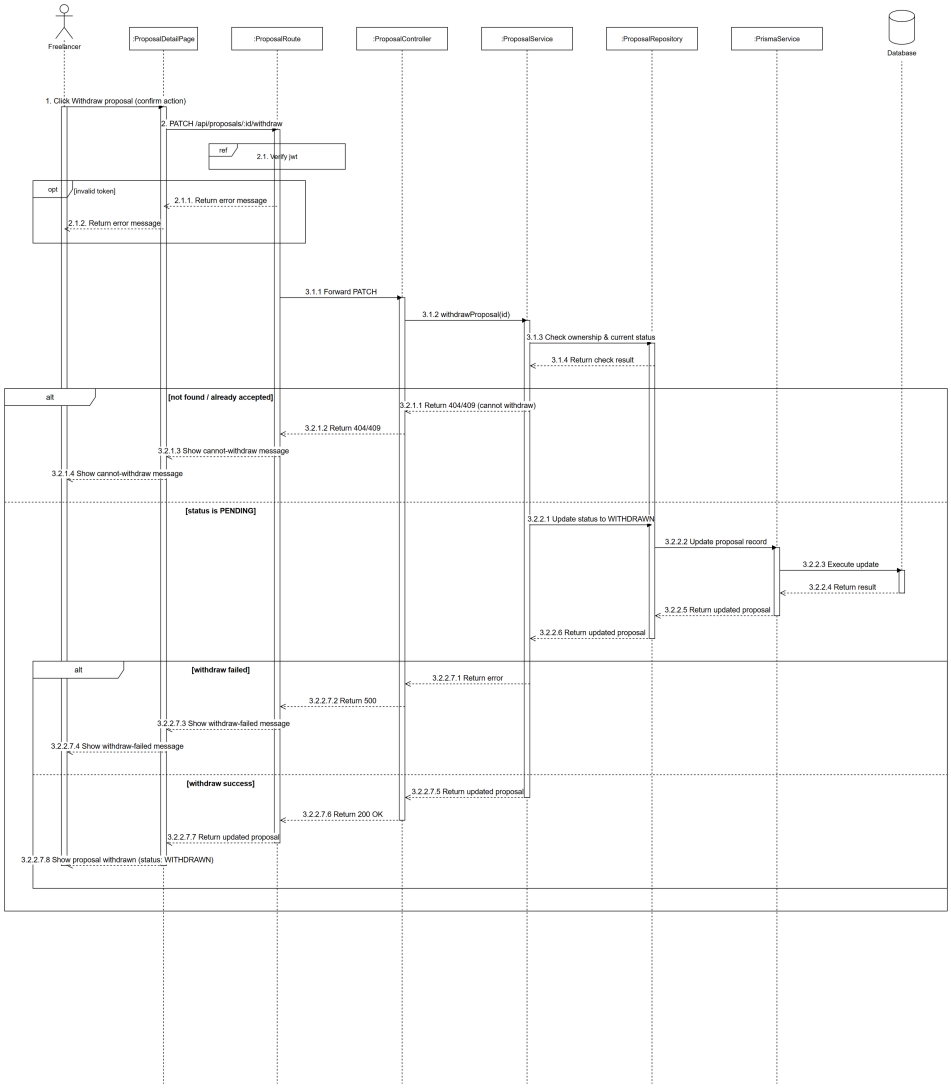

Figure 127: Sequence diagram of UC-32.05 Withdraw Proposal

3.17.7 UC-32.06 – Accept Proposal

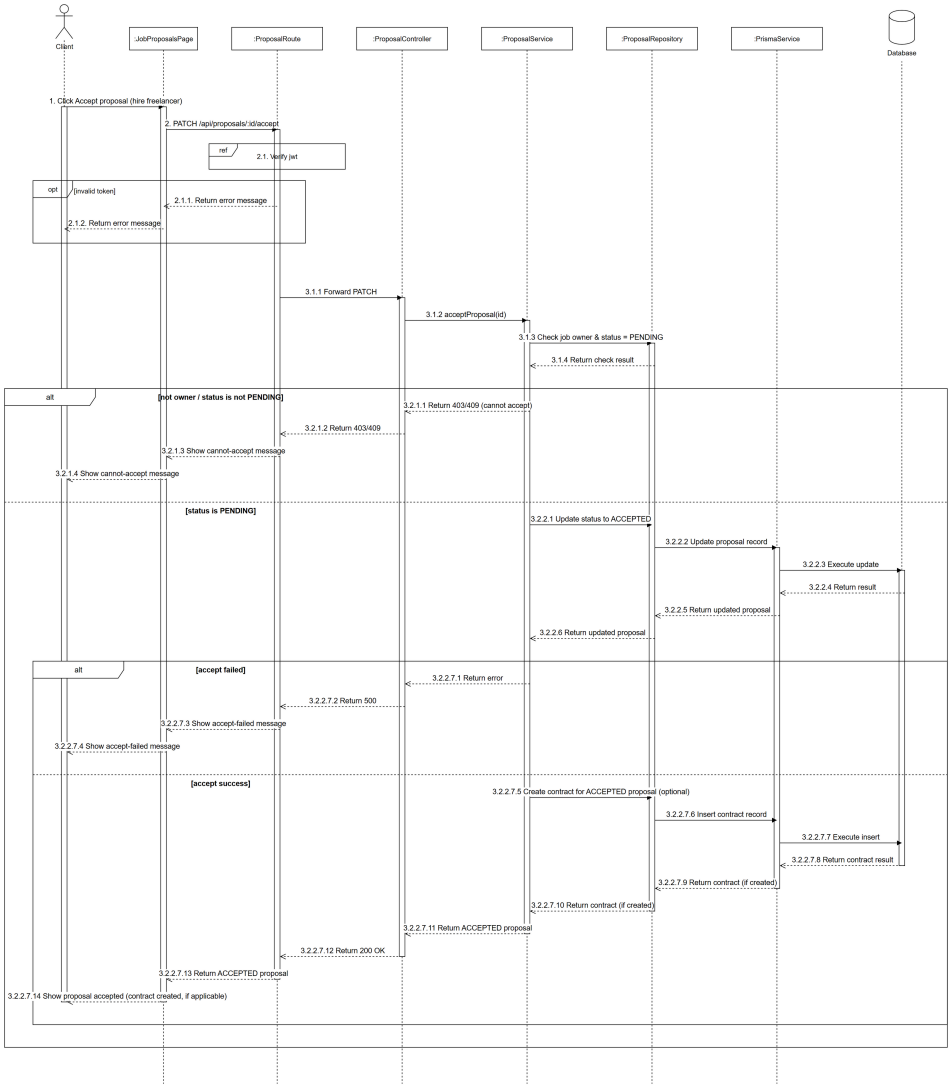

Figure 128: Sequence diagram of UC-32.06 Accept Proposal

3.17.8 UC-32.07 – Reject Proposal

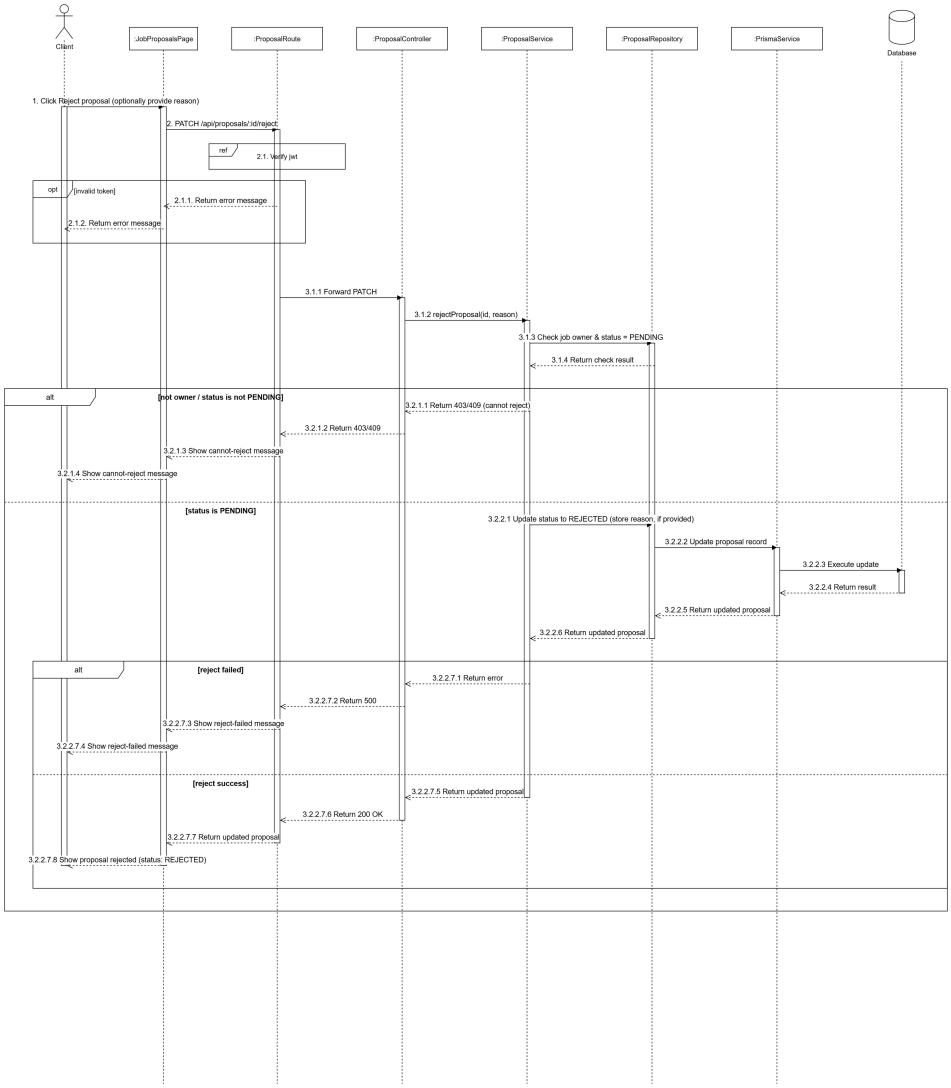

Figure 129: Sequence diagram of UC-32.07 Reject Proposal

3.17.9 UC-32.08 – View Job Proposal List

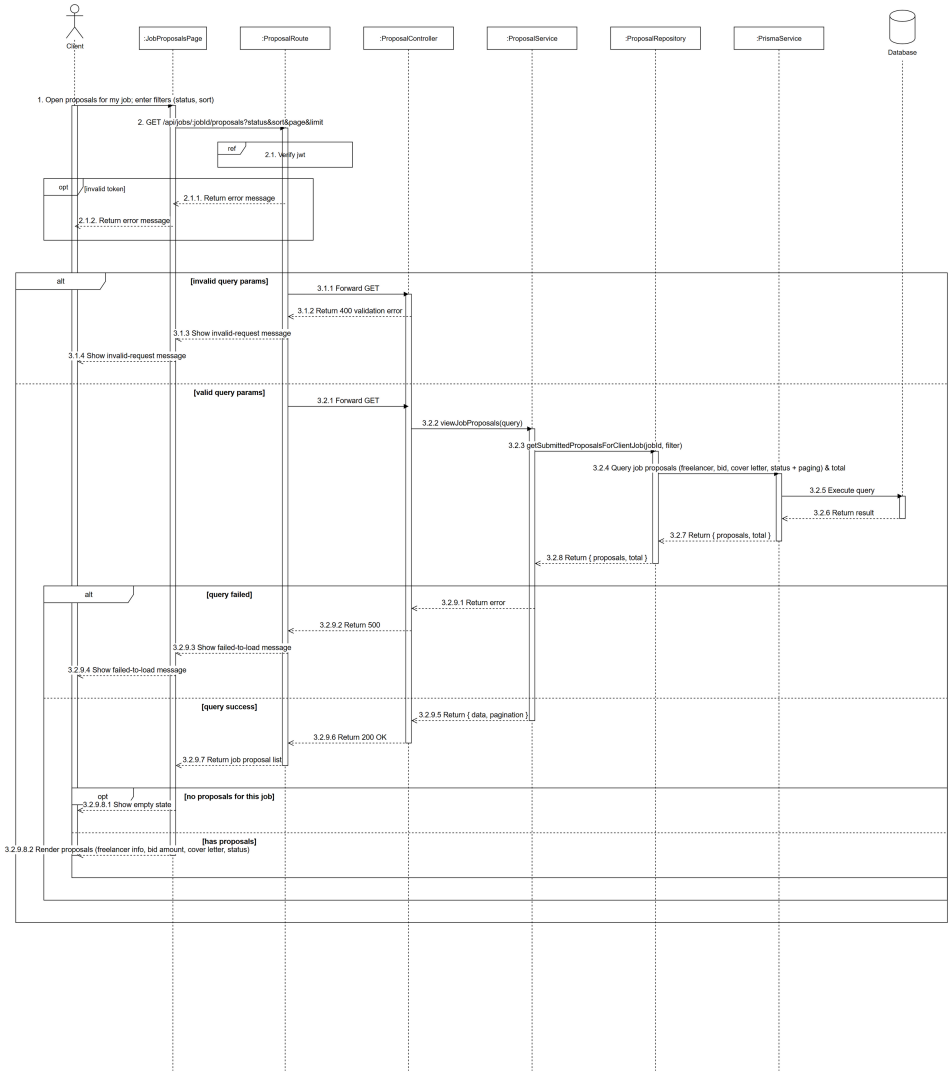

Figure 130: Sequence diagram of UC-32.08 View Job Proposal List

3.17.10 UC-32.09 – View Proposal Detail

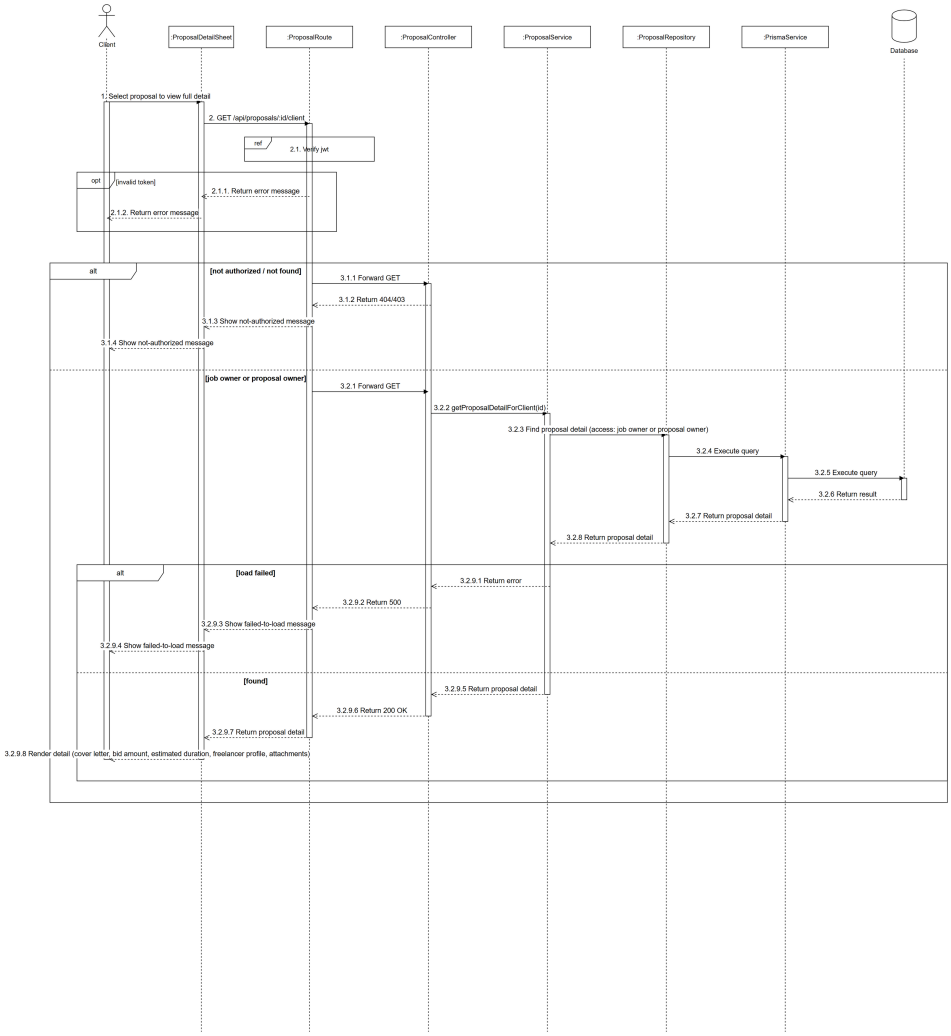

Figure 131: Sequence diagram of UC-32.09 View Proposal Detail

3.18 User Management

User Management provides the administrator with a searchable user catalogue, a consolidated account detail view, and dedicated editors for the Client and Freelancer profiles attached to an account.

3.18.1 Class Diagram

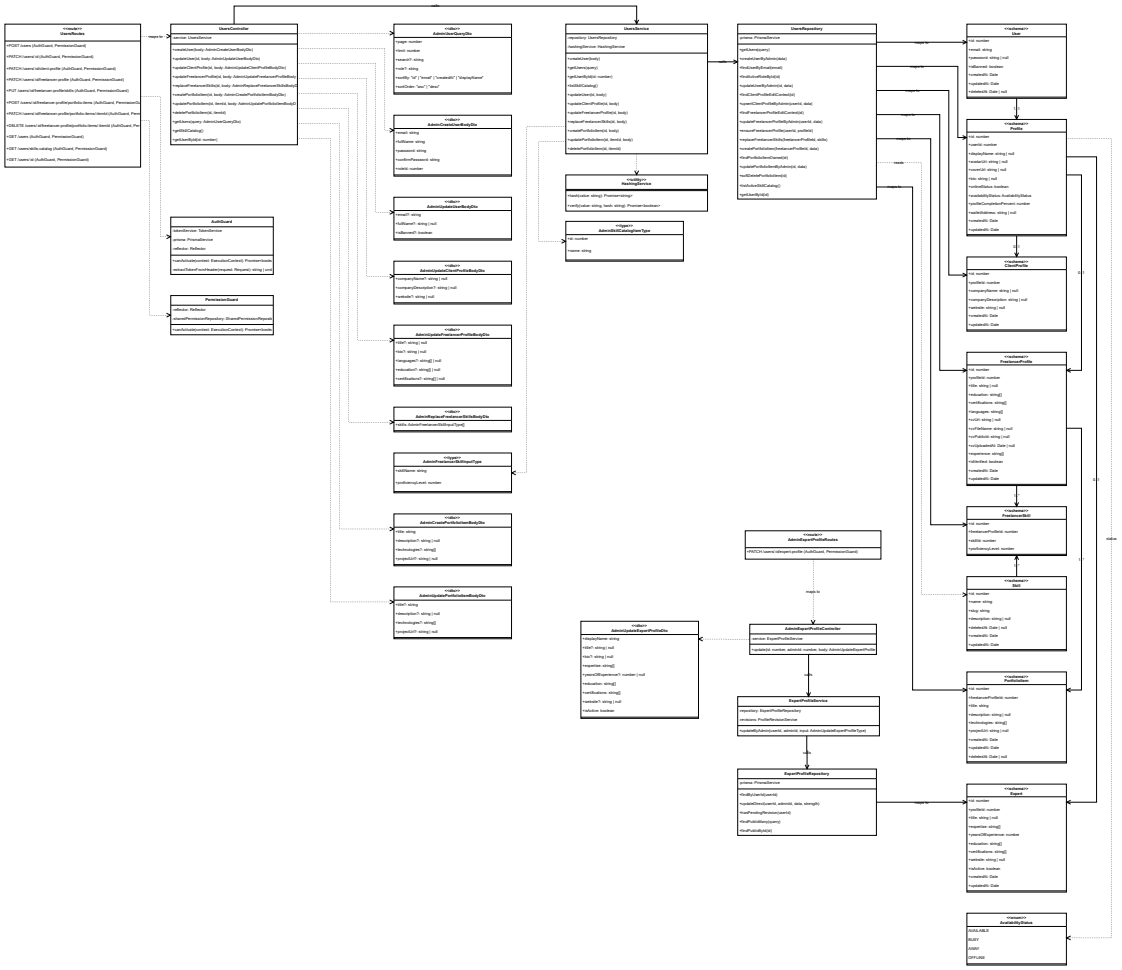

Figure 132: Class diagram of User Management

3.18.2 UC-28.01 – View User List

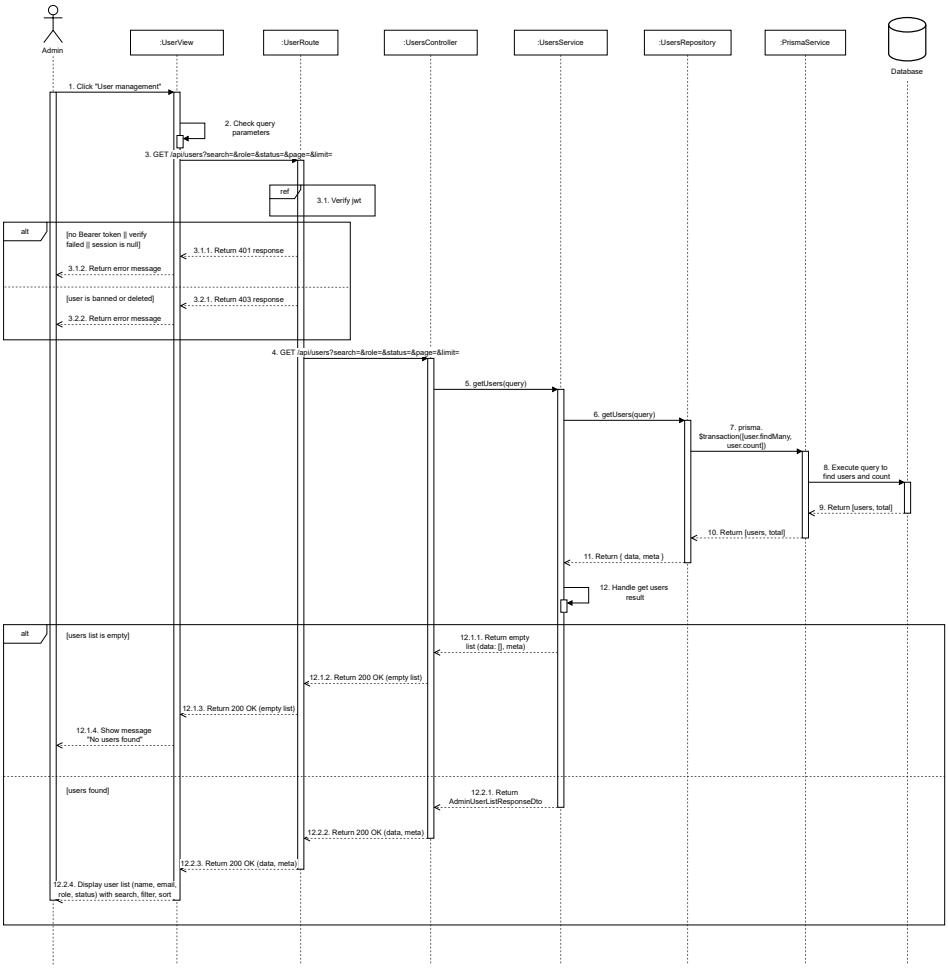

Figure 133: Sequence diagram of UC-28.01 View User List

3.18.3 UC-28.02 – View User Detail

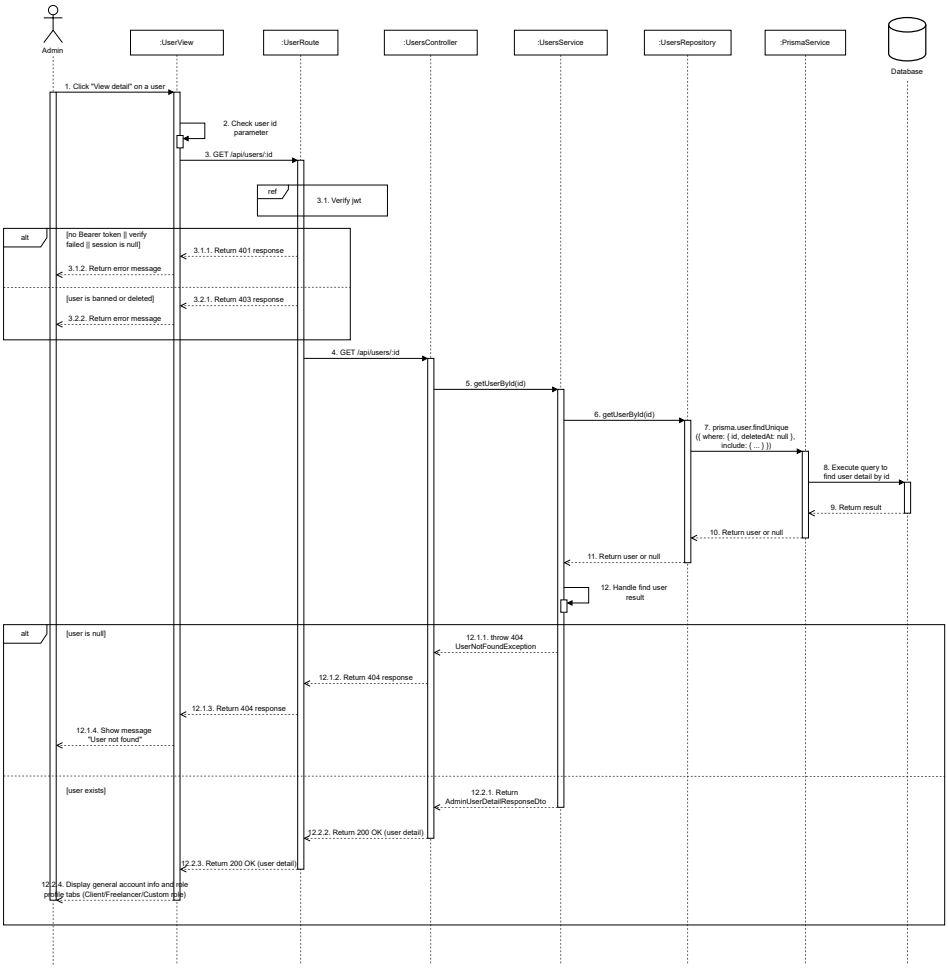

Figure 134: Sequence diagram of UC-28.02 View User Detail

3.18.4 UC-28.03 – Create User

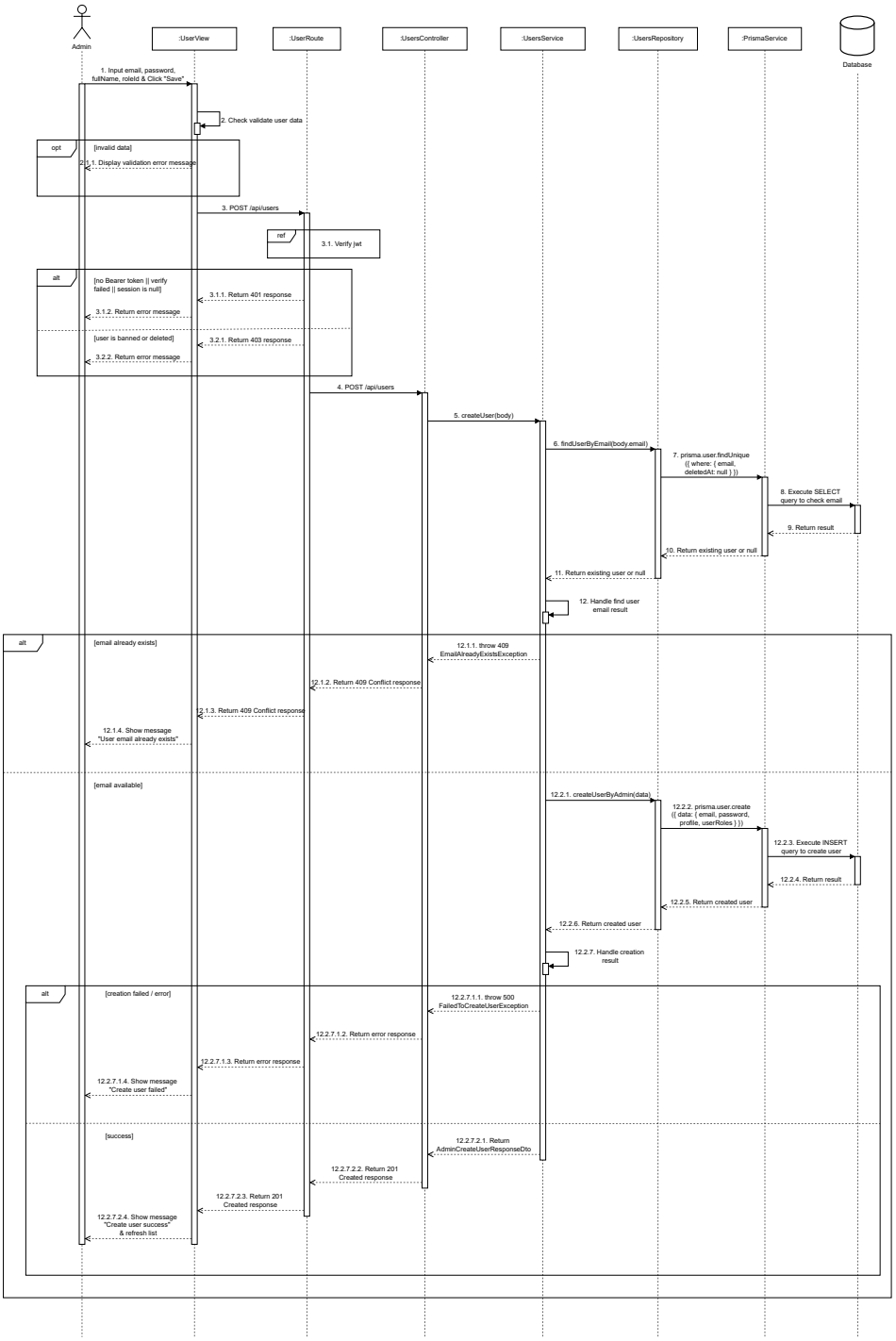

Figure 135: Sequence diagram of UC-28.03 Create User

3.18.5 UC-28.04 – Update User

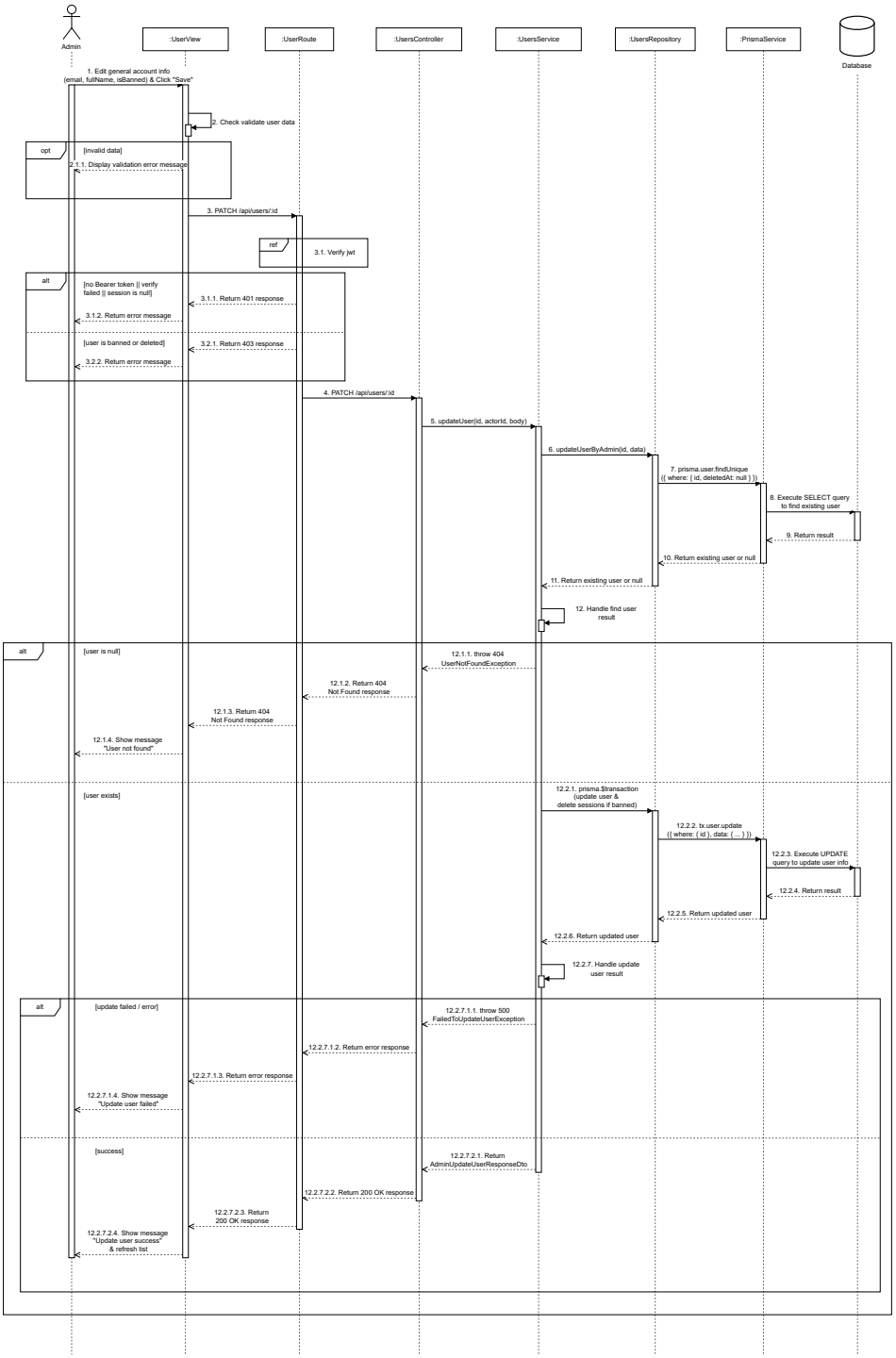

Figure 136: Sequence diagram of UC-28.04 Update User

3.18.6 UC-28.05 – Update Client Profile

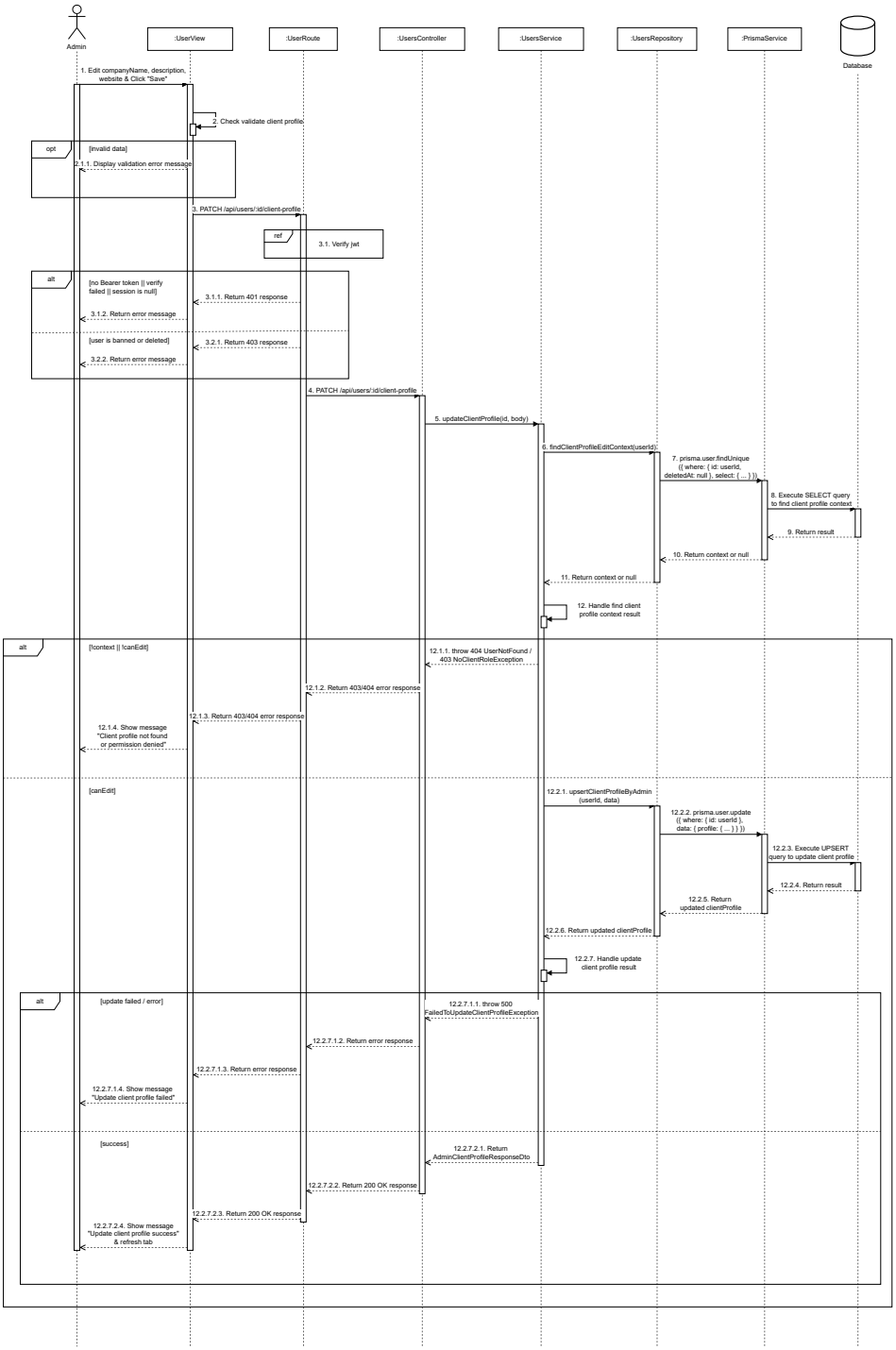

Figure 137: Sequence diagram of UC-28.05 Update Client Profile

3.18.7 UC-28.06 – Update Freelancer Basic Info

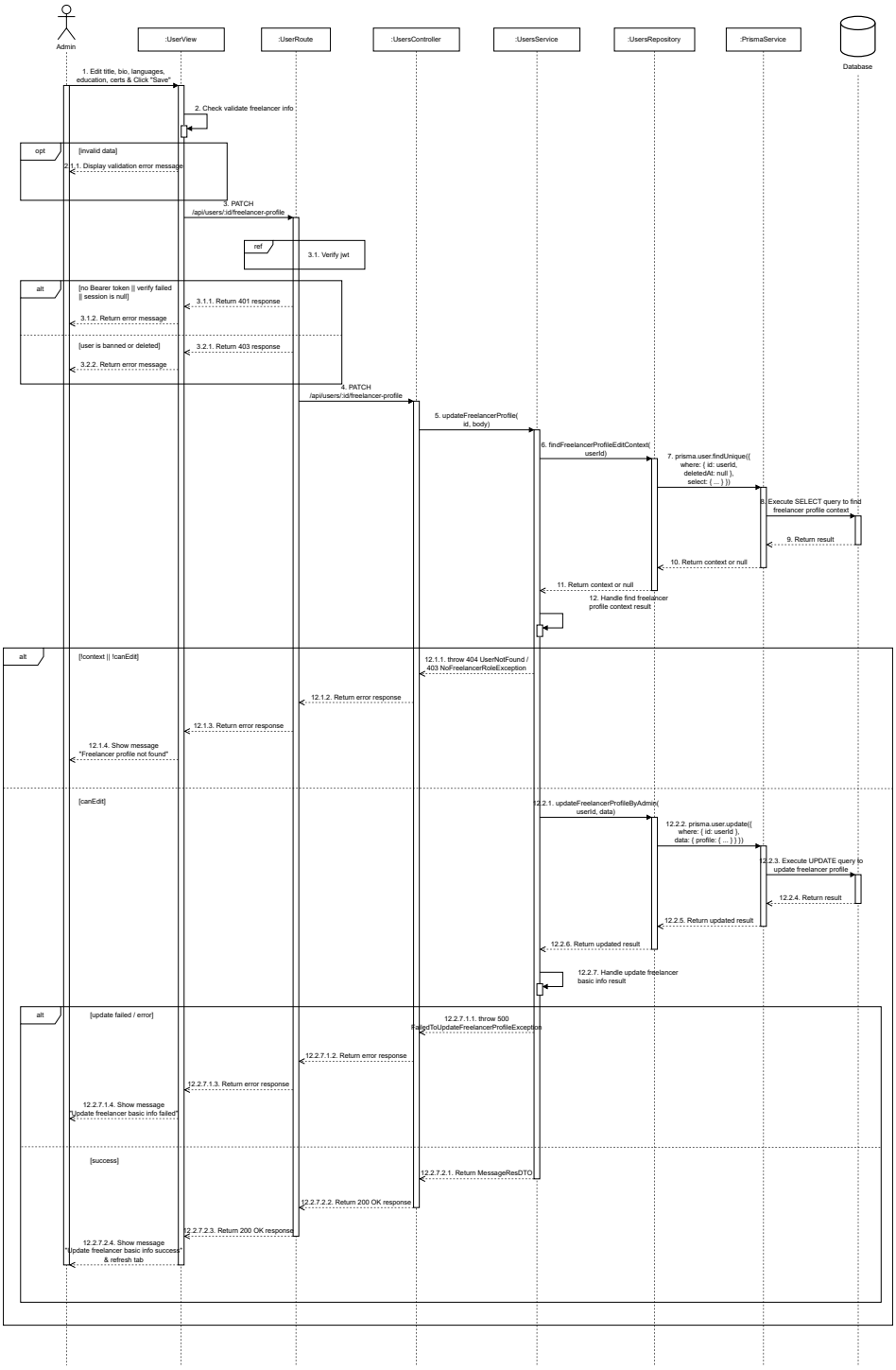

Figure 138: Sequence diagram of UC-28.06 Update Freelancer Basic Info

3.18.8 UC-28.07 – Update Freelancer Skills

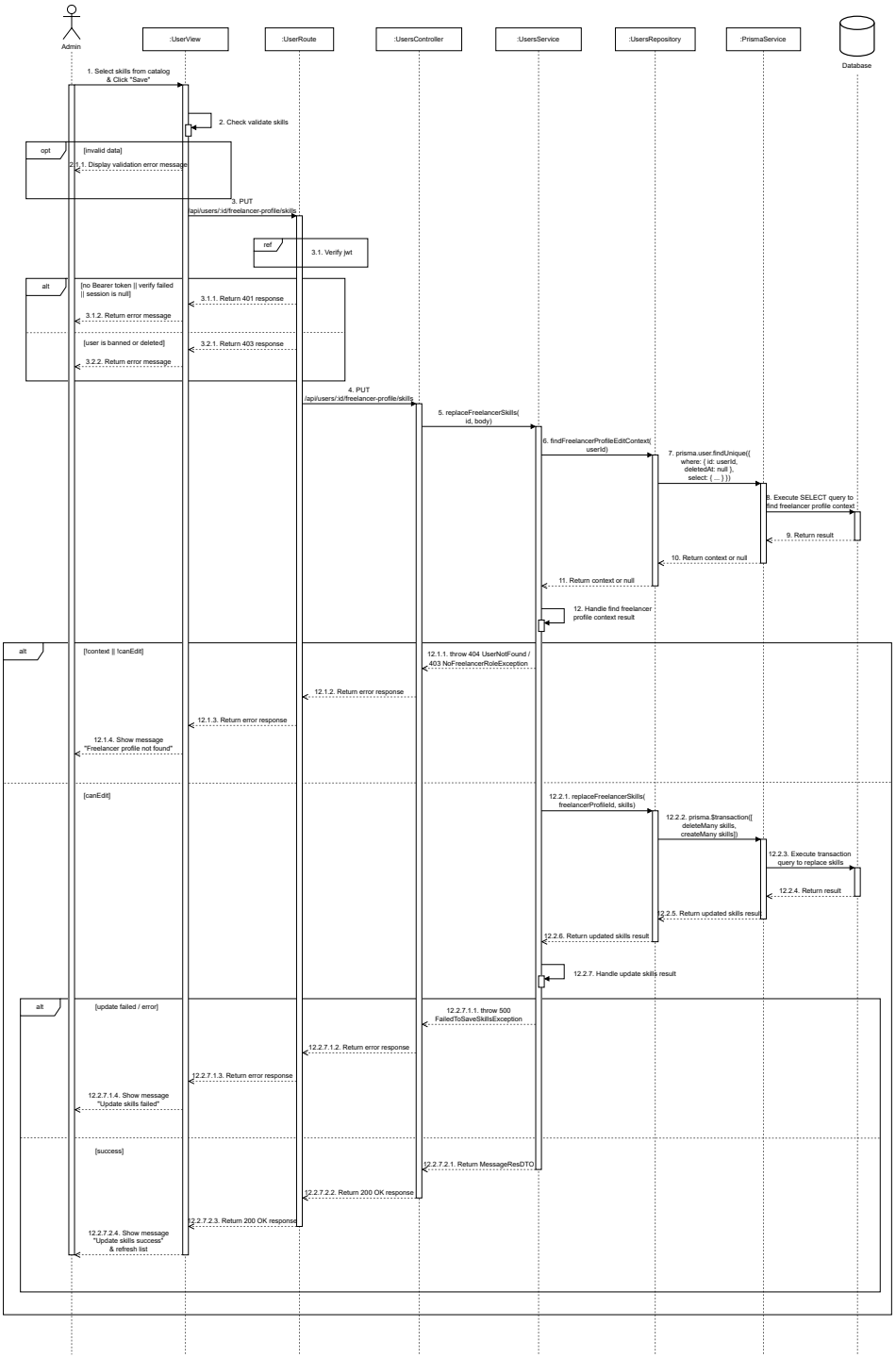

Figure 139: Sequence diagram of UC-28.07 Update Freelancer Skills

3.18.9 UC-28.08 – Update Freelancer Portfolio

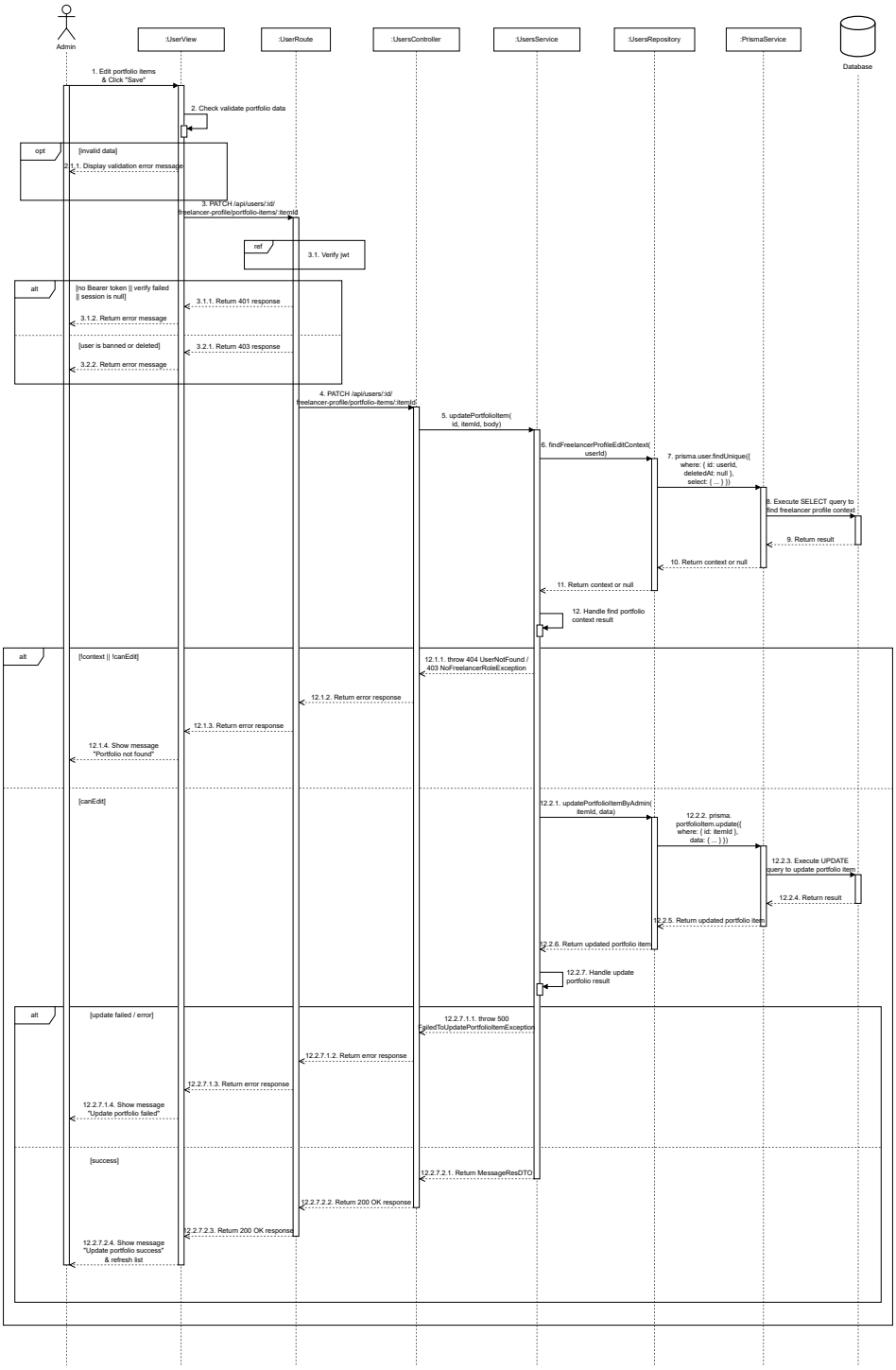

Figure 140: Sequence diagram of UC-28.08 Update Freelancer Portfolio

3.19 Banner Management

Banner Management maintains the advertisement banners shown in configured page positions. The module supports image upload, lifecycle dates, activation state, and hard deletion.

3.19.1 Class Diagram

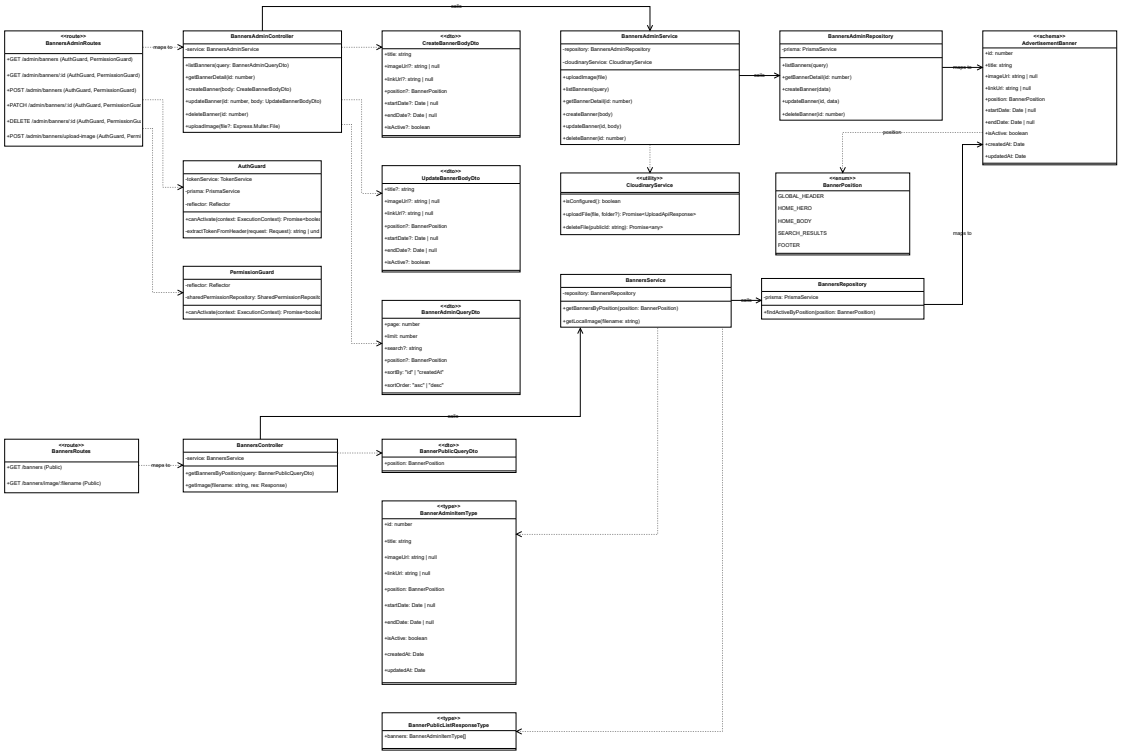

Figure 141: Class diagram of Banner Management

3.19.2 UC-37.01 – View Banner List

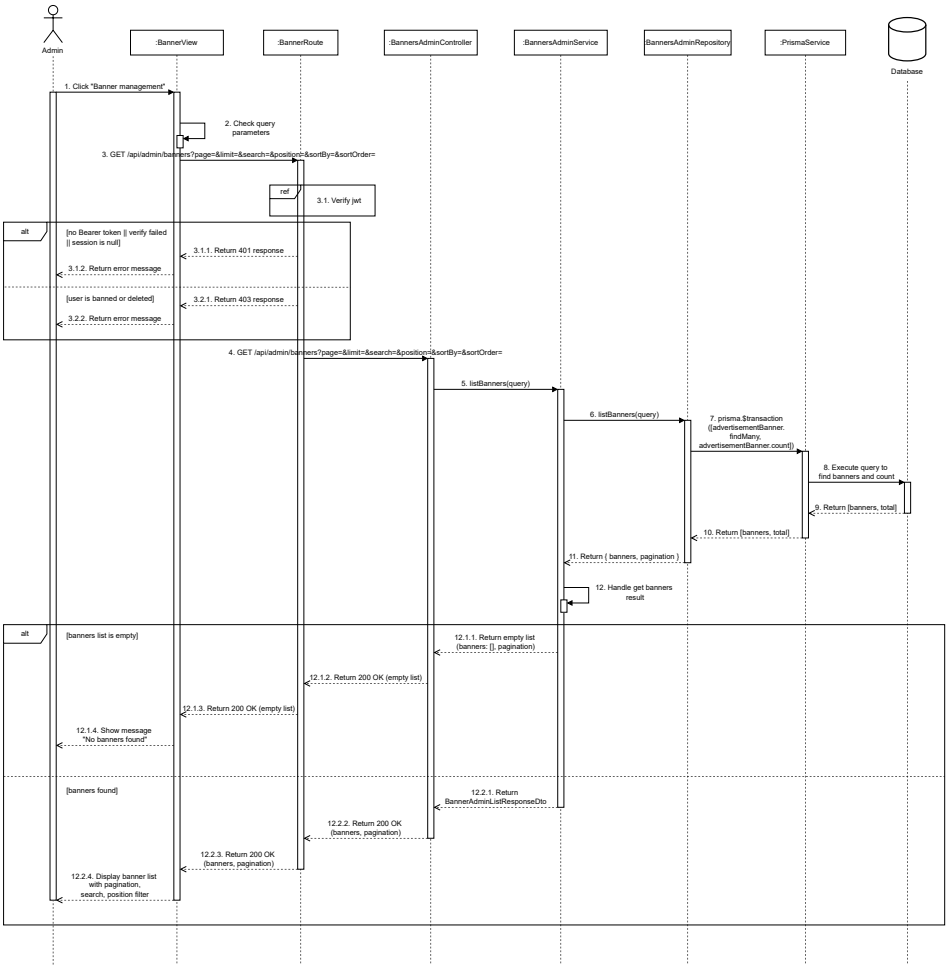

Figure 142: Sequence diagram of UC-37.01 View Banner List

3.19.3 UC-37.02 – View Banner Detail

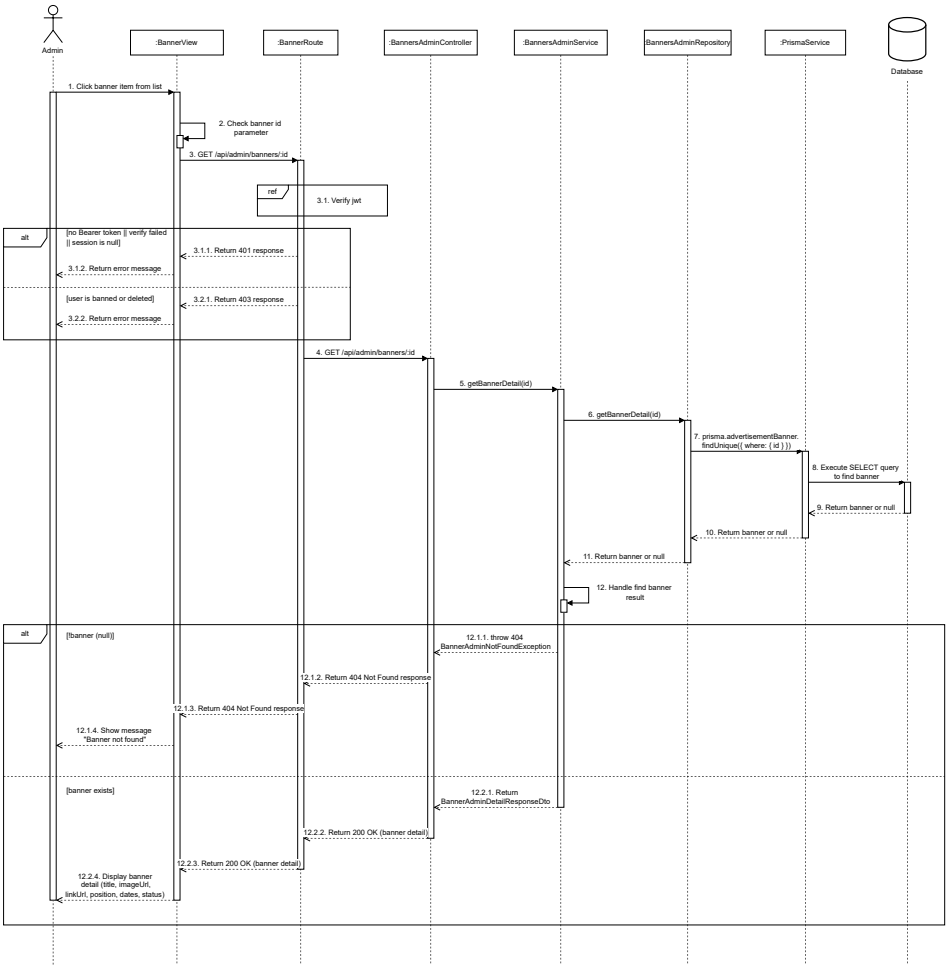

Figure 143: Sequence diagram of UC-37.02 View Banner Detail

3.19.4 UC-37.03 – Create Banner

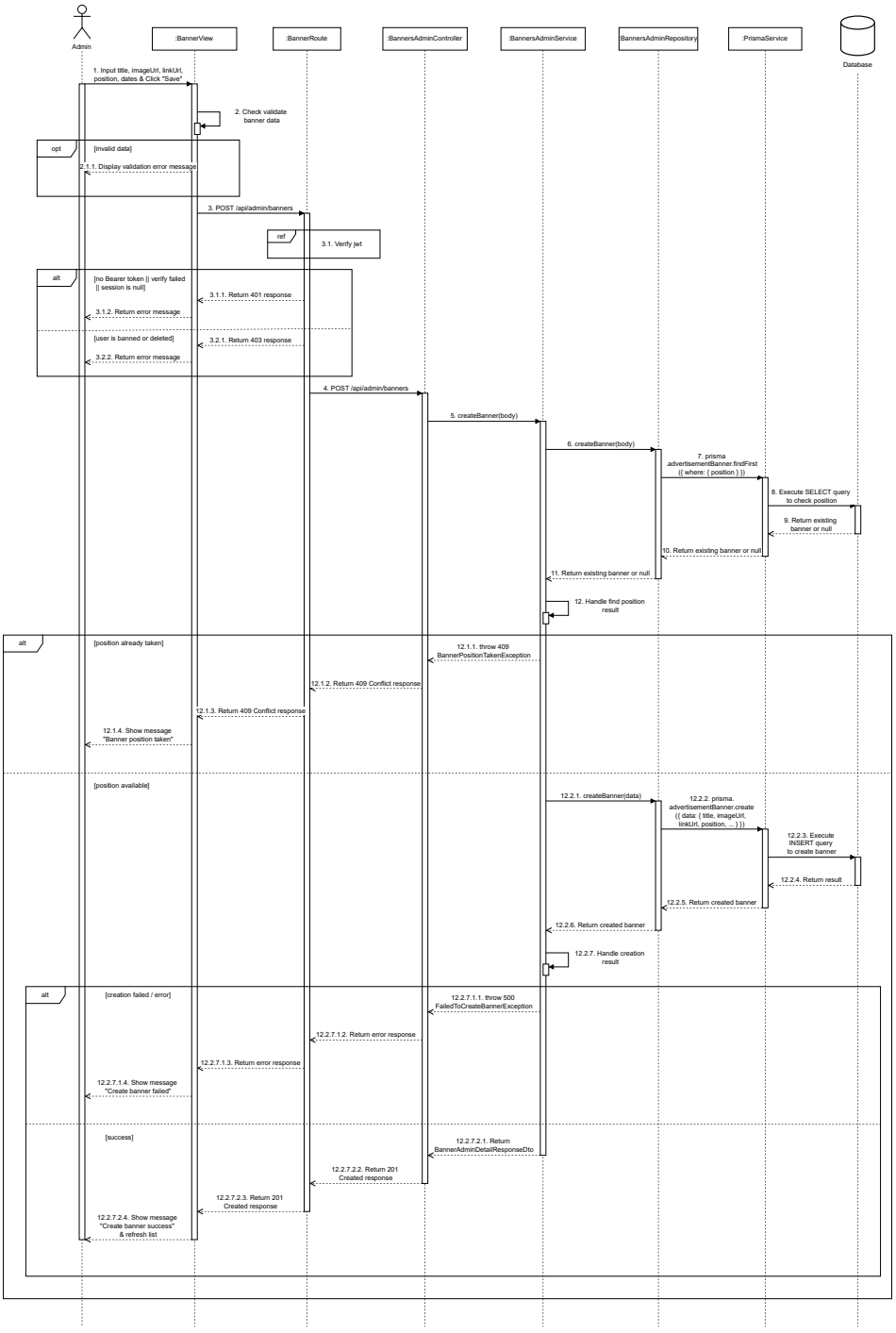

Figure 144: Sequence diagram of UC-37.03 Create Banner

3.19.5 UC-37.04 – Update Banner

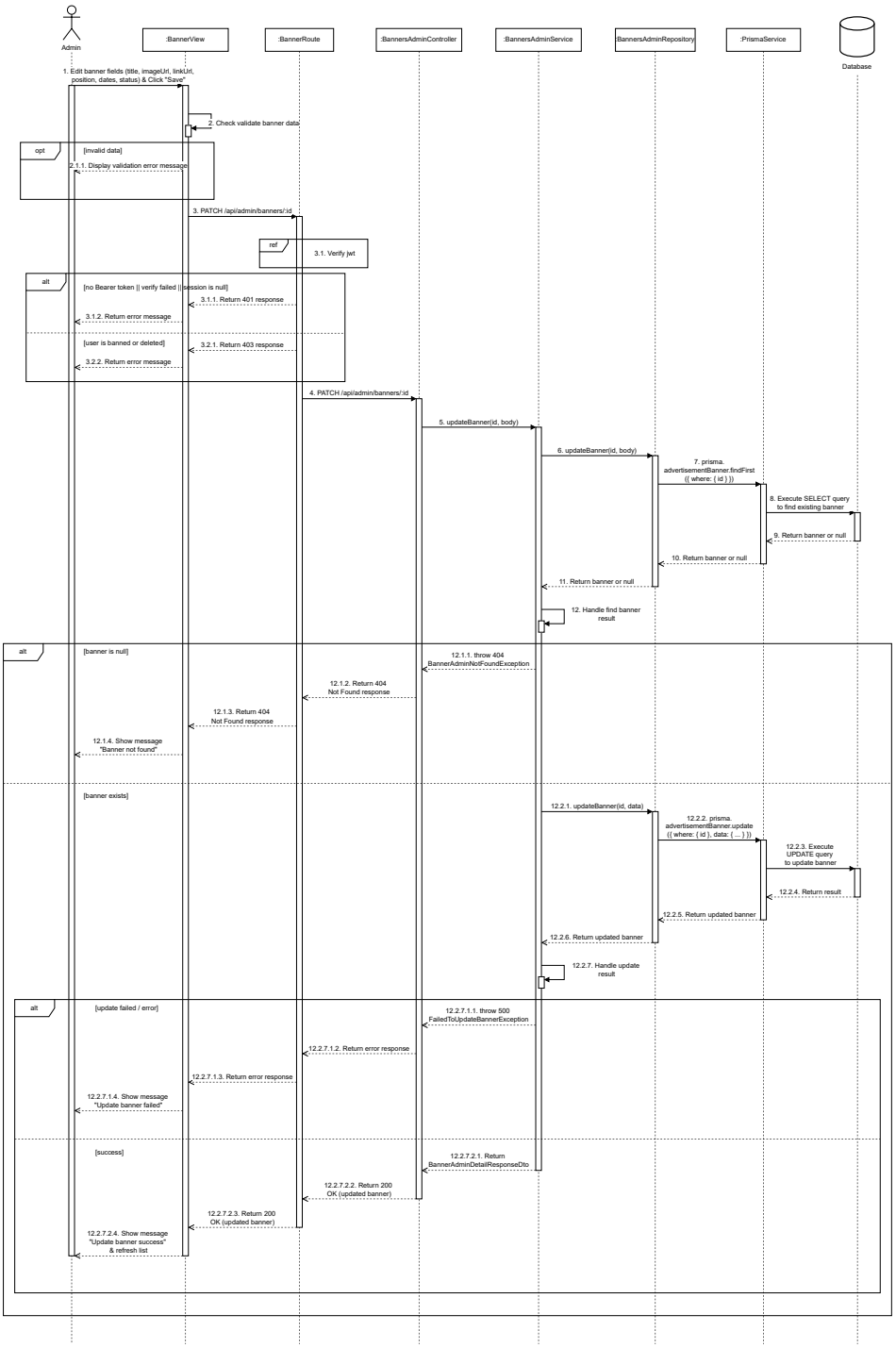

Figure 145: Sequence diagram of UC-37.04 Update Banner

3.19.6 UC-37.05 – Delete Banner

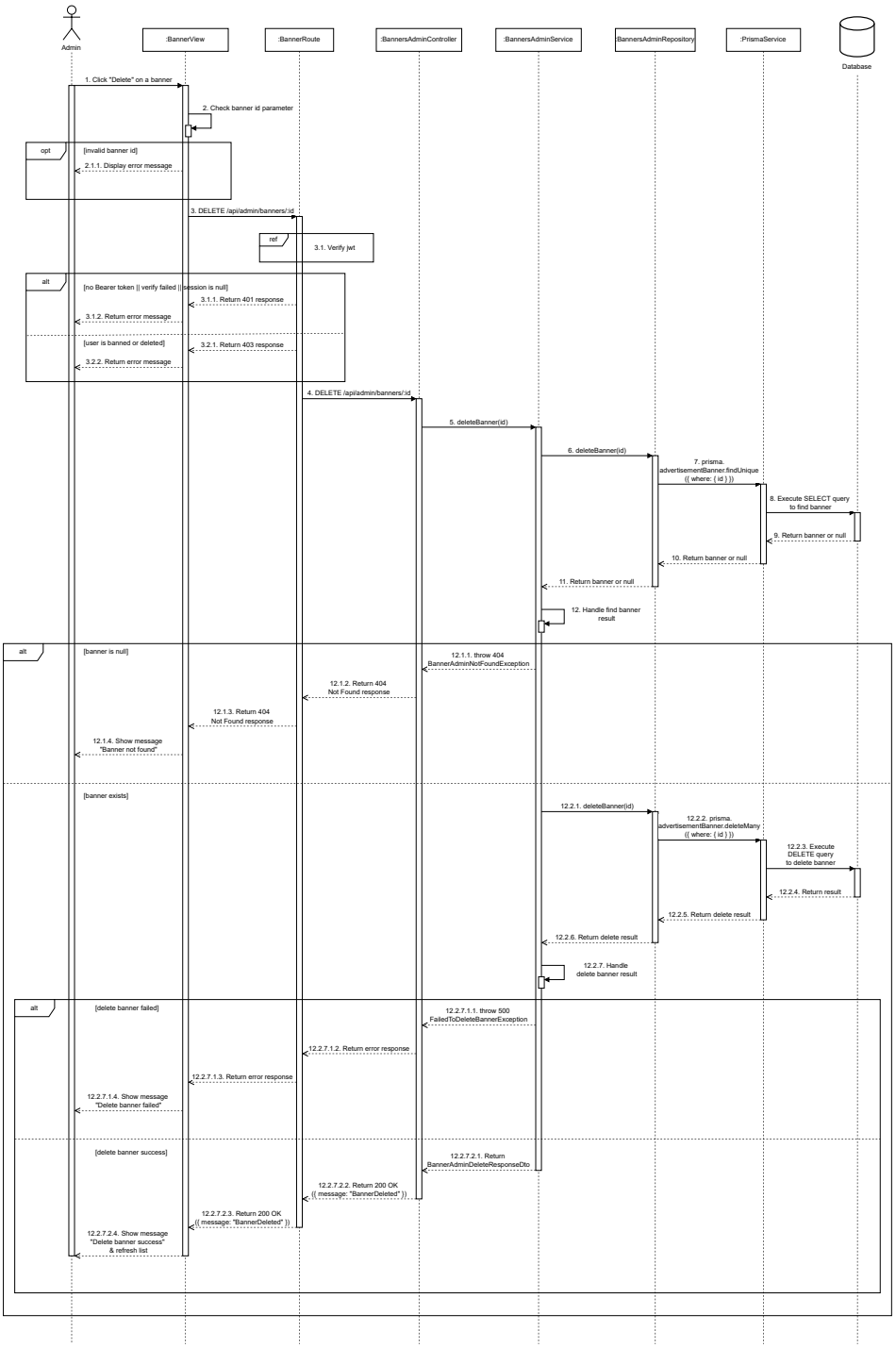

Figure 146: Sequence diagram of UC-37.05 Delete Banner

3.20 Skill Management

Skill Management owns the master skill catalogue used by freelancer profiles and job postings. Skills are soft-deleted so that existing references remain valid, and a deleted skill can be restored.

3.20.1 Class Diagram

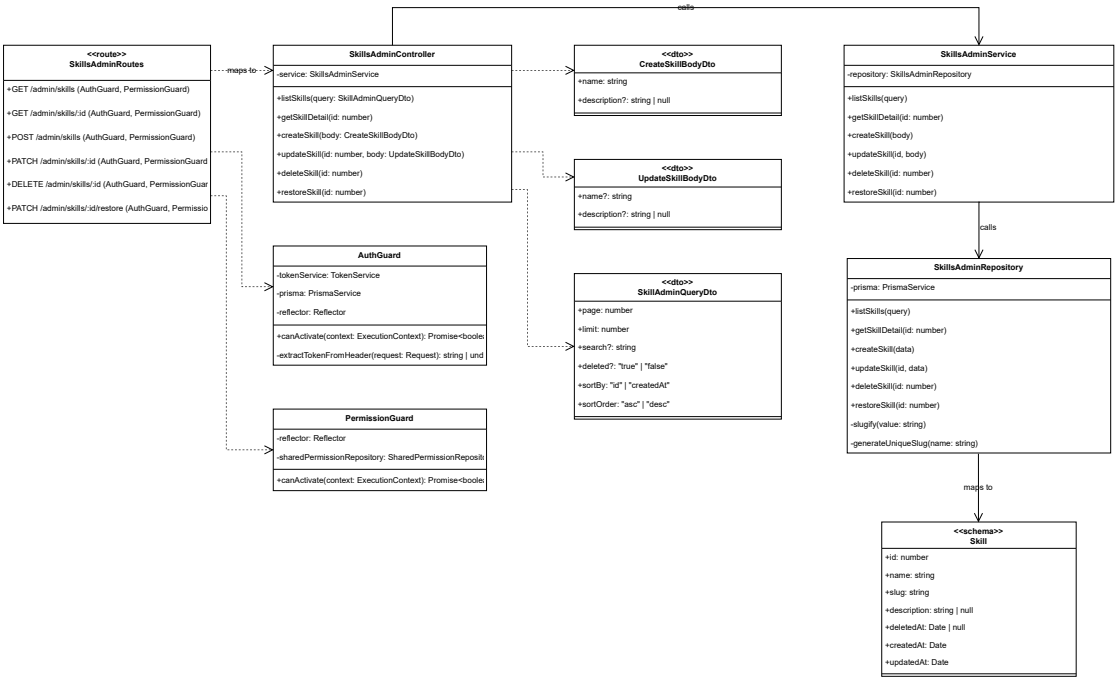

Figure 147: Class diagram of Skill Management

3.20.2 UC-38.01 – View Skill List

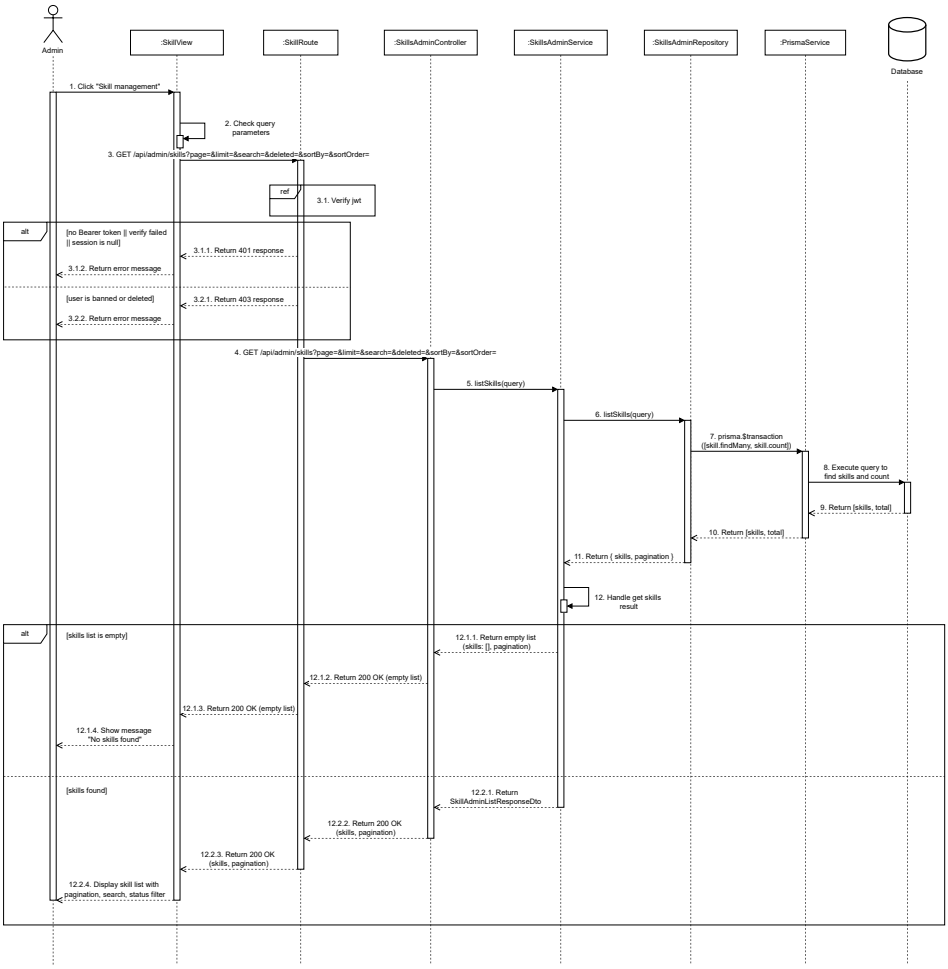

Figure 148: Sequence diagram of UC-38.01 View Skill List

3.20.3 UC-38.02 – View Skill Detail

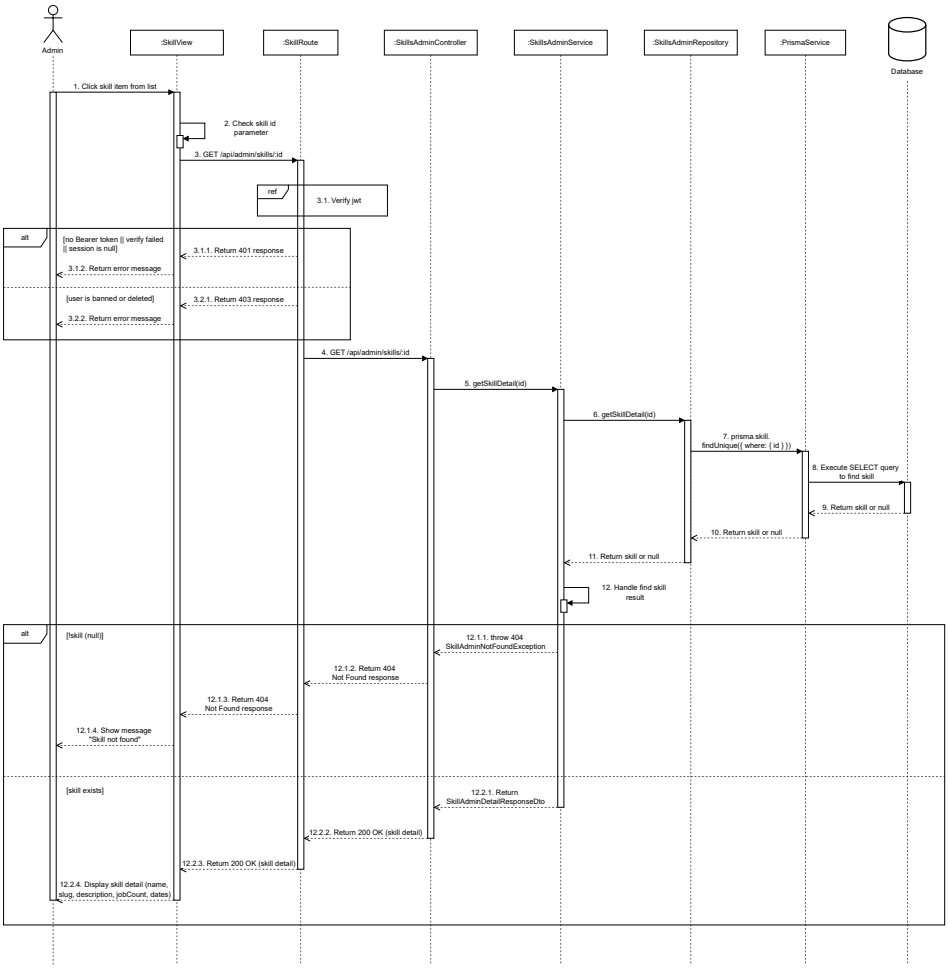

Figure 149: Sequence diagram of UC-38.02 View Skill Detail

3.20.4 UC-38.03 – Create Skill

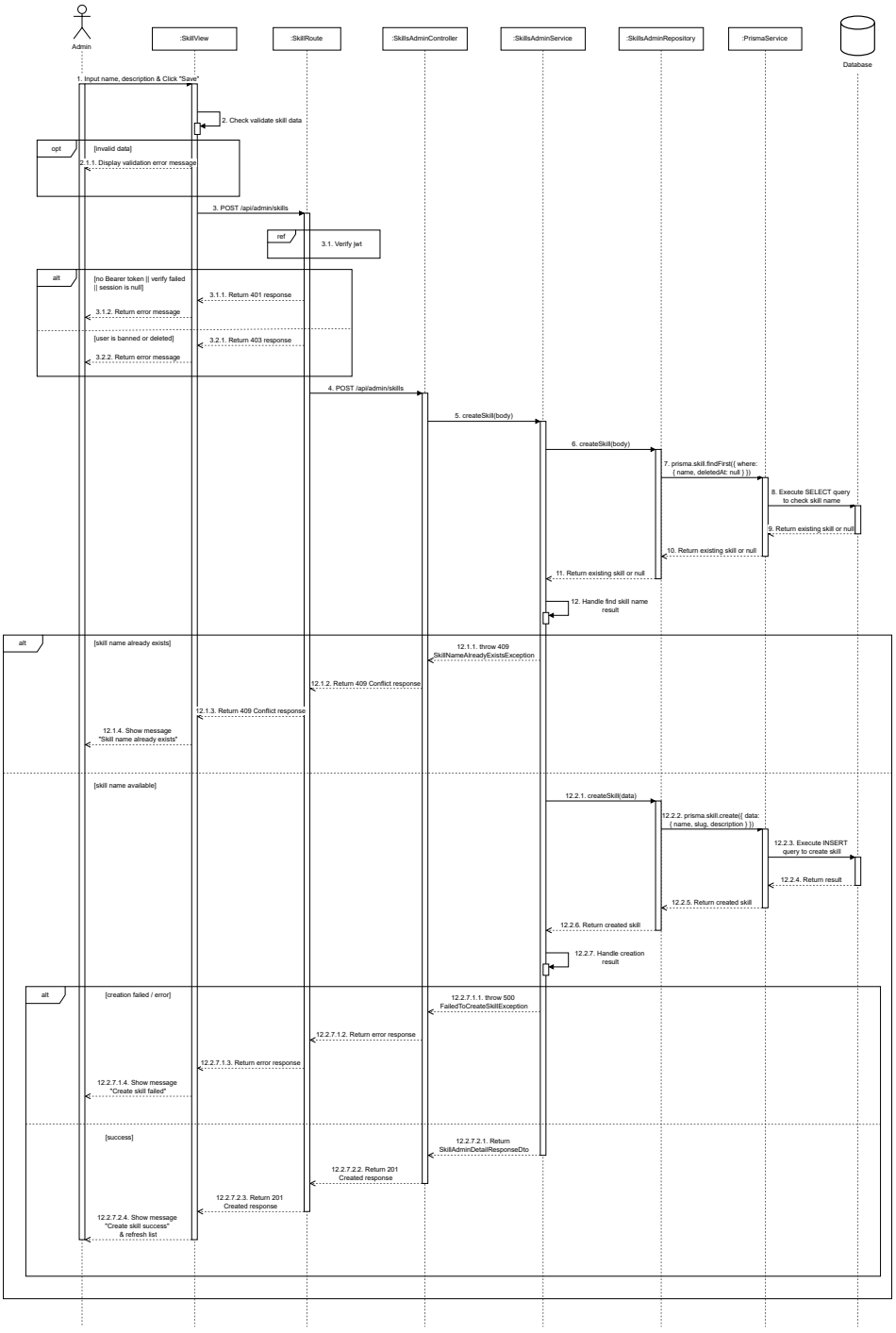

Figure 150: Sequence diagram of UC-38.03 Create Skill

3.20.5 UC-38.04 – Update Skill

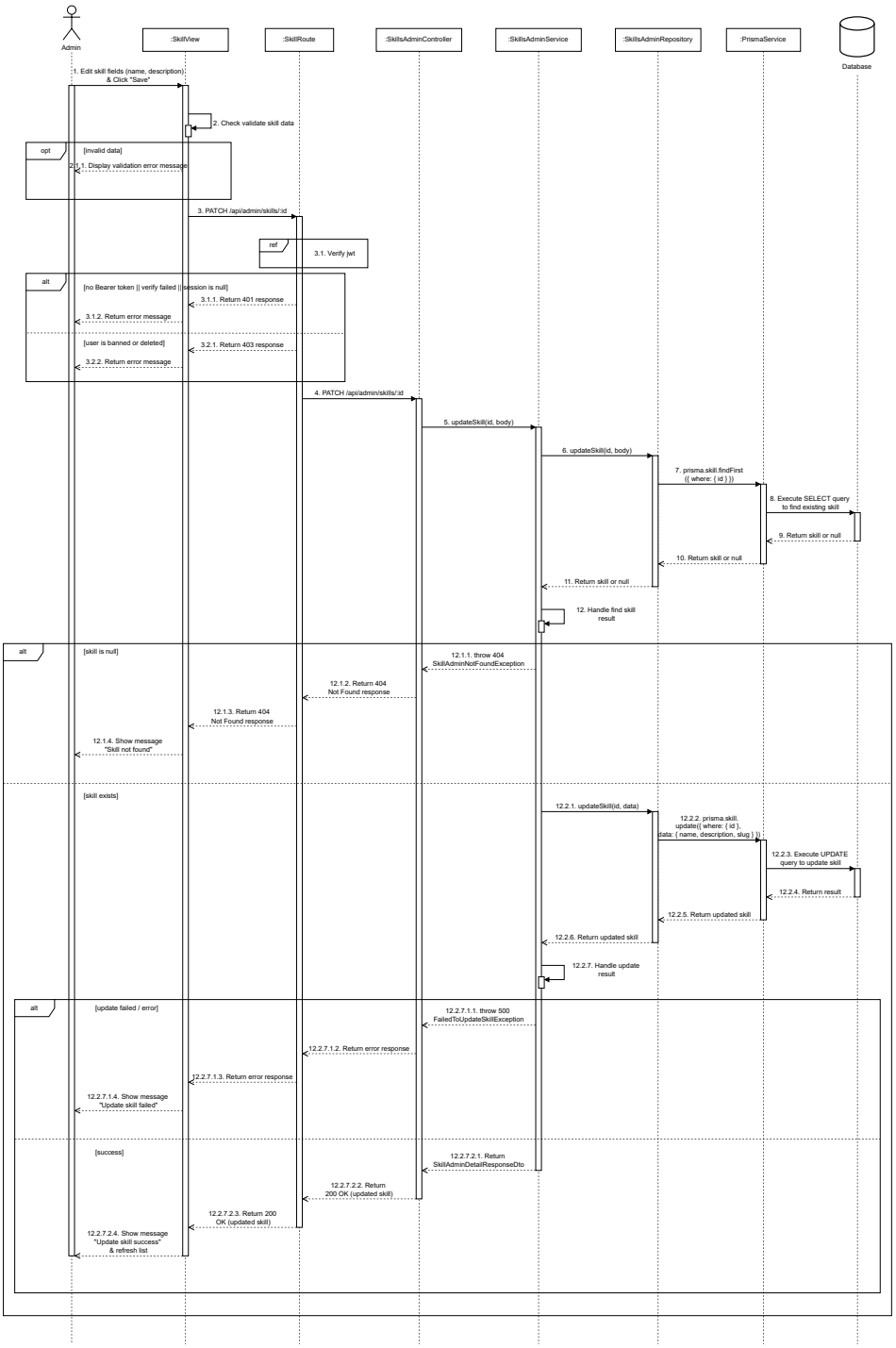

Figure 151: Sequence diagram of UC-38.04 Update Skill

3.20.6 UC-38.05 – Delete Skill

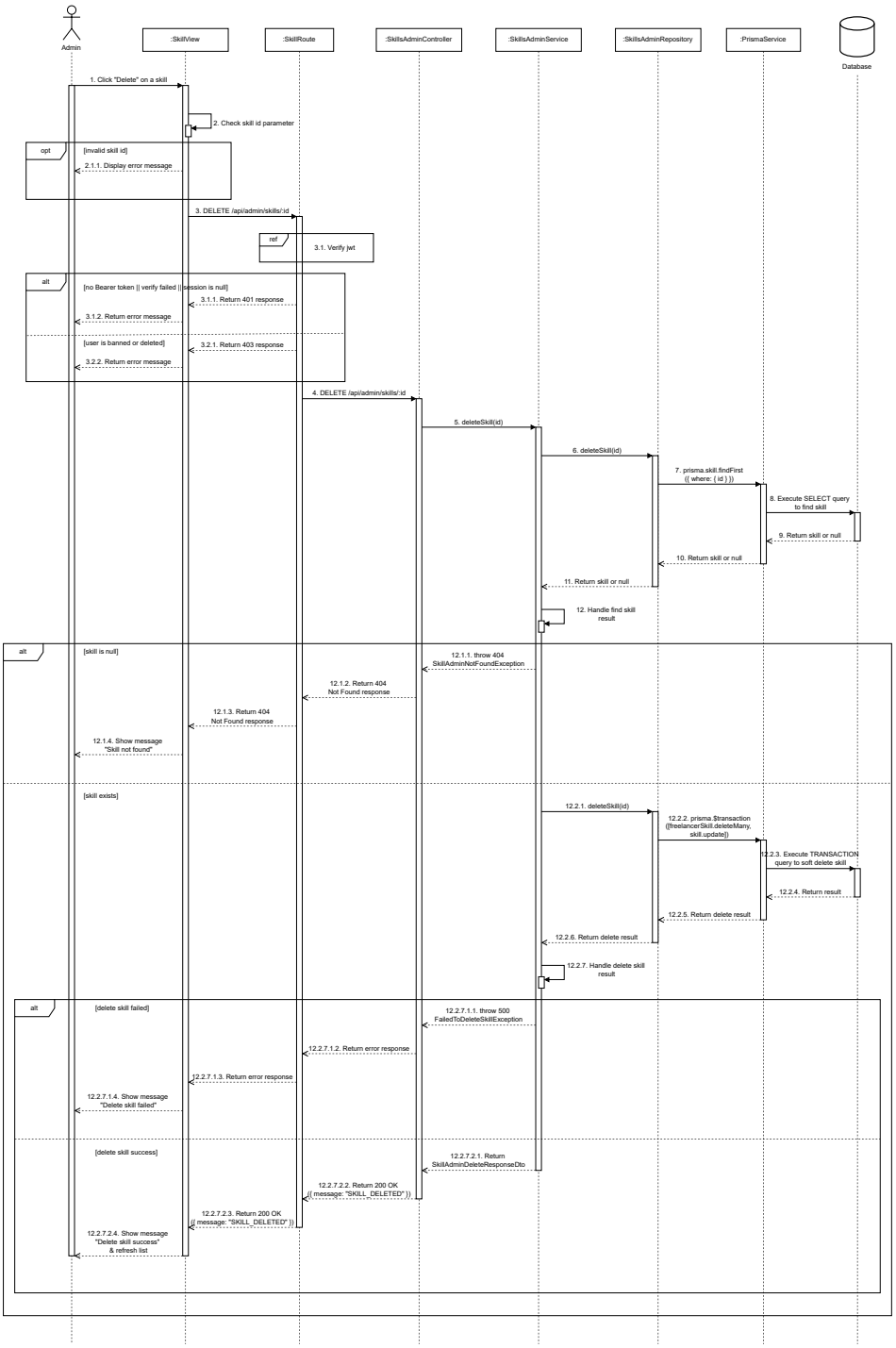

Figure 152: Sequence diagram of UC-38.05 Delete Skill

3.20.7 UC-38.06 – Restore Skill

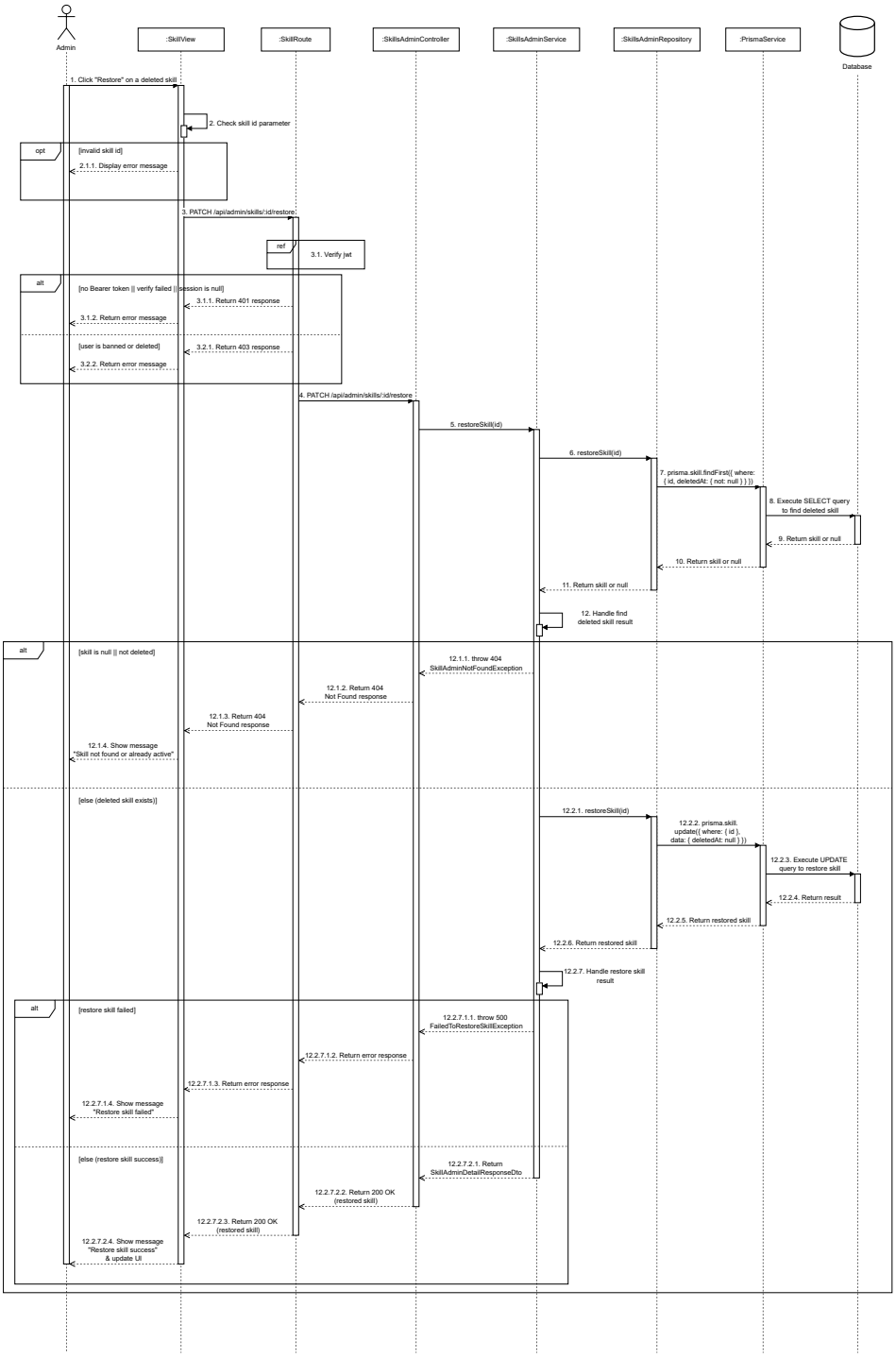

Figure 153: Sequence diagram of UC-38.06 Restore Skill

3.21 Expert Profile Management

The administrator-side Expert Profile editor updates an expert account directly. Any pending self-service revision is superseded, and the account owner is notified of the administrative change.

3.21.1 Class Diagram

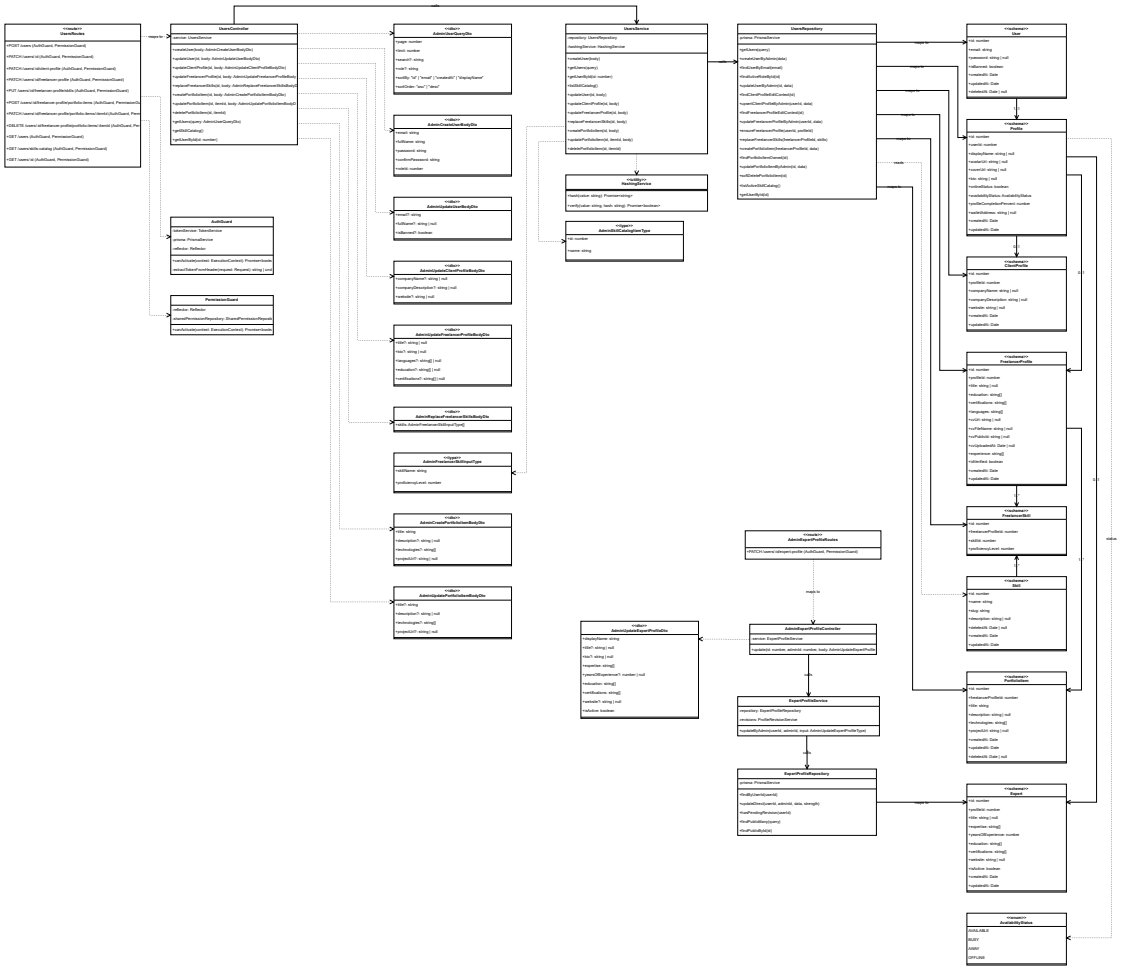

Figure 154: Class diagram of Expert Profile Management

3.21.2 UC-28.09 – Update Expert Profile

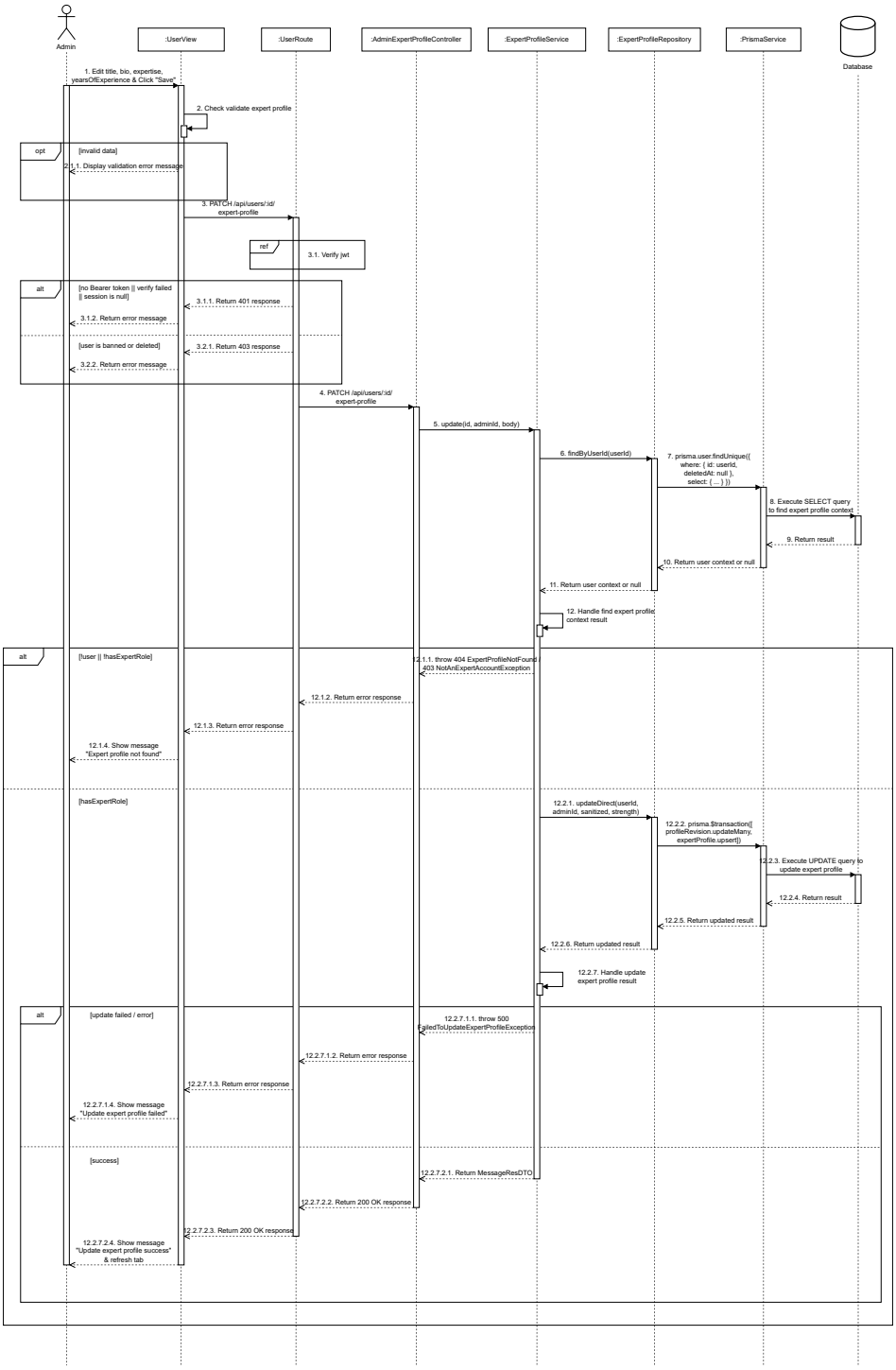

Figure 155: Sequence diagram of UC-28.09 Update Expert Profile

3.22 Job Category Management

Job Category Management maintains the taxonomy used by job postings. A category can be created, updated, soft-deleted, and restored; deletion is blocked while active jobs still reference the category.

3.22.1 Class Diagram

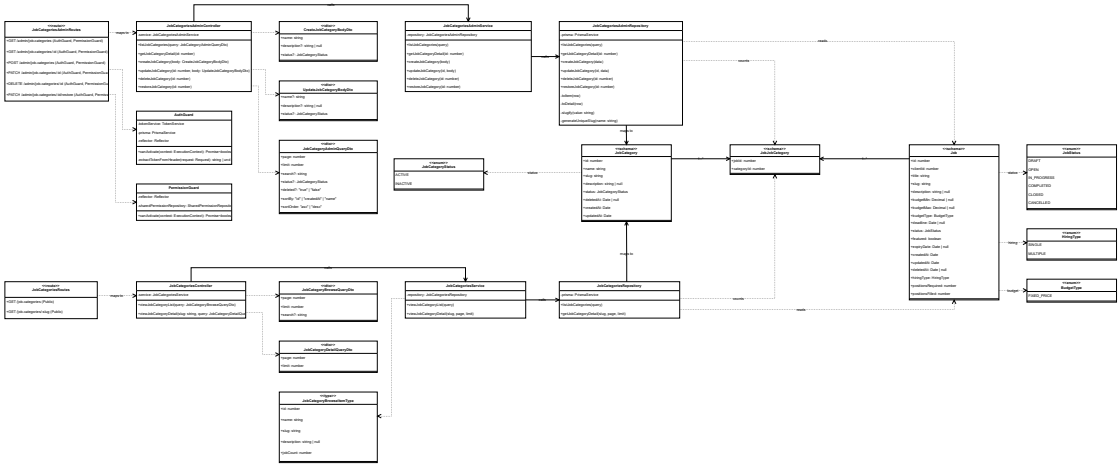

Figure 156: Class diagram of Job Category Management

3.22.2 UC-46.01 – View Job Category List

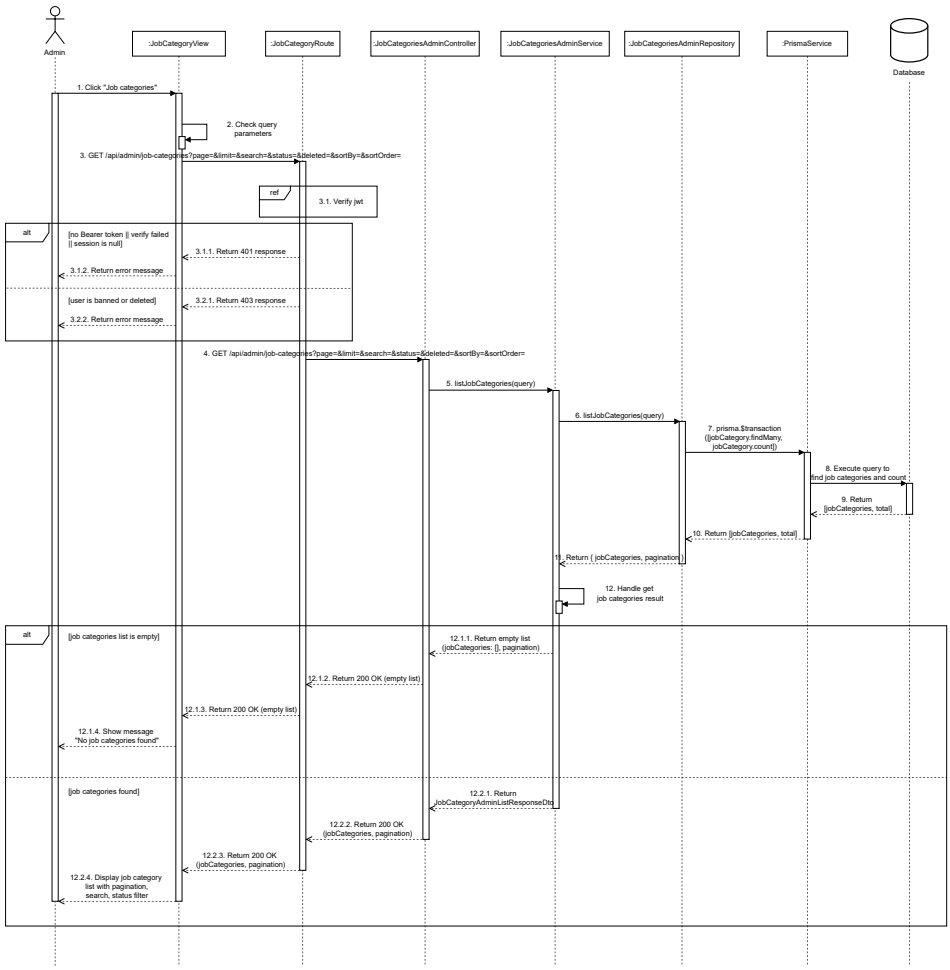

Figure 157: Sequence diagram of UC-46.01 View Job Category List

3.22.3 UC-46.02 – View Job Category Detail

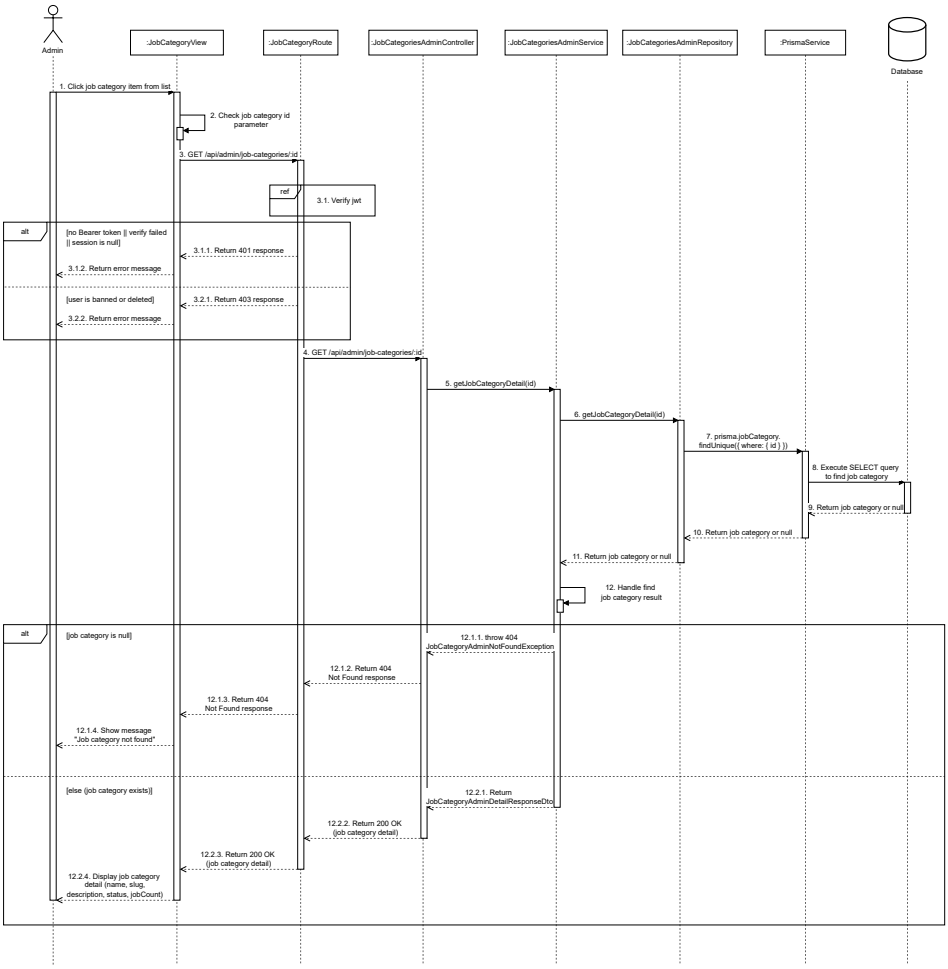

Figure 158: Sequence diagram of UC-46.02 View Job Category Detail

3.22.4 UC-46.03 – Create Job Category

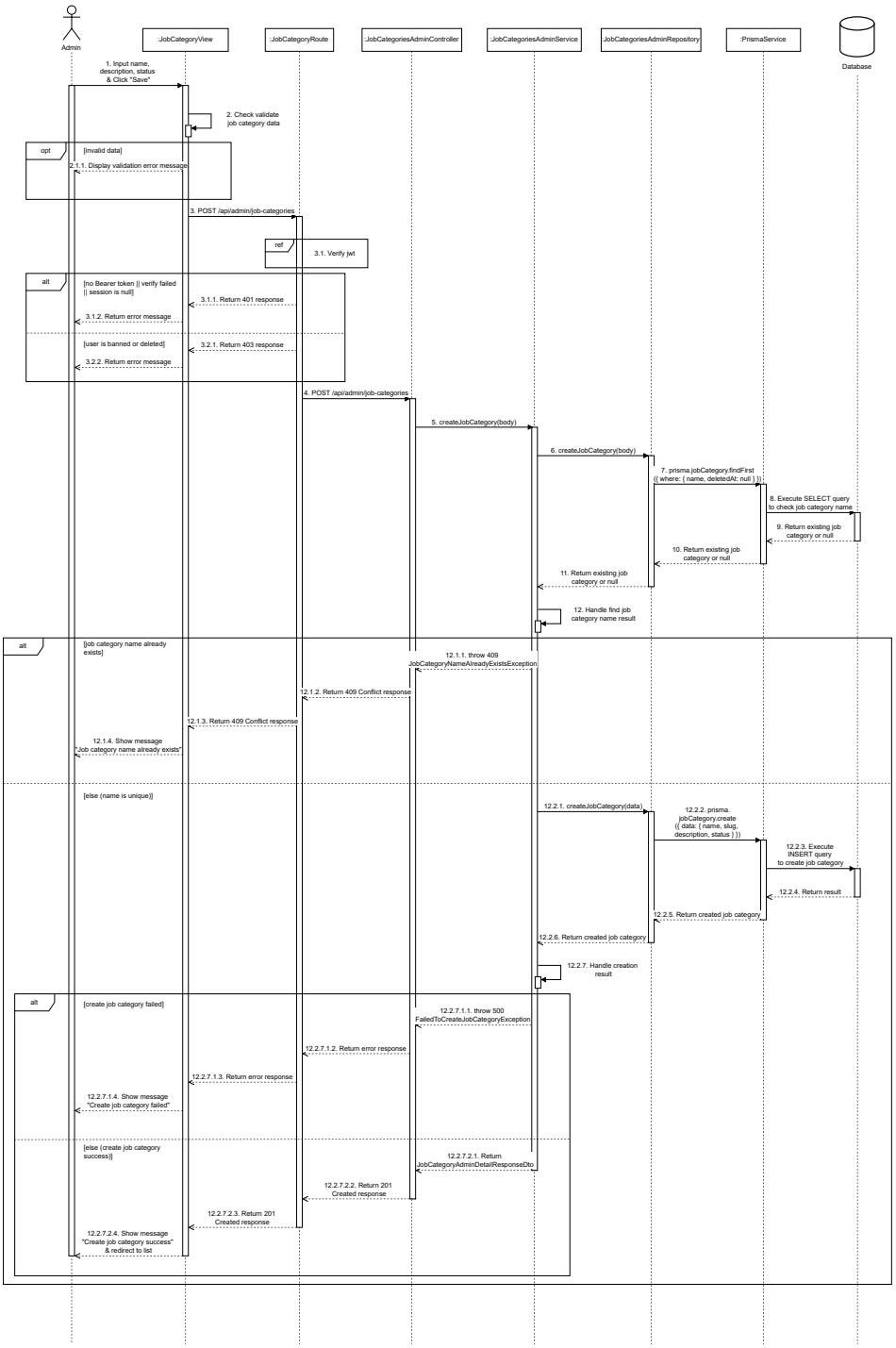

Figure 159: Sequence diagram of UC-46.03 Create Job Category

3.22.5 UC-46.04 – Update Job Category

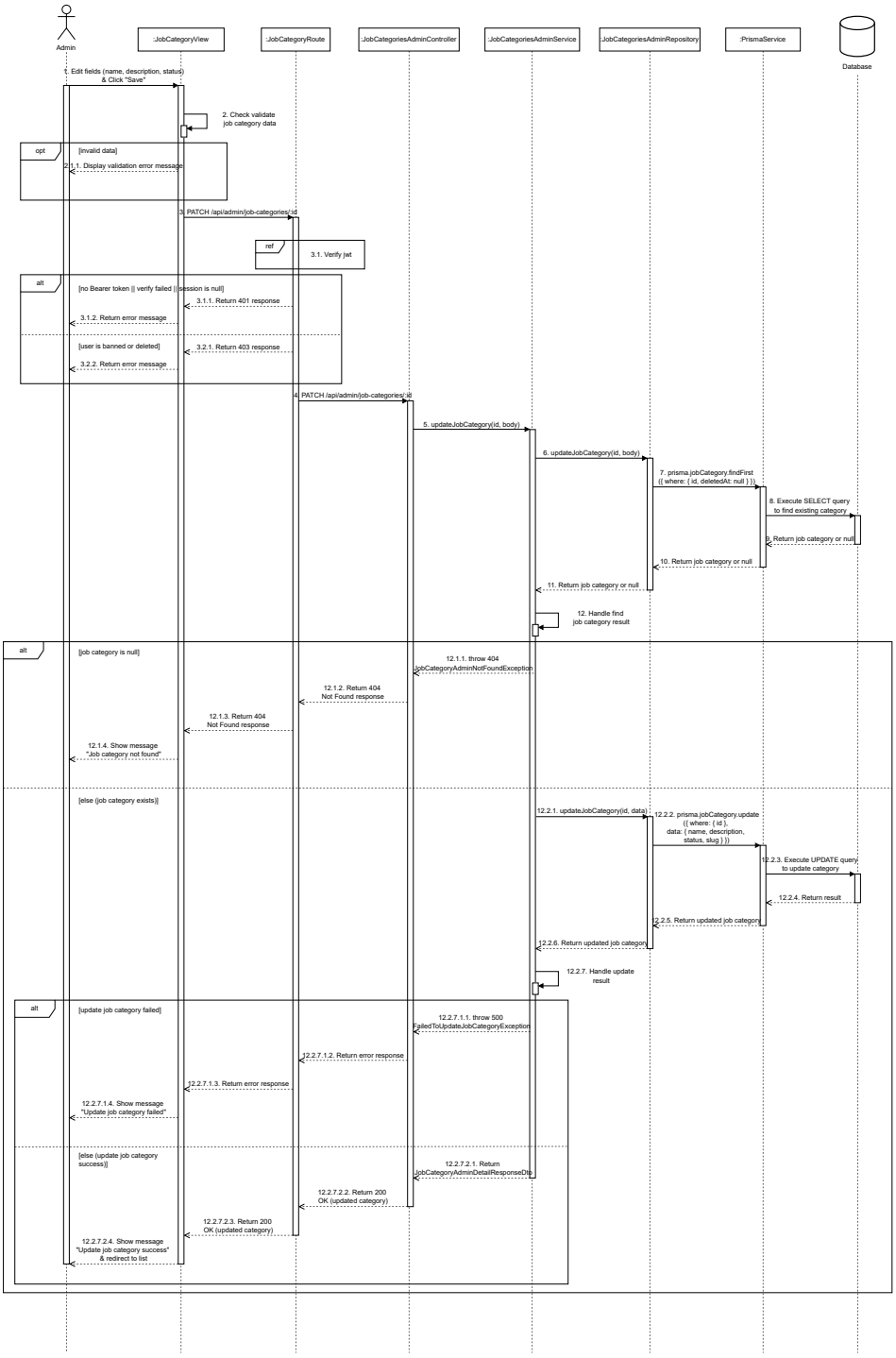

Figure 160: Sequence diagram of UC-46.04 Update Job Category

3.22.6 UC-46.05 – Delete Job Category

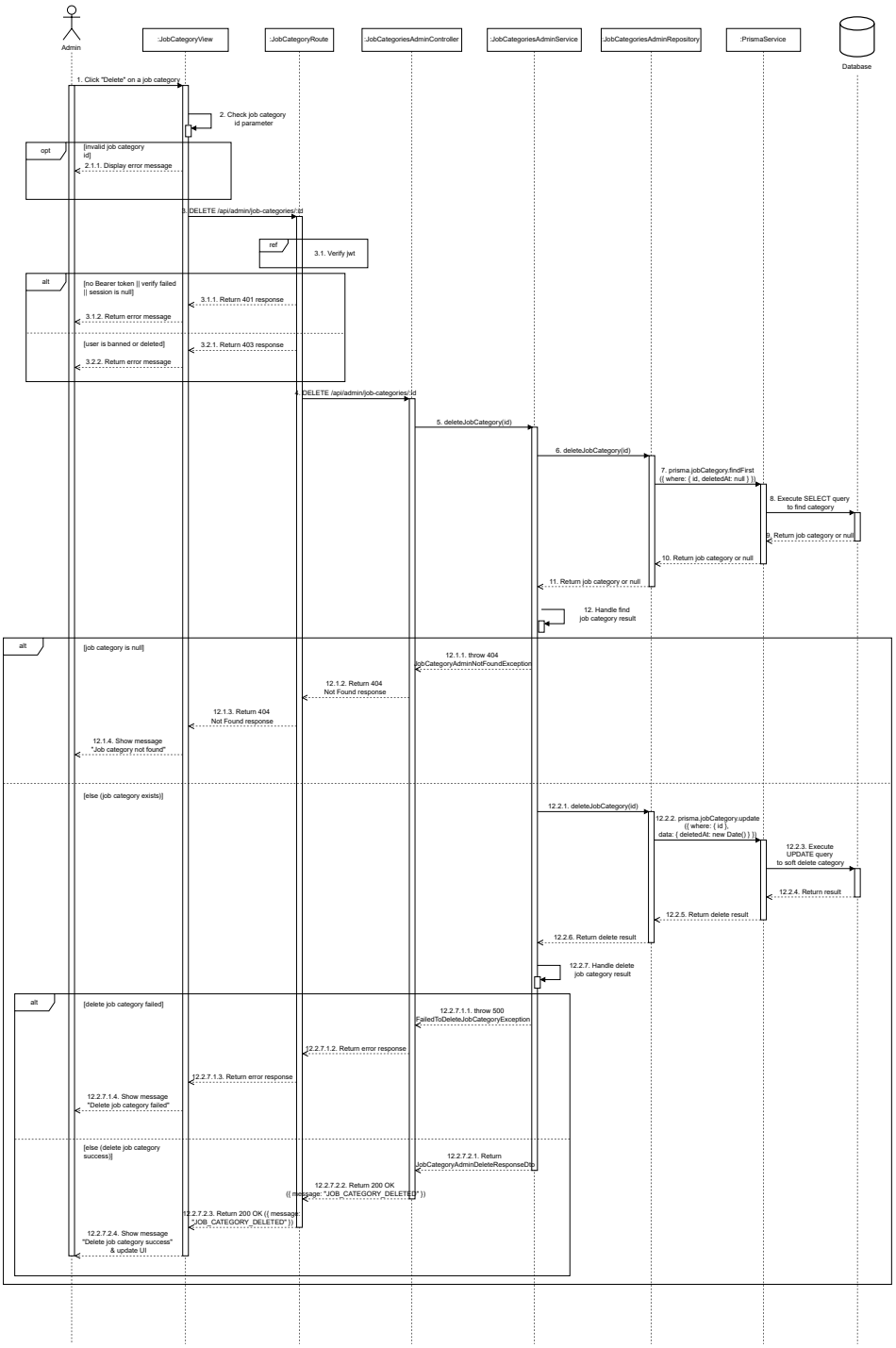

Figure 161: Sequence diagram of UC-46.05 Delete Job Category